

MACHINE LEARNING

Without the Magic

From vectors and derivatives to a Mixture-of-Experts transformer, every idea built from scratch: intuition first, then the math, then working PyTorch.

A 23-section course · complete edition

Contents

PART 1

- 1 Linear Algebra for ML
- 2 Calculus for ML
- 3 Probability & Statistics
- 4 Information Theory

PART 2

- 5 What Learning Means
- 6 Logistic Regression
- 7 Overfitting & Regularization
- 8 A Tour of Classical Models

PART 3

- 9 Neurons, Layers, Activations
- 10 Backpropagation by Hand
- 11 Build an MLP in NumPy
- 12 Training Dynamics & Optimizers

PART 4

- 13 Tensors & Autograd
- 14 nn.Module & the Training Loop
- 15 Debugging & Good Habits

PART 5

- 16 Convolutional Networks
- 17 Sequence Models: RNNs & LSTMs
- 18 Attention & Transformers
- 19 Language Modeling & GPT
- 20 Training at Scale

PART 6

- 21 Reading a Paper Like an Engineer
- 22 Mixture of Experts from Scratch
- 23 Where to Go Next

Linear Algebra for ML

*A neural network is, underneath, a stack of vectors being pushed through matrices.
Learn to read that sentence and you can read almost anything in ML.*

28 min read **You'll learn:** vectors · dot products · matrices as transformations · norms · eigenvalues

Every piece of data you will ever feed a model — an image, a sentence, a row in a spreadsheet — first becomes a **list of numbers**. Linear algebra is simply the set of rules for working with those lists efficiently. It is the grammar of machine learning, and the good news is you already have the intuition; we just need to attach the notation to it.

◆ THE ONE IDEA TO HOLD ONTO

Data becomes **vectors**. Operations on data become **matrices**. Learning is the search for the right numbers to put inside those matrices. That's the whole game — everything below is detail.

1 Vectors: a point, and an arrow

◆ INTUITION FIRST

A **vector** is just an ordered list of numbers. That's the boring definition. The useful one: a vector is an *arrow* in space that points from the origin (the center, position zero) to some spot. The numbers tell you how far to walk along each direction to reach that spot.

🔼 EVERYDAY PARALLEL

Think of a grocery order: *2 apples, 1 loaf of bread, 3 coffees*. Written as a vector that's `[2, 1, 3]`. Each slot always means the same thing (slot 1 is always apples), and the order never changes. A vector is nothing scarier than a labelled shopping list where you've agreed to drop the labels and just remember the order.

We write a vector as a column of numbers and give it a bold, lowercase name:

$$\mathbf{x} = \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix} \in \mathbb{R}^3$$

In words: " $\mathbf{x}$ is a vector living in $\mathbb{R}^3$ " — that symbol just means "a list of 3 real numbers." The little 3 is the *dimension*: how many numbers are in the list. An image with 784 pixels is a vector in $\mathbb{R}^{784}$.

◆ NOW THE MATH

Two things you can do to vectors, and both are exactly what they look like. You can **add** them (slot by slot) and you can **scale** them (multiply every slot by one number, called a *scalar*):

$$\mathbf{a} + \mathbf{b} = \begin{bmatrix} a_1 + b_1 \\ a_2 + b_2 \end{bmatrix}, \quad c \mathbf{a} = \begin{bmatrix} c a_1 \\ c a_2 \end{bmatrix}$$

In words: adding walks one arrow, then the other, tip to tail. Scaling by c stretches the arrow to c times its length (and flips it if c is negative). A **scalar** is just a plain single number, as opposed to a list.

2 The dot product: how aligned are two arrows?

◆ INTUITION FIRST

The **dot product** takes two vectors and returns a single number that measures *how much they point the same way*. Big positive number: they point in similar directions. Zero: they're at right angles (totally unrelated). Negative: they point in opposing directions. This one operation is doing quiet work inside almost every model you'll ever build.

🚩 EVERYDAY PARALLEL

A dating app scores your profile against someone else's: *+3 for shared love of hiking, +2 for both liking jazz, -1 because one of you hates cats*. Add those up and you get a single compatibility score. That summed-up score **is** a dot product – multiply matching slots, then add everything together. A high score means “aligned”; a score near zero means “no strong relationship either way.”

◆ NOW THE MATH

Multiply the vectors slot by slot, then add up all the results:

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^n a_i b_i = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n$$

In words: the big Σ (“sigma”) just says “add up the following as **i** counts from 1 to **n**.” So $\mathbf{a} \cdot \mathbf{b}$ = (pair up the slots, multiply each pair, sum the products). The output is always a single number, no matter how long the vectors are.

There is a second, geometric formula for the exact same number, and it's the one that reveals *why* the dot product measures alignment:

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta$$

In words: the dot product equals (length of **a**) × (length of **b**) × the cosine of the angle **θ** between them. Because **cos** is 1 when the angle is 0° and drops to 0 at 90°, the dot product is largest when the arrows are parallel and exactly zero when they are perpendicular. That's the alignment meter, made precise.

DEFINITION

Two vectors are **orthogonal** when their dot product is zero. “Orthogonal” is the mathematician's word for *at a right angle*, i.e. carrying no shared direction. It shows up constantly – it's the precise way to say “these two features are unrelated.”

◆ THE SAME THING IN PYTORCH

›_ DOT PRODUCT, TWO WAYS

```
import torch

a = torch.tensor([1.0, 2.0, 3.0])
b = torch.tensor([4.0, 5.0, 6.0])

# element-wise multiply, then sum – literally the definition
print((a * b).sum())          # tensor(32.)    = 1*4 + 2*5 + 3*6

# the idiomatic one-liner every codebase uses
print(torch.dot(a, b))        # tensor(32.)
```

CHECK YOURSELF

Vectors $\mathbf{u} = [1, 0]$ and $\mathbf{v} = [0, 1]$: what is $\mathbf{u} \cdot \mathbf{v}$, [Reveal ▼](#) and what does the result tell you geometrically?

ANSWER

$\mathbf{u} \cdot \mathbf{v} = (1)(0) + (0)(1) = 0$. A dot product of zero means the two arrows are **orthogonal** – at a perfect right angle. And indeed, $[1, 0]$ points along the x-axis while $[0, 1]$ points straight up the y-axis; they share no direction at all. This is exactly why $[1, 0]$ and $[0, 1]$ are called the standard *basis* of the 2D plane: two independent directions you can mix to reach any point.

3 Matrices as transformations

◆ INTUITION FIRST

A **matrix** is a grid of numbers – a stack of vectors, if you like. But the real insight, the one that makes deep learning click, is this: **a matrix is a machine that takes a vector in and returns a different vector out**. It rotates, stretches, squishes, and reflects space. When you multiply a matrix by a vector, you are asking the machine “where does this point end up after the transformation?”

🔼 EVERYDAY PARALLEL

A photo-editing slider is a transformation. “Increase brightness” takes every pixel and maps it to a new value. “Rotate 90°” takes every point and sends it somewhere new, in a consistent, predictable way. A matrix is exactly that: one fixed rule applied to every input vector. Feed it a different picture, same rule runs – that reusable rule is the whole point.

◆ NOW THE MATH

A matrix has rows and columns; we name it with a bold capital and write its shape as (*rows* × *columns*):

$$\mathbf{W} = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} \in \mathbb{R}^{2 \times 2}$$

To transform a vector, we do **matrix–vector multiplication**. The rule: each output slot is the dot product of one *row* of the matrix with the input vector.

$$\mathbf{W}\mathbf{x} = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} w_{11}x_1 + w_{12}x_2 \\ w_{21}x_1 + w_{22}x_2 \end{bmatrix}$$

In words: the top output number is (row 1) · (x). The bottom output number is (row 2) · (x). So a matrix is just *several dot products stacked up* — and you already understand dot products. Row 1 of **W** asks one question about **x**; row 2 asks another; the answers stack into the output vector.

◆ WHY THIS IS THE WHOLE OF DEEP LEARNING

A single layer of a neural network is exactly $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$ — transform the input with a matrix **W**, then shift it by a vector **b** (the *bias*). Stack a few of these with a bit of nonlinearity between them and you have a deep network — we build neurons, layers, and nonlinearity properly in [Section 9](#). When we say a model “learns,” we mean it is searching for the numbers to write inside **W** and **b**.

Matrix × matrix = do one transformation, then the next

Multiplying two matrices means *composing* their transformations: apply the right one first, then the left one. This is why order matters ($\mathbf{AB} \neq \mathbf{BA}$ in general — rotating then

stretching is not the same as stretching then rotating). The shape rule is the one thing you must memorize, because 90% of real ML bugs are shape mismatches:

THE SHAPE RULE

To multiply $(m \times n) \times (n \times p)$ the **inner numbers must match** (both n). The result has the shape of the **outer numbers**: $(m \times p)$. Read it as: “the middle two cancel, the outer two survive.”

◆ THE SAME THING IN PYTORCH

>_ A MATRIX TRANSFORMING A VECTOR

```
import torch

W = torch.tensor([[2.0, 0.0],      # stretch x by 2, leave y alone
                  [0.0, 1.0]])
x = torch.tensor([3.0, 4.0])

y = W @ x                        # '@' is matrix multiplication in Python
print(y)                        # tensor([6., 4.]) → x doubled, y unchanged

# shapes are everything. print them when confused:
A = torch.randn(2, 3)           # (2 x 3)
B = torch.randn(3, 5)           # (3 x 5) inner 3s match
print((A @ B).shape)            # torch.Size([2, 5]) → outer numbers survive
```

▲ THE BUG YOU WILL HIT FIRST

`RuntimeError: mat1 and mat2 shapes cannot be multiplied (2x3 and 5x3)`. The inner numbers (3 and 5) don't match. Ninety percent of the time the fix is to *transpose* one matrix – flip its rows and columns with `.T` – so the inner dimensions line up. When in doubt, `print(tensor.shape)` on every line.

CHECK YOURSELF

You have a batch of 64 images, each flattened to a 784-length vector, stored as a matrix **X** of shape (64, 784).

Reveal ▼

You want to pass them through a layer with weight matrix **W**. What shape must **W** be to produce 10 outputs per image, and what's the output shape?

ANSWER

We compute **X @ W** with **X** of shape (64, 784). For the inner numbers to match, **W** must start with 784: its shape is (784, 10). By the shape rule, $(64, 784) \times (784, 10) \rightarrow$ the outer numbers survive, so the output is (64, 10): 10 numbers for each of the 64 images. This is exactly the shape of a linear layer that turns pixels into 10 class scores – you just designed the last layer of a digit classifier.

4 Norms: measuring the size of a vector

◆ INTUITION FIRST

A **norm** answers a simple question: “how big is this vector?” It collapses a whole list of numbers into a single measure of length or magnitude. We need this constantly – to measure how far a prediction is from the truth, or how large a model's weights have grown.

🚩 EVERYDAY PARALLEL

Two ways to measure the distance from your home to a friend's flat. The **L2** way is the straight-line “as the crow flies” distance – what a ruler on a map shows. The **L1** way is the “taxicab” distance – you can only drive along the street grid, so you add up the blocks east plus the blocks north. Same start and end, two honest but different notions of “how far,” and ML uses both on purpose.

◆ NOW THE MATH

The **L2 norm** (the default, “Euclidean length”) is the Pythagorean theorem in any number of dimensions – square each slot, add them, take the square root:

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2} = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$$

The **L1 norm** just adds up the sizes (absolute values) of the slots – no squaring, no square root:

$$\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i| = |x_1| + |x_2| + \cdots + |x_n|$$

In words: L2 punishes one big value harshly (because it squares it), so it prefers many small values over one large one. L1 treats all values evenly and, as you'll see in [Section 7](#), has a knack for pushing weights all the way to exactly zero. That single difference is the entire reason we have two flavours of *regularization* (a technique for keeping models simple), so keep it in your pocket.

◆ THE SAME THING IN PYTORCH

```
import torch
x = torch.tensor([3.0, 4.0])

print(torch.norm(x, p=2))    # tensor(5.)    sqrt(3^2 + 4^2) = sqrt(25)
print(torch.norm(x, p=1))    # tensor(7.)    |3| + |4|
```

5 Eigenvalues: the directions a matrix leaves alone

◆ INTUITION FIRST

We said a matrix stretches and rotates space. For most input directions, the arrow comes out pointing somewhere new. But almost every matrix has a few special directions that it *doesn't* rotate – it only stretches or shrinks them, leaving them pointing the same way. Those special directions are **eigenvectors**, and the amount each one gets stretched is its **eigenvalue**. (“Eigen” is German for “own” or “characteristic” – these are the matrix's *own* favoured directions.)

🚩 EVERYDAY PARALLEL

Spin a globe. Almost every city on the surface swings around to a new position. But the two poles stay put — they sit exactly on the axis of rotation. That axis is the globe's eigenvector: the one direction the “spin” transformation leaves pointing the same way. Eigenvectors are the poles of whatever a matrix does.

◆ NOW THE MATH

The defining equation is short and says exactly what we described in words:

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$$

In words: “applying the matrix **A** to its special vector **v** gives back the *same* vector **v**, merely scaled by the number **λ** (lambda).” The transformation **A** — which normally rotates things — acts on **v** like plain multiplication by a number. No new direction, just a stretch by **λ**.

Why should you care, before we've even built a model? Because eigenvalues quietly explain a lot of ML behaviour you'll meet later:

- **PCA** (Principal Component Analysis, a way to compress data) finds the eigenvectors of your data — the directions along which the data varies most, so you can keep those and throw the rest away.
- **Exploding and vanishing gradients** (a training failure we'll hit in [Section 17](#)) happen when repeated matrix multiplications have eigenvalues bigger or smaller than 1 — the same stretch applied over and over blows up or fades to nothing.

◆ SEEING IT IN PYTORCH

```
import torch

A = torch.tensor([[2.0, 0.0],
                  [0.0, 3.0]])           # a pure stretch: x by 2, y by 3

vals = torch.linalg.eigvals(A).real
print(vals)                             # tensor([2., 3.]) the two
stretch factors

# check the definition on the x-axis direction v = [1, 0]:
v = torch.tensor([1.0, 0.0])
```

```
print(A @ v)
eigenvalue 2
```

```
# tensor([2., 0.]) = 2 * v →
```

CHECK YOURSELF

A matrix has an eigenvalue of 0.5 along some direction. If you apply that matrix to a vector pointing in that direction, over and over, 20 times – what happens to the vector's length? Reveal ▼

ANSWER

Each application multiplies the length by 0.5, so after 20 applications the length is

$0.5^{20} \approx 0.00000095$ of what it started as – practically zero. The vector *vanishes*.

Flip the eigenvalue to 2 instead and you'd get $2^{20} \approx 1,000,000$ × – it *explodes*. This is precisely the vanishing/exploding-gradient problem that makes deep and recurrent networks hard to train, and it's why eigenvalues near 1 are so prized. You just previewed one of the deepest issues in the whole course.

6 What to carry forward

- **Vectors** are lists of numbers you can picture as arrows; data becomes vectors.
- The **dot product** multiplies matching slots and sums them – an alignment meter that returns one number.
- A **matrix** is a reusable transformation; matrix–vector multiply is just several dot products stacked. One neural layer is $\mathbf{W}\mathbf{x} + \mathbf{b}$.
- The **shape rule**: inner dimensions must match, outer dimensions survive. Print shapes to kill bugs.
- **Norms** measure vector size (L2 = straight line, L1 = taxicab); **eigenvalues** are the stretch factors along a matrix's favoured directions.

Next we'll learn how to *change* those numbers on purpose – which means calculus, and the single most important idea in the whole course: the gradient.

Calculus for ML

The chain rule is not a dusty calculus footnote. It is backpropagation – the exact rule that lets a network of millions of numbers figure out how to improve itself.

32 min read You'll learn: derivatives · partial derivatives · gradients · the chain rule

Last section we learned to *represent* data as vectors and to push it through matrices. But a model that never changes is useless. Learning means nudging the numbers inside those matrices, over and over, in the direction that makes the model a little less wrong. Calculus is the mathematics of **change** – it is the tool that tells us which way, and how hard, to nudge. This one section contains the single most important idea in the whole course, so we'll go slowly and derive everything.

◆ THE ONE IDEA TO HOLD ONTO

A **derivative** tells you which way to push a number to make an output go up or down. A neural network learns by asking, for every one of its numbers, “if I wiggle you a tiny bit, does the error get better or worse?” – then wiggling accordingly. That question is a derivative, and answering it efficiently for millions of numbers at once is the entire trick behind training. Everything below builds to that.

1 Derivatives: the slope at a single instant

◆ INTUITION FIRST

A **derivative** measures how fast something is changing *right now*. If a quantity y depends on an input x , the derivative answers: “if I increase x by a tiny amount, how much does y move, and in which direction?” Geometrically it is the **slope** of the curve at one point – how steeply it rises or falls at that exact spot. A positive slope means “going up”; a negative slope means “going down”; a slope of zero means “flat, for a moment.”

🚗 EVERYDAY PARALLEL

Look at your car's **speedometer**. The odometer tells you total distance travelled; the speedometer tells you how fast that distance is changing *at this instant* – that is a derivative. Sixty km/h means “if this instant held, you'd cover 60 km in the next hour.” You don't have to wait an hour to know your speed now; the derivative reads the rate of change in the moment. Speed is the derivative of distance.

◆ NOW THE MATH

Here is the honest problem. Slope is “rise over run” – change in output divided by change in input – but at a *single point* there is no run to divide by. The clever fix: measure the slope over a tiny step of width h , then shrink h toward zero and see what number the slope settles on. That settling-down process is a *limit* (the value an expression approaches as you push a variable toward something). This gives the definition of the derivative:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

In words: take two points on the curve a tiny distance h apart — one at x , one at $x+h$. The fraction is (change in output) ÷ (change in input): the slope of the little line joining them. As h shrinks to nothing ($h \rightarrow 0$), that little line hugs the curve tighter and tighter, until its slope becomes the slope *at the single point* x . The result $f'(x)$ (read “f-prime of x”) is the derivative.

Let's actually compute one from scratch, so the limit stops being abstract. Take $f(x) = x^2$ and grind the definition:

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} (2x + h) \\ &= 2x \end{aligned}$$

In words: we expanded $(x+h)^2$, the two lone x^2 terms cancelled, then we divided every surviving term by h . That left $2x + h$. Now let h fall to zero and the leftover h simply vanishes,

leaving $2x$. So the slope of x^2 at any point is $2x$: at $x = 3$ the curve rises with slope 6; at $x = 0$ the slope is 0 (the bottom of the bowl, momentarily flat). We didn't guess — we derived it.

DEFINITION

The **derivative** of a function is another function that reports the original's slope at every point. We write it $f'(x)$ or $\frac{df}{dx}$ (read “dee-f dee-x”), which literally means “a tiny change in f per tiny change in x .” Both notations mean the same thing.

◆ THE SAME THING IN PYTORCH

PyTorch never runs the limit by hand — it applies known rules like our $2x$ automatically. You mark an input as something to track, compute an output, and call `.backward()` to get the derivative:

>_ THE DERIVATIVE OF x^2 AT $x = 3$

```
import torch

x = torch.tensor(3.0, requires_grad=True)  # "watch this number"
y = x ** 2                                #  $f(x) = x^2$ 

y.backward()                               # compute  $df/dx$  at  $x = 3$ 
print(x.grad)                             # tensor(6.)  $\rightarrow 2*x = 2*3 = 6$ 
```

In words: `requires_grad=True` tells PyTorch to remember how `y` was built from `x`. `backward()` walks that record and fills `x.grad` with the derivative — exactly the $2x = 6$ we derived. You'll rarely differentiate by hand again, but you must know what the machine is computing.

CHECK YOURSELF

Using the limit definition, what is the derivative of the constant function $f(x) = 5$? What does the answer mean?

Reveal ▼

ANSWER

Plug in: $f(x + h) = 5$ and $f(x) = 5$, so the top of the fraction is $5 - 5 = 0$.

That gives $0 / h = 0$ for every non-zero h , so the limit is **0**. Meaning: a flat, horizontal line has slope zero everywhere – a constant never changes, so its rate of change is nothing. This is why *bias* terms don't affect the slope with respect to an input, and why constants drop out when you differentiate.

2 Partial derivatives: change one input, freeze the rest

◆ INTUITION FIRST

Real models don't depend on a single number – they depend on thousands. A function like $f(x, y)$ has two inputs, and “the slope” is ambiguous: slope as we change x , or as we change y ? A **partial derivative** answers one at a time. It measures how the output changes as you nudge *one single input*, while holding all the others perfectly still. Nothing new to fear – it's the ordinary derivative from the start of this section, just with everything except the chosen variable treated as a frozen constant.

☂ EVERYDAY PARALLEL

Your shower has two taps: **hot** and **cold**. The water temperature depends on both. To learn what the hot tap alone does, you leave the cold tap exactly where it is and turn only the hot one, feeling how the temperature responds. That is a partial derivative: the effect of one input while the other is held fixed. Turn only the cold tap instead and you're measuring the other partial. Each tap has its own separate influence, and you probe them one at a time.

◆ NOW THE MATH

We swap the straight d for a curly ∂ (“partial”) to signal “there are other inputs, but I’m holding them still.” To take $\frac{\partial f}{\partial x}$, treat every other variable as a plain number. Take $f(x, y) = x^2y + 3y$:

$$\frac{\partial f}{\partial x} = 2xy, \quad \frac{\partial f}{\partial y} = x^2 + 3$$

In words: for $\partial f / \partial x$ we treat y as a frozen constant. Then x^2y differentiates like $(\text{constant}) \cdot x^2$, giving $2xy$, and the $3y$ term has no x in it at all, so it’s constant and drops to zero. For $\partial f / \partial y$ we freeze x instead: x^2y is $(\text{constant}) \cdot y$, whose slope in y is just that constant x^2 , and $3y$ contributes 3 . Same rules as before — only one variable moves at a time.

◆ THE SAME THING IN PYTORCH

>_ TWO PARTIALS AT ONCE

```
import torch

x = torch.tensor(2.0, requires_grad=True)
y = torch.tensor(5.0, requires_grad=True)

f = x**2 * y + 3*y          # f(x, y) = x^2 * y + 3y
f.backward()                # fills in BOTH partials

print(x.grad)               # tensor(20.) → 2*x*y = 2*2*5 = 20
print(y.grad)               # tensor(7.) → x^2 + 3 = 4 + 3 = 7
```

In words: one call to `backward()` computes the partial with respect to every tracked input in a single sweep. That’s the whole reason training is feasible: the machine gets the influence of *all* the knobs together, not one slow experiment per knob.

CHECK YOURSELF

For $f(x, y) = 4x + y^3$, what is $\partial f / \partial x$? Why does the y^3 term disappear?

Reveal ▼

ANSWER

$\partial f / \partial x = 4$. When we differentiate with respect to x , we freeze y – so y^3 is just some fixed constant number, and the derivative of any constant is zero (we saw this at the start of this section). The $4x$ term has slope 4 in x . So $\partial f / \partial x = 4$. The y^3 term only shows up if you ask for $\partial f / \partial y$, which would be $3y^2$.

3 The gradient: the direction of steepest climb

◆ INTUITION FIRST

If a function has many inputs, collect *all* its partial derivatives into one vector. That vector is the **gradient**, written ∇f (the symbol ∇ is called “nabla” or just “del”). The gradient is more than bookkeeping: it points in the direction where the function **increases fastest**, and its length says how steep that increase is. Flip its sign and you get the direction of steepest *decrease* – which, when the function is our error, is exactly the way we want to move.

☂ EVERYDAY PARALLEL

You're on a mountainside in **thick fog**, trying to get down. You can't see the valley, but you can feel the ground under your feet. You sense which way tilts most steeply downward, take a step that way, then feel again and repeat. The steepest direction under your feet is the (negative) gradient; the fog is your model's ignorance of the whole landscape. You never see the map – you only ever feel the local slope, and that is enough to walk downhill.

◆ NOW THE MATH

For a function of n inputs, the gradient is the column of every partial derivative, one per input:

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$$

In words: slot i of the gradient is “how much the output changes when I nudge input i .” Stacked together, these slots form an arrow in input-space. That arrow aims at the fastest uphill direction — big values mean inputs that matter a lot right now, near-zero values mean inputs that barely move the output at this point.

DEFINITION

Gradient descent is the recipe for minimizing a function: from wherever you stand, compute the gradient, then take a small step in the *opposite* direction (downhill). Repeat. Because the negative gradient is the steepest way down, each step lowers the value a little, and enough steps carry you toward a minimum. In ML the function being minimized is the **loss** (a number measuring how wrong the model is — smaller is better), so descending it means the model gets better.

The step itself is one short update rule, applied again and again:

$$\theta \leftarrow \theta - \alpha \nabla L$$

In words: θ (“theta”) stands for the model’s parameters — all the numbers inside those weight matrices from Section 1. ∇L is the gradient of the loss L with respect to those parameters: the uphill-toward-more-error direction. We subtract it, so we move toward *less* error. α (“alpha”) is the *learning rate*: a small positive number setting how big each step is. The arrow $\leftarrow$ means “compute the right side, then overwrite θ with it.” Do this thousands of times and the parameters drift to values that make the loss small.

▲ WATCH OUT

The learning rate α is the step size, and it is delicate. Too **small** and training crawls – thousands of tiny steps to get anywhere. Too **large** and you overshoot the valley on every step, bouncing from wall to wall or flying off to infinity (the loss becomes `NaN` – “not a number”). Picking α is the first knob you’ll tune on almost every model.

◆ THE SAME THING IN PYTORCH

> GRADIENT DESCENT ON $F(X) = X^2$, WALKING TO THE BOTTOM

```
import torch

x = torch.tensor(10.0, requires_grad=True)  # start far from the minimum
lr = 0.1                                    # learning rate (alpha)

for step in range(5):
    loss = x ** 2                          # we want to minimize this
    loss.backward()                        # grad = df/dx = 2x
    with torch.no_grad():                  # don't track the update itself
        x -= lr * x.grad                  # theta ← theta - alpha * grad
    x.grad.zero_()                         # clear grad before the next step
    print(f"x = {x.item():.4f}")

# x marches 10 → 8 → 6.4 → ... toward 0, the minimum of x^2
```

In words: each loop reads the slope at the current spot, steps against it by `lr * grad`, and the value slides down the bowl toward zero. Clearing `x.grad` matters because PyTorch *adds* new gradients onto old ones; forget it and last step's slope contaminates this one.

CHECK YOURSELF

You're minimizing a loss and the gradient with respect to a weight is $+4$. With learning rate 0.1 , does the weight go up or down, and by how much? Reveal ▼

ANSWER

The update is $\theta \leftarrow \theta - \alpha \cdot \nabla L = \theta - 0.1 \cdot (+4) = \theta - 0.4$. The weight goes **down by 0.4**. A positive gradient means “increasing this weight increases the loss,” so to reduce the loss we push the weight the other way – downward. The minus sign in the update rule is precisely what turns “which way is uphill” into “step downhill.”

4 The chain rule: this is backpropagation

◆ INTUITION FIRST

Neural networks are **compositions**: functions wrapped inside functions inside functions. The input feeds layer 1, whose output feeds layer 2, whose output feeds layer 3, and so on to the loss at the end. To train, we need the derivative of that final loss with respect to a weight buried deep inside – many layers away from the output. The **chain rule** is the tool that reaches back through the whole stack. Its idea is beautifully simple: the derivative of a chain of functions is the *product* of the little derivatives along the chain. Each link contributes its own local rate, and you multiply them together.

🌂 EVERYDAY PARALLEL

You're changing money: **US dollars → euros → pounds**. Say 1 dollar buys 0.9 euros, and 1 euro buys 0.85 pounds. How many pounds does 1 extra dollar get you? You *multiply* the rates: $0.9 \times 0.85 = 0.765$ pounds per dollar. Each conversion is a link in a chain, and the overall rate is the product of the link rates. Gears mesh the same way: if one gear turns twice as fast as the next, and that one twice as fast as the last, the first spins four times the last – rates multiply along the chain. That multiplying-of-rates is the chain rule.

◆ NOW THE MATH

Suppose y depends on u , and u depends on x — a two-link chain $x \rightarrow u \rightarrow y$. The chain rule says:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

In words: “how y changes as x moves” equals “how y changes as u moves” times “how u changes as x moves.” The middle variable u cancels visually, like a fraction — a handy way to remember it. Each factor is a *local* derivative, easy on its own; the chain rule glues them by multiplication.

Let's work a full single-variable example. Take $y = (3x + 1)^2$. We split it into the inner and outer pieces:

$$u = 3x + 1, \quad y = u^2$$

Now the two local derivatives, each trivial on its own:

$$\frac{dy}{du} = 2u, \quad \frac{du}{dx} = 3$$

Multiply them, then substitute $u = 3x + 1$ back in:

$$\begin{aligned} \frac{dy}{dx} &= \frac{dy}{du} \cdot \frac{du}{dx} \\ &= 2u \cdot 3 \\ &= 6u \\ &= 6(3x + 1) \\ &= 18x + 6 \end{aligned}$$

In words: we found the outer function's slope ($2u$, the derivative of a square), the inner function's slope (3 , the derivative of $3x + 1$), and *multiplied* them. Then we put $u = 3x + 1$ back so the answer is in terms of the original x . We can check it: expanding $(3x + 1)^2 = 9x^2 + 6x + 1$, whose ordinary derivative is $18x + 6$ — a perfect match. The chain rule gave the same answer without ever expanding the square.

The multivariable chain rule: sum over every path

In a network a variable usually influences the output through *several* routes at once. When that happens, the chain rule gains one addition: differentiate along each path, then **add the paths up**. If L depends on a and b , and both of those depend on x :

$$\frac{dL}{dx} = \frac{\partial L}{\partial a} \cdot \frac{da}{dx} + \frac{\partial L}{\partial b} \cdot \frac{db}{dx}$$

In words: x reaches L two ways — through a and through b . Each route is a chain-rule product (multiply the rates along it), and because both routes carry influence, we *sum* them. More paths, more terms in the sum. Every knob's total effect is the sum of its effects along all routes to the output. That's the whole rule.

◆ WHY THIS IS THE BEATING HEART OF ML

Backpropagation is nothing but the chain rule applied to a neural network. The network is a long composition of layers; the loss sits at the far end. To learn, we need the gradient of the loss with respect to every weight, however deep. Backprop computes it by starting at the loss and multiplying local derivatives layer by layer as it walks *backward* — exactly $\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx}$, chained across the whole network, summing wherever paths merge. When PyTorch's `.backward()` ran in the code above, this is precisely what it did. We build it by hand and see every multiplication in [Section 10](#).

◆ THE SAME THING IN PYTORCH

>_ THE CHAIN RULE, COMPUTED FOR US

```
import torch

x = torch.tensor(2.0, requires_grad=True)
y = (3*x + 1) ** 2           # the composed function from above

y.backward()                # chain rule runs automatically
print(x.grad)               # tensor(42.) → 18*x + 6 = 18*2 + 6 = 42
```

In words: PyTorch recorded that y was built as a square of $(3x + 1)$. `backward()` walked that chain outer-to-inner, multiplying $2u$ by 3 just as we did by hand, and delivered 42 — matching our derived $18x + 6$ at $x = 2$. The rule you just learned is literally the engine under the hood.

CHECK YOURSELF

Let $y = (x^2 + 1)^3$. Using the chain rule with inner $u = x^2 + 1$, find dy/dx . Reveal ▼

ANSWER

Outer: $y = u^3$ gives $dy/du = 3u^2$. Inner: $u = x^2 + 1$ gives $du/dx = 2x$.

Multiply them: $dy/dx = 3u^2 \cdot 2x = 6x \cdot u^2 = 6x(x^2 + 1)^2$. Notice how the answer stays compact – the chain rule spared us from expanding a cube. This is exactly why networks can be deep: each layer just contributes one more local derivative to multiply into the chain.

5 The Jacobian: a gradient for every output at once

◆ INTUITION FIRST

So far the output was a single number. But a layer usually maps a vector of inputs to a vector of outputs, so there are many partial derivatives to track: each output slot has its own slope against each input slot. Arrange all of them in a grid – one row per output, one column per input – and you get the **Jacobian** matrix. It is simply the gradient idea scaled up: where a single-output function has one gradient *vector*, a multi-output function has a whole *matrix* of partials. The multivariable chain rule for vectors then becomes plain matrix multiplication of Jacobians – which is why Section 1's shape rules and this section's chain rule meet inside every real network, and why backprop is, at bottom, a sequence of matrix products.

6 What to carry forward

- A **derivative** is an instantaneous rate of change – the slope at a single point, defined by shrinking a step h to zero. The slope of x^2 is $2x$.

- A **partial derivative** is the ordinary derivative with respect to one input while all others are frozen – the hot tap while the cold stays put.
- The **gradient** ∇f stacks every partial into a vector that points uphill fastest. **Gradient descent** steps the other way to shrink a loss: $\theta \leftarrow \theta - \alpha \nabla L$, with α the learning rate.
- The **chain rule** multiplies local derivatives along a composition, and sums over paths when they merge. This *is* backpropagation – the exact mechanism we build in [Section 10](#).
- The **Jacobian** generalizes the gradient to many outputs, turning the vector chain rule into matrix multiplication.

You now hold the two halves of ML's foundation: linear algebra to move data, and calculus to improve the model. Next we bring in the third – probability – so a model can express not just an answer, but how sure it is.

Probability & Statistics

Models don't output answers, they output beliefs. Probability is the language we use to reason carefully about things we are not sure of – and, quietly, it's where every loss function is born.

30 min read **You'll learn:** distributions · expectation · variance · Bayes' rule · maximum likelihood

Machine learning lives in a world of uncertainty. Will this email turn out to be spam? What digit is in this blurry photo? A good model never bangs its fist on the table with a single certain answer – it hands you a **degree of belief**: “90% spam,” “probably a 7, maybe a 1.” Probability is the mathematics of those graded beliefs, and statistics is how we tune them from data. This section builds that toolkit from the ground up.

◆ THE ONE IDEA TO HOLD ONTO

A model outputs a **distribution** – a spread of belief across the possible answers, not one answer. Everything in this section is either a way to *describe* such a distribution (expectation, variance), a common *shape* one takes (Bernoulli, Gaussian), or a way to *update or fit* one from evidence (Bayes, maximum likelihood). Hold that map in your head and the symbols fall into place.

1 Random variables & distributions

◆ INTUITION FIRST

A **random variable** is just a number whose value is uncertain until you look – the outcome of some process you can't predict exactly. Roll a die and call the result X : before the roll, X could be any of 1–6. A **distribution** is the full accounting of that uncertainty: it tells you, for every possible value, how likely that value is. If a random variable is the question (“what will

come up?”), the distribution is the complete answer key with the odds written next to each option.

Random variables come in two flavours, and the difference matters for every formula below:

- **Discrete** – the values come in separate, countable lumps: 1, 2, 3... You can list them. A die, a coin, the number of emails in your inbox.
- **Continuous** – the values slide smoothly along a range with no gaps: any real number in some interval. A temperature, a height, a waiting time.

🚩 EVERYDAY PARALLEL

A **die roll** versus **tomorrow's temperature**. The die is discrete: six neat outcomes, each with a clean $1/6$ chance, and it makes sense to ask “what's the probability of exactly a 4?” The temperature is continuous: it could be 21.0° , 21.3° , 21.3007° ... Asking “what's the chance it's *exactly* $21.30000...^\circ$?” is the wrong question – there are infinitely many values, so any single one has essentially zero chance. Instead you ask “what's the chance it lands *between* 21° and 22° ?” That shift – from exact values to ranges – is the whole difference between the two worlds.

◆ NOW THE MATH

For a **discrete** variable we use a *probability mass function*, written $P(X = x)$ – the amount of probability sitting *on* each specific value. Two honest rules hold: every mass is between 0 and 1, and all the masses add up to exactly 1 (something must happen).

$$0 \leq P(X = x) \leq 1, \quad \sum_x P(X = x) = 1$$

In words: each outcome gets a share of belief between “impossible” (0) and “certain” (1), and the shares of all outcomes together make one whole. For a fair die, $P(X = x) = 1/6$ for each of the six faces, and $6 \times \frac{1}{6} = 1$. The function is called “mass” because you can picture a lump of probability weight parked on each value.

For a **continuous** variable, single values have zero probability, so we use a *probability density function* $p(x)$ instead. Density is not probability itself – it's probability *per unit of x*, like grams per centimetre. To get an actual probability you must sum the density over a range, which for a smooth curve means taking an **integral** (the area under the curve):

$$P(a \leq X \leq b) = \int_a^b p(x) dx, \quad \int_{-\infty}^{\infty} p(x) dx = 1$$

In words: the chance that X falls between **a** and **b** is the *area* under the density curve over that stretch. The $\int$ sign is just a continuous cousin of the $\sum$ sign — “add up over a smooth range” instead of “add up over a list.” And the total area under the whole curve is 1, the continuous version of “the masses sum to 1.”

DEFINITION

A **distribution** is the complete description of how likely each value of a random variable is. For discrete variables it's given by a **PMF** (mass on each value); for continuous variables by a **PDF** (a density curve whose *area* gives probability). Both always sum/integrate to 1.

◆ THE SAME THING IN PYTORCH

›_ A DISCRETE DISTRIBUTION (A LOADED DIE) AND SAMPLING FROM IT

```
import torch

# a PMF over 6 faces – six numbers that must sum to 1
probs = torch.tensor([0.10, 0.10, 0.10, 0.10, 0.10, 0.50]) # face 6 is
loaded
print(probs.sum()) # tensor(1.0000) → a valid
distribution

die = torch.distributions.Categorical(probs=probs)

# draw 10,000 rolls and see the loaded face show up ~half the time
samples = die.sample((10_000,))
print((samples == 5).float().mean()) # ~0.50 (face index 5 = the
'6')
```

CHECK YOURSELF

Someone hands you a continuous PDF and asks “what is $P(X = 3.0)$ exactly?” Why is the honest answer 0, and what should they have asked instead?

Reveal ▼

ANSWER

For a continuous variable there are infinitely many possible values packed into any interval, so the probability piled on any single exact point is zero – the area under a curve over a zero-width slice is zero. The density $p(3.0)$ can be large, but density is not probability. The right question is about a *range*, e.g. $P(2.9 \leq X \leq 3.1)$, which is the area under the curve between those bounds. This is the single most common confusion when people first meet PDFs.

2 Expectation and variance

◆ INTUITION FIRST

Once you have a distribution, the first thing you want is a one-number summary: “on average, what value should I expect?” That's the **expectation** (also called the *mean* or *expected value*), written $\mathbb{E}[X]$. It is the long-run average you'd converge to if you repeated the random process forever. Crucially, it's a *weighted* average: outcomes that are more likely pull the average toward them harder.

🚩 EVERYDAY PARALLEL

A slot machine pays *0 dollars most spins, but 100 dollars one time in fifty*. What do you win **per spin on average**? Not 100 dollars (that's rare), not 0 dollars (you do sometimes win). It's each payout weighted by how often it happens:

$\frac{49}{50} \times \$0 + \frac{1}{50} \times \$100 = \$2$ per spin. That 2 dollars is the expectation. The casino sets the machine so your expected win is less than what you paid to play – the “house edge” is just an expectation calculation running in their favour, spin after spin.

◆ NOW THE MATH

For a discrete variable, expectation is the probability-weighted sum of the values:

$$\mathbb{E}[X] = \sum_x x P(X = x)$$

In words: go through every possible value x , multiply it by how likely it is, and add all those products up. Likely values contribute more; rare ones barely nudge the total. The result is the balance point of the distribution — the spot where it would sit level on a seesaw.

Let's derive one concretely. A **fair die**: values 1–6, each with probability $1/6$.

$$\begin{aligned}\mathbb{E}[X] &= 1 \cdot \frac{1}{6} + 2 \cdot \frac{1}{6} + 3 \cdot \frac{1}{6} + 4 \cdot \frac{1}{6} + 5 \cdot \frac{1}{6} + 6 \cdot \frac{1}{6} \\ &= \frac{1}{6} (1 + 2 + 3 + 4 + 5 + 6) \\ &= \frac{1}{6} \cdot 21 = 3.5\end{aligned}$$

In words: the expected roll is 3.5. Note you can never actually *roll* a 3.5 — the expectation is not a possible outcome, it's the long-run average. Roll ten thousand times and the running average will settle very close to 3.5.

The mean tells you *where* a distribution sits. The next question is *how spread out* it is — do outcomes cluster tightly around the mean, or scatter wildly? That's the **variance**, written $\text{Var}[X]$. We measure each outcome's distance from the mean $\mu = \mathbb{E}[X]$, *square* it (so that being too high and too low both count as spread, and never cancel out), and take the expected value of that:

$$\text{Var}[X] = \mathbb{E}[(X - \mu)^2]$$

In words: variance is the average squared distance from the mean. Big variance means outcomes routinely land far from the average (a wild, unpredictable variable); small variance means they hug it (a steady, reliable one). We square the distances so that a swing of $+3$ and a swing of -3 both count as spread rather than erasing each other. The square root of variance, σ , is the *standard deviation* — the same idea back in the original units.

🔼 EVERYDAY PARALLEL

Two basketball players both average 20 points a game – same expectation. But one scores 19, 21, 20, 20, 20 (a **reliable** player, tiny variance), while the other scores 2, 40, 5, 38, 15 (a **streaky** player, huge variance). The mean can't tell them apart; variance is exactly the number that does. A coach picking who to trust in the final seconds cares about variance as much as the average.

◆ THE SAME THING IN PYTORCH

```
import torch

# simulate the fair die and check both summaries empirically
rolls = torch.randint(1, 7, (100_000,)).float() # 100k rolls of 1..6

print(rolls.mean()) # ~3.50 matches E[X] we derived by hand
print(rolls.var(unbiased=False)) # ~2.92 the spread around 3.5

# reliable vs streaky: same mean, very different variance
reliable = torch.tensor([19., 21., 20., 20., 20.])
streaky = torch.tensor([ 2., 40.,  5., 38., 15.])
print(reliable.mean(), streaky.mean()) # both 20.0
print(reliable.var(), streaky.var()) # ~0.5 vs ~326.5
```

CHECK YOURSELF

A bet pays **+\$10** with probability 0.2 and **-\$3** with probability 0.8. What is the expected payout, and should you take the bet?

Reveal ▼

ANSWER

$E[\text{payout}] = (0.2)(+10) + (0.8)(-3) = 2.0 - 2.4 = -\0.40 . On average you lose 40 cents every time you play, even though the win is bigger than the loss – because the loss happens four times as often. A rational one-number decision says skip it. (In real life you'd also weigh variance and how much you can afford to lose, but the expectation is the first thing to check.)

3 Two distributions you'll meet everywhere

Most of ML leans on a small handful of standard distribution shapes. Two of them show up so often they're worth knowing by heart.

The Bernoulli: one yes-or-no event

◆ INTUITION FIRST

The **Bernoulli distribution** is the simplest one there is: a single event with exactly two outcomes, 1 (“success”) or 0 (“failure”), governed by one number p – the probability of the 1. A coin flip, a click-or-no-click, a spam-or-not-spam decision. Whenever a model answers a single yes/no question, a Bernoulli is hiding underneath.

☂ EVERYDAY PARALLEL

A single coin flip with a possibly-bent coin. If the coin comes up heads 70% of the time, then $p = 0.7$: heads (1) has probability 0.7, tails (0) has probability 0.3. That one number p is the entire distribution – nothing else to know.

◆ NOW THE MATH

Its PMF can be written as a single tidy line that works for both outcomes:

$$P(X = x) = p^x (1 - p)^{1-x}, \quad x \in \{0, 1\}$$

In words: plug in $x = 1$ and the formula collapses to p (the chance of a 1); plug in $x = 0$ and it collapses to $1 - p$ (the chance of a 0). The exponents are a clever switch: whichever outcome you ask about, the other term's exponent becomes 0 and vanishes (anything to the power 0 is 1). Its mean is $\mathbb{E}[X] = p$ and its variance is $p(1 - p)$ — biggest at $p = 0.5$, the most unpredictable coin.

The Gaussian: the bell curve

◆ INTUITION FIRST

The **Gaussian** (also called the **Normal** distribution) is the famous bell curve: a smooth hump, tall in the middle and tailing off symmetrically on both sides. It's the default shape for a continuous quantity that clusters around a typical value with random wobble either side — heights, measurement errors, exam scores, sensor noise.

🚩 EVERYDAY PARALLEL

The heights of adults in a city. Most people are near the average, say 170 cm. Fewer are 185 cm, fewer still 200 cm; the same thinning-out happens toward the short end. Plot how many people sit at each height and you get a bell: crowded in the middle, sparse in the tails, roughly symmetric. That's a Gaussian in the wild.

◆ NOW THE MATH

The Gaussian PDF is controlled by just two numbers — its mean μ and its standard deviation σ :

$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

In words: μ (mu) is the *centre* — where the peak sits, the typical value. σ (sigma) is the *width* — how fat the bell is, i.e. how much things spread around the centre. The heart of the formula is $-(x - \mu)^2$: the further x is from the centre, the more negative that gets, and $\exp$ of a big negative number is tiny — so probability density falls off fast as you move away from μ . The $\frac{1}{\sigma\sqrt{2\pi}}$ out front is just a scaling factor that forces the total area to equal 1.

◆ WHY THE GAUSSIAN IS EVERYWHERE

The deep reason is the **Central Limit Theorem**: when you add up (or average) lots of small, independent random effects, the total tends toward a Gaussian – almost regardless of the shape of the individual effects. A person's height is the sum of countless small genetic and environmental nudges; measurement error is the sum of many tiny disturbances. Because so much of the world is “many little random pushes added together,” the bell curve appears again and again – and that's why ML so often assumes noise is Gaussian, and models it that way.

◆ THE SAME THING IN PYTORCH

```
import torch

# a Gaussian centred at mu=170 with spread sigma=10 (think: heights in cm)
normal = torch.distributions.Normal(loc=170.0, scale=10.0)

samples = normal.sample((100_000,))
print(samples.mean(), samples.std())    # ~170.0, ~10.0 → recovers mu, sigma

# about 68% of a Gaussian lies within one sigma of the mean:
within_1sigma = ((samples > 160) & (samples < 180)).float().mean()
print(within_1sigma)                    # ~0.68 the famous 68% rule
```

CHECK YOURSELF

A Bernoulli has $p = 0.5$ and another has $p = 0.9$.

Reveal ▼

Which one is more uncertain – harder to predict on any single trial – and how does the variance $p(1-p)$ confirm it?

ANSWER

The $p = 0.5$ coin is more uncertain: it's a genuine coin flip, no way to guess. The $p = 0.9$ one is easy – just guess “1” and you're right 90% of the time. The variance confirms it: $0.5 \times 0.5 = 0.25$ versus $0.9 \times 0.1 = 0.09$. Variance $p(1-p)$ peaks exactly at $p = 0.5$ and shrinks toward the extremes, which matches the intuition that a near-certain event carries little surprise.

4 Conditional probability & Bayes' rule

◆ INTUITION FIRST

Conditional probability, written $P(A \mid B)$ and read “the probability of *A* *given* *B*,” is how likely *A* is *once you already know B has happened*. New information shrinks the world of possibilities, and the odds shift accordingly. Learning that it's cloudy raises the chance of rain; the raw chance of rain and the chance-of-rain-given-clouds are two different numbers.

🌂 EVERYDAY PARALLEL

The chance a stranger is over six feet tall is modest. But *given* that you're told they play professional basketball, that chance rockets up. The fact didn't change the person – it changed *your information*, and so it changed the probability you should assign. That's all a conditional probability is: an updated belief in the light of a known fact.

◆ NOW THE MATH

Start from the *joint probability* – the chance that *both* *A* and *B* happen, written $P(A, B)$. There are two equally valid ways to build that joint up out of a condition, depending on

which event you condition on:

$$P(A, B) = P(A | B) P(B) = P(B | A) P(A)$$

In words: the chance of both A and B is the chance of B, times the chance of A once B is known — “B happens, and then A happens given B.” You could equally tell the story the other way round (A first, then B given A). Both routes reach the same joint, so the two products are equal.

Now set those two expressions for the same joint equal to each other and divide by $P(B)$. That single step of algebra *is* **Bayes' rule**:

$$P(A | B) P(B) = P(B | A) P(A)$$
$$P(A | B) = \frac{P(B | A) P(A)}{P(B)}$$

In words: Bayes' rule lets you *flip* a conditional. If you know how likely the evidence B is when A is true, plus how common A is to begin with, you can work out how likely A is once you've seen B. It's the formula for updating a belief in the light of evidence.

THE THREE WORDS OF BAYES

The **prior** $P(A)$ is your belief in A *before* the evidence. The **likelihood** $P(B | A)$ is how well A explains the evidence B — how probable the evidence would be if A were true. The **posterior** $P(A | B)$ is your updated belief in A *after* seeing the evidence. Bayes is the machine that turns prior + likelihood into posterior.

🌂 EVERYDAY PARALLEL

A **medical test** for a rare disease. You take it; it comes back *positive*. Your gut screams “I'm sick.” But Bayes says: hold on — factor in how rare the disease is to begin with (the prior), and how often the test cries wolf on healthy people (false positives). The frightening number and the true number can be very far apart. Let's put real figures on it.

A concrete number: the base-rate effect

Suppose a disease affects **1 in 1000** people. A test is quite good: if you *have* the disease it comes back positive 99% of the time (the likelihood), and if you're *healthy* it wrongly comes back positive just 5% of the time (the false-positive rate). You test positive. What's the real chance you're sick?

Let D = “has disease,” $+$ = “tests positive.” We want the posterior $P(D \mid +)$. Bayes needs the total chance of a positive result, $P(+)$, which is positives from sick people *plus* positives from healthy people:

$$\begin{aligned} P(+) &= \underbrace{P(+ \mid D) P(D)}_{\text{true positives}} + \underbrace{P(+ \mid \neg D) P(\neg D)}_{\text{false positives}} \\ &= (0.99)(0.001) + (0.05)(0.999) \\ &= 0.00099 + 0.04995 = 0.05094 \\ P(D \mid +) &= \frac{P(+ \mid D) P(D)}{P(+)} = \frac{(0.99)(0.001)}{0.05094} \approx 0.0194 \end{aligned}$$

In words: even after a positive result on a 99%-accurate test, your chance of actually being sick is only about **2%**. Why so low? Because the disease is rare (the prior $P(D) = 0.001$ is tiny). Out of a positive pile, the vast majority come from the huge healthy population's 5% false-positive rate, which swamps the few true positives. This is the **base-rate effect**: ignore how rare something is to begin with, and you'll wildly overreact to evidence. Bayes forces you to keep the prior in the picture.

◆ THE SAME THING IN PYTORCH

```
import torch

p_disease = torch.tensor(0.001)    # prior: 1 in 1000
p_pos_sick = torch.tensor(0.99)    # likelihood: true-positive rate
p_pos_well = torch.tensor(0.05)    # false-positive rate

# total probability of a positive test (sick + healthy sources)
p_pos = p_pos_sick * p_disease + p_pos_well * (1 - p_disease)

posterior = (p_pos_sick * p_disease) / p_pos
print(posterior)    # tensor(0.0194) → ~2%, not 99%
```

CHECK YOURSELF

If the disease were far more common – say 1 in 10 people (prior 0.1) instead of 1 in 1000 – would a positive test make you more or less confident you're sick? Roughly what happens to the posterior? Reveal ▼

ANSWER

Much more confident. Redo the sum with $P(D) = 0.1$: $P(+) = (0.99)(0.1) + (0.05)(0.9) = 0.099 + 0.045 = 0.144$, so $P(D | +) = 0.099 / 0.144 \approx 0.69$ – about 69%, versus 2% before. The test never changed; only the prior did.

When the base rate is high, the true positives no longer get drowned out by false positives. This is exactly why context matters: the same evidence means very different things depending on how common the thing was to begin with.

5 Maximum likelihood: fitting a distribution to data

◆ INTUITION FIRST

So far the distributions came with their numbers already filled in. But in real ML you have *data* and you need to find the numbers yourself. **Maximum Likelihood Estimation** (MLE) is the workhorse method for that, and its idea is beautifully simple: **pick the parameters that make the data you actually observed as probable as possible**. Of all the coins that *could* have produced your flips, MLE picks the one that would produce them most readily.

🌂 EVERYDAY PARALLEL

You hear a muffled band playing through a wall and try to guess the song. You run through candidates and pick the one that would most naturally produce the notes you're hearing – the song under which “these notes” are least surprising. MLE does the same with numbers: it scans the possible parameter values and keeps the one under which your observed data is least surprising, i.e. most probable.

◆ NOW THE MATH

Let's derive it fully for a coin. You flip a coin with unknown heads-probability p and observe H heads out of N flips. Each flip is a Bernoulli, and because the flips are independent, the probability of the whole sequence is the product of the individual probabilities. That product, viewed as a function of the unknown p , is the *likelihood*:

$$L(p) = p^H (1 - p)^{N-H}$$

In words: to get exactly this data you need H heads (each contributing a factor of p) and $N - H$ tails (each contributing $1 - p$). Multiply them all together and you get the chance of the observed sequence for a given p . We want the p that makes this as large as possible.

Multiplying many probabilities gives a tiny, awkward number, and products are painful to differentiate. So we take the **logarithm** first. Two reasons the log is a free win: it turns products into sums (much easier calculus), and because **log** is a steadily increasing function, *the p that maximizes $\log L$ is the exact same p that maximizes L* – we lose nothing by working with the log.

$$\ell(p) = \log L(p) = H \log p + (N - H) \log(1 - p)$$

In words: the log pulled the exponents down into multipliers and turned the product into a sum. This $\ell(p)$ is the *log-likelihood*, and it's what we'll actually maximize.

To find the maximum, use calculus from [Section 2](#): take the derivative with respect to p , set it to zero (the slope is flat at a peak), and solve. Recall $\frac{d}{dp} \log p = \frac{1}{p}$:

$$\begin{aligned} \frac{d\ell}{dp} &= \frac{H}{p} - \frac{N - H}{1 - p} = 0 \\ \frac{H}{p} &= \frac{N - H}{1 - p} \\ H(1 - p) &= (N - H)p \\ H - Hp &= Np - Hp \\ H &= Np \\ \hat{p} &= \frac{H}{N} \end{aligned}$$

In words: the best estimate is $\hat{p} = H/N$ — the fraction of flips that came up heads. Flip 100 times, see 70 heads, and MLE says the most likely coin is one with $p = 0.7$. The little hat on $\hat{p}$

marks it as an *estimate from data*, not the true hidden value. What feels like plain common sense — “just count the heads” — turns out to be exactly what the maximum-likelihood principle prescribes. That's reassuring: the fancy method agrees with your intuition on the easy case, which is why you can trust it on the hard ones.

◆ THE SAME THING IN PYTORCH

```
import torch

# 100 flips, 70 heads: recover p by maximizing the log-likelihood with
# gradient ascent
H, N = 70.0, 100.0
p = torch.tensor(0.5, requires_grad=True)  # start with a fair-coin
# guess

for _ in range(500):
    log_lik = H * torch.log(p) + (N - H) * torch.log(1 - p)
    log_lik.backward()  # d(log_lik)/dp, from
    # Section 2
    with torch.no_grad():
        p += 0.001 * p.grad  # step UP-hill (maximize)
        p.grad.zero_()

print(p)  # tensor(0.70...) → matches the closed-form p_hat =
H/N = 0.7
```

◆ WHERE LOSS FUNCTIONS COME FROM

Notice what we maximized: a log-likelihood. Now flip the sign. Maximizing $\log L$ is the same as *minimizing* $-\log L$, and that quantity, **negative log-likelihood**, is precisely the loss function most classifiers are trained on — you'll meet it as *cross-entropy* in [Section 4](#). The Gaussian version, where you do this same derivation for a Normal instead of a Bernoulli, collapses into the *mean-squared-error* loss behind [Section 5](#)'s linear regression. Training a model is maximum likelihood in disguise: “find the parameters that make the data most probable” is the same sentence as “find the parameters that minimize the loss.”

CHECK YOURSELF

You flip a coin 10 times and get 10 heads. MLE says $\hat{p} = 10/10 = 1.0$ – a coin that *never* lands tails. Why is that suspicious, and what does it hint about the limits of pure MLE?

ANSWER

MLE only listens to the data in front of it, and with a small sample that data can be misleading. Ten heads in a row is perfectly plausible from a fair coin (about a 0.1% chance), yet MLE confidently declares $p = 1.0$, meaning it now assigns probability *zero* to ever seeing a tail – a reckless certainty from ten flips. The fix is to fold in a **prior** belief (“coins are usually roughly fair”) and use Bayes, which pulls the estimate back toward 0.5 until enough data overwhelms it. This tug-of-war between what the data says and what you believed beforehand is the bridge from maximum likelihood to full Bayesian learning.

6 What to carry forward

- A model outputs a **distribution** – belief spread over outcomes. Discrete ones use a **PMF** (mass on each value); continuous ones use a **PDF** (area under a curve gives probability).
- **Expectation** $\mathbb{E}[X]$ is the probability-weighted average – where the distribution balances. **Variance** $\mathbb{E}[(X - \mu)^2]$ is its spread – reliable versus streaky.
- The **Bernoulli** models one yes/no event via a single p ; the **Gaussian** is the bell curve set by mean μ and spread σ , and it appears everywhere because sums of many small random effects tend toward it.
- **Bayes' rule** $P(A | B) = \frac{P(B|A)P(A)}{P(B)}$ updates a *prior* belief with *likelihood* into a *posterior*. Never ignore the base rate.
- **MLE** picks the parameters that make the observed data most probable; for a coin it gives the plain count $\hat{p} = H/N$. Minimizing negative log-likelihood *is* where loss functions

come from.

Next we turn probability into a way to *measure information and surprise* – which leads straight to entropy, cross-entropy, and the loss functions we just previewed.

Information Theory

By the end of this page you will have derived, not been told, why cross-entropy is the loss function for classification. Every step is shown; nothing is asserted.

26 min read

You'll learn: information & surprise · entropy · cross-entropy · KL divergence · why cross-entropy is THE classification loss

Every time a model makes a prediction, we need a single number that says **how wrong it was**. That number is the loss, and for classification the loss is almost always *cross-entropy*. Most courses hand you the formula and move on. We are going to build it from nothing – starting with a question a child could ask, “*why is some news surprising and other news boring?*” – and end up, honestly and step by step, at the exact loss function that trains every image classifier and every language model on Earth.

◆ THE ONE IDEA TO HOLD ONTO

Information is **surprise**, and surprise is **measured in the length of the message you'd need to announce an event**. Entropy is your average surprise, cross-entropy is your average surprise when you trained on the wrong beliefs, and the gap between them – the wasted effort – is exactly what a classifier learns to shrink.

1 Information: surprise, measured

◆ INTUITION FIRST

Start with a plain observation about the news. A headline is “big news” exactly when it tells you something you did *not* expect. “Dog bites man” is not a story – dogs bite, it happens all the time, you learned almost nothing. “Man bites dog” is a story – it is rare, and precisely because it is rare it carries a lot of **information**. So information is not about how important

an event feels; it is about how *unlikely* it was. The rarer the event, the more you learn when it happens.

🚂 EVERYDAY PARALLEL

A friend who says “the sun rose this morning” has told you nothing – probability ≈ 1 , zero surprise. A friend who says “it snowed in the desert” has told you a lot – tiny probability, huge surprise. Same number of words spoken, wildly different amount of information delivered. Information is the *gap between what you expected and what happened*.

◆ NOW THE MATH

We want a number – call it **information content**, or *surprise* – that takes the probability $p(x)$ of an event x and grows as that probability shrinks. Let's not pull a formula out of a hat; let's demand the properties any honest measure of surprise *must* have, then see what function is forced on us.

1. **Certain events are boring.** If $p(x) = 1$, the surprise must be exactly 0. You learn nothing from what was guaranteed.
2. **Rarer means more surprising.** As $p(x)$ goes down, surprise must go up. The measure decreases with probability.
3. **Independent surprises add up.** If two unrelated things happen – a coin lands heads *and* a die rolls a six – the total surprise should be the surprise of the first *plus* the surprise of the second. (Two independent events. “Independent” means the outcome of one tells you nothing about the other.)

That third rule is the strict one. For independent events, probabilities *multiply*: $p(\text{both}) = p(x) p(y)$. But we want the surprises to *add*. We need a function that turns multiplication on the inside into addition on the outside. There is exactly one family of functions that does that – the **logarithm**, because $\log(ab) = \log a + \log b$. So the surprise must be a log of the probability. To make it *decrease* as probability rises (rule 2) and be non-negative, we put a minus sign in front:

$$I(x) = -\log p(x) = \log \frac{1}{p(x)}$$

In words: the information in an event is “minus the log of how likely it was.” Because probabilities are between 0 and 1, their log is negative, so the minus sign makes $I(x)$ positive. A sure thing ($p = 1$) gives $-\log 1 = 0$ — no surprise. A one-in-a-million event gives a big positive number. And the two equal forms show the same idea two ways: *surprise is the log of “one over the probability,”* i.e. the log of how many equally-likely possibilities you just ruled out.

DEFINITION

The **information content** (also called **surprise** or **surprisal**) of an outcome x is $I(x) = -\log p(x)$. When the log is base 2 the unit is **bits**; when it is the natural log (base e) the unit is **nats**. One bit is the surprise of a single fair coin flip.

Why is one fair coin flip worth exactly one bit? Plug in $p = \frac{1}{2}$ with base-2 log:
 $-\log_2 \frac{1}{2} = \log_2 2 = 1$. One yes/no answer, one bit. Eight independent coin flips carry $8 \times 1 = 8$ bits — which is why a byte (8 bits) can name exactly $2^8 = 256$ equally likely things.

◆ THE SAME THING IN PYTORCH

> SURPRISE IN BITS AND NATS

```
import torch

# a rare event vs a common one
p_rare = torch.tensor(1/1000) # one-in-a-thousand
p_common = torch.tensor(0.9) # happens most of the time

# base-2 log → answer in BITS
print(-torch.log2(p_rare)) # tensor(9.9658) ~10 bits of surprise
print(-torch.log2(p_common)) # tensor(0.1520) barely a fifth of a bit

# natural log → answer in NATS (what PyTorch losses use internally)
print(-torch.log(p_rare)) # tensor(6.9078) same event, different unit
```

CHECK YOURSELF

A weather app says tomorrow's rain has probability $1/8$. Reveal ▼

In bits, how surprised should you be if it rains – and why does that number feel like “how many yes/no questions”?

ANSWER

$-\log_2(1/8) = \log_2(8) = 3$ bits. The reason 3 feels like three yes/no questions: with equally-likely options, each yes/no question cuts the possibilities in half. One-in-eight is the same as “which of 8 boxes?”, and you can pin down one of 8 boxes with exactly 3 halvings ($8 \rightarrow 4 \rightarrow 2 \rightarrow 1$). Surprise in bits *is* the number of ideal yes/no questions the event's outcome answers.

2 Entropy: your average surprise

◆ INTUITION FIRST

Surprise is defined for a *single* outcome. But usually we care about a whole **distribution** – a source that keeps producing outcomes, like a coin you keep flipping or a language that keeps emitting words. **Entropy** is the answer to: “*on average, how surprised will I be per outcome from this source?*” It is the mean surprise, weighted by how often each outcome actually happens.

🌂 EVERYDAY PARALLEL

Think of the game **20 Questions**. Entropy is the average number of yes/no questions you need to guess the answer. If I'm thinking of a fair coin's result, one question settles it – entropy 1. If I'm thinking of one of 1,024 equally likely names, you need about 10 questions – entropy 10. A predictable source (I *always* pick “heads”) needs zero questions; you already know. **Entropy is uncertainty measured in questions.**

◆ NOW THE MATH

To average the surprise, we weight each outcome's surprise — $\log p(x)$ by its own probability $p(x)$ and add them all up. That weighted average is written with the expectation

symbol $\mathbb{E}$ (“expected value” = long-run average):

$$H(p) = \mathbb{E}_{x \sim p}[-\log p(x)] = -\sum_x p(x) \log p(x)$$

In words: go through every possible outcome x ; take its surprise $-\log p(x)$; multiply by how often it happens, $p(x)$; sum over all outcomes. Common outcomes are surprising rarely but counted often; rare outcomes are very surprising but counted seldom. Entropy balances the two into one number: *the surprise you should expect per draw*.

Let's actually compute it for a coin that lands heads with probability p (and tails with $1 - p$).

There are only two outcomes, so the sum has two terms:

$$H(p) = -p \log_2 p - (1 - p) \log_2(1 - p)$$

Now read off the extremes, which is where the meaning lives:

Coin	p (heads)	Entropy H (bits)	What it means
Two-headed (rigged)	1	0	No uncertainty — you always know the result.
Heavily biased	0.1	0.47	Usually tails; little uncertainty.
Fair	0.5	1	Maximum uncertainty — a full bit per flip.
Two-tailed (rigged)	0	0	No uncertainty again — always tails.

Notice the shape: entropy is **zero at both ends** (a rigged coin holds no surprise) and **largest exactly in the middle**, at $p = 0.5$. Uncertainty peaks when the outcomes are equally likely and you have the least idea what's coming.

We can *show* that the peak sits at $p = 0.5$ rather than just eyeball the table. Using the natural log for the calculus and differentiating $H(p) = -p \ln p - (1 - p) \ln(1 - p)$ with respect to p :

$$\begin{aligned}
\frac{dH}{dp} &= -(\ln p + 1) - (-\ln(1-p) - 1) \\
&= -\ln p - 1 + \ln(1-p) + 1 \\
&= \ln \frac{1-p}{p}.
\end{aligned}$$

In words: the slope of the entropy curve is the log of “tails-odds over heads-odds.” Set the slope to 0 to find the top of the hill: $\ln \frac{1-p}{p} = 0$ means $\frac{1-p}{p} = 1$, i.e. $1-p = p$, i.e. $p = 0.5$. The single flat point of the curve is dead centre — proving the fair coin is the most uncertain coin there is.

DEFINITION

Bits vs nats. Entropy is the same quantity whichever log you use; only the unit changes. Base-2 log gives **bits** (natural for “yes/no questions” and file sizes); natural log gives **nats**. They differ only by a constant factor ($1 \text{ nat} \approx 1.44 \text{ bits}$). Machine learning code uses nats, because the natural log is the one calculus likes.

◆ THE SAME THING IN PYTORCH

›_ ENTROPY OF A COIN ACROSS ALL BIASES

```
import torch

def coin_entropy_bits(p):
    # clamp keeps log(0) from blowing up; 0*log0 is defined as 0
    p = torch.clamp(p, 1e-9, 1 - 1e-9)
    return -(p * torch.log2(p) + (1 - p) * torch.log2(1 - p))

for p in [0.0, 0.1, 0.5, 0.9, 1.0]:
    print(f"p={p:.1f}  H={coin_entropy_bits(torch.tensor(p)):.3f} bits")
# p=0.0  H=0.000  p=0.1  H=0.469  p=0.5  H=1.000
# p=0.9  H=0.469  p=1.0  H=0.000  → peak in the middle
```

CHECK YOURSELF

A fair six-sided die has entropy $\log_2(6) \approx 2.585$ bits. Reveal ▼

Why is that *more* than the fair coin's 1 bit, and what would drop the die's entropy below that?

ANSWER

The die has six equally likely outcomes instead of two, so you need more yes/no questions to pin one down – about 2.585 on average – hence higher entropy. Entropy is maximised when all outcomes are equally likely: for k equal options it is exactly $\log_2(k)$. To *lower* the die's entropy you'd make it unfair – say it lands on 6 half the time – because now you can guess “6?” first and often be right, so fewer questions are needed on average. Predictability always lowers entropy.

3 Cross-entropy: surprise under the wrong beliefs

◆ INTUITION FIRST

Entropy assumed you *knew* the true probabilities p . But a model doesn't know them – it only has a *guess*, a predicted distribution we'll call q . **Cross-entropy** asks the honest question: *if the world really runs on p , but I plan for it as if it runs on q , how surprised will I be on average?* It is your average surprise when your beliefs are q but reality is p .

🚩 EVERYDAY PARALLEL

You travel to a country and pack the *wrong* phrasebook – it's for a neighbouring language. You can still get by: many phrases overlap, so you're understood often enough. But every mismatch costs you extra effort – repeating yourself, pointing, guessing. The *true* language is p ; your phrasebook is q . Cross-entropy is the total effort you spend speaking reality through the wrong book. Pack a better-matched book (a better q) and the daily effort drops.

◆ NOW THE MATH

The recipe is the same as entropy – average the surprise, weighted by how often each outcome happens – with **one change**. The outcomes still happen with the true frequency $p(x)$ (reality decides how often things occur). But the surprise you actually *feel* is computed from your belief $q(x)$, because that's the probability you assigned. So we weight by p but take the log of q :

$$H(p, q) = \mathbb{E}_{x \sim p}[-\log q(x)] = - \sum_x p(x) \log q(x)$$

In words: outcomes arrive at their real rate $p(x)$, but each one startles you by $-\log q(x)$ — the surprise *your* model assigned it. If your guess q matched reality p perfectly, this would collapse back into ordinary entropy $H(p)$. Any mismatch makes it bigger. Cross-entropy is never smaller than entropy — a fact we prove in the next section.

◆ READ THE TWO ARGUMENTS CAREFULLY

The order matters: $H(p, q)$ weights by the **first** argument p (what's true) and takes the log of the **second** argument q (what you believe). Swap them and you get a different number. Keep saying it to yourself: “*weighted by truth, surprised by belief*.”

◆ THE SAME THING IN PYTORCH

>_ CROSS-ENTROPY BETWEEN A TRUE AND A PREDICTED DISTRIBUTION

```
import torch

# true distribution p over 3 classes, and two guesses q
p      = torch.tensor([0.7, 0.2, 0.1])  # reality
q_ok   = torch.tensor([0.6, 0.3, 0.1])  # a decent guess
q_bad  = torch.tensor([0.1, 0.2, 0.7])  # confidently wrong

def cross_entropy(p, q):
    return -(p * torch.log(q)).sum()    # nats

print(cross_entropy(p, p))              # tensor(0.8018) = H(p), the floor
print(cross_entropy(p, q_ok))          # tensor(0.8286) a little above the
floor
print(cross_entropy(p, q_bad))         # tensor(1.9694) far above → bad model
```

CHECK YOURSELF

In the code above, `cross_entropy(p, p)` gave the *smallest* value of the three. Why can no guess `q` ever beat using the true `p` itself?

Reveal ▼

ANSWER

Because the least-surprising code for a source is the one built from its *true* probabilities – that's what entropy `H(p)` is, the theoretical minimum average surprise. Any other belief `q` assigns at least one outcome a probability that's too low, and you pay for that with extra surprise when it happens. So `H(p, q) ≥ H(p)` always, with equality only when `q = p`. The exact proof of this inequality is the whole point of the next section.

4 KL divergence: the extra cost of being wrong

◆ INTUITION FIRST

Cross-entropy $H(p, q)$ is your total surprise with a wrong belief. Entropy $H(p)$ is the unavoidable surprise even with a perfect belief. The *difference* is the part you could have avoided – the pure **waste** caused by believing q instead of p . That waste is the **Kullback-Leibler divergence**, written $D_{\text{KL}}(p||q)$.

🌂 EVERYDAY PARALLEL

Back to the phrasebook. Cross-entropy is the *total* effort you spend communicating. Entropy is the effort a fluent local *still* spends – language is never effortless, some work is irreducible. KL divergence is the **extra** effort you burn purely because your phrasebook is wrong: the repeats, the pointing, the confused looks. Get the right book and KL drops to zero; you're down to the irreducible effort and not a jot more.

◆ NOW THE MATH

By definition it is the gap between cross-entropy and entropy. Subtract, and the two sums combine termwise into a single clean sum:

$$\begin{aligned}
D_{\text{KL}}(p\|q) &= H(p, q) - H(p) \\
&= \left(- \sum_x p(x) \log q(x) \right) - \left(- \sum_x p(x) \log p(x) \right) \\
&= \sum_x p(x) (\log p(x) - \log q(x)) \\
&= \sum_x p(x) \log \frac{p(x)}{q(x)}.
\end{aligned}$$

In words: for each outcome, look at the ratio of what's true to what you believed, $p(x)/q(x)$; take its log; weight by how often the outcome happens, $p(x)$; sum up. Where your belief matched reality the ratio is 1 and its log is 0 — no penalty. Wherever you were wrong the ratio strays from 1 and adds a positive cost.

Why KL divergence is never negative (Gibbs' inequality)

We keep claiming “cross-entropy $\geq$ entropy” and “the extra cost is never negative.” Let's actually prove it, because the whole story hangs on it. The claim is $D_{\text{KL}}(p\|q) \geq 0$, known as **Gibbs' inequality**. The clean proof uses one fact about the logarithm and one fact about averages.

The fact about the logarithm: $\log$ is a *concave* function — it bulges upward, so any straight chord between two points on the curve lies *below* the curve. A concrete consequence we'll use is the simple bound $\log t \leq t - 1$ for all $t > 0$ (the line $t - 1$ is the tangent to $\log$ at $t = 1$, and a concave curve never rises above its tangent). Now watch:

$$\begin{aligned}
-D_{\text{KL}}(p\|q) &= \sum_x p(x) \log \frac{q(x)}{p(x)} && \text{(flip the ratio; the minus sign flips the log)} \\
&\leq \sum_x p(x) \left(\frac{q(x)}{p(x)} - 1 \right) && \text{(apply } \log t \leq t - 1 \text{ with } t = \frac{q(x)}{p(x)}) \\
&= \sum_x (q(x) - p(x)) \\
&= \sum_x q(x) - \sum_x p(x) = 1 - 1 = 0.
\end{aligned}$$

In words: we showed $-D_{\text{KL}} \leq 0$, which is the same as $D_{\text{KL}} \geq 0$. The last line used the only thing we know for sure about probability distributions: they each sum to 1. So the extra cost of a wrong belief is *always* zero or positive — never a bonus. And equality ($D_{\text{KL}} = 0$) holds only when the bound $\log t = t - 1$ is tight for every term, which happens only at $t = 1$, i.e. only when $q(x) = p(x)$ everywhere. Belief matches reality exactly, or you pay.

◆ THIS IS THE PILLAR

Gibbs' inequality is *why* minimising cross-entropy is a sensible goal. Since $H(p, q) = H(p) + D_{\text{KL}}(p||q)$ and $H(p)$ is a fixed constant the model cannot change, pushing cross-entropy down is **exactly** pushing D_{KL} down – dragging your predicted distribution q toward the truth p . The lowest it can go is zero, reached only when $q = p$.

▲ WATCH OUT: KL IS NOT A DISTANCE

KL divergence is **asymmetric**: in general $D_{\text{KL}}(p||q) \neq D_{\text{KL}}(q||p)$. The cost of using the wrong phrasebook depends on which language is actually being spoken. So never call KL a “distance” – a real distance is symmetric, and this is not. It's a directed measure of “extra surprise if the truth is p and you planned for q .”

◆ THE SAME THING IN PYTORCH

>_ KL IS THE GAP, AND IT IS ASYMMETRIC

```
import torch

p = torch.tensor([0.7, 0.2, 0.1])
q = torch.tensor([0.4, 0.4, 0.2])

def H(p, q=None):                                # entropy if q is None, else
cross-entropy
    q = p if q is None else q
    return -(p * torch.log(q)).sum()

kl_pq = H(p, q) - H(p)                           # extra cost if truth is p,
belief is q
kl_qp = H(q, p) - H(q)                           # the reversed question

print(kl_pq)   # tensor(0.1838)
print(kl_qp)   # tensor(0.1920) → different number: KL is asymmetric
print(kl_pq ≥ 0, kl_qp ≥ 0) # tensor(True) tensor(True) → Gibbs holds
```

CHECK YOURSELF

If a model's predicted distribution q exactly equals the truth p , what are the values of $H(p, q)$, $D_{KL}(p||q)$, and the training loss – and can the loss ever reach zero for a genuinely uncertain source?

Reveal ▼

ANSWER

When $q = p$: $D_{KL}(p||q) = 0$ (no wasted effort), so $H(p, q) = H(p) + 0 = H(p)$ – cross-entropy bottoms out at the source's own entropy. The *loss* here is the cross-entropy, so its floor is $H(p)$, not zero. For a genuinely uncertain source ($H(p) > 0$) the loss can never hit zero no matter how perfect the model – some surprise is irreducible. It reaches zero only when the truth itself is certain (one-hot), which is exactly the classification case we turn to now.

5 The punchline: why cross-entropy is THE classification loss

◆ INTUITION FIRST

Everything above was general. Now specialise it to **classification** – the task of putting an input into one of several categories (is this image a cat, a dog, or a bird?). Here the truth is not fuzzy: each training example belongs to *exactly one* class. That single fact collapses cross-entropy into something startlingly simple, and reveals that “minimise cross-entropy” and “make the correct answer as likely as possible” are *the very same instruction*.

🔼 EVERYDAY PARALLEL

Imagine grading a multiple-choice test where only one answer is correct. You don't care how confident the student was about the wrong options – you only care *how much probability they put on the right one*. Full marks for “100% sure, and correct.” Harsh penalty for “only 5% on the correct answer.” Cross-entropy is exactly this grader: it ignores the wrong boxes and scores you purely on the confidence you placed on the true one.

◆ NOW THE MATH — THE COLLAPSE

In classification the true distribution p for a single example is **one-hot**: it puts probability 1 on the correct class c and 0 on every other class. That is, $p(c) = 1$ and $p(x) = 0$ for $x \neq c$. (“One-hot” = a vector that is all zeros except a single 1 marking the right answer.) Substitute that into the cross-entropy sum and watch every term but one vanish:

$$\begin{aligned} H(p, q) &= - \sum_x p(x) \log q(x) \\ &= - \left[p(c) \log q(c) + \sum_{x \neq c} p(x) \log q(x) \right] \\ &= - \left[1 \cdot \log q(c) + \sum_{x \neq c} 0 \cdot \log q(x) \right] \\ &= - \log q(c). \end{aligned}$$

In words: because the truth is 1 for the correct class and 0 everywhere else, every wrong-class term is multiplied by 0 and disappears. What survives is a single term: *minus the log of the probability your model assigned to the correct class*. That's it. The whole cross-entropy of a labelled example is just $-\log q(\text{correct class})$ — the surprise of the right answer under your model.

◆ **CROSS-ENTROPY = NEGATIVE LOG-LIKELIHOOD**

$-\log q(c)$ is precisely the **negative log-likelihood** of the correct label. The *likelihood* is how probable the model thinks the true answer is; the loss is minus its log. So minimising cross-entropy is **maximising the likelihood** of the correct answers – the very principle of **Maximum Likelihood Estimation** we met in [Section 3](#). Two ideas that looked unrelated – “shortest code” from information theory and “most probable model” from statistics – turn out to be the same loss.

Over a whole dataset of N independent examples, the likelihoods multiply, so the log-likelihoods add, and the training loss is the *average* negative log-likelihood – one cross-entropy term per example:

$$L = \frac{1}{N} \sum_{i=1}^N -\log q_i(c_i)$$

In words: for each training example i take the model's probability on that example's true class c_i , take its negative log (the surprise), and average over all N examples. Drive this down and you are, example by example, making the model less surprised by the right answers.

Where do the probabilities q come from? The softmax

A neural network's last layer doesn't output probabilities – it outputs a vector of raw, unbounded real numbers called **logits** (one score per class; they can be negative, huge, anything). To turn logits $z_1, \dots, z_K$ into a valid probability distribution q , we pass them through the **softmax** function – exponentiate each score (making it positive) and divide by the total (making the whole set sum to 1):

$$q_k = \text{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

In words: e^{z_k} makes every score positive and blows up the gaps so the largest logit dominates; dividing by the sum normalises them into probabilities that add to 1. The class with the biggest logit gets the biggest probability. Softmax is the bridge from “raw network scores” to “a distribution q you can feed to cross-entropy.”

◆ SOFTMAX + CROSS-ENTROPY: THE STANDARD PAIRING

Feed the network's logits through softmax to get q , then score q with cross-entropy — $-\log q(c)$. This softmax-then-cross-entropy pair is the default output of virtually every classifier. It's so common — and so much more numerically stable when fused — that PyTorch bundles both into a single function, `nn.CrossEntropyLoss`, which takes the *raw logits* directly. You almost never apply softmax by hand before the loss; the loss does it for you, safely.

◆ THE SAME THING IN PYTORCH

>_ FROM RAW LOGITS TO THE CLASSIFICATION LOSS, TWO EQUIVALENT WAYS

```
import torch
import torch.nn.functional as F

# a batch of 2 examples, 3 classes; these are raw LOGITS from a network
logits = torch.tensor([[2.0, 0.5, -1.0],      # example 0
                      [0.1, 0.2, 3.0]])      # example 1
targets = torch.tensor([0, 2])                # true classes: 0 and 2 (one-hot, given as index)

# --- the manual way: softmax → pick correct class → -log → average ---
q = F.softmax(logits, dim=1)                  # logits → probabilities per row
p_correct = q[range(2), targets]              # q assigned to each true class
manual = (-torch.log(p_correct)).mean()       # mean negative log-likelihood
print(manual)                                # tensor(0.1755)

# --- the idiomatic way: one fused, numerically-stable call on RAW logits ---
builtin = F.cross_entropy(logits, targets)    # softmax + NLL, fused
print(builtin)                                # tensor(0.1755) → identical
```

▲ THE CLASSIC DOUBLE-SOFTMAX BUG

`nn.CrossEntropyLoss` / `F.cross_entropy` apply softmax *internally*. If you also call `softmax` on your model's output first, you softmax twice – squashing the logits toward uniform, so the model trains slowly and badly. Rule: feed **raw logits** to the loss, never pre-softmaxed probabilities.

CHECK YOURSELF

Your classifier puts probability `0.9` on the correct class for one image and `0.1` on the correct class for another. Which image contributes more to the loss, by how much (in nats), and what does that say about how cross-entropy treats confident mistakes?

Reveal ▼

ANSWER

Loss per example is `-log(q_correct)`. For the first: `-log(0.9) ≈ 0.105` nats. For the second: `-log(0.1) ≈ 2.303` nats. The second image contributes about `2.303 / 0.105 ≈ 22×` more loss. Because `-log` shoots toward infinity as the correct-class probability approaches 0, cross-entropy **punishes confident wrongness savagely**: putting almost no probability on the true answer is enormously costly, which is exactly what pushes the model to stop being confidently wrong. This same loss, applied to “predict the next word,” is what trains language models – see [Section 19](#).

6 What to carry forward

- **Information / surprise** is $-\log p(x)$: rare events carry more, and the log makes independent surprises add up.
- **Entropy** $H(p) = -\sum p \log p$ is your average surprise – uncertainty in “yes/no questions,” maximal when outcomes are equally likely.

- **Cross-entropy** $H(p, q) = - \sum p \log q$ is your average surprise when reality is p but you believed q : weighted by truth, surprised by belief.
- **KL divergence** $D_{\text{KL}}(p\|q) = H(p, q) - H(p) \geq 0$ is the pure extra cost of a wrong belief (Gibbs' inequality, proved via $\log t \leq t - 1$) – and it is asymmetric, so not a distance.
- **The punchline:** for one-hot labels cross-entropy collapses to $-\log q(\text{correct class})$ = negative log-likelihood, so minimising it *is* maximum likelihood (Section 3). Softmax turns logits into q ; cross-entropy reappears in its binary form in Section 6, and the full softmax + cross-entropy pair in Section 11 and Section 19.

You now hold the loss function that trains nearly every classifier ever built – and you *derived* it, from “why is snow in the desert surprising?” all the way to `F.cross_entropy`. Next we build our first full model – linear regression – and put the gradient from Section 2 to work reducing a real loss, end to end.

What Learning Means

Every model you will ever train is secretly just three ingredients – a hypothesis, a loss, and an optimizer – and linear regression is the cleanest place to watch all three get solved by hand.

30 min read

You'll learn: hypothesis classes · loss functions · optimization · linear regression (closed form & gradient descent)

"Training a model" sounds mysterious until you break it into three questions, none of which is mysterious at all: what kinds of guesses are we willing to make, how do we score a guess as good or bad, and how do we search for the best one? Get comfortable with those three questions and you can read the training loop of almost any model in this course – including the giant ones later on.

◆ THE ONE IDEA TO HOLD ONTO

Learning is **search**. You fix a family of candidate functions, you fix a number that measures how wrong each candidate is, and then you search that family for the candidate with the lowest wrongness. Every algorithm in this course is a different way of doing that search.

1 The learning triad: hypothesis, loss, optimization

◆ INTUITION FIRST

Three separate ingredients make up every learning algorithm you'll meet. First, a **hypothesis class**: the family of functions you're willing to consider as "the answer" – maybe only straight lines, maybe anything a neural network can express. Second, a **loss**: a single number that says how wrong one particular function's guesses are, computed by comparing its predictions to the real answers. Third, **optimization**: the search procedure that tries different functions

from the hypothesis class and steers toward the one with the lowest loss. Learning, end to end, is just those three things wired together.

EVERYDAY PARALLEL

You're tuning a recipe for tomato soup. The **hypothesis class** is every recipe you'd consider making – maybe you've decided it must be tomato, garlic, and cream in some proportions, so "recipe" really means "the three numbers: how much tomato, how much garlic, how much cream." The **loss** is a taste test: how far this batch is from the flavor you're aiming for (too salty, not creamy enough – turned into one "distance from perfect" score). **Optimization** is what you do next: taste, adjust the ratios a little in the direction that helped last time, taste again. You never see "the answer" printed anywhere – you find it by repeatedly tasting and nudging.

DEFINITION

A **hypothesis class** is the set of candidate functions a model is allowed to choose from. Restricting it is a choice you make before training even starts – e.g. "only straight lines" is a much smaller, simpler hypothesis class than "any curve a deep network can draw."

DEFINITION

A **loss function** takes one hypothesis's predictions and the true answers and returns a single non-negative number: 0 means "perfect," and bigger means "worse." Every training run is, underneath, minimizing some loss function.

DEFINITION

An **optimization** procedure is the algorithm that searches the hypothesis class for the member with the lowest loss. It never magically knows the answer; it only knows how to take the current guess and produce a slightly better one.

CHECK YOURSELF

In the soup analogy, what plays the role of the loss function – and would "zero loss" ever really happen?

Reveal ▼

ANSWER

The taste test's "distance from the target flavor" score is the loss. In practice you almost never hit exactly zero – there's always some noise (today's tomatoes are slightly different from yesterday's), so the goal is realistically "as low as we can get," not a mathematically perfect zero. The same is true of nearly every model you'll train on real data.

2 The hypothesis class: linear regression

◆ INTUITION FIRST

The simplest useful hypothesis class is: predictions are a **straight-line function** of the input. Linear regression restricts itself to exactly that family and nothing fancier – no curves, no bends.

🌂 EVERYDAY PARALLEL

Predicting a house's price from its size. Bigger houses tend to cost more, roughly in proportion – add a square foot, add a roughly fixed number of dollars. That's exactly the shape of a straight line: a starting price plus a rate per square foot.

◆ NOW THE MATH

With a single input x (say, size in square feet), the hypothesis is:

$$\hat{y} = wx + b$$

In words: $\hat{y}$ ("y-hat") is the model's predicted price. x is the house's size. w (the *weight*) is dollars added per extra square foot — the slope. b (the *bias*) is the baseline price when size is zero — the intercept. Learning this hypothesis means finding good numbers for w and b .

Real houses have more than one useful feature — size, number of bedrooms, age, distance to downtown. Stack those numbers into a vector $\mathbf{x}$ and give each one its own weight in a vector $\mathbf{w}$; the model is the same idea generalized:

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b = w_1x_1 + w_2x_2 + \cdots + w_dx_d + b$$

In words: $\mathbf{w}^\top \mathbf{x}$ is a dot product (Section 1) — multiply each feature by its own weight and add up the results — then shift by the single bias number b . One weight per feature, one shared bias for the whole prediction.

◆ THE SAME THING IN PYTORCH

```
import torch

w = torch.tensor([120.0])      # $ per extra sqft
b = torch.tensor([10000.0])    # baseline price
x = torch.tensor([1800.0])     # house size, sqft

y_hat = x * w + b
print(y_hat)                   # tensor([226000.])
```

CHECK YOURSELF

With $\hat{y} = 120x + 10000$, what price does the model predict for a 1,800 sqft house, and what do w and b represent in plain words?

Reveal ▼

ANSWER

$\hat{y} = 120 \times 1800 + 10000 = \mathbf{226,000}$. Here $\mathbf{w} = 120$ means "each extra square foot adds 120 dollars to the predicted price," and $\mathbf{b} = 10000$ means "a hypothetical zero-square-foot house would be priced at 10,000 dollars" — a baseline that mostly exists to shift the line up or down, not a literal claim about tiny houses.

3 The loss: Mean Squared Error

◆ INTUITION FIRST

We need one number that says "across every house in our data, how wrong were this line's guesses?" **Mean Squared Error** (MSE) does this: for each house, subtract the true price from the predicted price, square that difference, and average the squares over all houses.

◆ NOW THE MATH

$$L(w, b) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2, \quad \hat{y}_i = wx_i + b$$

In words: for each of the n houses, take (predicted price – true price), square it, then average all n squared errors into one number. Squaring does two jobs at once: it makes every error positive (so a 10-dollar over-guess and a 10-dollar under-guess don't cancel out when summed), and it punishes large mistakes far more than small ones.

WHY SQUARE, AND NOT ABSOLUTE VALUE?

Absolute value $|\hat{y}-y|$ would also make errors positive, but it isn't smooth: at exactly zero its slope jumps abruptly, which is awkward for gradient-based optimization (Section 2). Squaring is smooth everywhere and – as the next box shows – falls directly out of a simple assumption about noisy data, rather than being picked arbitrarily.

◆ WHERE MSE ACTUALLY COMES FROM

Suppose the true price is a straight line plus random noise: $y_i = wx_i + b + \varepsilon_i$, where each ε_i is drawn from a Gaussian (bell-curve) distribution with mean 0 (see [Section 3](#)). Then the probability of seeing price y_i given x_i is the Gaussian density centered at $\hat{y}_i = wx_i + b$:

$$p(y_i | x_i; w, b) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \hat{y}_i)^2}{2\sigma^2}\right)$$

Assuming the houses are independent, the likelihood of the *whole* dataset is the product of these densities. Taking the log turns the product into a sum (logs turn multiplication into addition) and the exponential into a plain term:

$$\log \mathcal{L}(w, b) = \sum_{i=1}^n \left[-\log \sqrt{2\pi\sigma^2} - \frac{(y_i - \hat{y}_i)^2}{2\sigma^2} \right] = -n \log \sqrt{2\pi\sigma^2} - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

In words: the first term doesn't involve **w** or **b** at all, and $\frac{1}{2\sigma^2}$ is just a positive constant. So *maximizing* this log-likelihood over **w, b** is exactly the same job as *minimizing* $\sum_i (y_i - \hat{y}_i)^2$ — which is Mean Squared Error, up to the constant $1/n$. Squared error isn't an arbitrary convention; it's what falls out the moment you assume Gaussian noise.

CHECK YOURSELF

Why does minimizing sum-of-squared-errors correspond to maximizing likelihood under Gaussian noise, rather than, say, minimizing it?

Reveal ▼

ANSWER

The log-likelihood is $-n \cdot \log(\sqrt{2\pi\sigma^2}) - (1/2\sigma^2) \cdot \sum (y - \hat{y})^2$. The second term is subtracted, and its size grows with the squared errors. So making the squared errors *bigger* makes the log-likelihood *smaller*, and making them *smaller* makes the log-likelihood *larger*. To maximize likelihood you therefore have to minimize the sum of squared errors — the minus sign is what flips "minimize error" into "maximize likelihood."

4 Optimization, take one: solving directly

◆ INTUITION FIRST

Because the MSE loss for a straight line is a smooth bowl shape (one single lowest point, no bumps), we don't have to search for the bottom by trial and error — we can use calculus to jump straight to it, the same way you'd find the minimum of a simple parabola by setting its slope to zero and solving.

◆ NOW THE MATH

First, a trick to handle the bias $\mathbf{b}$ without carrying it separately: glue a column of 1s onto the front of the data matrix, and glue $\mathbf{b}$ onto the front of the weight vector. Call the augmented versions $\tilde{\mathbf{X}}$ (shape $n \times (d + 1)$) and $\tilde{\mathbf{w}}$ (length $d + 1$, with $\tilde{w}_0 = b$). Now $\tilde{\mathbf{X}}\tilde{\mathbf{w}}$ produces every prediction $\hat{y}_i = wx_i + b$ at once, and the loss is:

$$L(\tilde{\mathbf{w}}) = \frac{1}{n} \sum_{i=1}^n (\tilde{\mathbf{x}}_i \cdot \tilde{\mathbf{w}} - y_i)^2$$

In words: $\tilde{\mathbf{x}}_i$ is row i of $\tilde{\mathbf{X}}$ — one house's (augmented) features. This is exactly the same MSE as before, just written with the bias folded in.

Differentiate with respect to one weight $\tilde{w}_k$ using the chain rule (derivative of a square, times derivative of the inside):

$$\frac{\partial L}{\partial \tilde{w}_k} = \frac{1}{n} \sum_{i=1}^n 2 (\tilde{\mathbf{x}}_i \cdot \tilde{\mathbf{w}} - y_i) \tilde{x}_{i,k}$$

In words: for every house, take its residual (predicted – actual) and multiply by that house's k -th feature value, then average and double. Stacking this expression over every k reassembles a vector, giving the full gradient in matrix form:

$$\nabla_{\tilde{\mathbf{w}}} L = \frac{2}{n} \tilde{\mathbf{X}}^\top (\tilde{\mathbf{X}}\tilde{\mathbf{w}} - \mathbf{y})$$

In words: compute every residual at once ($\tilde{\mathbf{X}}\tilde{\mathbf{w}} - \mathbf{y}$, a length- n vector), then multiply by $\tilde{\mathbf{X}}^\top$, which — being a matrix — does exactly the "multiply by each feature column and sum over houses" step from the line above, for every weight simultaneously.

Now solve for the minimum by setting the gradient to the zero vector:

$$\begin{aligned}
\frac{2}{n} \tilde{\mathbf{X}}^\top (\tilde{\mathbf{X}} \tilde{\mathbf{w}} - \mathbf{y}) &= \mathbf{0} \\
\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} \tilde{\mathbf{w}} - \tilde{\mathbf{X}}^\top \mathbf{y} &= \mathbf{0} \\
\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} \tilde{\mathbf{w}} &= \tilde{\mathbf{X}}^\top \mathbf{y} \\
\tilde{\mathbf{w}} &= (\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}})^{-1} \tilde{\mathbf{X}}^\top \mathbf{y}
\end{aligned}$$

In words: multiply both sides by $n/2$ (drops out), move the constant term to the other side, then multiply both sides by the inverse of $\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}$ to isolate $\tilde{\mathbf{w}}$. This is the **normal equation**: one formula that hands you the exact best-fit weights, no searching required.

▲ WATCH OUT

The inverse $(\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}})^{-1}$ only exists when $\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}$ is non-singular – roughly, when no feature is an exact linear combination of the others (e.g. one column being exactly twice another). In practice, real code never calls an explicit matrix-inverse function; it calls a linear-solve routine instead, which is faster and numerically safer for the same answer.

◆ THE SAME THING IN NUMPY

```
import numpy as np

def fit_normal_equation(X, y):
    n = X.shape[0]
    X_aug = np.hstack([np.ones((n, 1)), X])      # bias trick: prepend a
    # column of 1s
    w_aug = np.linalg.solve(X_aug.T @ X_aug, X_aug.T @ y)  # solve,
    # don't explicitly invert
    b, w = w_aug[0], w_aug[1:]
    return w, b

X = np.array([[650.0], [800.0], [1200.0], [1500.0]])  # house size, sqft
y = np.array([150_000.0, 180_000.0, 240_000.0, 300_000.0])

w, b = fit_normal_equation(X, y)
print(w, b)  # array([176.6...]), 22...  -- exact best-fit line, in one
call
```

CHECK YOURSELF

Suppose two of your features are "size in sqft" and "size in square meters" – one is just the other times a constant. What breaks in the normal equation, and why?

Reveal ▼

ANSWER

$\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}$ becomes singular (non-invertible), because the two columns carry identical information – infinitely many combinations of their two weights produce the exact same predictions, so there's no single "best" solution to land on. Geometrically, the two features point in the same direction, so the loss bowl isn't a bowl anymore along that direction – it's flat, like a valley floor rather than a single pit. The usual fix, *ridge regularization*, adds a small penalty that makes the matrix invertible again (a technique from [Section 7](#)).

5 Optimization, take two: gradient descent

◆ INTUITION FIRST

The normal equation is exact, but it needs to invert a $(d + 1) \times (d + 1)$ matrix, which gets expensive – and sometimes impossible – as the number of features d grows huge. **Gradient descent** instead starts from any guess and repeatedly takes a small step downhill on the loss surface, using the gradient (Section 2 of [this course](#)) as a compass that always points toward steepest increase – so we step in the opposite direction.

☂ EVERYDAY PARALLEL

You're hiking down a mountain in thick fog. You can't see the valley floor, but you can feel which way the ground slopes under your feet right now. So you take a step in the steepest downhill direction, feel the new slope, take another step, and repeat. You never need a map of the whole mountain – just the local slope, over and over. It's the same foggy hillside from [Section 2](#); here we finally walk all the way down it.

◆ NOW THE MATH

We already derived the gradient above: $\nabla_{\tilde{\mathbf{w}}} L = \frac{2}{n} \tilde{\mathbf{X}}^\top (\tilde{\mathbf{X}} \tilde{\mathbf{w}} - \mathbf{y})$. Gradient descent uses it not to solve for zero directly, but to take a small step against it:

$$\tilde{\mathbf{w}} \leftarrow \tilde{\mathbf{w}} - \alpha \nabla_{\tilde{\mathbf{w}}} L$$

In words: the arrow means "replace the old value with this new one." α (alpha) is the *learning rate*, a small positive number controlling step size. We move a little bit in the direction *opposite* the gradient, because the gradient points toward increasing loss and we want to decrease it. Repeat this update many times and $\tilde{\mathbf{w}}$ slides down the loss bowl toward its bottom.

◆ WHEN GRADIENT DESCENT BEATS THE CLOSED FORM

The normal equation costs roughly $O(d^3)$ to invert a $d \times d$ matrix – for a dataset with a million features (e.g. a bag-of-words text representation), that's completely infeasible, and such matrices are often sparse in a way that inversion destroys. Gradient descent instead only needs cheap matrix–vector products at each step, can process data in small batches instead of loading everything into memory at once, and works even when $\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}$ isn't invertible at all. The trade: it gives an approximate answer that improves gradually, instead of the closed form's exact answer in one shot.

◆ THE SAME THING IN PYTORCH

```
import torch

X = torch.tensor([[650.], [800.], [1200.], [1500.]])
y = torch.tensor([150_000., 180_000., 240_000., 300_000.])

# gradient descent is sensitive to feature scale -- normalize first
X_mean, X_std = X.mean(), X.std()
Xn = (X - X_mean) / X_std

w = torch.zeros(1, requires_grad=True)
b = torch.zeros(1, requires_grad=True)
lr = 0.1

for step in range(500):
    y_hat = Xn.squeeze() * w + b          # forward pass (the hypothesis)
    loss = ((y_hat - y) ** 2).mean()      # MSE
```

```

loss.backward()                                # autograd fills w.grad, b.grad

with torch.no_grad():                          # the update step itself isn't
    part of the graph
        w -= lr * w.grad
        b -= lr * b.grad
        w.grad.zero_()
        b.grad.zero_()

print(w.item(), b.item(), loss.item())

```

CHECK YOURSELF

You're training on a bag-of-words dataset with 1,000,000 features. Why is gradient descent the practical choice over the normal equation here? Reveal ▼

ANSWER

The normal equation needs $(\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}})^{-1}$, a $1,000,000 \times 1,000,000$ matrix inverse – roughly 10^{18} operations and enormous memory, completely impractical. Gradient descent instead touches the data only through matrix–vector products, which stay cheap and can even exploit the sparsity that bag-of-words features usually have (most entries are zero). It trades an exact one-shot answer for a sequence of cheap, scalable steps.

6 What to carry forward

- Learning = **hypothesis class** (what functions we allow) + **loss** (how we score wrongness) + **optimization** (how we search) – every algorithm in this course is this triad in a different costume.
- Linear regression's hypothesis is $\hat{y} = \mathbf{w}^\top \mathbf{x} + b$, a straight line (or plane, or hyperplane) fit to the data.

- **MSE** squares and averages the errors – and isn't an arbitrary choice: it's the maximum-likelihood loss when noise is Gaussian.
- The **normal equation** $\tilde{\mathbf{w}} = (\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}})^{-1} \tilde{\mathbf{X}}^\top \mathbf{y}$ solves for the exact minimum directly, by setting the gradient to zero.
- **Gradient descent** instead walks downhill step by step – slower per problem, but the only option once the data is too large or too sparse to invert.

Next we'll keep the same triad but swap the hypothesis and the loss: predicting a category instead of a number, in [Logistic Regression](#).

Logistic Regression

One S-shaped curve turns a raw number into a probability, and one loss function turns wrong probabilities into a clean, learnable signal.

24 min read You'll learn: sigmoid · probabilities · decision boundaries · binary cross-entropy

Linear regression predicts a number: a price, a temperature, a score. Plenty of real questions instead want a category: spam or not spam, will this patient relapse or not, does this photo contain a cat. Logistic regression takes almost the exact same machinery as linear regression and bends it, with one extra ingredient, into a probability.

◆ THE ONE IDEA TO HOLD ONTO

Compute a plain linear score, same as linear regression. Then squash that score through a curve that only ever outputs numbers between 0 and 1, so it can be read as "probability of class 1." Everything else in this section is either building that curve or building the loss that teaches it.

1 Classification vs. regression

◆ INTUITION FIRST

Regression predicts a continuous number — anything on a sliding scale. **Classification** predicts which of a fixed set of categories something belongs to. The output isn't "how much," it's "which one."

🔼 EVERYDAY PARALLEL

A weather forecast does both jobs side by side. "Tomorrow's high will be 22°C" is regression – a number on a continuous scale, and being off by half a degree is a small mistake. "Will it rain tomorrow: yes or no" is classification – there is no "half a rain," just one of two buckets, and the model's job is to say which bucket is more likely.

CHECK YOURSELF

Is "will this email be marked as spam?" a regression or a classification problem, and why?

Reveal ▼

ANSWER

Classification – the answer is one of two discrete categories (spam / not spam), not a number on a continuous scale. Logistic regression is built exactly for this two-category ("binary") case.

2 The sigmoid: turning any number into a probability

◆ INTUITION FIRST

A linear score $w\mathbf{x} + b$ can be any real number – huge, tiny, negative, whatever the data demands. But a probability has to live strictly between 0 and 1. The **sigmoid function** is a squashing curve that takes any real number in and always returns a number in that range, so we can hand it a raw linear score and get back something we're allowed to call "probability."

🔼 EVERYDAY PARALLEL

Think of it as a dimmer switch built to never quite reach fully off or fully on – the light can get extremely dim or extremely bright, but it always stays a real, positive amount of light. Or picture a student's chance of passing an exam as study hours climb: near zero hours, the chance sits near 0%; past some point, a few extra hours barely move the needle because it's already close to 100%. The chance rises in a smooth S-shape, never quite touching 0% or 100%.

◆ NOW THE MATH

Given any real number z (called the *logit*), the sigmoid is:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

In words: e^{-z} shrinks toward 0 as z grows large and blows up toward infinity as z becomes very negative. So the whole fraction crawls toward $1/(1 + 0) = 1$ for large positive z , and toward $1/(1 + \infty) \approx 0$ for very negative z — exactly the squashing behavior we wanted.

Two properties worth deriving, because we'll lean on both later. First, the midpoint:

$$\sigma(0) = \frac{1}{1 + e^0} = \frac{1}{1 + 1} = \frac{1}{2}$$

In words: a logit of exactly zero maps to a probability of exactly one-half — total uncertainty, the tipping point of the curve.

Second, the sigmoid's derivative — needed later to derive the cross-entropy gradient. Rewrite $\sigma(z) = (1 + e^{-z})^{-1}$ and apply the chain rule:

$$\begin{aligned} \frac{d\sigma}{dz} &= -(1 + e^{-z})^{-2} \cdot (-e^{-z}) \\ &= \frac{e^{-z}}{(1 + e^{-z})^2} \\ &= \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} \\ &= \sigma(z)(1 - \sigma(z)) \end{aligned}$$

In words: the last line uses the fact that $1 - \sigma(z) = \frac{e^{-z}}{1 + e^{-z}}$ (subtract the fraction from 1 and simplify). So the slope of the sigmoid at any point is just $\sigma(z)$ times $(1 - \sigma(z))$ — a tidy self-

referential formula that's largest at $z = 0$ (where the curve is steepest) and shrinks toward 0 at either extreme (where the curve is nearly flat).

◆ THE SAME THING IN PYTORCH

```
import torch

z = torch.tensor([-5.0, 0.0, 5.0])
print(torch.sigmoid(z))    # tensor([0.0067, 0.5000, 0.9933])
```

CHECK YOURSELF

What is $\sigma(0)$, and why does that make it the natural cutoff for the decision boundary? [Reveal ▼](#)

ANSWER

$\sigma(0) = 0.5$ exactly. Since a probability of 0.5 is the point of maximum uncertainty (equally likely to be either class), the logit value that produces it – $z = 0$ – is the natural line between "lean toward class 1" and "lean toward class 0."

3 From logit to probability to decision boundary

◆ INTUITION FIRST

Logistic regression computes the same linear score as linear regression, then passes it through the sigmoid. The point where the model flips from predicting "class 0" to predicting "class 1" turns out to be a perfectly straight line (or flat plane, in higher dimensions) – nothing curvy about where the decision itself is made.

◆ NOW THE MATH

The score, called the logit, is exactly linear regression's formula:

$$z = \mathbf{w}^\top \mathbf{x} + b$$

Pass it through the sigmoid to get a probability of belonging to class 1:

$$p = \sigma(z) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}$$

In words: z can be any real number depending on x ; p is that number squashed into a genuine probability between 0 and 1.

The prediction rule is: predict class 1 when $p \geq 0.5$. Because $\sigma'(z) = \sigma(z)(1 - \sigma(z)) \geq 0$ everywhere, the sigmoid is non-decreasing – it never turns downward – and we showed $\sigma(0) = \frac{1}{2}$. Combining those two facts:

$$p \geq \frac{1}{2} \iff \sigma(z) \geq \sigma(0) \iff z \geq 0 \iff \mathbf{w}^\top \mathbf{x} + b \geq 0$$

In words: because the sigmoid only ever goes up (or stays flat) as z increases, and it crosses exactly one-half at $z = 0$, asking "is the probability at least a half" is exactly the same question as "is the raw logit non-negative." The messy curved probability threshold reduces to a simple straight-line threshold on the underlying score.

The boundary itself – where the model is perfectly on the fence, p exactly 0.5 – is the set of points where $\mathbf{w}^\top \mathbf{x} + b = 0$. With two features x_1, x_2 this rearranges into the equation of a straight line:

$$w_1 x_1 + w_2 x_2 + b = 0 \implies x_2 = -\frac{w_1}{w_2} x_1 - \frac{b}{w_2}$$

In words: that's just $y = mx + c$ in disguise, with slope $-w_1/w_2$ and intercept $-b/w_2$. With three features it would be a flat plane instead of a line; with more, a flat *hyperplane* – but always flat, never curved, no matter how curved the probability surface around it looks.

CHECK YOURSELF

With $\mathbf{w} = [2, -1]$ and $b = -3$, is the point $\mathbf{x} = [2, 1]$ on the decision boundary, and which class does it favor?

Reveal ▼

ANSWER

$z = (2)(2) + (-1)(1) + (-3) = 4 - 1 - 3 = 0$. Since $z = 0$, this point sits exactly on the decision boundary – $\sigma(0) = 0.5$, so the model is perfectly torn between the two classes, favoring neither.

4 The loss: binary cross-entropy

◆ INTUITION FIRST

You might guess we'd reuse Mean Squared Error on the probability output — predicted probability minus true label, squared. It turns out that choice quietly breaks optimization. Instead, classification uses **binary cross-entropy**, a loss built specifically for comparing probabilities.

DEFINITION

A function is **convex** when its graph is bowl-shaped: exactly one lowest point, with the surface sloping downward toward it from every direction and no other dips or flat plateaus to get stuck in. Gradient descent is guaranteed to find the true minimum of a convex loss; on a non-convex loss it can stall on a flat shelf or a false valley.

◆ NOW THE MATH

Here is exactly why squared error misbehaves on top of a sigmoid. Let $L_{sq} = (\hat{y} - y)^2$ with $\hat{y} = \sigma(z)$. By the chain rule (Section 2), using the sigmoid's own derivative $\sigma'(z) = \hat{y}(1 - \hat{y})$ from earlier in this section:

$$\frac{\partial L_{sq}}{\partial z} = 2(\hat{y} - y) \sigma'(z) = 2(\hat{y} - y) \hat{y}(1 - \hat{y})$$

In words: the gradient carries a factor of $\hat{y}(1 - \hat{y})$, the sigmoid's own slope. Suppose the true label is $y = 1$ but the model is confidently — and badly — wrong, predicting $\hat{y} \approx 0$ (a very negative logit). Then $\hat{y}(1 - \hat{y}) \approx 0 \times 1 = 0$, so the *entire gradient collapses to nearly zero* — right when the model most needs a large correction. That vanishing signal on confidently-wrong predictions is what makes squared error's loss surface here riddled with near-flat shelves instead of one clean bowl: exactly the non-convexity described above.

Binary cross-entropy fixes this. For a true label $y \in \{0, 1\}$ and predicted probability $\hat{y} = \sigma(z)$ (see [Section 4](#) for where cross-entropy comes from more generally):

$$L(\hat{y}, y) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

In words: exactly one of the two bracketed terms is "active" for any given example — if $y = 1$ the loss is $-\log \hat{y}$ (small when $\hat{y}$ is close to 1, and shoots toward infinity as $\hat{y} \rightarrow 0$); if $y = 0$ it's

— $\log(1 - \hat{y})$ (small when $\hat{y}$ is close to 0, exploding as $\hat{y} \rightarrow 1$). Being confidently wrong is now punished by an unboundedly growing loss, not a shrinking gradient.

Now derive its gradient with respect to the logit z , step by step. First differentiate with respect to $\hat{y}$:

$$\begin{aligned}\frac{\partial L}{\partial \hat{y}} &= -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} \\ &= \frac{-y(1-\hat{y}) + (1-y)\hat{y}}{\hat{y}(1-\hat{y})} \\ &= \frac{-y + y\hat{y} + \hat{y} - y\hat{y}}{\hat{y}(1-\hat{y})} \\ &= \frac{\hat{y} - y}{\hat{y}(1-\hat{y})}\end{aligned}$$

Then apply the chain rule through $\hat{y} = \sigma(z)$, using $\partial \hat{y} / \partial z = \hat{y}(1 - \hat{y})$:

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \cdot \hat{y}(1 - \hat{y}) = \hat{y} - y$$

In words: the messy $\hat{y}(1 - \hat{y})$ terms cancel exactly, leaving the cleanest possible gradient: predicted probability minus true label. No vanishing, no exploding — just a signal directly proportional to how wrong the prediction is.

Finally, since $z = \mathbf{w}^\top \mathbf{x} + b$, we have $\partial z / \partial w_j = x_j$, so by the chain rule one more time:

$$\frac{\partial L}{\partial \mathbf{w}} = (\hat{y} - y) \mathbf{x}, \quad \frac{\partial L}{\partial b} = \hat{y} - y$$

In words: the gradient with respect to each weight is (error) $\times$ (that feature's value) — the exact same clean shape as linear regression's gradient, even though we started from a completely different loss. This is why logistic regression's training loop looks almost identical to linear regression's.

CHECK YOURSELF

True label $y = 1$, model predicts $\hat{y} = 0.2$ (confidently wrong). What is $\partial L / \partial z$, and which direction does gradient descent push the logit?

Reveal ▼

ANSWER

$\partial L / \partial z = \hat{y} - y = 0.2 - 1 = -0.8$. Gradient descent updates z (indirectly, through w and b) by subtracting a positive multiple of this gradient, so z moves by $-(-0.8) \cdot (\text{something positive}) = \text{a positive amount}$ — the logit is pushed *up*, which pushes $\hat{y}$ toward 1, correcting the mistake. A more confidently wrong prediction (say $\hat{y} = 0.01$) gives an even larger-magnitude gradient (-0.99), a bigger correction — exactly the opposite of the vanishing-gradient problem squared error had.

5 Training logistic regression in PyTorch

◆ THE SAME THING IN PYTORCH

```
import torch
import torch.nn as nn

X = torch.tensor([[2.0], [4.0], [6.0], [8.0]]) # hours studied
y = torch.tensor([0., 0., 1., 1.]) # failed / passed

w = torch.zeros(1, requires_grad=True)
b = torch.zeros(1, requires_grad=True)
lr = 0.1

# BCEWithLogitsLoss combines sigmoid + binary cross-entropy in one
# numerically
# stable call -- computing log(sigmoid(z)) directly avoids log(0) errors
loss_fn = nn.BCEWithLogitsLoss()

for step in range(1000):
    z = X.squeeze() * w + b # the logit -- do NOT apply sigmoid
    # before this loss
```

```

loss = loss_fn(z, y)

loss.backward()
with torch.no_grad():
    w -= lr * w.grad
    b -= lr * b.grad
    w.grad.zero_()
    b.grad.zero_()

probs = torch.sigmoid(X.squeeze() * w + b)  # convert logits to
probabilities for reporting
print(probs, loss.item())

```

▲ WATCH OUT

Don't apply `torch.sigmoid` yourself and then feed the result into `nn.BCELoss` if you can instead use `nn.BCEWithLogitsLoss` directly on the raw logit – the combined version is numerically stable (it never actually computes $\log(0)$ even when the sigmoid saturates near 0 or 1), while doing the two steps separately can silently produce `nan` losses on confidently wrong predictions.

6 What to carry forward

- **Classification** predicts a category, not a number; logistic regression handles the two-category case.
- The **sigmoid** $\sigma(z) = 1/(1 + e^{-z})$ squashes any real logit into a valid probability, with $\sigma(0) = 0.5$ and derivative $\sigma(z)(1 - \sigma(z))$.
- The **decision boundary** ($p = 0.5$) is exactly where the logit $\mathbf{w}^\top \mathbf{x} + b$ crosses zero – always flat, a line or hyperplane.
- **Binary cross-entropy** replaces squared error because squared error's gradient vanishes on confidently-wrong predictions; cross-entropy's gradient simplifies cleanly to $(\hat{y} - y)\mathbf{x}$.

Back in [Section 5](#) we solved a regression problem two ways; here we saw the same hypothesis-loss-optimization triad reused for classification, just with a squashing function and a different

loss.

Overfitting & Regularization

A model that scores 100% on its training data has told you nothing yet – this section is about the honest number, the one measured on data the model has never seen, and how to protect it.

26 min read **You'll learn:** generalization · bias–variance · L1/L2 regularization · train/val/test · cross-validation

Every model can be made to fit the data it was trained on almost perfectly – that part is easy, and it is also nearly meaningless. What we actually care about is whether the model works on **new** data it has never touched: tomorrow's customer, the next photo, a sentence it hasn't read. This section is about that gap, why it opens up, and the concrete tools that close it.

◆ THE ONE IDEA TO HOLD ONTO

The only score that matters is performance on data the model never saw during training. Every idea below – splitting your data, cross-validation, regularization – exists purely to protect that one honest number from being fooled.

1 Generalization: doing well on data you haven't seen

◆ INTUITION FIRST

Generalization is a model's ability to perform well on new, unseen examples – not just the ones it was trained on. A model that generalizes has learned the actual pattern in the data. A model that hasn't has instead memorized the specific examples, including their noise and quirks.

🔼 EVERYDAY PARALLEL

Two students prepare for a history exam. One **memorizes** last year's exam paper word for word – every date, every answer, in order. The other actually **understands** the period: causes, consequences, how events connect. Give both students last year's exact paper and the memorizer wins easily. Give them this year's paper, with the same material asked differently, and the memorizer collapses while the one who understood does fine. The memorizer *overfit* to one specific paper; the other *generalized* to the subject itself.

DEFINITION

Overfitting is when a model fits the training data too closely – including its noise and one-off quirks – so training error keeps dropping while error on new data gets worse.

Underfitting is the opposite: the model is too simple to capture even the real pattern, so it does poorly on both the training data and new data. In between sits **model**

capacity – a rough measure of how flexible a model is, i.e. how complicated a pattern it's able to represent. A straight line has low capacity; a deep tree with a thousand splits has very high capacity.

Capacity is the dial that controls both failure modes. Turn it too low and the model can't represent the real pattern – it underfits. Turn it too high and the model has so much flexibility that, with only a finite training set, it starts fitting the random noise in that specific set – it overfits. There's a sweet spot in between, and finding it is most of what this section is about.

◆ SEEING IT HAPPEN

>_ FITTING THE SAME NOISY DATA WITH THREE MODEL CAPACITIES

```

import numpy as np

rng = np.random.default_rng(0)
x = np.linspace(-3, 3, 20)
y = 0.5 * x**2 + rng.normal(0, 1, size=x.shape)    # true pattern: a
                                                    # parabola, plus noise

x_test = np.linspace(-3, 3, 200)
y_test = 0.5 * x_test**2                          # noise-free, for a
                                                    # fair test score

for degree in (1, 2, 15):
    coeffs = np.polyfit(x, y, degree)
    train_mse = np.mean((np.polyval(coeffs, x) - y) ** 2)
    test_mse = np.mean((np.polyval(coeffs, x_test) - y_test) ** 2)
    print(f"degree {degree:2d}: train MSE {train_mse:.3f}    test MSE
{test_mse:.3f}")

# degree 1 (a line): can't bend to match a parabola → both train and
test error stay high (underfit)
# degree 2 (a parabola): matches the true pattern → train and test
error both low and close together
# degree 15: bends through nearly every noisy point → train error near
zero, test error much worse
#           than degree 2 -- it learned the noise, not the pattern
(overfit)

```

CHECK YOURSELF

A model reaches 99% accuracy on its training set but only 61% on its validation set. Is this overfitting or underfitting, and what's a sensible first fix? Reveal ▼

ANSWER

This is **overfitting** – near-perfect training performance next to much worse performance on data the model didn't train on is the signature. The model has enough capacity to memorize the training set rather than learn the general pattern. Sensible first fixes: reduce capacity (a smaller model, fewer tree splits), add regularization (covered later in this section), collect more training data (harder to memorize a bigger set), or stop training earlier before it starts fitting noise.

2 Train / validation / test — and cross-validation

◆ INTUITION FIRST

If you only ever measure your model on the data it trained on, you have no way to detect overfitting – the memorizer always looks brilliant on the paper it memorized. So we split the data into three separate pots, each with a different job, and we never let them mix.

☂ EVERYDAY PARALLEL

Studying for a real exam has three phases. You do **practice questions** to learn the material (the **training set**). You then sit a **mock exam** to see how you're doing and decide what to restudy or which strategy to change (the **validation set**). Finally, you sit the **real exam** exactly once (the **test set**) – its whole value comes from the fact that you never adjusted your studying based on it. If you somehow saw the real exam early and kept "improving" based on it, your final grade would no longer tell you anything honest about how well you actually know the material.

DEFINITION

The **training set** is what the model directly learns its numbers from. The **validation set** is held out during training and used to make decisions – which model, which settings (called **hyperparameters**: choices you pick before training, like capacity or the regularization strength below). The **test set** is touched exactly once, at the very end, to report a final, unbiased number. If you tune anything based on the test set, you've quietly turned it into a second validation set, and your reported score stops being trustworthy.

◆ NOW THE MATH

With only one validation split, your estimate of "how good is this model" depends on which examples happened to land in that split – a small, unlucky validation set can make a good model look bad, or vice versa. **k-fold cross-validation** fixes this by rotating which slice is held out, so every example gets used for validation exactly once:

$$\text{CV score} = \frac{1}{k} \sum_{i=1}^k \text{score}(\text{model trained on all folds except } i, \text{ evaluated on fold } i)$$

In words: split the training data into **k** equal-sized folds (5 and 10 are the common choices). Train **k** separate times, each time leaving out a different fold to validate on, then average the **k** validation scores. Every example gets validated on exactly once, so the final average is far less sensitive to one unlucky split — and you also get a spread across the **k** scores, telling you how stable the estimate is.

◆ THE SAME THING IN CODE

```
import numpy as np
from sklearn.model_selection import KFold

X = np.arange(20).reshape(10, 2)          # 10 training examples, 2
features each
kf = KFold(n_splits=5, shuffle=True, random_state=0)

for fold, (train_idx, val_idx) in enumerate(kf.split(X)):
    print(f"fold {fold}: {len(train_idx)} rows train, {len(val_idx)} rows
validate")

# 5 folds of 10 rows → each validation fold has 2 rows, each training
```


fold has the other 8.

By the end, every one of the 10 rows has served as validation data exactly once.

CHECK YOURSELF

While tuning a model, you check test-set accuracy after every change and keep the version that scores highest on the test set. What's wrong with this, even though you never trained on the test set directly?

Reveal ▼

ANSWER

You never trained on it, but you **selected** based on it — and selection is a form of fitting. Across enough attempts, some version will score well on the test set purely by luck (matching its particular noise), and picking "the best on the test set" is indistinguishable from tuning on it. The reported number stops estimating real-world performance and starts estimating "how well did I get lucky on this one fixed sample." The fix is to do all that tuning against the validation set (or cross-validation), and touch the test set only once, at the very end, to report the final number.

3 The bias–variance tradeoff

◆ INTUITION FIRST

Overfitting and underfitting are two flavors of a deeper split: a model's test error comes from two separate sources, **bias** and **variance**, and they trade off against each other as capacity changes.

📌 EVERYDAY PARALLEL

Picture a dartboard. **High bias** is a thrower whose darts land in a tight little cluster – but that cluster is off in the upper-left corner, nowhere near the bullseye. Consistent, but consistently wrong: a systematic error. **High variance** is a thrower whose darts scatter all over the board – sometimes near the bullseye, sometimes far from it, with no consistency at all. A great model is the rare thrower who is both consistent *and* centered on the bullseye; a bad one can fail in either direction, or both.

DEFINITION

Bias is systematic error: the model's assumptions are too simple to capture the real pattern, so its predictions are wrong in the same direction no matter what training data it sees (an underfitting straight line, forced to fit a curve, is high-bias). **Variance** is sensitivity to the particular training set: retrain on a slightly different sample and the predictions swing wildly (a deep, unconstrained tree that carves itself around every training point is high-variance).

◆ NOW THE MATH

Suppose the true relationship is $y = f(x) + \epsilon$, where ϵ is random noise with mean 0 and variance σ^2 (irreducible – no model can predict pure noise). We train a model on a random training set $\mathbf{D}$, getting a prediction $\hat{f}_D(x)$ at some test point x . Let $\bar{f}(x) = \mathbb{E}_D[\hat{f}_D(x)]$ be the *average* prediction you'd get if you retrained on many different random training sets. Then the expected squared error at x , averaged over training sets, splits cleanly into three pieces:

$$\begin{aligned}\mathbb{E}_D[(y - \hat{f}_D(x))^2] &= \mathbb{E}_D[((f(x) - \bar{f}(x)) + (\bar{f}(x) - \hat{f}_D(x)) + \epsilon)^2] \\ &= \underbrace{(f(x) - \bar{f}(x))^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}_D[(\bar{f}(x) - \hat{f}_D(x))^2]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{irreducible noise}}\end{aligned}$$

In words: the cross terms vanish when you average over many retrainings, because the noise ϵ has mean zero and is independent of the data, and $\hat{f}_D(x)$ deviates from its own average $\bar{f}(x)$ in both directions equally. What's left is three additive pieces: how far off-center your average prediction is (**bias²**), how much predictions swing between different training sets (**variance**), and noise you could never predict no matter how good the model is (**σ^2**). Total test error is the sum of all three — you can trade bias for variance, but you can never drive either below zero, and you can never remove the noise floor.

◆ THE TRADEOFF

Raising model capacity (a deeper tree, more parameters, a higher-degree polynomial) typically **lowers bias** – the model can now represent the true pattern more closely – but **raises variance** – it also has more freedom to chase the noise in whatever training set it happens to see. Total test error, plotted against capacity, is usually U-shaped: too little capacity and bias dominates, too much and variance dominates, with a minimum somewhere in the middle. Regularization, up next, is a way to buy back some of that variance without fully giving up capacity.

CHECK YOURSELF

A single straight line is fit to clearly curved data, and separately, a deep unregularized decision tree is fit to the same data. Which one is high-bias/low-variance, and which is low-bias/high-variance?

Reveal ▼

ANSWER

The straight line is **high-bias, low-variance**: it's too simple to bend to the curve (systematic error, the same wrong shape no matter what sample you train it on), but retrain it on a different sample and you'll get almost the same line back – it's stable. The deep tree is **low-bias, high-variance**: it's flexible enough to fit the curve closely on average, but it also carves itself around whichever specific noisy points it happened to see, so a different training sample produces a noticeably different tree.

4 Regularization: penalizing the size of the weights

◆ INTUITION FIRST

Regularization is a way to fight overfitting directly: add an extra term to the loss that penalizes weights for being *large*, on top of the usual term that penalizes wrong predictions. A model can still use every weight, but it now pays a price for using it aggressively, which pushes it

toward simpler, smoother solutions that are less likely to be chasing noise. The penalty is measured with the norms from [Section 1](#) – L2 or L1, applied to the weights.

☂ EVERYDAY PARALLEL

Put yourself on a strict monthly budget. With plenty of room to spend, you buy everything that seems even a little useful. Under a tight budget, you're forced to make choices: some small, marginal purchases get **cut to zero** entirely (you decide you don't need them at all), while the purchases you keep, you trim down a little across the board to stay under the limit. L1 regularization behaves like the first kind of budgeting – it zeroes out the marginal weights entirely. L2 behaves like the second – it trims everything down a bit, but rarely cuts anything all the way to nothing.

◆ NOW THE MATH

L2 regularization (also called *ridge*, or *weight decay* when used with gradient descent) adds the squared L2 norm of the weights to the loss:

$$L_{\text{ridge}}(\boldsymbol{\theta}) = L_{\text{data}}(\boldsymbol{\theta}) + \lambda \|\boldsymbol{\theta}\|_2^2 = L_{\text{data}}(\boldsymbol{\theta}) + \lambda \sum_i \theta_i^2$$

In words: L_{data} is the ordinary loss measuring wrong predictions; λ (lambda, a hyperparameter you choose) controls how strongly we punish large weights. Take the gradient of this loss with respect to one weight θ_i and plug it into a gradient descent step with learning rate α :

$$\theta_i \leftarrow \theta_i - \alpha \left(\frac{\partial L_{\text{data}}}{\partial \theta_i} + 2\lambda \theta_i \right) = \underbrace{(1 - 2\alpha\lambda)}_{\text{shrink factor} < 1} \theta_i - \alpha \frac{\partial L_{\text{data}}}{\partial \theta_i}$$

In words: every single update multiplies the weight by a shrink factor slightly less than 1, *before* applying the usual data-driven update. That's why L2 is nicknamed "weight decay" — the weight quietly decays toward zero every step, on top of whatever the data pushes it toward. Because the shrink is proportional to the weight's own size, it acts on every weight, but never fully zeroes one out — a smaller and smaller nudge as the weight approaches zero, never quite finishing.

L1 regularization (also called *lasso*) instead adds the L1 norm:

$$L_{\text{lasso}}(\boldsymbol{\theta}) = L_{\text{data}}(\boldsymbol{\theta}) + \lambda \|\boldsymbol{\theta}\|_1 = L_{\text{data}}(\boldsymbol{\theta}) + \lambda \sum_i |\theta_i|$$

In words: the penalty's gradient with respect to θ_i is just $\lambda \cdot \text{sign}(\theta_i)$ — a *constant* push toward zero, the same size whether the weight is huge or tiny. Unlike L2's shrink-by-a-fraction, this is shrink-by-a-fixed-amount every step, so a small weight can get pushed straight through zero and the optimizer sets it to exactly zero and leaves it there. That's L1 doing *feature selection*: deciding certain inputs are simply not worth using.

◆ WHY L1 ZEROES WEIGHTS AND L2 DOESN'T: THE GEOMETRIC REASON

Both penalties can be viewed as a budget: keep the total penalty under some limit t while minimizing the data loss. The L2 budget $\sum \theta_i^2 \leq t$ traces out a smooth **circle** (or sphere, in higher dimensions) around the origin. The L1 budget $\sum |\theta_i| \leq t$ traces out a **diamond** — a shape with sharp corners that sit exactly on the axes (where one weight is zero and the rest aren't). As the loss's contour lines expand outward from their unconstrained best point, the first place they touch the constraint region is, for a smooth circle, almost never exactly on an axis — but for a diamond, the corners stick out further than the flat edges, so contours very often make first contact right at a corner. Touching at a corner means at least one weight is *exactly* zero. The circle has no corners to land on, so L2 shrinks everything smoothly instead.

◆ THE SAME THING IN PYTORCH

›_ L2 VIA `WEIGHT_DECAY` , L1 ADDED TO THE LOSS BY HAND

```

import torch
import torch.nn as nn

model = nn.Linear(10, 1)
criterion = nn.MSELoss()
x = torch.randn(32, 10)
y = torch.randn(32, 1)

# L2 / ridge / "weight decay": built into the optimizer. Every step also
# shrinks each weight by a factor of (1 - lr * weight_decay), pulling it
toward 0.
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, weight_decay=1e-4)
loss = criterion(model(x), y)
loss.backward()
optimizer.step()

# L1 / lasso: |w| has no clean gradient at w = 0, so there's no optimizer
# flag for it -- add the penalty to the loss by hand instead.
optimizer.zero_grad()
l1_penalty = sum(p.abs().sum() for p in model.parameters())
loss = criterion(model(x), y) + 1e-4 * l1_penalty
loss.backward()
optimizer.step()

```

CHECK YOURSELF

You train the same linear model twice, once with L1 and once with L2 regularization, at strengths tuned to give similar validation accuracy. The L1 model ends up with many weights at exactly 0.0000; the L2 model has no exact zeros, just many small values. Why?

Reveal ▼

ANSWER

L1's penalty gradient is a constant push toward zero regardless of the weight's size, so small weights get driven straight through zero and stay there – geometrically, the diamond-shaped L1 constraint region has corners sitting exactly on the axes, and the loss contours tend to first touch the constraint at one of those corners, which means some weight is exactly zero. L2's penalty gradient is proportional to the weight's own size, so the push gets weaker and weaker as the weight shrinks – it approaches zero but the constraint region is a smooth circle with no corners to land on, so it never quite gets there. This is why L1 is used for feature selection and L2 is used for general-purpose smoothing.

5 What to carry forward

- **Generalization** means doing well on unseen data. **Overfitting** fits the training data's noise; **underfitting** is too simple to fit the real pattern. **Capacity** is the dial between them.
- **Train / validation / test** have separate jobs: train learns the weights, validation tunes decisions, test is touched exactly once for an honest final number. **k-fold cross-validation** rotates the held-out slice so every example validates exactly once.
- Test error decomposes into **bias² + variance + irreducible noise**. More capacity usually trades bias for variance; the goal is the sweet spot that minimizes the sum.
- **L2 (ridge)** shrinks every weight smoothly by a fraction each step ("weight decay"). **L1 (lasso)** pushes by a constant amount and can drive weights to exactly zero, because its

constraint region is a diamond with corners on the axes rather than a smooth circle.

Next, we zoom out from any single algorithm and take a broad tour of the classical models – kNN, decision trees, ensembles, and SVMs – to build intuition for when and why each one is the right tool, in [Section 8](#).

A Tour of Classical Models

Before neural networks, machine learning already worked — meet the four ideas that still quietly win on everyday tabular data.

16 min read You'll learn: k-nearest neighbors · decision trees · ensembles · SVM intuition

This section is a fast tour, on purpose. No derivations — just one solid intuition and one everyday parallel per model, so you know what each one is doing and when to reach for it. We'll go deep on neural networks for the rest of the course; these classical models are still the right choice for a huge share of real, structured (tabular) data problems.

◆ THE ONE IDEA TO HOLD ONTO

Each model below makes a different bet about how to turn "examples I've seen" into "a prediction for something new." Knowing the bet each one makes is enough to know when to use it — you don't need to derive it to use it well.

1 k-Nearest Neighbors: ask the closest examples

◆ INTUITION FIRST

k-Nearest Neighbors (kNN) has almost no "training" step at all — it just remembers every labeled example. To predict on something new, it finds the **k** stored examples closest to it (measured by ordinary distance — the L2 norm from [Section 1](#)) and takes a majority vote of their labels.

🔦 EVERYDAY PARALLEL

You've just moved to a new neighborhood and want a restaurant recommendation. You don't consult a general-purpose critic – you ask the few neighbors who live *closest* to you, on the theory that people nearby tend to have similar taste and know the same local spots. Whatever most of them recommend is your answer. That's kNN: no theory about restaurants in general, just "ask the nearest ones and go with the majority."

▲ WATCH OUT: THE CURSE OF DIMENSIONALITY

With very many features, almost every pair of points ends up roughly the same distance apart, so "closest" stops meaning much – kNN quietly degrades as dimensions pile up.

◆ THE SAME THING IN CODE

```
from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)           # "fit" just stores the data -- no
real learning happens
pred = model.predict(X_new)           # for each new point: find its 5
closest neighbors, vote
```

CHECK YOURSELF

Why does kNN get slower at prediction time as your training set grows, unlike a trained decision tree or SVM?

Reveal ▼

ANSWER

Because kNN has no real training phase – all the "work" is deferred to prediction time. Every single prediction requires measuring the distance from the new point to *every* stored training example to find the closest **k**. A decision tree or SVM instead spends effort once, up front, to build a compact rule (a tree of splits, or a boundary defined by a few support vectors), so predicting afterward is fast regardless of how large the original training set was.

2 Decision trees: a flowchart of yes/no questions

◆ INTUITION FIRST

A **decision tree** predicts by asking a series of simple yes/no questions about the features ("is age > 30?", "is income > 50k?"), branching on the answer, and repeating until it reaches a leaf that holds the prediction. Training a tree just means choosing, at each branch, the question that best separates the classes.

🌂 EVERYDAY PARALLEL

It's the game of **20 Questions**. "Is it alive?" "Is it bigger than a breadbox?" Each answer cuts the space of possibilities roughly in half, and a handful of well-chosen questions narrows a huge range of options down to one guess. A decision tree is exactly this game played automatically, with the questions chosen from the training data.

Trees are wonderfully **interpretable** – you can print the whole flowchart and read off exactly why it made a prediction. But left alone, they **overfit easily**: given enough questions, a tree can carve out a tiny leaf for every single training example, memorizing the data instead of learning the pattern (exactly the failure mode from [Section 7](#)).

◆ THE SAME THING IN CODE

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(max_depth=4)    # cap the depth so it can't
ask endless questions
model.fit(X_train, y_train)
```

CHECK YOURSELF

Why does limiting `max_depth` help a decision tree generalize better?

Reveal ▼

ANSWER

Depth controls the tree's capacity: an unlimited tree can keep splitting until every leaf holds a single training example, which means it has memorized the training set's noise along with its pattern (high variance). Capping the depth forces the tree to stop asking questions early, keeping only the splits that separate large, meaningful groups – trading a bit of training accuracy for a model that holds up on new data.

3 Ensembles: combine many weak models

◆ INTUITION FIRST

A single tree is unstable and prone to overfitting. **Ensembles** fix this by combining many models instead of trusting one, in two different ways.

🔼 EVERYDAY PARALLEL

Bagging (used in **random forests**) is the wisdom-of-the-crowd fact that a county fair uses to guess an ox's weight: ask one person and they might be way off, but average hundreds of independent guesses and the errors cancel out, leaving a surprisingly accurate number. A random forest trains many trees, each on a random subset of the data and features, then averages their votes. **Boosting** is different: it's like a series of tutors, where each new tutor looks specifically at the mistakes the previous tutors' combined advice got wrong, and focuses entirely on fixing those. Each new model is trained to correct the ensemble's current errors, and the models are added together rather than simply averaged.

◆ THE SAME THING IN CODE

```
from sklearn.ensemble import RandomForestClassifier,
GradientBoostingClassifier

forest = RandomForestClassifier(n_estimators=200)    # bagging: many
trees, average their votes
forest.fit(X_train, y_train)

boosted = GradientBoostingClassifier(n_estimators=200) # boosting: each
tree fixes the last one's errors
boosted.fit(X_train, y_train)
```

CHECK YOURSELF

A random forest and a gradient boosting model both use decision trees underneath. What's the key structural difference in how they combine those trees?

Reveal ▼

ANSWER

A random forest trains its trees **independently and in parallel**, each on a random subset of data and features, then averages their votes – the trees don't know about each other. Gradient boosting trains its trees **sequentially**: each new tree is trained specifically to correct the errors the ensemble-so-far is still making, then gets added on top. Bagging mainly reduces variance (averaging cancels noise); boosting mainly reduces bias (each step directly attacks remaining errors).

4 SVM intuition: the widest street between classes

◆ INTUITION FIRST

A **support vector machine (SVM)** for classification looks for the line (or, in higher dimensions, the flat boundary called a *hyperplane*) that separates the two classes with the **widest possible margin** – the biggest empty gap on either side before you hit a training point.

☂ EVERYDAY PARALLEL

Imagine drawing the widest possible straight road between two neighborhoods, without the road touching a single house on either side. Many roads would separate the neighborhoods, but the widest one gives you the most buffer if a new house is built close to the boundary later. An SVM finds exactly that widest street; the houses closest to its edges – the ones that would get clipped first if the road narrowed – are called the *support vectors*, and they're the only points that actually determine where the road goes.

Real data is rarely neatly separable by a straight line. The **kernel trick** handles this in one clean move: it lifts the data into a higher-dimensional space (for example, adding each point's squared distance from the center as a new feature) where a straight boundary in that higher space can separate classes that were tangled together in the original space – then that boundary maps back down to a curved one in the space you started with. Crucially, the kernel trick computes the effect of this lift without ever explicitly building the higher-dimensional points, which is what keeps it fast.

◆ THE SAME THING IN CODE

```
from sklearn.svm import SVC

model = SVC(kernel="rbf")      # "rbf" kernel = the lift-into-higher-
                               # dimensions trick
model.fit(X_train, y_train)
```

CHECK YOURSELF

Why are only the "support vectors" – and not every training point – needed to describe the final SVM boundary?

Reveal ▼

ANSWER

The boundary is defined purely by finding the widest possible margin, and the width of that margin is limited only by whichever points sit closest to it on each side. Move a point that's already far from the boundary and the widest street doesn't change at all – it was never constraining the margin. Move one of the closest points and the street's position and width shift immediately. So only those closest points (the support vectors) matter for describing the boundary; the rest could be deleted after training with no effect on the model.

5 When to use what

Model	Strength	Weakness	When to reach for it
k-Nearest Neighbors	Simple, no training step, adapts to any shape of boundary	Slow at prediction time on large data; degrades in high dimensions	Quick baseline on small, low-dimensional datasets
Decision tree	Interpretable, handles mixed feature types, no feature scaling needed	Overfits easily; unstable (small data changes → different tree)	When you (or a regulator) need to read the exact rule
Random forest (bagging)	Much more stable and accurate than a single tree; resists overfitting	Less interpretable; slower and heavier than one tree	Strong, low-effort default for tabular data
Gradient boosting	Usually the most accurate classical model on tabular data	More hyperparameters to tune; trains sequentially (slower to fit); can overfit if pushed too far	Squeezing out the best possible accuracy on structured data
SVM	Strong with a clear margin; kernel trick handles non-linear boundaries; works with few samples relative to features	Doesn't scale well to huge datasets; needs feature scaling	Small-to-medium datasets, especially with a clean or kernel-recoverable margin

6 What to carry forward

- **kNN** predicts by majority vote among the closest stored examples – no real training.
- **Decision trees** are readable yes/no flowcharts that overfit easily on their own.
- **Ensembles** fix that: bagging/random forests average many independent trees (wisdom of the crowd); boosting chains models that each fix the last one's mistakes.

- **SVMs** find the widest-margin boundary, and the kernel trick lets that boundary be curved by lifting data into a higher dimension first.
- None of this required deep math – the same intuitions (closeness, splitting, voting, margins) carry a long way in practice.

Next we leave the classical toolkit behind and build the **neuron** – the single unit every deep network is made of – in [Section 9](#).

Neurons, Layers, Activations

A neuron is a weighted vote passed through a switch – and without that switch, depth is a lie.

28 min read

You'll learn: the artificial neuron · layers as matrices · activation functions · why nonlinearity is essential

Everything you have built so far – vectors, matrices, dot products – was preparation for this one small object: the **neuron**. A neuron takes a handful of numbers, weighs them, adds them up, and then passes the result through a little switch. Wire a few thousand of these together and you have a neural network. This section builds the neuron, stacks neurons into layers, and then proves the single most important fact about them: the switch is not optional. Remove it and a hundred layers collapse into one.

◆ THE ONE IDEA TO HOLD ONTO

One neuron is $\text{activation}(\mathbf{w}^\top \mathbf{x} + b)$ – a weighted sum passed through a nonlinear switch. A layer runs many neurons at once, which is just a matrix multiply. And the switch – the **activation** – is the only thing that lets stacking layers buy you real power. That's the whole section; everything below is why.

1 The artificial neuron: a weighted vote through a switch

◆ INTUITION FIRST

A **neuron** makes one small decision. It looks at several input numbers, decides how much each one matters, adds up its weighted opinion, and then squashes that total through a switch that decides how strongly to “fire.” Big weighted total → strong output. Small or negative total →

weak or zero output. That's it. The cleverness of a network is not in any one neuron; it's in wiring many simple ones together.

🚂 EVERYDAY PARALLEL

Deciding whether to go outside is a weighted vote. The weather matters a lot to you (big weight), whether you have plans matters somewhat (medium weight), your energy level matters a little (small weight). You tally up the weighted evidence – and you only actually get off the couch once that tally crosses some personal threshold. Below the threshold you stay in; above it you go. A neuron is exactly that: weigh the factors, sum them, fire only past a point.

◆ NOW THE MATH

Write the inputs as a vector $\mathbf{x} \in \mathbb{R}^n$ (a list of n numbers). Give the neuron a matching list of **weights** $\mathbf{w} \in \mathbb{R}^n$ and a single extra number b , the **bias**. The neuron computes:

$$y = \text{activation}(\mathbf{w}^\top \mathbf{x} + b) = \text{activation}\left(\sum_{i=1}^n w_i x_i + b\right)$$

In words: the piece $\mathbf{w}^\top \mathbf{x}$ is just the *dot product* from [Section 1](#) — multiply each input by its weight and add the results, giving one number (“how much do I care, in total?”). Then add the bias b to shift that number up or down, and finally push the whole thing through the **activation** — the switch — to get the neuron's output y .

DEFINITION

A **weight** w_i says how much input x_i matters, and in which direction — a large positive weight means “this input pushes the output up,” a negative weight means “this input pushes it down.” The **bias** b is a constant offset added to every input; it sets the neuron's default lean, i.e. how big the weighted sum must be before the neuron starts to fire. The **activation** is a fixed nonlinear function — the switch — applied to that sum.

The raw number *before* the activation, $z = \mathbf{w}^\top \mathbf{x} + b$, shows up so often it has a name: the **pre-activation** (some texts call it the *logit* when it is the final score before a probability). Keep the two apart in your head: z is the weighted vote, $y = \text{activation}(z)$ is the vote after the switch.

◆ THE SAME THING IN PYTORCH

>_ ONE NEURON BY HAND

```
import torch

x = torch.tensor([0.6, -1.2, 2.0])    # three inputs
w = torch.tensor([1.5, 0.3, -0.8])    # a weight per input
b = torch.tensor(0.1)                # one bias

z = torch.dot(w, x) + b                # pre-activation: the weighted
vote
y = torch.relu(z)                     # push it through the switch
(ReLU)

print(z)    # tensor(-1.05)    0.6*1.5 + (-1.2)*0.3 + 2.0*(-0.8) + 0.1
print(y)    # tensor(0.)       negative vote → neuron stays silent
```

CHECK YOURSELF

If every weight in a neuron were **0**, what would its output be for *any* input, and what does that tell you about the bias?

Reveal ▼

ANSWER

With all weights zero, $\mathbf{w}^\top \mathbf{x} = 0$ no matter what $\mathbf{x}$ is, so the pre-activation is just $\mathbf{b}$ and the output is a constant $\text{activation}(\mathbf{b})$ – the neuron ignores its inputs entirely. That's the point of the bias: it's the neuron's output when the weighted evidence is silent, its baseline lean. Weights decide how the inputs move the output; the bias decides where the output sits when the inputs say nothing.

2 Layers: many neurons at once is one matrix multiply

◆ INTUITION FIRST

One neuron gives you one number. Usually you want many — one neuron asking “does this look like a wheel?”, another “does this look like a window?”, another “is there fur?”. A **layer** is simply a row of neurons, all looking at the same input at the same time, each with its own weights and bias, each producing its own number. Their outputs stack into a new vector, which becomes the input to the next layer.

🌂 EVERYDAY PARALLEL

A loan application lands on a desk with a panel of reviewers. One reviewer cares mostly about income, another mostly about credit history, another about existing debt. They all read the *same* application but weigh it differently, and each writes down a score. The panel's set of scores is richer than any single reviewer's opinion. A layer is that panel: same input to everyone, different weights per member, a vector of scores out.

◆ NOW THE MATH

Here is the beautiful part. Stacking neurons doesn't need any new machinery — it is exactly the matrix–vector multiply from [Section 1](#). Put each neuron's weight vector as a *row* of a matrix **W**, collect the biases into a vector **b**, and one line does the whole layer:

$$\mathbf{h} = \text{activation}(\mathbf{W}\mathbf{x} + \mathbf{b})$$

In words: row **i** of **W** is the weights of neuron **i**, so **Wx** computes every neuron's weighted vote in a single multiply (one dot product per row). Add the bias vector, apply the activation to every slot, and out comes **h** — the layer's output vector, one number per neuron.

The shapes are the whole story, so let's pin them down. Suppose the input has **n** features and the layer has **m** neurons:

$$\underbrace{\mathbf{W}}_{m \times n} \underbrace{\mathbf{x}}_{n \times 1} + \underbrace{\mathbf{b}}_{m \times 1} = \underbrace{\mathbf{h}}_{m \times 1}$$

In words: **W** has one row per neuron (**m** rows) and one column per input feature (**n** columns). By the shape rule — inner numbers match, outer numbers survive — an $(m \times n)$ matrix times an $(n \times 1)$ vector gives an $(m \times 1)$ vector. So a layer maps **n** numbers in to **m** numbers out; you pick **m** when you design the network.

In practice you rarely push one example through at a time. You send a whole **batch** (a bundle of examples processed together for speed). Stack **N** input vectors as the rows of a matrix **X** of

shape $(N \times n)$, and the layer becomes $\mathbf{H} = \text{activation}(\mathbf{XW}^\top + \mathbf{b})$, producing an $(N \times m)$ output – one row of answers per example. (This row-of-examples convention is why PyTorch's `Linear` layer stores its weight as $(m \times n)$ and multiplies by $\mathbf{W}^\top$ under the hood.)

CHECK YOURSELF

A layer takes a 784-length image vector and has 128 neurons. What are the shapes of $\mathbf{W}$, $\mathbf{b}$, and the output $\mathbf{h}$ for a single image?

Reveal ▼

ANSWER

The layer maps $n = 784$ inputs to $m = 128$ neurons. So $\mathbf{W}$ is $(128, 784)$ – one row per neuron, one column per input feature – the bias $\mathbf{b}$ is $(128,)$ – one per neuron – and the output $\mathbf{h}$ is $(128,)$. The 784 “cancels” in the multiply and the 128 survives. You've just sized the first hidden layer of an image classifier.

3 Activation functions: the shape of the switch

◆ INTUITION FIRST

The activation is the switch each neuron passes its weighted vote through. Different switches have different personalities – some are soft and gentle, some are hard on/off, some let strong signals through untouched. The choice matters because it decides two things: what range the output can take, and – crucially for learning – how the **gradient** (the slope that tells training which way to nudge the weights, from `backprop`) behaves. A switch whose slope goes flat is a switch through which learning cannot flow.

🔼 EVERYDAY PARALLEL

Think of a dimmer versus a light switch. A **dimmer** (like sigmoid or tanh) eases smoothly from off to fully on, but near the two ends it barely responds – twist it a lot and almost nothing changes. A plain **light switch** is hard on/off. And a **ReLU** is like a diode: it blocks anything negative completely, but passes positive signal straight through at full strength, no dimming. Each feels different to “tune,” and that tuning feel *is* the gradient.

◆ NOW THE MATH

The **sigmoid** squashes any number into the range (0, 1) – handy when you want an output you can read as a probability:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad \text{range } (0, 1)$$

In words: huge positive z gives an output near 1, huge negative z gives near 0, and $z = 0$ gives exactly 0.5. The catch: at both far ends the curve flattens out, so its slope (gradient) shrinks to nearly zero — this is called **saturation**, and it starves learning.

The **tanh** is a rescaled sigmoid centred on zero, with range (−1, 1):

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \quad \text{range } (-1, 1)$$

In words: same S-shape as sigmoid but symmetric around zero, so its outputs are centred (a negative input gives a negative output). That centering makes it a bit nicer to train than sigmoid — but it still saturates and flattens at the two ends.

The **ReLU** (“rectified linear unit”) is almost embarrassingly simple: keep the input if it's positive, otherwise output zero.

$$\text{ReLU}(z) = \max(0, z) \quad \text{range } [0, \infty)$$

In words: for any positive z the output is just z and the slope is exactly 1 — the gradient passes through undamped, so learning never stalls on the positive side. For negative z the output is 0 and the slope is 0 (the neuron is “off”). No expensive e^x , no saturation for positive inputs — that combination is why ReLU became the default hidden-layer activation.

ReLU has one flaw: a neuron stuck in the negative zone has zero gradient and may never wake up – a **dead neuron**. **Leaky ReLU** fixes this by letting a small trickle through on the negative side instead of hard zero:

$$\text{LeakyReLU}(z) = \max(\alpha z, z), \quad \alpha \approx 0.01 \quad \text{range } (-\infty, \infty)$$

In words: positive inputs behave exactly like ReLU (slope 1); negative inputs get a gentle slope α (say 0.01) instead of 0, so the neuron always keeps a faint gradient and can recover instead of dying. α (alpha) is just the small leak factor.

Finally, worth knowing by name: **GELU** (“Gaussian Error Linear Unit”) is a smooth, curved cousin of ReLU that bends gently through zero instead of kinking sharply. It is the default activation inside modern transformer models (the architecture behind today’s large language models). You don’t need its formula yet – just recognise the name when it appears later.

Activation	Formula	Output range	Gradient behaviour
Sigmoid	$\frac{1}{1 + e^{-z}}$	(0, 1)	Saturates both ends → tiny gradients, slow learning
Tanh	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	(-1, 1)	Zero-centred but still saturates both ends
ReLU	$\max(0, z)$	[0, ∞)	Slope 1 for $z > 0$, 0 for $z < 0$; cheap, no positive saturation
Leaky ReLU	$\max(\alpha z, z)$	$(-\infty, \infty)$	Like ReLU but small slope α on the negative side → no dead neurons
GELU	smooth ReLU-like curve	$\approx (-0.17, \infty)$	Smooth everywhere; default in transformers

◆ THE SAME THING IN PYTORCH

›_ EVERY SWITCH ON THE SAME INPUTS


```
import torch

z = torch.tensor([-2.0, -0.5, 0.0, 0.5, 2.0])

print(torch.sigmoid(z))           # (0,1): [0.12, 0.38, 0.50,
0.62, 0.88]
print(torch.tanh(z))              # (-1,1): [-0.96, -0.46, 0.00,
0.46, 0.96]
print(torch.relu(z))              # [0,inf):[0.00, 0.00, 0.00,
0.50, 2.00]
print(torch.nn.functional.leaky_relu(z)) # negatives leak: [-0.02,
-0.005, 0, 0.5, 2]
```

▲ WATCH OUT

Don't reach for sigmoid or tanh in deep hidden layers by default. Because they flatten at the ends, stacking many of them makes gradients shrink layer after layer until the early layers barely learn – the **vanishing gradient** problem. Sigmoid still has a proper home: the *final* layer of a binary classifier, where you genuinely want a 0-to-1 probability. For the hidden layers in between, start with ReLU.

CHECK YOURSELF

A neuron uses ReLU and its pre-activation is $z = -3.0$. Reveal ▼

What is its output, and what is the gradient flowing back through it during training?

ANSWER

Since $z = -3.0 < 0$, $\text{ReLU}(z) = \max(0, -3) = 0$ – the neuron is silent. And ReLU's slope for negative inputs is **0**, so no gradient flows back through this neuron on this example: it learns nothing from it. If the neuron sits in the negative zone for every example, it's a *dead neuron* that never recovers – the exact problem Leaky ReLU's small negative slope is designed to prevent.

4 Why nonlinearity is not optional

◆ INTUITION FIRST

Here is the claim, and it is worth proving because it justifies the entire existence of activation functions: **if you stack layers with no activation between them, the whole stack is exactly equal to a single layer.** A hundred plain matrix multiplies in a row have no more power than one. The switch is the only thing that makes depth mean anything.

🚩 EVERYDAY PARALLEL

Lay straight rulers end to end, as many as you like. Every ruler is straight, so the whole chain is still one straight line – you never bend it into a curve, no matter how many rulers you add. A linear layer is a straight ruler. To draw a curve you need a bend somewhere, and the activation is the only place a bend can enter. No bend, no curve; no nonlinearity, no real depth.

◆ NOW THE MATH

Take two layers with **no** activation. The first maps $\mathbf{x}$ to $\mathbf{h}$, the second maps $\mathbf{h}$ to $\mathbf{y}$:

$$\mathbf{h} = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1, \quad \mathbf{y} = \mathbf{W}_2\mathbf{h} + \mathbf{b}_2$$

Now substitute the first equation into the second and simplify, one honest step at a time:

$$\begin{aligned} \mathbf{y} &= \mathbf{W}_2\mathbf{h} + \mathbf{b}_2 \\ &= \mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 && \text{(plug in } \mathbf{h} \text{)} \\ &= \mathbf{W}_2\mathbf{W}_1\mathbf{x} + \mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2 && \text{(multiply through)} \\ &= \underbrace{(\mathbf{W}_2\mathbf{W}_1)}_{\mathbf{W}'}\mathbf{x} + \underbrace{(\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2)}_{\mathbf{b}'} && \text{(group the constants)} \\ &= \mathbf{W}'\mathbf{x} + \mathbf{b}' \end{aligned}$$

In words: the two weight matrices multiply together into one new matrix $\mathbf{W}' = \mathbf{W}_2\mathbf{W}_1$, and the two biases fold into one new bias $\mathbf{b}' = \mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2$. The result, $\mathbf{y} = \mathbf{W}'\mathbf{x} + \mathbf{b}'$, is a *single* linear layer. The two-layer network and this one-layer network compute the identical function.

◆ THE PUNCHLINE

Nothing in that derivation used the number two. Chain 100 activation-free layers and the same collapse happens 99 times over: the product $\mathbf{W}_{100} \cdots \mathbf{W}_2 \mathbf{W}_1$ is just *some* matrix $\mathbf{W}'$, and all the biases fold into one $\mathbf{b}'$. A 100-layer linear network is **exactly** as expressive as one layer – it can only ever draw straight lines (technically, flat planes). All those extra parameters buy you nothing. The activation is what breaks the collapse: because $\text{activation}(\mathbf{W}_2 \text{activation}(\mathbf{W}_1 \mathbf{x}))$ can't be flattened into one matrix, each real layer adds genuine new power.

So what *can* a network do once you put the nonlinearity back? Far more than you might guess. The **universal approximation theorem** states that a network with just one hidden layer – as long as it has a nonlinear activation and enough neurons – can approximate *any* continuous function to any accuracy you want. “Approximate” means get as close as you like everywhere; “continuous function” means any smooth input-to-output rule without sudden jumps. In plain terms: the bend the activation provides, repeated across enough neurons, can be assembled into any curve at all. That theorem is why neural networks are worth building – and it is silent, worthless, the moment you delete the activation.

◆ THE SAME THING IN PYTORCH

›_ WATCH TWO LINEAR LAYERS COLLAPSE INTO ONE

```
import torch

W1 = torch.randn(4, 3); b1 = torch.randn(4)
W2 = torch.randn(2, 4); b2 = torch.randn(2)
x  = torch.randn(3)

# two "layers" with NO activation between them
y_two = W2 @ (W1 @ x + b1) + b2

# the single equivalent layer, built from the proof
W_eq = W2 @ W1                # (2x4)(4x3) → (2x3)
b_eq = W2 @ b1 + b2           # folded bias
y_one = W_eq @ x + b_eq

print(torch.allclose(y_two, y_one))  # True → same function, one layer
```

CHECK YOURSELF

A colleague builds a 10-layer network but forgets to add any activation functions. During training the loss drops a little then plateaus, and the model does no better than plain linear regression. Why?

Reveal ▼

ANSWER

Without activations, all ten layers collapse into a single linear map $\mathbf{W}'\mathbf{x} + \mathbf{b}'$ by the proof above. The model literally *is* a linear model, so it can never beat one – the nine extra layers add parameters but zero expressive power. It plateaus wherever a straight-line fit plateaus. The fix is one line per layer: wrap each hidden layer's output in a nonlinearity like ReLU, and the collapse can no longer happen.

5 A real layer in PyTorch

◆ INTUITION FIRST

You now have every piece: weights, bias, matrix multiply, activation. PyTorch bundles the weighted sum and bias into `nn.Linear` and keeps the activation separate, so you compose them yourself. Let's run a batch through one hidden layer with ReLU and watch the shapes flow – the shapes are the proof that everything above is wired correctly.

🔧 EVERYDAY PARALLEL

It's a two-stage assembly line. Station one (`Linear`) takes each incoming part, weighs and combines it, and hands on a rough result. Station two (`ReLU`) is quality control: anything below the bar is set to zero, the rest passes through unchanged. Same two stations run on every item on the belt (every example in the batch), in lockstep.

◆ THE SAME THING IN PYTORCH

›_ A HIDDEN LAYER'S FORWARD PASS, WITH SHAPES

```

import torch
import torch.nn as nn

batch_size, in_features, out_features = 32, 784, 128

layer = nn.Linear(in_features, out_features)    # holds W (128x784) and b (128,)
relu = nn.ReLU()

x = torch.randn(batch_size, in_features)      # a batch: 32 examples of
784 numbers
print("input :", x.shape)                     # torch.Size([32, 784])

z = layer(x)                                  # weighted sum + bias: x
@ W.T + b
print("linear:", z.shape)                     # torch.Size([32, 128])

h = relu(z)                                   # apply the switch
element-wise
print("relu :", h.shape)                     # torch.Size([32, 128])

# proof the activation actually clipped the negatives:
print("min after relu:", h.min().item())      # 0.0 → nothing negative
survives

```

In words: the batch goes in as (32, 784), `Linear` turns each 784-number example into 128 numbers giving (32, 128), and `ReLU` keeps that shape while zeroing every negative entry. Each row is one example's journey through the layer; the batch dimension (32) just rides along untouched.

▲ WATCH OUT

The `in_features` you pass to `nn.Linear` must equal the last dimension of your input, and the next layer's `in_features` must equal this layer's `out_features`. Mismatch and PyTorch throws a shape error the instant you call the layer. When a network won't run, print the shape after every layer – the break is always where two numbers that should match, don't.

6 What to carry forward

- A **neuron** is $\text{activation}(\mathbf{w}^\top \mathbf{x} + b)$ – a weighted vote (weights say how much each input matters) plus a bias (baseline lean), pushed through a nonlinear switch.
- A **layer** runs many neurons at once and is just one matrix multiply:
 $\mathbf{h} = \text{activation}(\mathbf{W}\mathbf{x} + \mathbf{b})$, mapping $\mathbf{n}$ inputs to $\mathbf{m}$ outputs.
- **Activations** differ by output range and gradient behaviour. **ReLU** ($\max(0, z)$) is the default hidden-layer switch: cheap and non-saturating for positive inputs. Sigmoid/tanh saturate; Leaky ReLU avoids dead neurons; GELU is the transformer default.
- **The proof:** stacked linear layers with no activation collapse to one layer – $\mathbf{W}'\mathbf{x} + \mathbf{b}'$. Nonlinearity is what makes depth real, and with it an MLP can approximate any continuous function.

We have a network that can compute a forward pass. But its weights are random, so its answers are nonsense. Next we learn how to *fix* those weights – how the gradient flows back through every neuron and every activation to teach the whole stack at once. That's [backpropagation](#), and it's where the network finally starts to learn.

Backpropagation by Hand

Backprop is just the chain rule with good bookkeeping. We'll compute every gradient in a real network by hand, with actual numbers – nothing left as “it can be shown.”

34 min read

You'll learn: the computational graph · the chain rule flowing backward · a full worked numerical example · the local-gradient rule

A neural network learns by nudging its weights in the direction that lowers the loss. To nudge a weight the right way, we need one number for it: the **gradient** – how much the loss would change if we wiggled that weight a hair. A real network has millions of weights, so we need millions of these numbers, and we need them fast, over and over. **Backpropagation** is the algorithm that computes all of them in a single sweep backward through the network. This is the engine room of deep learning: once it clicks, most of what follows is variations on this one idea.

◆ THE ONE IDEA TO HOLD ONTO

Backprop is the [chain rule from Section 2](#) applied to a graph, with the intermediate results **stored and reused** instead of recomputed. That reuse is the whole trick: it turns “millions of expensive experiments” into “one cheap backward pass.” Everything below is just careful bookkeeping of that idea.

1 The goal, and why the obvious method fails

◆ INTUITION FIRST

Suppose you want to know how each weight affects the loss. The most obvious idea: *try it*. Take one weight, add a tiny amount to it, run the whole network again, and see how much the loss moved. Divide the change in loss by the change in weight – that ratio is (approximately)

the gradient for that one weight. Then put the weight back and do the next one. It works, and it needs no calculus at all. The problem is speed.

🔧 EVERYDAY PARALLEL

Imagine a mixing desk with a million volume sliders feeding one final loudspeaker, and you want to know how much each slider affects the total loudness. The brute-force way: nudge slider one, replay the whole song, measure the loudness, write it down, reset. Then slider two. Replay the *entire* song again. A million replays for one reading of the desk. You'd never finish. Backprop is the sound engineer who, in a single playthrough, works out every slider's influence at once – by listening backward from the speaker to each source.

◆ NOW THE MATH

Write the loss as L (a single number telling us how wrong the network is) and a weight as w . The brute-force (“finite difference”) estimate of the gradient is:

$$\frac{\partial L}{\partial w} \approx \frac{L(w + \varepsilon) - L(w)}{\varepsilon}$$

In words: $\partial L / \partial w$ means “how fast the loss L changes as we change the weight w .” We estimate it by adding a tiny bump ε (epsilon, a small number like 10^{-5}) to the weight, seeing how much L moves, and dividing. The catch: computing $L(w + \varepsilon)$ means running the *entire* forward pass of the network.

Now count the cost. If the network has P weights, and one forward pass costs C units of work, then estimating every gradient this way costs about:

$$P \times C \quad \text{units of work.}$$

In words: one full forward pass *per weight*. For a small model with a million weights ($P = 10^6$), that is a million forward passes just to take a single learning step. Modern models have billions of weights and take hundreds of thousands of steps. The brute-force method is not slow — it is impossible.

◆ WHAT BACKPROP BUYS YOU

Backprop computes **all** P gradients in the cost of roughly **one** extra pass – a backward one – no matter how many weights there are. It does this by reusing partial results: the gradient at a later layer is built from work already done, so nothing is recomputed from scratch. This is a classic case of *dynamic programming* (solving a big problem by storing and reusing the answers to smaller overlapping pieces).

CHECK YOURSELF

A network has 5 million weights and one forward pass takes 2 milliseconds. Roughly how long does one gradient step cost with brute force, versus with backprop?

Reveal ▼

ANSWER

Brute force: one forward pass per weight, so $5,000,000 \times 2 \text{ ms} = 10,000,000 \text{ ms} \approx 2.8 \text{ hours}$ for a single learning step – and you need thousands of steps. **Backprop:** one forward pass plus one backward pass, roughly $2 \times 2 \text{ ms} = 4 \text{ ms}$. That is the difference between “trains overnight” and “never finishes.” Same gradients, wildly different cost, because backprop reuses intermediate results instead of throwing them away.

2 The computational graph

◆ INTUITION FIRST

To organise the bookkeeping we draw the network as a **computational graph**: a diagram of little boxes (called *nodes*), one per simple operation – multiply, add, apply the sigmoid, and so on. Arrows (the *edges*) carry numbers from one box into the next. Feeding the input in and letting numbers flow left-to-right is the **forward pass** – it produces the prediction and the loss. Then we go right-to-left, and that return trip is the **backward pass** – it produces the gradients.

🔼 EVERYDAY PARALLEL

A dish comes out too salty, and the head chef wants to assign blame up the kitchen chain. The final saltiness is the loss. Working backward: the plating cook barely touched the salt, so little blame flows to them; the cook who poured the stock added most of it, so most of the blame flows there; and whoever *measured* the stock's saltiness in the first place inherits blame in proportion to how much the stock mattered. Each station receives blame equal to (blame passed down from above) $\times$ (how much this station actually influenced the next step). That single rule, applied station by station backward through the kitchen, is backprop.

◆ NOW THE MATH

Here is the exact graph we will compute with. One input x flows through two layers. Each layer does a *linear step* (multiply by a weight, add a bias) and then a *nonlinear step* (the sigmoid, written σ). The final node compares the output to the target y and produces the loss L :

$$x \longrightarrow \underbrace{z_1 = w_1x + b_1}_{\text{linear}} \longrightarrow \underbrace{a_1 = \sigma(z_1)}_{\text{nonlinear}} \longrightarrow \underbrace{z_2 = w_2a_1 + b_2}_{\text{linear}} \longrightarrow \underbrace{a_2 = \sigma(z_2)}_{\text{nonlinear}} \longrightarrow \underbrace{L = \frac{1}{2}(a_2 - y)^2}_{\text{loss}}$$

In words: x is the input number. w_1, b_1 are the first layer's weight and bias; z_1 is its raw output (the *pre-activation*) and a_1 is that value squashed by the sigmoid into the range $(0, 1)$ (the *activation*). w_2, b_2, z_2, a_2 are the same for the second layer, and a_2 is the network's final prediction. y is the target we wish a_2 equalled, and L is the squared-error loss — half the squared gap. The $\frac{1}{2}$ is a convenience that cancels when we differentiate.

DEFINITION

The **forward pass** runs the graph left-to-right, computing and **storing** every intermediate value (z_1, a_1, z_2, a_2, L). The **backward pass** runs it right-to-left, computing the gradient of L with respect to every stored value and every weight. The forward pass must happen first because the backward pass *reuses* those stored values — that is exactly where the speed comes from.

CHECK YOURSELF

Why must we keep the forward-pass values in memory instead of discarding them once we have the loss?

Reveal ▼

ANSWER

Because the backward pass needs them. As we'll see, the gradient for w_2 turns out to be $\delta_2 \cdot a_1$ – it literally multiplies by the stored activation a_1 . If we threw a_1 away, we'd have to recompute it, defeating the purpose. This is why training uses far more memory than just running a model for predictions (*inference*): training must hold the whole forward pass in memory, ready for the return trip.

3 The core rule at every node

◆ INTUITION FIRST

Every node in the graph obeys the same tiny rule, and once you see it you have seen all of backprop. A node receives, from the node just downstream of it, a number that says “here is how much the final loss cares about *my* output.” The node then knows one purely local fact: “how much does my output change when my input changes?” Multiply those two together and you get “how much the final loss cares about my *input*” – which is exactly the message to pass one step further back.

🔑 EVERYDAY PARALLEL

Money changing hands down a supply chain. The shop tells the wholesaler “each extra unit you send raises my profit by 2 dollars” (the gradient flowing in). The wholesaler knows a purely local fact: “for every extra kilo of raw material I buy, I can make 3 units” (the local gradient). So the wholesaler tells the farmer “each extra kilo you grow is worth $2 \times 3 = 6$ dollars to the final profit.” Nobody needs the full picture – each link only multiplies the message from downstream by its own local conversion rate and passes it on.

◆ NOW THE MATH

This is the chain rule, stated as a rule you can apply mechanically. Consider any node computing an output u from an input t . Let the loss be L . Then:

$$\underbrace{\frac{\partial L}{\partial t}}_{\text{gradient sent to input}} = \underbrace{\frac{\partial L}{\partial u}}_{\text{gradient from above}} \times \underbrace{\frac{\partial u}{\partial t}}_{\text{local gradient}}$$

In words: (gradient the loss has for my output) $\times$ (how my output responds to my input) = (gradient the loss has for my input). The middle term, $\partial u / \partial t$, is the **local gradient** — it depends only on what *this* node does, and can be worked out without knowing anything about the rest of the network. That locality is why backprop is modular: each operation just needs to know its own derivative.

◆ THE LOCAL-GRADIENT RULE

gradient in (from above) $\times$ local gradient = gradient out (to inputs)

Every node applies exactly this. Backprop is nothing more than starting at the loss (whose gradient with respect to itself is 1) and applying this rule, node by node, all the way back to the weights. If a value feeds *several* nodes, its gradient is the **sum** of the messages coming back from each of them — blame from multiple children adds up.

We will lean on the local gradients of the three operations in our graph. Two are immediate; the third is the sigmoid, whose derivative has a famously tidy form:

$$\begin{aligned} \text{multiply } u = wt : \quad & \frac{\partial u}{\partial t} = w, & \frac{\partial u}{\partial w} = t \\ \text{add } u = t + b : \quad & \frac{\partial u}{\partial t} = 1, & \frac{\partial u}{\partial b} = 1 \\ \text{sigmoid } a = \sigma(z) : \quad & \frac{\partial a}{\partial z} = \sigma(z)(1 - \sigma(z)) = a(1 - a) \end{aligned}$$

In words: a multiply node passes gradient through scaled by the *other* input (so $z_2 = w_2 a_1$ sends gradient to a_1 scaled by w_2 , and to w_2 scaled by a_1). An add node passes gradient through unchanged (local gradient 1). And the sigmoid's local gradient is just $a(1 - a)$ — computed straight from the activation a we already stored on the forward pass. No re-evaluating exponentials on the way back.

CHECK YOURSELF

At a node $u = w \cdot t$, the gradient arriving from above is Reveal ▼

$\partial L / \partial u = -3$, and the stored inputs are $w = 2$, $t = 5$.

What gradients does the node send to t and to w ?

ANSWER

Apply the rule twice. To the input t : local gradient is $w = 2$, so $\partial L / \partial t = -3 \times 2 = -6$. To the weight w : local gradient is $t = 5$, so $\partial L / \partial w = -3 \times 5 = -15$. Notice the multiply node “swaps” its inputs on the way back – each branch is scaled by the *other* value. That swap is worth memorising; it is the single most common node in any network.

4 The centrepiece: a full network, by hand

Now we do the real thing. We take the two-layer graph above, plug in concrete numbers, and compute **every** value forward and **every** gradient backward. Nothing is skipped; you can reproduce each number with a calculator. Here are the fixed ingredients:

Symbol	Meaning	Value
x	the single input	1.0
w_1, b_1	layer-1 weight and bias	0.5, 0.5
w_2, b_2	layer-2 weight and bias	1.5, -1.0
y	the target we want a_2 to hit	1.0

We round every result to four decimal places. The sigmoid is $\sigma(z) = 1/(1 + e^{-z})$.

Forward pass — compute and store every value

◆ NOW THE MATH

Node 1 – first linear step. Multiply the input by w_1 and add b_1 :

$$z_1 = w_1x + b_1 = (0.5)(1.0) + 0.5 = \mathbf{1.0000}$$

Node 2 – first sigmoid. Squash z_1 into $(0, 1)$:

$$a_1 = \sigma(1.0) = \frac{1}{1 + e^{-1.0}} = \frac{1}{1 + 0.3679} = \mathbf{0.7311}$$

Node 3 – second linear step. Multiply the activation by w_2 and add b_2 :

$$z_2 = w_2a_1 + b_2 = (1.5)(0.7311) + (-1.0) = 1.0966 - 1.0 = \mathbf{0.0966}$$

Node 4 – second sigmoid. This is the network's prediction:

$$a_2 = \sigma(0.0966) = \frac{1}{1 + e^{-0.0966}} = \frac{1}{1 + 0.9079} = \mathbf{0.5241}$$

Node 5 – the loss. Compare the prediction a_2 to the target $y = 1.0$:

$$L = \frac{1}{2}(a_2 - y)^2 = \frac{1}{2}(0.5241 - 1.0)^2 = \frac{1}{2}(-0.4759)^2 = \frac{1}{2}(0.2265) = \mathbf{0.1132}$$

In words: the network currently predicts 0.5241 but we wanted 1.0, so it is off by 0.4759, and the loss is 0.1132. Every one of these five numbers is now stored; the backward pass will reuse them.

EVERYDAY PARALLEL

The forward pass is like following a recipe end to end and tasting the final dish: too bland, a “wrongness” score of **0.1132**. We wrote down the state after every step – how salty the stock was, how it tasted after simmering – precisely so that, when the dish disappoints, we can walk back through our notes and see which step to blame. That walk back is next.

Backward pass — send the gradient back, node by node

◆ NOW THE MATH

We start at the loss and apply the local-gradient rule repeatedly. It helps to name the gradient arriving at each pre-activation; the standard name is δ (delta). So $\delta_2 = \partial L / \partial z_2$ and $\delta_1 =$

$\partial L / \partial z_1$. Watch how each step reuses a stored forward value.

Step A – through the loss node. Differentiate $L = \frac{1}{2}(a_2 - y)^2$ with respect to a_2 . The $\frac{1}{2}$ cancels the power's 2:

$$\frac{\partial L}{\partial a_2} = (a_2 - y) = 0.5241 - 1.0 = -\mathbf{0.4759}$$

In words: the loss wants a_2 to *increase* (the gradient is negative, and moving *against* a negative gradient means increasing a_2) — which makes sense, since 0.5241 is below the target 1.0.

Step B – through the second sigmoid. The local gradient is $a_2(1 - a_2)$, built from the stored a_2 . Multiply it by the gradient from above to get δ_2 :

$$\begin{aligned}\frac{\partial a_2}{\partial z_2} &= a_2(1 - a_2) = (0.5241)(0.4759) = 0.2494 \\ \delta_2 &= \frac{\partial L}{\partial z_2} = \frac{\partial L}{\partial a_2} \cdot \frac{\partial a_2}{\partial z_2} = (-0.4759)(0.2494) = -\mathbf{0.1187}\end{aligned}$$

In words: we took the message -0.4759 and scaled it by the sigmoid's local gradient 0.2494. That 0.2494 came for free from the stored a_2 — no exponentials recomputed. δ_2 is now the gradient of the loss with respect to the layer-2 pre-activation.

Step C – the layer-2 weight and bias. Recall $z_2 = w_2 a_1 + b_2$. The multiply node sends gradient to w_2 scaled by the *other* input a_1 (stored!); the add node sends gradient to b_2 unchanged:

$$\begin{aligned}\frac{\partial L}{\partial w_2} &= \delta_2 \cdot \frac{\partial z_2}{\partial w_2} = \delta_2 \cdot a_1 = (-0.1187)(0.7311) = -\mathbf{0.0868} \\ \frac{\partial L}{\partial b_2} &= \delta_2 \cdot \frac{\partial z_2}{\partial b_2} = \delta_2 \cdot 1 = -\mathbf{0.1187}\end{aligned}$$

In words: two of our four weight gradients are already done. The weight gradient reused the stored activation $a_1 = 0.7311$; the bias gradient is just δ_2 itself, because adding a bias passes gradient through untouched.

Step D – back into layer 1. The multiply node also sends gradient to its *other* input a_1 , scaled by w_2 :

$$\frac{\partial L}{\partial a_1} = \delta_2 \cdot \frac{\partial z_2}{\partial a_1} = \delta_2 \cdot w_2 = (-0.1187)(1.5) = -\mathbf{0.1780}$$

In words: we have crossed from layer 2 into layer 1. The blame for the loss has been handed to a_1 , the first layer's activation. Now we push it through the first sigmoid, exactly as we did the second.

Step E – through the first sigmoid. Same rule as Step B, now with the stored a_1 :

$$\frac{\partial a_1}{\partial z_1} = a_1(1 - a_1) = (0.7311)(0.2689) = 0.1966$$

$$\delta_1 = \frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} = (-0.1780)(0.1966) = -\mathbf{0.0350}$$

In words: δ_1 is the gradient of the loss with respect to the layer-1 pre-activation. Notice it is much smaller than δ_2 — the signal shrank as it passed back through the sigmoid (whose local gradient is at most 0.25). Stack many sigmoid layers and the signal keeps shrinking; that is the *vanishing gradient* problem in one glance.

Step F – the layer-1 weight and bias. Recall $z_1 = w_1x + b_1$. Same pattern as Step C, now with the stored input x :

$$\frac{\partial L}{\partial w_1} = \delta_1 \cdot \frac{\partial z_1}{\partial w_1} = \delta_1 \cdot x = (-0.0350)(1.0) = -\mathbf{0.0350}$$

$$\frac{\partial L}{\partial b_1} = \delta_1 \cdot \frac{\partial z_1}{\partial b_1} = \delta_1 \cdot 1 = -\mathbf{0.0350}$$

In words: all four weight gradients are done. We swept from the loss back to the very first weight, and at each node we did nothing but “gradient from above $\times$ local gradient,” reusing stored values. That entire sweep is one backward pass.

◆ ALL FOUR GRADIENTS, IN ONE PLACE

These are the numbers we came for — the slope of the loss along each weight:

$\frac{\partial L}{\partial w_1}$	$\frac{\partial L}{\partial b_1}$	$\frac{\partial L}{\partial w_2}$	$\frac{\partial L}{\partial b_2}$
−0.0350	−0.0350	−0.0868	−0.1187

Every one is negative, meaning: increasing any of these weights would *lower* the loss. So the update will increase them all.

One gradient-descent step — and the loss really drops

◆ NOW THE MATH

Gradient descent moves each weight a small step *against* its gradient (downhill on the loss). The step size is the *learning rate* α (alpha). Take $\alpha = 1.0$. The update rule for any weight is $w \leftarrow w - \alpha \partial L / \partial w$:

$$w_1 \leftarrow 0.5 - (1.0)(-0.0350) = \mathbf{0.5350}$$

$$b_1 \leftarrow 0.5 - (1.0)(-0.0350) = \mathbf{0.5350}$$

$$w_2 \leftarrow 1.5 - (1.0)(-0.0868) = \mathbf{1.5868}$$

$$b_2 \leftarrow -1.0 - (1.0)(-0.1187) = \mathbf{-0.8813}$$

In words: subtracting a negative gradient means *adding* — every weight grew slightly, exactly as the signs predicted. Now the honest test: does the loss actually fall? Run the forward pass again with the new weights.

New forward pass:

$$\begin{aligned} z_1 &= (0.5350)(1.0) + 0.5350 = 1.0700, & a_1 &= \sigma(1.0700) = 0.7446 \\ z_2 &= (1.5868)(0.7446) + (-0.8813) = 0.3002, & a_2 &= \sigma(0.3002) = 0.5745 \\ L_{\text{new}} &= \frac{1}{2}(0.5745 - 1.0)^2 = \frac{1}{2}(0.1811) = \mathbf{0.0905} \end{aligned}$$

In words: the loss fell from 0.1132 to **0.0905**, and the prediction rose from 0.5241 toward the target — from 0.5241 to 0.5745. One backward pass, one step, and the network is measurably less wrong. Repeat this loop thousands of times and you have training.

🚶 EVERYDAY PARALLEL

You are hiking down a foggy hill and can only feel the slope under your feet. The gradient is that felt slope; the learning rate is your stride length. You take one step straight downhill, re-feel the slope, and step again. You never see the whole valley, yet step by step you reach the bottom. Backprop is what tells you the slope; gradient descent is the step.

◆ THE SAME THING IN PYTORCH (MANUAL, NO AUTOGRAD)

›_ REPRODUCES EVERY NUMBER ABOVE BY HAND

```

import torch

def sigmoid(z):
    return 1.0 / (1.0 + torch.exp(-z))

# fixed ingredients
x = torch.tensor(1.0)
w1, b1 = torch.tensor(0.5), torch.tensor(0.5)
w2, b2 = torch.tensor(1.5), torch.tensor(-1.0)
y = torch.tensor(1.0)

# ---- forward pass: compute and keep every value ----
z1 = w1 * x + b1          # 1.0000
a1 = sigmoid(z1)          # 0.7311
z2 = w2 * a1 + b2          # 0.0966
a2 = sigmoid(z2)          # 0.5241
L = 0.5 * (a2 - y)**2      # 0.1132

# ---- backward pass: gradient in x local gradient, node by node ----
dL_da2 = (a2 - y)          # -0.4759
d2 = dL_da2 * a2 * (1 - a2) # delta2 = -0.1187
dL_dw2 = d2 * a1            # -0.0868
dL_db2 = d2                 # -0.1187
dL_da1 = d2 * w2            # -0.1780
d1 = dL_da1 * a1 * (1 - a1) # delta1 = -0.0350
dL_dw1 = d1 * x             # -0.0350
dL_db1 = d1                 # -0.0350

print(dL_dw1, dL_db1, dL_dw2, dL_db2)

# ---- one gradient-descent step (learning rate 1.0) ----
alpha = 1.0
w1, b1 = w1 - alpha*dL_dw1, b1 - alpha*dL_db1
w2, b2 = w2 - alpha*dL_dw2, b2 - alpha*dL_db2

a2_new = sigmoid(w2 * sigmoid(w1*x + b1) + b2)
print(0.5 * (a2_new - y)**2)    # 0.0905 → loss dropped

```

CHECK YOURSELF

Why is $\partial L / \partial b_1$ equal to $\partial L / \partial w_1$ here (both -0.0350), and would they still be equal if the input x were 2.0 instead of 1.0 ? Reveal ▼

ANSWER

Both are δ_1 times a local gradient. For the bias the local gradient is 1 (adding passes gradient through), giving $\partial L / \partial b_1 = \delta_1$. For the weight the local gradient is the input x , giving $\partial L / \partial w_1 = \delta_1 \cdot x$. They are equal here only because $x = 1.0$, so multiplying by it changes nothing. With $x = 2.0$ the weight gradient would double to $\delta_1 \cdot 2 = -0.0700$ while the bias gradient would stay -0.0350 . A weight's gradient always scales with the input feeding it; a bias's never does.

5 The same rules for a whole layer at once

◆ INTUITION FIRST

Real layers have many neurons, so weights form a matrix $\mathbf{W}$ and activations form vectors. The good news: the rules do not change at all. “Multiply node swaps its inputs” becomes “multiply by the transpose of the matrix.” The scalar δ becomes a vector $\boldsymbol{\delta}$, one entry per neuron. Everything you just did by hand now runs on whole layers in one matrix operation – which is exactly what a GPU is built to do fast.

🚩 EVERYDAY PARALLEL

Doing the whole payroll at once instead of one employee at a time. The per-person rule (hours $\times$ rate) never changed – you just stacked everyone into a table and ran the arithmetic in one sweep. Vectorising backprop is the same move: identical rule, applied to the entire layer in a single matrix multiply.

◆ NOW THE MATH

For a layer $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$ followed by an activation $\mathbf{a} = f(\mathbf{z})$, let $\boldsymbol{\delta} = \partial L / \partial \mathbf{z}$ be the gradient arriving at the layer's pre-activations. Then the three gradients we need are:

$$\begin{aligned}\frac{\partial L}{\partial \mathbf{W}} &= \boldsymbol{\delta} \mathbf{x}^\top && \text{(gradient for the weight matrix)} \\ \frac{\partial L}{\partial \mathbf{b}} &= \boldsymbol{\delta} && \text{(gradient for the bias vector)} \\ \frac{\partial L}{\partial \mathbf{x}} &= \mathbf{W}^\top \boldsymbol{\delta} && \text{(gradient to send to the previous layer)}\end{aligned}$$

In words: $\mathbf{x}^\top$ is the input row-vector (the little $\top$ means *transpose* — flip a column into a row). These are the exact scalar rules from Step C and Step D, wearing matrix clothes: the weight gradient is “ δ times the stored input,” the bias gradient is “just δ ,” and the gradient handed to the previous layer is “ δ scaled by the weights,” now with a transpose so the shapes line up.

To keep the chain going one more layer back, push that returned gradient through the previous layer's activation — multiply by its local gradient f' , element by element:

$$\boldsymbol{\delta}_{\text{prev}} = (\mathbf{W}^\top \boldsymbol{\delta}) \odot f'(\mathbf{z}_{\text{prev}})$$

In words: $\odot$ means multiply matching entries (the *element-wise* product). So each neuron's incoming gradient gets scaled by its own local slope f' . This one line, repeated layer to layer, **is** the backward pass for a deep network — the same “gradient in $\times$ local gradient = gradient out” you did by hand, vectorised.

◆ THE SAME THING IN PYTORCH (VECTORIZED LAYER)

```
import torch

# a layer: W is (out=3, in=4), one input vector x of length 4
W = torch.randn(3, 4)
x = torch.randn(4)
delta = torch.randn(3)          # gradient arriving at this layer's z

dL_dW = torch.outer(delta, x)   # delta @ x^T → shape (3, 4), matches W
dL_db = delta                   # → shape (3,)
dL_dx = W.T @ delta             # → shape (4,), sent to the previous layer

print(dL_dW.shape, dL_db.shape, dL_dx.shape)  # (3,4) (3,) (4,)
```

▲ WATCH OUT

The transposes are not decoration – they are what makes the shapes fit. $\partial L / \partial \mathbf{W}$ must have the *same shape* as $\mathbf{W}$ itself (you subtract it from $\mathbf{W}$ during the update), and $\partial L / \partial \mathbf{x}$ must match $\mathbf{x}$. If a backward pass throws a shape error, a misplaced transpose is the first suspect. When stuck, print shapes on every line, exactly as in [Section 1](#).

6 Trust, but verify: gradient checking

◆ INTUITION FIRST

Hand-derived gradients are easy to get subtly wrong – a dropped transpose, a sign flip. There is a dead-simple way to catch it: compare your fancy backprop gradient against the crude brute-force estimate from the start of this section. If they agree to several decimals, your backprop is almost certainly correct. This is the **numerical gradient check**, and it has saved more from-scratch networks than any other single trick.

👉 EVERYDAY PARALLEL

You add up a long shopping bill in your head, then also run it through the till. If both give £47.30, you trust your arithmetic. If they disagree, you know there is a bug *somewhere* before you rely on the total. The till is slow, so you don't use it for every bill – just to confirm your method once.

◆ NOW THE MATH

Use the **centred** difference – bump the weight both up and down by ε . It is far more accurate than the one-sided version we opened with:

$$\frac{\partial L}{\partial w} \approx \frac{L(w + \varepsilon) - L(w - \varepsilon)}{2\varepsilon}$$

In words: nudge the weight up, nudge it down, and take the average slope between. With $\varepsilon = 10^{-5}$ this typically matches the true gradient to 6–8 decimal places. It is far too slow to *train* with (one forward pass per weight — the very problem backprop solves), but it is perfect as a one-off check that backprop is wired correctly.

◆ CHECKING OUR EXAMPLE'S GRADIENT FOR W_1

```
import torch

def loss(w1, b1, w2, b2, x=torch.tensor(1.0), y=torch.tensor(1.0)):
    a1 = 1/(1 + torch.exp(-(w1*x + b1)))
    a2 = 1/(1 + torch.exp(-(w2*a1 + b2)))
    return 0.5 * (a2 - y)**2

w1, b1, w2, b2 = (torch.tensor(v) for v in (0.5, 0.5, 1.5, -1.0))
eps = 1e-5
num_grad_w1 = (loss(w1+eps, b1, w2, b2) - loss(w1-eps, b1, w2, b2)) /
(2*eps)

print(num_grad_w1)    # tensor(-0.0350)  -- matches our hand-computed
delta1 * x
```

In words: the numerical estimate lands on -0.0350 , the same value we derived by hand in Step F. That agreement is the proof our backward pass is correct. We build this into a proper debugging habit in [Section 15](#).

◆ YOU WILL NEVER WRITE THIS BY HAND AGAIN

Deriving gradients by hand is how you *understand* backprop — but in practice a library does it for you. [Autograd \(Section 13\)](#) records the computational graph as your forward code runs, then applies the exact local-gradient rule you just used at every node, automatically. And in [Section 11](#) we code a full multi-layer network's backward pass from scratch in NumPy, so you see there is no magic — only the bookkeeping you now understand.

CHECK YOURSELF

Your backprop gradient for some weight is `0.4000`, but the centred numerical check gives `-0.4000`. What kind of bug does that specific mismatch point to?

[Reveal ▼](#)**ANSWER**

A **sign error**. When the magnitudes match but the signs are opposite, you have almost certainly flipped a subtraction somewhere – a $y - a$ written as $a - y$, or a gradient added where it should be subtracted (or vice-versa) in the update. It is one of the most common backprop bugs, and gradient checking catches it instantly. If instead the *magnitudes* were wildly different, suspect a missing local-gradient factor (forgot to multiply by $a(1 - a)$) or a transpose in the vectorized form.

7 What to carry forward

- Training needs $\partial L / \partial w$ for every weight; brute force costs one forward pass *per weight* and is hopeless. Backprop gets them **all** in one backward pass by reusing stored forward values – dynamic programming.
- The **computational graph** is the network as nodes (operations) and edges (values). Forward computes and stores values; backward computes gradients.
- The **local-gradient rule** at every node: *gradient in (from above) $\times$ local gradient = gradient out (to inputs)*. If a value feeds several nodes, sum the incoming gradients.
- Handy local gradients: multiply swaps its inputs; add passes gradient through unchanged; sigmoid's is $a(1 - a)$, free from the stored activation.
- We computed a whole two-layer network by hand: forward to $L = 0.1132$, backward to four gradients, one step, and the loss fell to 0.0905 – verified in code.
- Vectorized, the same rules read $\partial L / \partial \mathbf{W} = \boldsymbol{\delta} \mathbf{x}^\top$, $\partial L / \partial \mathbf{b} = \boldsymbol{\delta}$, $\partial L / \partial \mathbf{x} = \mathbf{W}^\top \boldsymbol{\delta}$.

- **Gradient checking** – compare to $(L(w + \varepsilon) - L(w - \varepsilon))/2\varepsilon$ – is your safety net; a sign flip or dropped factor shows up at once.

Next, in [Section 11](#), we stop doing it by hand and build a full multi-layer network – forward and backward – from scratch in NumPy. You already know the one rule it runs on.

Build an MLP in NumPy

No PyTorch yet — just NumPy and the backprop math from Section 10, assembled into a network that actually learns.

30 min read **You'll learn:** forward pass · backward pass · the training loop · watching a real network learn

So far every idea has lived on its own page: a neuron here, a gradient there, a loss function somewhere else. In this section we **put all the pieces in one file** and press run. No framework, no magic — just NumPy arrays and arithmetic you can read line by line. By the end you'll have a small network that teaches itself to solve a problem no straight line can, and you'll understand every single number that flows through it.

◆ THE ONE IDEA TO HOLD ONTO

A neural network is not a mysterious black box. It is four steps in a loop: **guess, measure how wrong the guess is, work out which direction reduces the wrongness, take a small step that way.** Repeat a few thousand times and the guesses become good. Today we write those four steps in plain NumPy and watch them work.

1 The plan: beat a problem a line cannot

◆ INTUITION FIRST

Back in [Section 9](#) we saw that a single layer — a matrix multiply plus a shift — can only draw a **straight boundary** between classes. That is fine for data you can separate with a ruler, but the world is rarely that tidy. The classic example is **XOR** (short for "exclusive or"): a rule that says "output 1 when the two inputs *disagree*, and 0 when they agree."

input x₁	input x₂	label (XOR)
0	0	0 (agree)
0	1	1 (disagree)
1	0	1 (disagree)
1	1	0 (agree)

Try to draw one straight line that puts the two 1s on one side and the two 0s on the other. You can't – the 1s sit on opposite corners of a square, and so do the 0s. No single line separates them. This is the smallest possible problem that defeats a linear model, which makes it the perfect first opponent for a real network.

🚩 EVERYDAY PARALLEL

A light switch at the top of a staircase and another at the bottom. The light is on only when the two switches are in *different* positions; flip either one and the light flips too. "On when they disagree" is exactly XOR. You know instantly how it behaves, yet there is no way to describe it with a single "the more of this, the more of that" rule – you need a rule that *bends*. A network bends.

Our plan for the whole section: build a **2-layer MLP** (multi-layer perceptron – just a fancy name for a small stack of these matrix-plus-nonlinearity layers) in pure NumPy, and train it until it classifies all four XOR points correctly. Every gradient we use was derived in [Section 10](#); here we finally turn that math into code that runs.

DEFINITION

An **MLP** (multi-layer perceptron) is a network built from *layers*, where each layer does a matrix multiply plus a bias shift, and a **nonlinearity** (a bending function like ReLU) sits between the layers. The nonlinearity is what lets the whole thing curve its decision boundary instead of drawing one flat line.

2 The training loop, before any code

◆ INTUITION FIRST

Every network in this course, from this toy up to the largest language model, learns with the same four-step loop. Hold these four words in your head and the code below will feel obvious:

- **Predict** – push the inputs through the network to get a guess.
- **Measure** – compare the guess to the true answer with a *loss* (a single number saying how wrong we are).
- **Adjust** – find, for every weight, which way to nudge it to make the loss smaller. That "which way" is the *gradient*.
- **Repeat** – take a small step in that direction and go again.

🔧 EVERYDAY PARALLEL

Learning to shoot a basketball free throw. You **shoot** (predict). You **see the miss** – short and to the left (measure the error). Your body works out the **correction** – a bit more arc, aim slightly right (the gradient: which adjustment reduces the miss). You take one shot with that small change and **shoot again**. No single throw is perfect, but after a hundred the ball drops through cleanly. A network trains the exact same way, just with numbers instead of muscles.

◆ ONE PASS = ONE EPOCH

Running that four-step loop once over the whole dataset is called one *epoch*. We'll run a few thousand epochs. Each one makes the weights a tiny bit better, and the loss printed on screen ticks downward – that falling number is the network learning, live.

3 Building it, one block at a time

We'll assemble the network in small labelled pieces so every line has a reason. At the very end ([the full script below](#)) the same pieces are collected into one script you can paste and run.

(a) The dataset

First the data. XOR has just four points, so we can type them out by hand. `X` holds the four inputs (one per row); `y` holds their correct labels.

>_ THE FOUR XOR POINTS AND THEIR LABELS

```
import numpy as np

# each row is one example: [x1, x2]
X = np.array([[0.0, 0.0],
              [0.0, 1.0],
              [1.0, 0.0],
              [1.0, 1.0]])

# the correct answer for each row (XOR: 1 when inputs differ)
y = np.array([0, 1, 1, 0])

N, D = X.shape      # N = 4 examples, D = 2 features each
C = 2               # number of classes: "0" and "1"
```

We treat this as a **classification** problem with two classes ("output is 0" and "output is 1") rather than predicting a raw number. That choice is what lets us use softmax and cross-entropy in a moment, exactly as a real classifier does.

Cross-entropy compares two *probability lists*, so the label `1` needs to become a probability list too: "100% class 1, 0% class 0." That format is called **one-hot** – a list that is all zeros except for a single 1 in the correct slot.

>_ TURN EACH LABEL INTO A ONE-HOT ROW

```
# Y[i] is all zeros except a 1 in the correct class column
Y = np.zeros((N, C))
Y[np.arange(N), y] = 1.0

# y = [0, 1, 1, 0] → Y = [[1, 0],
#                           [0, 1],
#                           [0, 1],
#                           [1, 0]]
```

🔼 EVERYDAY PARALLEL

One-hot is a multiple-choice answer sheet where you fill in exactly one bubble. The label "the answer is B" becomes the row `[0, 1, 0, 0]` – a 1 under B, blanks everywhere else. Storing the answer this way lets us score it against the network's "how sure am I about each option?" output, option by option.

(b) Initialize the weights

The network has two layers, so it has two weight matrices and two bias vectors. Layer 1 turns the 2 input features into `H` hidden numbers; layer 2 turns those `H` numbers into 2 class scores. We fill the weights with **small random** values and set the biases to zero.

›_ TWO LAYERS: (2 → H) THEN (H → 2)

```
rng = np.random.default_rng(42)    # fixed seed so results repeat
H = 8                               # neurons in the hidden layer

# He initialization: spread scaled to the number of inputs (keeps signals
# sane)
W1 = rng.normal(0, np.sqrt(2.0 / D), size=(D, H))    # (2, 8)
b1 = np.zeros(H)                                     # (8,)
W2 = rng.normal(0, np.sqrt(2.0 / H), size=(H, C))    # (8, 2)
b2 = np.zeros(C)                                     # (2,)
```

Why *random* and not just zeros? If every weight in a layer starts identical, every neuron in that layer computes the exact same thing, receives the exact same gradient, and updates identically forever – they never split apart to learn different features. This is the **symmetry-breaking** problem, and a little randomness is the cure. We dig into why the specific *scale* matters (too big or too small both break training) in [Section 12](#).

🔼 EVERYDAY PARALLEL

Seat eight brand-new interns and give every one the identical starting notes and identical feedback. They'll all reach the identical conclusion – you've effectively hired one intern, eight times over. Give each a slightly different starting point and they explore different ideas, and the team as a whole covers far more ground. Random init is how the eight hidden neurons avoid becoming eight copies of the same neuron.

(c) The forward pass

The **forward pass** is the "predict" step: push **X** through both layers to get class scores. Between the layers sits **ReLU** (from [Section 9](#)) – the bending function that keeps positive numbers and replaces negatives with 0. That bend is the whole reason the network can beat XOR.

◆ NOW THE MATH

$$\begin{aligned}\mathbf{z}_1 &= \mathbf{X}\mathbf{W}_1 + \mathbf{b}_1 \\ \mathbf{a}_1 &= \text{ReLU}(\mathbf{z}_1) = \max(0, \mathbf{z}_1) \\ \mathbf{z}_2 &= \mathbf{a}_1\mathbf{W}_2 + \mathbf{b}_2 \\ \mathbf{p} &= \text{softmax}(\mathbf{z}_2)\end{aligned}$$

In words: multiply the inputs by the first weight matrix and add the bias to get **z₁** (the "pre-activation"). Bend it with ReLU to get the hidden values **a₁**. Multiply those by the second weight matrix and add the second bias to get the raw scores **z₂** (called *logits* — unnormalized, can be any size, positive or negative). Finally squash the logits into probabilities **p** with softmax.

Softmax turns a row of raw scores into a row of probabilities: it makes every number positive by exponentiating it, then divides by the total so the row sums to 1.

$$\text{softmax}(\mathbf{z})_j = \frac{e^{z_j}}{\sum_k e^{z_k}}$$

In words: for each score, raise **e** to that score (this makes it positive and blows up big scores more than small ones), then divide by the sum of all of them so the results are fractions that add to 100%. The biggest score gets the biggest probability.

🔼 EVERYDAY PARALLEL

Softmax is how you'd convert a panel of judges' raw enthusiasm into betting odds. Judge scores of 8, 6, and 2 aren't probabilities – they don't add to 100%. Softmax exponentiates and normalizes them into something like 82% / 15% / 3%: a clean set of odds where the loudest score gets the lion's share but nobody is left at exactly zero.

◆ THE SAME THING IN NUMPY

```
def relu(z):
    return np.maximum(0.0, z)

def softmax(z):
    # subtract each row's max before exp: same result, no overflow on big logits
    z = z - z.max(axis=1, keepdims=True)
    e = np.exp(z)
    return e / e.sum(axis=1, keepdims=True)

def forward(X, W1, b1, W2, b2):
    z1 = X @ W1 + b1          # (N,2)@(2,8) → (N,8)
    a1 = relu(z1)             # (N,8)
    z2 = a1 @ W2 + b2         # (N,8)@(8,2) → (N,2)
    probs = softmax(z2)       # (N,2), each row sums to 1
    return z1, a1, z2, probs  # keep z1 & a1 – the backward pass needs them
```

Notice the shapes threaded through the comments. `X @ W1` is $(N,2) \cdot (2,8) \rightarrow (N,8)$; then `a1 @ W2` is $(N,8) \cdot (8,2) \rightarrow (N,2)$. The inner numbers match at every step (the shape rule from [Section 1](#)), and the final output is one probability pair per example. We return `z1` and `a1` as well because – as you'll see next – the backward pass reuses these exact values instead of recomputing them.

(d) The loss: cross-entropy

The loss is the "measure" step: one number saying how wrong the predictions are. **Cross-entropy** (from [Section 4](#)) looks only at the probability the network assigned to the *correct* class and penalizes it for being anything less than 1.

◆ NOW THE MATH

$$L = -\frac{1}{N} \sum_{i=1}^N \log(p_{i, y_i})$$

In words: for each example i , look up the probability the model gave to the *true* class y_i , take its log, negate it, and average over all examples. If the model is confident and right ($p \approx 1$), $\log(1) = 0$ — no penalty. If it is confident and wrong ($p \approx 0$), $-\log(0)$ shoots toward infinity — a huge penalty. Cross-entropy rewards being both correct *and* sure.

◆ THE SAME THING IN NUMPY

```
def cross_entropy(probs, y):  
    # pick out prob of the correct class for each row, then -log, then  
    # average  
    correct = probs[np.arange(len(y)), y] # shape (N,)  
    return -np.mean(np.log(correct + 1e-12)) # +1e-12 guards log(0)
```

The tiny `1e-12` is a floor that stops `log(0)` from becoming `-inf` if a probability ever rounds to exactly zero. It changes the number negligibly but keeps the run from blowing up.

☂ EVERYDAY PARALLEL

Cross-entropy grades like a strict exam that scores your *confidence*, not just your answer. Tick the right box and say "I'm certain" → full marks. Tick the right box but hedge "maybe" → partial marks. Confidently tick the *wrong* box → a savage penalty. It teaches the network not only to be right, but to know when it's right.

(e) The backward pass

Now the heart of it: the **backward pass** computes the gradient of the loss with respect to every weight — the "which way should each number move?" step. Every line below is one application of the chain rule from [Section 10](#); we walk backward through the same operations the forward pass ran.

◆ NOW THE MATH

The chain starts with a small miracle. When softmax feeds straight into cross-entropy, their two derivatives combine and almost everything cancels, leaving one clean line:

$$\frac{\partial L}{\partial \mathbf{z}_2} = \frac{1}{N} (\mathbf{p} - \mathbf{Y})$$

In words: the gradient at the logits is simply *predicted probabilities minus the one-hot truth*, averaged over the batch. If the network said 0.7 for a class that should have been 1, that slot's gradient is $0.7 - 1 = -0.3$: "push this logit up." Where it's right, prediction minus truth is near zero: "leave it alone." This is the single most reused fact in all of deep-learning code.

From `dz2` we push the gradient back through each earlier operation. Each step pairs with a forward step and uses the rules from Section 10 – a matrix multiply's gradient re-uses the *other* factor, transposed; a bias's gradient is the sum over the batch; and ReLU passes the gradient through only where its input was positive:

$$\begin{aligned} \frac{\partial L}{\partial \mathbf{W}_2} &= \mathbf{a}_1^\top \mathbf{dz}_2 & \frac{\partial L}{\partial \mathbf{b}_2} &= \sum_i (\mathbf{dz}_2)_i \\ \frac{\partial L}{\partial \mathbf{a}_1} &= \mathbf{dz}_2 \mathbf{W}_2^\top & \mathbf{dz}_1 &= \mathbf{da}_1 \odot \mathbf{1}[\mathbf{z}_1 > 0] \\ \frac{\partial L}{\partial \mathbf{W}_1} &= \mathbf{X}^\top \mathbf{dz}_1 & \frac{\partial L}{\partial \mathbf{b}_1} &= \sum_i (\mathbf{dz}_1)_i \end{aligned}$$

In words, line by line: the gradient for $\mathbf{W}_2$ is the layer's input $\mathbf{a}_1$ (transposed) times the incoming gradient — the multiply rule from Section 10. The gradient for the bias $\mathbf{b}_2$ is just the incoming gradient summed down the batch. To pass gradient back into the hidden layer we multiply by $\mathbf{W}_2$ transposed, giving $\mathbf{da}_1$. ReLU only let positive values through on the way forward, so on the way back it only lets gradient through where $\mathbf{z}_1 > 0$ — that's the $\odot \mathbf{1}[\mathbf{z}_1 > 0]$ mask ($\odot$ means multiply element by element). Then $\mathbf{W}_1$ and $\mathbf{b}_1$ follow the very same two rules as the second layer.

◆ THE SAME THING IN NUMPY

```
def backward(X, y, z1, a1, probs, W2):
    N = len(y)
    Y = np.zeros_like(probs)
    Y[np.arange(N), y] = 1.0

    dz2 = (probs - Y) / N                # (N,2) softmax+CE shortcut
    dW2 = a1.T @ dz2                     # (8,N)@(N,2) → (8,2) matches W2
    db2 = dz2.sum(axis=0)                 # (2,) matches b2

    da1 = dz2 @ W2.T                     # (N,2)@(2,8) → (N,8)
    dz1 = da1 * (z1 > 0)                  # (N,8) ReLU gate: 0 where z1 ≤ 0
    dW1 = X.T @ dz1                      # (2,N)@(N,8) → (2,8) matches W1
    db1 = dz1.sum(axis=0)                 # (8,) matches b1
```

```
return dw1, db1, dw2, db2
```

Read the shape comments as a checklist: every gradient comes out the same shape as the thing it updates (`dw2` is `(8,2)` just like `W2`, and so on). When your own backward pass has a bug, this is the first thing to check – a gradient whose shape doesn't match its weight means a transpose is in the wrong place.

▲ WATCH OUT

The ReLU gate uses `z1` – the value *before* the bend – not `a1`. It happens that `(z1 > 0)` and `(a1 > 0)` give the same mask for plain ReLU, so either works here, but the rule you must remember is "gradient flows only where the neuron was active." Also don't forget the `/ N` in `dz2`: it must match the `1/N` average in the loss, or your gradients are `N` times too large and the steps overshoot.

(f) The update: stochastic gradient descent

The "adjust" step. We now know, for every weight, the direction that *increases* the loss (that's what the gradient points toward). So we step the opposite way – subtract a small fraction of the gradient. That fraction is the **learning rate**.

◆ NOW THE MATH

$$\mathbf{W} \leftarrow \mathbf{W} - \alpha \frac{\partial L}{\partial \mathbf{W}}$$

In words: replace each weight with itself minus (learning rate `α`, "alpha") times its gradient. The gradient says "uphill is this way"; the minus sign turns us downhill, toward smaller loss; `α` controls how big a step we take. Too small and learning crawls; too big and we leap past the bottom and bounce around. This one update, repeated, is *gradient descent*.

◆ THE SAME THING IN NUMPY

```
def sgd_step(params, grads, lr):
    for p, g in zip(params, grads):
        p -= lr * g          # in-place: nudge each weight downhill
```

Because our whole dataset is only four points, each epoch uses all of them at once. When you instead use a small *random* handful per step – which real datasets force you to do – the "random" makes it **stochastic** gradient descent (SGD). Same update rule, just fed a noisy sample of the data each time.

(g) The epoch loop

Now we thread the four steps together and run them a few thousand times. This is the loop we sketched earlier, finally in code:

> _ PREDICT → MEASURE → ADJUST → REPEAT

```
lr = 0.5
for epoch in range(2001):
    # 1. PREDICT
    z1, a1, z2, probs = forward(X, W1, b1, W2, b2)

    # 2. MEASURE
    loss = cross_entropy(probs, y)

    # 3. ADJUST – gradients, then step downhill
    dW1, db1, dW2, db2 = backward(X, y, z1, a1, probs, W2)
    sgd_step([W1, b1, W2, b2], [dW1, db1, dW2, db2], lr)

    # 4. watch it learn
    if epoch % 400 == 0:
        print(f"epoch {epoch:4d}  loss {loss:.4f}")
```

Running it prints the loss falling toward zero – the network getting less wrong every few hundred epochs:

```
epoch    0  loss 0.8471
epoch  400  loss 0.0059
epoch  800  loss 0.0022
epoch 1200  loss 0.0012
epoch 1600  loss 0.0008
epoch 2000  loss 0.0006
```

And the payoff – checking the final predictions against the truth:

```
pred = forward(X, W1, b1, W2, b2)[-1].argmax(axis=1)
print(pred)                                # [0 1 1 0]
print("accuracy:", (pred == y).mean())     # accuracy: 1.0
```

◆ THE PAYOFF

`[0 1 1 0]` – all four XOR points correct, 100% accuracy. A single linear layer *cannot* do this, no matter how long you train it, because no straight line separates the classes. The hidden layer with its ReLU bend can, and it learned to do so entirely on its own, just by rolling the four-step loop a couple thousand times. That is the whole idea of deep learning, demonstrated in code you can read top to bottom.

CHECK YOURSELF

If you delete the ReLU line – so `a1 = z1` with no bend between the two layers – will the network still solve XOR? Why or why not?

Reveal ▼

ANSWER

No. Without the bend, layer 2 applied to layer 1 is just $(X \cdot W1 + b1) \cdot W2 + b2$, which multiplies out to $X \cdot (W1 \cdot W2) + \text{const}$ – a single matrix $W1 \cdot W2$ times X , plus a shift. Two stacked linear layers collapse into **one** linear layer, and we already know one linear layer can't separate XOR. The nonlinearity is not decoration; it is the *only* reason depth buys you anything. Remove it and you're back to drawing a single straight line.

4 The complete script

Here is everything above collected into one file. It has no dependencies beyond NumPy – paste it into a `.py` file and run it, and you'll watch the loss fall and see the network hit 100% on XOR. Every line is one you've already met.

›_ MLP_XOR.PY — A 2-LAYER NETWORK, FROM SCRATCH

```

import numpy as np

# ---- 1. data -----
--
X = np.array([[0.0, 0.0],
              [0.0, 1.0],
              [1.0, 0.0],
              [1.0, 1.0]])
y = np.array([0, 1, 1, 0])      # XOR: 1 when the inputs differ
N, D = X.shape                  # 4 examples, 2 features
C = 2                           # two classes: "0" and "1"

# ---- 2. weights (small random; biases zero) -----
--
rng = np.random.default_rng(42)
H = 8                           # hidden neurons
W1 = rng.normal(0, np.sqrt(2.0 / D), size=(D, H))    # (2, 8)
b1 = np.zeros(H)                  # (8,)
W2 = rng.normal(0, np.sqrt(2.0 / H), size=(H, C))    # (8, 2)
b2 = np.zeros(C)                  # (2,)

# ---- 3. building blocks -----
--
def relu(z):
    return np.maximum(0.0, z)

def softmax(z):
    z = z - z.max(axis=1, keepdims=True)      # stability, same
result                                         # result
    e = np.exp(z)
    return e / e.sum(axis=1, keepdims=True)

def forward(X, W1, b1, W2, b2):
    z1 = X @ W1 + b1                    # (N, 8)
    a1 = relu(z1)                       # (N, 8)
    z2 = a1 @ W2 + b2                   # (N, 2) logits
    probs = softmax(z2)                 # (N, 2) each row sums to 1
    return z1, a1, z2, probs

def cross_entropy(probs, y):
    correct = probs[np.arange(len(y)), y]
    return -np.mean(np.log(correct + 1e-12))

def backward(X, y, z1, a1, probs, W2):

```

```

N = len(y)
Y = np.zeros_like(probs)
Y[np.arange(N), y] = 1.0
dz2 = (probs - Y) / N          # (N, 2) softmax + cross-entropy
shortcut
dW2 = a1.T @ dz2               # (8, 2)
db2 = dz2.sum(axis=0)          # (2,)
da1 = dz2 @ W2.T               # (N, 8)
dz1 = da1 * (z1 > 0)           # (N, 8) ReLU gate
dW1 = X.T @ dz1                # (2, 8)
db1 = dz1.sum(axis=0)          # (8,)
return dW1, db1, dW2, db2

# ---- 4. train -----
--
lr = 0.5
for epoch in range(2001):
    z1, a1, z2, probs = forward(X, W1, b1, W2, b2) # predict
    loss = cross_entropy(probs, y)                  # measure
    dW1, db1, dW2, db2 = backward(X, y, z1, a1, probs, W2) # adjust
    for p, g in zip([W1, b1, W2, b2], [dW1, db1, dW2, db2]):
        p -= lr * g                                # step downhill
    if epoch % 400 == 0:
        print(f"epoch {epoch:4d}  loss {loss:.4f}")

# ---- 5. results -----
--
pred = forward(X, W1, b1, W2, b2)[-1].argmax(axis=1)
print("predictions:", pred, " truth:", y)
print("accuracy:   ", (pred == y).mean())

```

▲ WATCH OUT

The `rng = np.random.default_rng(42)` seed makes this run reproducible – you'll get the exact loss numbers shown above. Change the seed and the random starting weights change, so the loss curve will look a little different, but it should still reach 100% within a couple thousand epochs. If a particular seed *stalls*, that's a real preview of [Section 12](#): initialization and learning rate genuinely decide whether training succeeds.

5 What to carry forward

- A network learns with one loop, four steps: **predict** → **measure** → **adjust** → **repeat**. Everything else is detail.
- The **forward pass** is matrix multiplies with a nonlinearity between them; the **ReLU bend** is the only reason two layers beat one – remove it and the layers collapse into a single line.
- **Softmax** turns raw scores (logits) into probabilities; **cross-entropy** scores those probabilities against the one-hot truth.
- The **backward pass** is Section 10's chain rule, one line per operation. The softmax+cross-entropy gradient is just $\mathbf{p} - \mathbf{Y}$, and every gradient comes out the same shape as the weight it updates.
- Weights start **small and random** (never all zeros – that never breaks symmetry), and **SGD** nudges them downhill by the learning rate each step.

You just wrote, by hand, everything a deep-learning framework does under the hood. Next, in Section 12, we make training itself reliable – initialization, optimizers, and learning-rate schedules – and then in Part 4 we hand exactly these steps to **PyTorch**, which computes the backward pass for us automatically, so we never have to derive those gradient lines again. But now, when it does, you'll know precisely what it's doing.

Training Dynamics & Optimizers

The same network can fail or fly depending on how you start it and how you step — here's the why behind the knobs.

32 min read

You'll learn: weight initialization (Xavier/He) · learning rates · SGD, momentum, Adam · LR schedules

You have a network. You have a loss (a single number saying how wrong the network is) and you have [backpropagation](#) to tell you which way each weight should move. So training is done, right? Not quite. Two choices you make *before* and *during* that loop decide whether it converges in a minute or never converges at all: the numbers you **start** the weights at, and the size of the **steps** you take. This section is about those two knobs — the ones that turn a working design into a working model.

◆ THE ONE IDEA TO HOLD ONTO

Backprop tells you the *direction* to move each weight. It says nothing about *where to begin* or *how far to step*. Those two decisions — initialization and the optimizer — are what "training dynamics" means, and getting them wrong breaks networks that are otherwise perfectly correct.

1 Initialization: where the weights start

◆ INTUITION FIRST

Before the first step of training, every weight in the network needs a starting value. It feels like it shouldn't matter much — training will fix them anyway, won't it? But the starting point secretly controls two things at once: whether the neurons can ever become *different* from each other, and whether the signal can travel through a deep stack of layers without either dying out or blowing up. Get either wrong and the network learns slowly, badly, or not at all.

🔼 EVERYDAY PARALLEL

Imagine a study group where everyone starts with the *identical* notes and always copies each other's answers. No matter how long they study, they stay perfect clones — the group is no smarter than one person. To actually cover the material, the members have to start from slightly different places so they can specialise. A neural network's neurons are the same: they must start *different* to end up doing different jobs.

Why not all zeros — the symmetry problem, proven

◆ NOW THE MATH

The tempting "safe" choice is to set every weight to zero. Let's prove why it destroys the network. Take one layer with two identical-looking neurons. Each neuron j computes a weighted sum of the same inputs $\mathbf{x}$, then passes it through an activation function ϕ (a simple nonlinear squashing function like ReLU or sigmoid):

$$a_j = \phi \left(\sum_i w_{ji} x_i + b_j \right)$$

In words: neuron j 's output is: multiply each input x_i by its weight w_{ji} , add them up, add the bias b_j , then squash. w_{ji} is the weight from input i into neuron j .

Now set every weight and bias to the same value — say zero. Then for *every* neuron j in the layer, the sum inside is the same number, so every neuron produces the *same output*:

$$w_{ji} = 0 \Rightarrow a_1 = a_2 = \dots = a_n = \phi(0)$$

That is already bad, but the killer is what backprop does next. The gradient that flows back into weight w_{ji} depends only on that neuron's input x_i and the error signal arriving at neuron j . Because all neurons in the layer are identical, they all receive the *identical* error signal, so every weight in the layer gets the *identical* gradient:

$$\frac{\partial L}{\partial w_{1i}} = \frac{\partial L}{\partial w_{2i}} = \dots = \frac{\partial L}{\partial w_{ni}}$$

In words: $\partial L / \partial w_{ji}$ means "how much the loss L changes if we nudge weight w_{ji} ." When every neuron is a copy, every one of these numbers is equal — so the optimizer nudges them all by the same amount.

Follow the consequence. If they start equal and always get updated by the same amount, they stay equal *forever*. The two neurons remain perfect clones for the entire run. A layer of 100 neurons behaves like a layer of *one*. This is the **symmetry problem**: nothing in the update rule can ever break the tie, so the neurons never differentiate.

◆ THE FIX

Break the symmetry by starting the weights at *small random* numbers. Now each neuron sees a slightly different weighted sum, receives a slightly different gradient, and is free to specialise. Random initialization isn't a nice-to-have – it's the only thing that lets a wide layer be more expressive than a single neuron. (Biases can safely start at zero; the random weights already break the tie.)

Why not too large, and not too small

◆ INTUITION FIRST

Fine – random. But *how big* random? This is the second job of initialization. A deep network passes the signal through many layers in a row. If each layer tends to *magnify* the signal a little, that magnification compounds layer after layer and the numbers explode to infinity. If each layer *shrinks* it a little, the signal fades to nothing before it reaches the end. Either way, learning stalls.

☂ EVERYDAY PARALLEL

This is a microphone next to its own speaker. Set the gain slightly too high and every pass through the loop is louder than the last – you get the screaming feedback squeal (exploding). Set it too low and the sound dies out to silence before you can hear it (vanishing). There is a narrow band of gain where the sound carries cleanly. Initialization is setting that gain for the network's signal.

This is exactly the eigenvalue story from [Section 1](#): passing a signal through a stack of layers is repeated multiplication by weight matrices, and repeated multiplication grows or decays according to whether the matrices stretch by more or less than 1. We want each layer to stretch by *about* 1 – to preserve the size of the signal – so it survives all the way through.

Deriving the right scale: variance preservation

◆ NOW THE MATH

Let's turn "stretch by about 1" into an actual formula. Consider one neuron's pre-activation (the weighted sum before the squashing function):

$$z = \sum_{i=1}^{n_{\text{in}}} w_i x_i$$

In words: z is the sum over all n_{in} incoming connections of weight times input. n_{in} (called *fan-in*) is simply how many inputs feed this neuron.

We want the output z to have roughly the same *spread* as the inputs, so the signal neither grows nor shrinks. "Spread" is measured by **variance** (a number saying how far values typically stray from their average). Assume the weights and inputs start centered at zero, are independent, and each input has variance $\text{Var}(x)$. The variance of a sum of independent zero-mean products adds up term by term:

$$\text{Var}(z) = \sum_{i=1}^{n_{\text{in}}} \text{Var}(w_i x_i) = n_{\text{in}} \text{Var}(w) \text{Var}(x)$$

In words: there are n_{in} terms in the sum, and each contributes $\text{Var}(w) \cdot \text{Var}(x)$ (the variance of a product of two independent zero-mean numbers). So the output variance is the input variance multiplied by the factor $n_{\text{in}} \cdot \text{Var}(w)$.

For the output spread to equal the input spread, that multiplying factor must be exactly 1:

$$n_{\text{in}} \text{Var}(w) = 1 \implies \boxed{\text{Var}(w) = \frac{1}{n_{\text{in}}}}$$

In words: initialize each weight from a distribution whose variance is **1 divided by the number of inputs**. A neuron with more inputs must start with *smaller* weights, because it is summing more terms. That single line is the seed of every modern init scheme.

Xavier / Glorot initialization refines this by noticing that signal flows *two* ways – forward (the activations) and backward (the gradients). The forward pass wants $\text{Var}(w) = 1/n_{\text{in}}$; the backward pass wants $\text{Var}(w) = 1/n_{\text{out}}$. Glorot's compromise averages the two:

$$\text{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

In words: use the average of the fan-in and fan-out constraints. `n_out` (*fan-out*) is how many neurons this one feeds. Xavier is designed for activations that are symmetric around zero, like `tanh` and sigmoid.

He (Kaiming) initialization fixes Xavier for ReLU. ReLU ($\max(0, z)$ – it zeroes out every negative input) throws away half the signal on average, halving the variance at each layer. He compensates by doubling the scale:

$$\text{Var}(w) = \frac{2}{n_{\text{in}}}$$

In words: the extra factor of 2 exactly cancels the halving that ReLU causes, so the variance is preserved again. This is why the numerator is 2, not 1.

WHICH TO USE, IN ONE LINE

Use **He (Kaiming)** initialization with **ReLU** and its relatives (the default for almost every modern network). Use **Xavier (Glorot)** with **tanh** or **sigmoid** activations. Both come in a "normal" flavour (draw from a bell curve with the target variance) and a "uniform" flavour (draw from a flat range with the same variance); the difference rarely matters.

◆ THE SAME THING IN PYTORCH

›_ HE INIT FOR A RELU LAYER — AND WHAT PYTORCH DOES BY DEFAULT

```
import torch
import torch.nn as nn

layer = nn.Linear(256, 128)  # fan_in = 256, fan_out = 128

# He / Kaiming init, the right choice when a ReLU follows this layer
nn.init.kaiming_normal_(layer.weight, nonlinearity='relu')
nn.init.zeros_(layer.bias)  # biases can safely start at zero

# check the empirical variance is about 2 / fan_in = 2/256 ~ 0.0078
print(layer.weight.var().item())  # ~0.0078

# For a tanh/sigmoid layer you'd use Xavier instead:
# nn.init.xavier_normal_(layer.weight)
```

Good news: `nn.Linear` and `nn.Conv2d` already apply a Kaiming-style init out of the box, so you rarely call these by hand. You reach for them when you build a custom layer, or when a network mysteriously refuses to learn and you suspect the starting scale.

▲ WATCH OUT

Two silent killers. **All-zero weights** – the symmetry trap above; the network trains but never beats a single-neuron layer. **Default-scale weights that are too big** for a very deep net – the loss shows up as `NaN` within a few steps because the activations overflowed. If your loss goes to `NaN` immediately, suspect the initialization (or the learning rate, next) before anything else.

CHECK YOURSELF

You build a 50-layer network with ReLU activations but initialize every weight from a standard normal (variance 1, ignoring fan-in). What happens in the forward pass, and why?

Reveal ▼

ANSWER

The activations **explode**. With variance 1 per weight and a fan-in of, say, 256, each layer multiplies the signal variance by roughly $n_{in} \cdot \text{Var}(w) = 256$ (before ReLU). Compounded over 50 layers that is an astronomically large factor, so the pre-activations overflow to `inf`, and the loss becomes `NaN` on the first forward pass. The fix is to scale the variance down to $2/n_{in}$ (He init), which makes each layer's multiplying factor about 1 instead of 256 – the signal then travels all 50 layers intact. This is the eigenvalue "repeated multiplication" story from [Section 1](#) in action.

2 The learning rate: how far to step

◆ INTUITION FIRST

Gradient descent moves each weight a little bit in the downhill direction that backprop found. The **learning rate** (written α , and sometimes called the *step size*) is the single number that says *how big* "a little bit" is. It is the most important hyperparameter you will tune, and it has a real sweet-spot: too big and the steps overshoot and the loss diverges; too small and training crawls, taking forever to reach the bottom.

🌂 EVERYDAY PARALLEL

You're walking downhill in thick fog, trying to reach the bottom of a valley, and you can only feel the slope right under your feet. Take *giant* strides and you'll leap clear over the valley and land higher up the far slope – bouncing back and forth, never settling. Take *tiny* baby steps and you'll get there eventually, but the sun will set first. The learning rate is the length of your stride in that fog.

◆ NOW THE MATH

The update rule for one weight (or the whole parameter vector θ) is just:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L$$

In words: the arrow means "replace the left side with the right side." Take the current parameters θ , subtract the gradient ∇L (the uphill direction) scaled by the learning rate α . Subtracting the uphill direction means stepping downhill. α controls the length of that step.

Why does too-large α diverge? On a simple bowl-shaped loss, if the step is larger than the curvature can absorb, each step lands on the opposite wall of the bowl *higher* than before. The loss then grows every step instead of shrinking – the tell-tale sign that your learning rate is too high.

▲ WATCH OUT

The classic learning-rate signatures, worth memorising: loss going to **NaN** or rising = **too high**, lower it 10×. Loss falling but painfully slowly, almost a flat line = **too low**, raise it. A good first guess for Adam is **1e-3**; for plain SGD, **1e-1** to **1e-2**. Always tune the learning rate before fiddling with anything fancier.

CHECK YOURSELF

Your training loss zig-zags up and down every step and slowly trends upward. Learning rate too high or too low?

Reveal ▼

ANSWER

Too high. The zig-zag is the optimizer overshooting the valley and bouncing off the opposite wall each step, landing higher than it started – exactly the giant-strides-in-fog failure. Divide the learning rate by 10 and the bouncing usually turns into smooth descent. (A loss that falls but very slowly, with no bouncing, would be the opposite problem: too low.)

3 Optimizers: smarter ways to step

Plain gradient descent takes the raw gradient and steps. Optimizers are recipes that *reshape* that step to make training faster and more stable. We'll build up the three you actually meet – SGD, momentum, and Adam – each one fixing a flaw in the last.

SGD and why we use minibatches

◆ INTUITION FIRST

The "true" gradient is the average of the gradient over your *entire* training set. Computing that for every single step is wildly expensive when you have millions of examples. **Stochastic Gradient Descent (SGD)** – "stochastic" just means "random" – takes a shortcut: estimate the gradient from a small random handful of examples, called a **minibatch**, and step on that estimate. It's noisy, but it's cheap, and you get to take hundreds of steps in the time one full-dataset step would cost.

🚩 EVERYDAY PARALLEL

To learn whether a national election is leaning left or right, you don't phone all 50 million voters — you poll a random 1,000. The poll is a little off from the true number (that's the noise), but it's a thousand times faster and cheap enough to repeat every day. A minibatch is that opinion poll: a small random sample that gives a good-enough read on which way to step.

◆ NOW THE MATH

SGD is the update from the last section, computed on a minibatch $\mathcal{B}$ instead of the whole set:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L_{\mathcal{B}}(\theta)$$

In words: same downhill step as before, but the gradient $\nabla_{\mathbf{L}} \mathbf{B}$ is measured on just the current minibatch $\mathbf{B}$ — a noisy but unbiased estimate of the true gradient. "Unbiased" means that on average, over many batches, it points the right way.

The noise is not purely a cost. Its little random kicks can bump the optimizer out of shallow bad spots (poor local dips in the loss) that a perfectly smooth gradient would get stuck in. Cheap *and* occasionally helpful.

Momentum: remember where you were going

◆ INTUITION FIRST

Pure SGD has a weakness: it only knows the slope *right now*. In a long narrow valley it wastes energy bouncing across the walls instead of rolling along the floor, and it stalls on flat patches. **Momentum** fixes this by keeping a running memory of the recent gradients and stepping in that accumulated direction — so consistent downhill directions build up speed, and little side-to-side wobbles cancel out.

📌 EVERYDAY PARALLEL

Roll a heavy bowling ball down a bumpy hill. A tiny dip or pebble doesn't stop it – its built-up momentum carries it straight over. And because it's heavy, random little bumps to the left and right mostly cancel, so it holds a steady line toward the bottom. That accumulated "want to keep going this way" is exactly what momentum adds to the raw gradient.

◆ NOW THE MATH

Momentum keeps a **velocity** vector v , which is an *exponentially-decaying moving average (EMA)* of past gradients – a running average that weights recent gradients more and lets old ones fade:

$$\begin{aligned}v &\leftarrow \beta v + \nabla_{\theta} L \\ \theta &\leftarrow \theta - \alpha v\end{aligned}$$

In words: first line — the new velocity is β (beta, typically 0.9) times the old velocity, *plus* the fresh gradient. Because we keep 90% of the old velocity each step, past gradients pile up and decay slowly. Second line — step in the direction of that accumulated velocity, not the raw gradient. A steady downhill direction reinforces itself step after step and gains speed; noisy sideways components average toward zero.

DEFINITION

An **EMA** (exponentially-decaying moving average) blends a running value with each new observation: $v \leftarrow \beta v + (1 - \beta) g$. A high β (like 0.9) means a long memory (each value influences many future steps); a low β means a short memory. It shows up three times in this section – in momentum, and twice inside Adam.

Adam: momentum plus a personal step size for every weight

◆ INTUITION FIRST

Momentum fixed *direction*. There's a second problem it leaves alone: different weights need different step sizes. Some sit on steep parts of the loss (small steps, please) and some on nearly flat parts (bigger steps, please), and a single global learning rate can't suit both. **Adam** ("Adaptive Moment Estimation") solves both at once – it uses momentum for direction *and*

gives every single parameter its own automatically-tuned step size. It's the default optimizer for most of deep learning for a reason.

🌂 EVERYDAY PARALLEL

A smart hiker descending a mountain. She *remembers* the general downhill direction she's been heading and keeps her momentum (that's the first part). But she also *adapts her stride to the ground*: long confident strides on smooth open slopes, short careful steps on steep rocky bits where a big step would send her tumbling. Different terrain, different step – per foot, per moment. Adam gives every weight that kind of terrain-aware footing.

◆ NOW THE MATH

Adam keeps *two* running averages of the gradient $g = \nabla_{\theta} L$. Let t be the step count, starting at 1. First, the **first moment** m – an EMA of the gradient itself, i.e. plain momentum, tracking *which way* to go:

$$m \leftarrow \beta_1 m + (1 - \beta_1) g$$

In words: blend the running direction m with the fresh gradient. β_1 is usually 0.9. This is the "momentum" half — the smoothed direction.

Second, the **second moment** v – an EMA of the gradient *squared* (each component squared, elementwise), tracking *how big and bouncy* each parameter's gradient has been:

$$v \leftarrow \beta_2 v + (1 - \beta_2) g^2$$

In words: blend the running squared-gradient v with the fresh squared gradient. β_2 is usually 0.999 (a very long memory). A parameter whose gradient has been large and jumpy builds a large v ; a parameter with small steady gradients has a small v . This is the per-parameter terrain sensor.

There's a cold-start problem. Both m and v start at zero, so for the first several steps they're biased toward zero – too small – which would make the early steps wrong. The **bias correction** scales them back up to their true size:

$$\hat{m} = \frac{m}{1 - \beta_1^t}, \quad \hat{v} = \frac{v}{1 - \beta_2^t}$$

In words: divide by $(1 - \beta^t)$. At step $t = 1$ this divisor is small (like 0.1), so it *inflates* the tiny early estimates back to full size. As t grows, β^t shrinks toward 0, the divisor approaches 1, and the correction quietly switches off. It's a warm-up patch for the first dozen or so steps.

Finally the update – momentum direction $\hat{m}$, divided by the per-parameter scale $\sqrt{\hat{v}}$:

$$\theta \leftarrow \theta - \alpha \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$$

In words: step in the smoothed direction $\hat{m}$, but divide each parameter's step by the square root of its own $\hat{v}$. A weight with a big, bouncy gradient (large $\hat{v}$) gets a *smaller* step; a weight with small steady gradients gets a *larger* one. ϵ (epsilon, about $1e-8$) is a tiny number added only to avoid dividing by zero. So $\hat{m}$ answers "which way," and $\sqrt{\hat{v}}$ hands each parameter its own adaptive step size — that's the whole trick.

◆ ADAM IN ONE BREATH

$\mathbf{m}$ is a smoothed *direction* (momentum). $\sqrt{\mathbf{v}}$ is a per-parameter *step-size dial* (adaptivity).

Bias correction fixes the too-small first few steps. Put them together and you get an optimizer that mostly "just works" at learning rate `1e-3` without hand-tuning per-layer — which is why it's everyone's default.

◆ THE SAME THING IN PYTORCH

>_ THE THREE OPTIMIZERS, SAME INTERFACE

```

import torch

model = torch.nn.Linear(64, 10)

# plain SGD
opt = torch.optim.SGD(model.parameters(), lr=1e-2)

# SGD with momentum (beta = 0.9)
opt = torch.optim.SGD(model.parameters(), lr=1e-2, momentum=0.9)

# Adam: betas = (beta1, beta2), eps as derived above
opt = torch.optim.Adam(model.parameters(), lr=1e-3,
                        betas=(0.9, 0.999), eps=1e-8)

# the training step is identical whichever you chose:
loss = model(torch.randn(32, 64)).sum() # stand-in for a real loss
opt.zero_grad() # clear last step's gradients (they accumulate
                # otherwise)
loss.backward() # backprop fills every .grad
opt.step()      # the optimizer applies its update rule

```

Notice the loop is the same three calls regardless of optimizer – `zero_grad`, `backward`, `step`. The optimizer object hides all the momentum and adaptivity math behind that one `step()`.

▲ WATCH OUT

Forgetting `opt.zero_grad()` is the most common training bug there is. PyTorch *accumulates* gradients into `.grad` by design, so if you skip the reset, this step's gradient is added on top of last step's leftover – your updates quietly go wrong and the model won't converge. Clear the gradients every single step.

CHECK YOURSELF

In Adam, one weight consistently has much larger gradients than the others. Compared to a plain fixed learning rate, does Adam take bigger or smaller steps for that weight, and which term causes it?

Reveal ▼

ANSWER

Smaller steps. A consistently large gradient builds up a large second moment v (the EMA of squared gradients), and the update divides by $\sqrt{v}$. Dividing by a big number shrinks that weight's step. This is the per-parameter adaptivity: Adam automatically reins in the jumpy, high-gradient weights while letting the calm, small-gradient weights take relatively larger steps – so a single global learning rate no longer has to suit every parameter at once.

4 Learning-rate schedules: change the step over time

◆ INTUITION FIRST

Even the best fixed learning rate is a compromise. Early in training you want *big* steps to cover ground fast; late in training, near the bottom, you want *small* steps so you can settle precisely into the minimum instead of bouncing around it. A **learning-rate schedule** changes α as training goes on – usually starting higher and shrinking it over time.

🌂 EVERYDAY PARALLEL

Parking a car. You approach the spot quickly (big steps), then slow right down for the final few centimetres so you don't bump the curb (small steps). Or think of cooling metal in blacksmithing: hot and mobile at first so it can reshape, then cooled slowly so it settles into a stable, precise form. Lowering the learning rate is that same "move fast, then finish gently."

◆ NOW THE MATH

Three schedules you'll meet constantly. **Step decay** multiplies the learning rate by a factor γ (gamma, e.g. 0.1) every fixed number of epochs (an *epoch* = one full pass over the training data):

$$\alpha_t = \alpha_0 \cdot \gamma^{\lfloor t/s \rfloor}$$

In words: start at α_0 ; every s epochs, drop the rate by the factor γ . The $\lfloor \cdot \rfloor$ (floor) just counts how many full drops have happened so far. Simple staircase: flat, then a sudden cut, then flat again.

Cosine decay lowers the rate smoothly along the shape of a cosine curve, from α_0 down to (near) zero over the total training length T :

$$\alpha_t = \frac{1}{2} \alpha_0 \left(1 + \cos \frac{\pi t}{T} \right)$$

In words: at the start ($t=0$) the cosine is 1, so $\alpha_t = \alpha_0$; at the end ($t=T$) the cosine is -1 , so $\alpha_t = 0$. In between it glides down a smooth S-shaped curve — no abrupt staircase steps. This is the most popular modern choice.

Warmup is the opposite move, used only at the very start: begin at a tiny learning rate and *raise* it over the first few hundred or thousand steps before letting a decay schedule take over. Why? Early on, the weights are freshly random and the gradient estimates are wild; a big step right away can throw the model into a bad region it never recovers from. Warmup lets the running averages (Adam's m and v) stabilise before you trust them with full-size steps.

◆ WHY TRANSFORMERS ALWAYS WARM UP

Large **transformer** models are famously touchy in their first few hundred steps — without warmup the loss often diverges immediately. So the standard recipe is **linear warmup then cosine (or inverse-square-root) decay**: ramp the learning rate up gently, then glide it back down. You'll see this exact schedule again in [Section 18](#); now you know what every line of it is doing and why.

◆ OPTIMIZER PLUS A SCHEDULER

> _ ADAM WITH COSINE DECAY — THE COMMON MODERN COMBO

```

import torch

model = torch.nn.Linear(64, 10)
opt = torch.optim.Adam(model.parameters(), lr=1e-3)

# cosine decay from 1e-3 down to ~0 over the whole run (T_max epochs)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=100)

for epoch in range(100):
    for x, y in train_loader:          # each minibatch
        opt.zero_grad()
        loss = loss_fn(model(x), y)
        loss.backward()
        opt.step()                     # update weights
    scheduler.step()                   # lower the LR once per epoch

# inspect the current learning rate at any time:
print(epoch, scheduler.get_last_lr()[0])

```

The rule of thumb: call `opt.step()` every minibatch, but call `scheduler.step()` once per epoch (unless the scheduler's docs say otherwise – a few per-step schedulers exist). The scheduler simply overwrites the `lr` that the optimizer reads on its next step.

CHECK YOURSELF

Your model trains fine with a fixed learning rate but the loss stalls and jitters in the last third of training, never quite settling. What single change is most likely to help, and why?

Reveal ▼

ANSWER

Add a **decay schedule** (step or cosine) so the learning rate shrinks later in training. The late-stage jitter is the optimizer taking steps too large to settle into the narrow minimum – it keeps overshooting by a little, like trying to park at full speed. Lowering α over time lets it inch into the bottom precisely. Cosine decay to near zero is the usual go-to, and it typically buys a small but real bump in final accuracy for free.

5 What to carry forward

- **Never initialize weights to all zeros** – identical neurons get identical gradients and stay clones forever (the symmetry problem). Use small random weights.
- Scale that randomness to **preserve variance** across layers: $\text{Var}(w) \approx 1/n_{\text{in}}$. Use **He** ($2/n_{\text{in}}$) with ReLU, **Xavier** ($2/(n_{\text{in}} + n_{\text{out}})$) with tanh/sigmoid.
- The **learning rate** is your most important knob: too high diverges, too low crawls. Tune it first; a good default is `1e-3` for Adam.
- **Optimizers** build up: SGD steps on a cheap noisy minibatch gradient; **momentum** adds an EMA of past gradients for direction and speed; **Adam** adds a per-parameter step size ($\sqrt{\hat{v}}$) plus bias correction.
- **Schedules** shrink the learning rate over time – step or cosine decay to settle precisely, with **warmup** at the start for touchy models like transformers.

You now have every moving part of the training loop: a model, a loss, gradients, a starting point, and a stepping rule. Next we move into PyTorch: first the tensor and autograd machinery (Section 13), then we assemble and train a real network and learn to read its diagnostics when it misbehaves (Sections 14–15).

Tensors & Autograd

Everything you hand-derived in Section 10 – the graph, the chain rule, the backward pass – is about to be built and run for you automatically. You'll re-derive a small graph by hand, then watch autograd reproduce your gradients to the decimal.

26 min read **You'll learn:** tensors · broadcasting · autograd · the computation graph

A **tensor** is PyTorch's one and only data structure – every input, weight, and output you will ever touch is one. Underneath it's refreshingly boring: a grid of numbers with a shape. What makes it interesting is that PyTorch can quietly track every operation you do to a tensor and, on request, hand you back the exact gradient – the thing you spent Section 10 computing by hand.

◆ THE ONE IDEA TO HOLD ONTO

A tensor is just an n-dimensional array. **Autograd** is the part of PyTorch that, as you do arithmetic on tensors, silently builds the same kind of computation graph you drew by hand in [Section 10](#) – then runs the same backward pass to fill in every gradient, automatically.

1 Tensors: arrays that can grow another dimension

◆ INTUITION FIRST

A **tensor** is a number, or a list of numbers, or a grid of numbers, or a stack of grids of numbers – the same idea, just allowed to keep growing dimensions. A single number is a *scalar* (rank 0). A list is a *vector* (rank 1, one you already know from [Section 1](#)). A grid is a *matrix* (rank 2). A stack of matrices is a rank-3 tensor, and so on – PyTorch doesn't care where you stop.

🔼 EVERYDAY PARALLEL

Think of a spreadsheet. One cell is a scalar. One row is a vector. The whole sheet is a matrix. Now imagine a *workbook* – several sheets stacked behind each other, all the same shape. That stack is a cube, a rank-3 tensor. Nothing new happened, you just kept stacking the same kind of thing one level deeper. That's all a tensor ever is.

◆ NOW THE MATH

Every tensor carries three pieces of information you should check whenever something breaks:

- **shape** – how many entries along each dimension, e.g. `(64, 3, 28, 28)` for 64 color images of 28×28 pixels.
- **dtype** – what kind of number is stored (`float32`, `int64`, ...). Mixing dtypes in one operation is a common source of errors.
- **device** – where the numbers physically live: the CPU's memory, or a GPU's memory. We'll come back to this at the end.

◆ THE SAME THING IN PYTORCH

>_ MAKING TENSORS AND READING THEIR SHAPE

```
import torch

scalar = torch.tensor(3.0)                # rank 0
vector = torch.tensor([1.0, 2.0, 3.0])    # rank 1, shape (3,)
matrix = torch.tensor([[1.0, 2.0],
                       [3.0, 4.0]])        # rank 2, shape (2, 2)
cube    = torch.zeros(2, 3, 4)            # rank 3, shape (2, 3, 4)

print(vector.shape, vector.dtype, vector.device)
# torch.Size([3]) torch.float32 cpu
```

CHECK YOURSELF

A batch of 32 grayscale images, each 28×28 pixels. What rank is the tensor, and what's a reasonable shape?

Reveal ▼

ANSWER

Rank 3, shape `(32, 28, 28)` – one dimension for "which image in the batch" (32), and two for the pixel grid (28, 28). If the images were color you'd add a channel dimension, giving rank 4: `(32, 3, 28, 28)`.

2 Broadcasting: stretching a small tensor to fit a big one

◆ INTUITION FIRST

Broadcasting is the rule PyTorch uses to let you combine tensors of different shapes without writing a loop. If one tensor is smaller along some dimension, PyTorch mentally "repeats" it along that dimension until the shapes match, then does the operation. No memory is actually copied – it just behaves as if it were.

📌 EVERYDAY PARALLEL

A spreadsheet of sale amounts, one row per transaction, and you want to apply a single 8% tax rate to every row. You don't write the rate 10,000 times – you write it once and the spreadsheet applies it to every row for you. Broadcasting is exactly that: one small tensor (the tax rate) automatically applied across every row of a bigger one (the transactions).

◆ NOW THE MATH

PyTorch compares two shapes from the *right-hand side*, dimension by dimension. Two dimensions are compatible if they are equal, or if one of them is 1 (a size-1 dimension gets stretched to match). If every pair is compatible, the shapes broadcast together:

$(4, 3)$ and $(3,)$ $\longrightarrow$ compatible: $(3,)$ acts like $(4, 3)$

In words: a matrix with 4 rows and 3 columns, combined with a plain vector of length 3, works because the vector's single dimension (3) matches the matrix's last dimension (3). PyTorch treats the vector as if it were copied into all 4 rows.

◆ THE SAME THING IN PYTORCH

>_ APPLYING ONE ROW TO EVERY ROW — THE TAX-RATE EXAMPLE

```
import torch

sales = torch.tensor([[100.0, 200.0, 300.0],  # 4 transactions,
                      [ 50.0,  75.0,  10.0],  # 3 line items each
                      [400.0, 100.0,  20.0],
                      [ 10.0,  10.0,  10.0]]) # shape (4, 3)

tax_rate = torch.tensor([0.08, 0.08, 0.08]) # shape (3,) – or just
0.08, a scalar

print(sales * tax_rate) # shape (4, 3): the rate applied to every row,
no loop
```

▲ THE BUG YOU WILL HIT FIRST

`RuntimeError: The size of tensor a (3) must match the size of tensor b (4) at non-singleton dimension 1`. Broadcasting only stretches dimensions that are exactly size 1 – it will never silently guess how to match a 3 against a 4. Print both `.shape`s and check which dimension refuses to line up.

CHECK YOURSELF

You have `x` of shape `(64, 10)` (a batch of 64 rows of 10 scores) and `b` of shape `(10,)` (a bias per column). Does `x + b` broadcast, and if so, to what shape? Reveal ▼

ANSWER

Yes. Comparing from the right: 10 vs 10 match, and the leftover dimension on `x` (64) simply stays. The result has shape `(64, 10)` – the same bias vector added to every one of the 64 rows. This is exactly how a linear layer's bias term $\mathbf{W}\mathbf{x} + \mathbf{b}$ gets applied to a whole batch at once.

3 Autograd: the payoff of Section 10

◆ INTUITION FIRST

In [Section 10](#) you built a computation graph by hand – nodes for each operation, arrows for how values flow forward, then walked the arrows backward applying the chain rule to get every gradient. **Autograd** is PyTorch doing exactly that same bookkeeping for you. Every time you do arithmetic on a tensor that has `requires_grad=True`, PyTorch records the operation as a node in a graph. Call `.backward()` and it walks that graph backward, the same direction you walked by hand, filling in `.grad` on every leaf.

🔧 EVERYDAY PARALLEL

Doing your own taxes by hand once teaches you exactly how each deduction moves the final number. Tax software then does the identical arithmetic for you every year after – same rules, same order of operations, just automated. Section 10 was doing your taxes by hand. Autograd is the software. It isn't a different, cleverer method – it's the same method, run for you.

DEFINITION

A tensor with `requires_grad=True` tells PyTorch "track every operation done to me, because I'll want a gradient with respect to this value later." Calling `.backward()` on a final scalar output triggers the backward pass; the result for each tracked tensor lands in its `.grad` attribute.

◆ THE SAME THING IN PYTORCH

>_ THE MINIMUM WORKING EXAMPLE

```
import torch

x = torch.tensor(3.0, requires_grad=True)
y = x ** 2 + 2 * x          # y = x^2 + 2x

y.backward()                # walk the graph backward from y
print(x.grad)               # tensor(8.) because dy/dx = 2x + 2 =
                             2(3)+2 = 8
```

Worked connection: a tiny graph by hand and by autograd — the same chain-rule walk as Section 10

Take a small expression in the same spirit as [Section 10](#)'s network – multiply two values, add a third, then multiply the result by a fourth. Different numbers from Section 10's sigmoid neuron, but the exact same backward procedure.

$$e = a \cdot b, \quad d = e + c, \quad L = d \cdot f$$

In words: three operations chained together, ending in a single number **L**. This is a tiny neural network in disguise: a multiply (like a weight times an input), an add (like a bias), then another multiply (like a second layer).

With `a = 2`, `b = -3`, `c = 10`, `f = -2`, the forward pass gives `e = -6`, `d = 4`, `L = -8`. Now apply the chain rule backward from **L**, exactly as in Section 10:

$$\frac{\partial L}{\partial d} = f = -2,$$

$$\frac{\partial L}{\partial e} = \frac{\partial L}{\partial d} \cdot \frac{\partial d}{\partial e} = -2 \cdot 1 = -2,$$

$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial e} \cdot \frac{\partial e}{\partial a} = -2 \cdot b = -2 \cdot (-3) = 6,$$

$$\frac{\partial L}{\partial f} = d = 4$$

$$\frac{\partial L}{\partial c} = \frac{\partial L}{\partial d} \cdot \frac{\partial d}{\partial c} = -2 \cdot 1 = -2$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial e} \cdot \frac{\partial e}{\partial b} = -2 \cdot a = -2 \cdot 2 = -4$$

In words: start at the output and multiply local derivatives backward along the graph, exactly the chain-rule walk from Section 10. Each variable's gradient is "how much did the final number move, per unit change in me" — **a**'s gradient of 6 means nudging **a** up by a tiny amount would push **L** up by about 6 times that amount.

Now build the identical graph in PyTorch and let autograd do the same walk:

```
import torch

a = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(-3.0, requires_grad=True)
c = torch.tensor(10.0, requires_grad=True)
f = torch.tensor(-2.0, requires_grad=True)

e = a * b
d = e + c
L = d * f

L.backward()

print(a.grad, b.grad, c.grad, f.grad)
# tensor(6.) tensor(-4.) tensor(-2.) tensor(4.) ← identical to the
# hand derivation above
```

Same numbers, to the decimal. That's the whole trick of autograd: it is not a different or approximate technique, it is the literal chain-rule walk from Section 10, executed by a machine that never drops a term.

CHECK YOURSELF

In the worked example, why is `c.grad` exactly equal to `e`'s upstream gradient, both -2 ?

Reveal ▼

ANSWER

Because `d = e + c`, and the local derivative of a sum with respect to either input is always 1 (`∂d/∂e = 1`, `∂d/∂c = 1`). The chain rule multiplies the upstream gradient by that local derivative, so both `e` and `c` simply inherit `d`'s gradient unchanged. Addition never rescales a gradient – it only ever copies it to both of its inputs.

4 Graph mechanics: leaves, detaching, and not tracking at all

◆ INTUITION FIRST

A few practical rules keep autograd from doing more (or less) work than you want. Knowing them is the difference between code that trains correctly and code that silently accumulates garbage or crashes with a confusing error.

DEFINITION

A **leaf tensor** is one you created directly (like `a` and `b` above) rather than one produced by an operation on other tracked tensors (like `e` or `d`). Only leaf tensors accumulate a `.grad` by default – the model's actual parameters (weights, biases) are always leaves, which is exactly what you want, since those are the numbers you'll update.

◆ NOW THE MATH

Two ways to stop autograd from tracking, for different reasons:

- `.detach()` – returns the same numbers as a new tensor that is no longer connected to the graph. Use it when you want a value (say, to log or plot) without dragging the whole

history of how it was computed along with it.

- `torch.no_grad()` – a context manager that turns off graph-building entirely for everything inside it. Use it for *inference* (making predictions, not training): there's no backward pass coming, so building the graph would only waste memory and time.

◆ THE SAME THING IN PYTORCH

```
import torch

w = torch.tensor(1.5, requires_grad=True)
x = torch.tensor(2.0)

y = w * x
print(y.requires_grad)      # True -- built from a tracked tensor

with torch.no_grad():
    y_infer = w * x          # no graph built at all -- for prediction,
    not training
    print(y_infer.requires_grad) # False

y_detached = y.detach()     # same value as y, graph history dropped
print(y_detached.requires_grad) # False
```

▲ GRADIENTS ACCUMULATE — ZERO THEM

Calling `.backward()` does not *overwrite* `.grad`, it **adds** to whatever was already there. Run the backward pass twice in a row without resetting, and you'll silently add two steps' worth of gradient together – a subtle bug that trains, just wrong. This is why every training loop calls something like `optimizer.zero_grad()` before each backward pass; we'll wire that into a full loop in [Section 14](#).

```
x = torch.tensor(2.0, requires_grad=True)

y = x ** 2
y.backward()
print(x.grad)      # tensor(4.)    dy/dx = 2x = 4

y = x ** 2
y.backward()
```

```
print(x.grad)      # tensor(8.)  NOT 4 again -- it accumulated: 4 + 4

x.grad.zero_()     # reset before the next backward pass
y = x ** 2
y.backward()
print(x.grad)      # tensor(4.)  correct, now that we cleared it first
```

CHECK YOURSELF

You call `loss.backward()` three times in a row on the same weights, forgetting to zero the gradient between calls, where each call would individually produce a gradient of 0.5. What ends up in `.grad`?

Reveal ▼

ANSWER

`1.5` – gradients accumulate by addition, so three backward passes of 0.5 each sum to `0.5 + 0.5 + 0.5 = 1.5`. If your training loss is doing something inexplicable, "forgot to call `zero_grad()`" is one of the first things to check.

5 What to carry forward

- A **tensor** is an n-dimensional array with a `shape`, `dtype`, and `device` – scalar, vector, matrix, or higher, all the same idea.
- **Broadcasting** stretches a smaller tensor across a bigger one when their shapes are compatible from the right, no loop required.
- **Autograd** builds the same kind of computation graph you built by hand in Section 10, and `.backward()` runs the exact same chain-rule walk – autograd matched the by-hand gradients above to the decimal.
- Only **leaf tensors** (like model parameters) accumulate `.grad`; use `.detach()` to drop history and `torch.no_grad()` for inference.

- **Gradients accumulate** across calls to `.backward()` – always zero them before the next backward pass.

Next we'll stop wiring graphs by hand entirely and let `nn.Module` organize the layers, the parameters, and the training loop itself.

nn.Module & the Training Loop

Six lines – `zero_grad`, `forward`, `loss`, `backward`, `step` – are the entire engine behind every PyTorch model you will ever train; once you can annotate every line, you can read anyone's code.

26 min read **You'll learn:** `nn.Module` · Dataset & DataLoader · the canonical training loop · GPU basics

You now know tensors and autograd cold: you can build a computation graph and get gradients out of it. `nn.Module` is nothing more than a tidy way to organize many such graphs – the weights, the layers, the forward computation – into a reusable object. Everything from here on is organization, not new math.

◆ THE ONE IDEA TO HOLD ONTO

`nn.Module` is a box that remembers its own parameters and knows how to compute its own output. The **training loop** is a short, fixed recipe that repeatedly feeds data through that box, measures how wrong it was, and nudges the parameters to be less wrong. Nothing in a PyTorch project is more important than these two ideas.

1 nn.Module: a container that tracks its own parts

◆ INTUITION FIRST

`nn.Module` is PyTorch's base class for "a thing that has learnable numbers and a way to use them." You subclass it, declare your layers in `__init__`, and write how data flows through them in `forward`. In return, PyTorch automatically finds every parameter you declared – no matter how deeply nested – so you never have to track them by hand.

🔧 EVERYDAY PARALLEL

Lego blocks. A single brick (one `nn.Linear` layer) does one small thing. Snap several bricks together in a fixed order and you get something bigger – a wall, a car, a model.

`nn.Module` is both the brick and the rule for snapping bricks together: a module can contain other modules, and the outside world just sees one bigger module.

◆ NOW THE MATH

A tiny MLP (multi-layer perceptron, a stack of linear layers with a nonlinearity between them) with one hidden layer computes:

$$\mathbf{h} = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1), \quad \hat{\mathbf{y}} = \mathbf{W}_2\mathbf{h} + \mathbf{b}_2$$

In words: transform the input $\mathbf{x}$ with the first layer's weights and bias, squash it through a nonlinearity σ (so the network can learn curves, not just straight lines), then transform that hidden result with a second layer to get the prediction $\hat{\mathbf{y}}$. Each $\mathbf{W}$ and $\mathbf{b}$ here is exactly the kind of leaf, learnable tensor from [Section 13](#).

◆ THE SAME THING IN PYTORCH

>_ A TWO-LAYER MLP AS AN NN.MODULE

```

import torch
import torch.nn as nn

class TinyMLP(nn.Module):
    def __init__(self, in_features, hidden, out_features):
        super().__init__() # required: wires up
PyTorch's bookkeeping
        self.layer1 = nn.Linear(in_features, hidden) # holds W1, b1 --
created and tracked for you
        self.act = nn.ReLU() # the nonlinearity
sigma
        self.layer2 = nn.Linear(hidden, out_features) # holds W2, b2

    def forward(self, x):
        h = self.act(self.layer1(x)) # h = sigma(W1 x + b1)
        return self.layer2(h) # y_hat = W2 h + b2

model = TinyMLP(in_features=784, hidden=64, out_features=10)

x = torch.randn(32, 784) # a batch of 32 flattened 28x28 images
y_hat = model(x) # calls forward() under the hood
print(y_hat.shape) # torch.Size([32, 10])

for name, p in model.named_parameters():
    print(name, p.shape) # layer1.weight, layer1.bias, layer2.weight,
layer2.bias -- found automatically

```

DEFINITION

Calling `model(x)` is what actually runs `forward` – never call `model.forward(x)` directly. The wrapper that `__call__` provides also handles some bookkeeping (like hooks) that you'd otherwise skip.

CHECK YOURSELF

Why must `self.layer1 = nn.Linear(...)` be assigned as an attribute inside `__init__`, rather than created fresh inside `forward` every call?

Reveal ▼

ANSWER

Because `nn.Module` discovers its parameters by scanning the attributes set in `__init__`. If you built a fresh `nn.Linear` inside `forward`, you'd get brand-new, randomly initialized weights on every single forward pass – nothing would ever be learned, since the "same" weights you trained a moment ago would already be gone. The weights have to persist as attributes of the module to be found by `.parameters()` and updated by the optimizer.

2 Dataset & DataLoader: feeding the model in bite-sized batches

◆ INTUITION FIRST

A **Dataset** is just an object that knows two things: how many examples it has, and how to fetch the n -th one. A **DataLoader** wraps it and handles the fiddly parts – grouping examples into minibatches, shuffling their order each epoch, and handing you one batch at a time in a simple loop.

🚂 EVERYDAY PARALLEL

A Dataset is a filing cabinet of index cards – ask for card #47 and you get it. A DataLoader is the assistant who grabs a random handful of cards at a time, hands you that stack, shuffles the cabinet's order overnight, and repeats. You never touch the cabinet directly; you just say "next stack, please."

Why bother with small stacks instead of processing everything at once? As covered in [Section 12](#), computing the exact gradient over the entire dataset before every single update is slow

and memory-hungry; a **minibatch** – a random, modest-sized slice of the data – gives a noisy but cheap-to-compute estimate of that gradient, and taking many small, noisy steps gets you to a good answer faster than taking few, perfectly accurate ones.

◆ THE SAME THING IN PYTORCH

```
import torch
from torch.utils.data import Dataset, DataLoader

class ToyDataset(Dataset):
    def __init__(self, X, y):
        self.X, self.y = X, y

    def __len__(self):
        return len(self.X)          # how many examples total

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx] # one example, by index

X = torch.randn(1000, 784)        # 1000 fake flattened images
y = torch.randint(0, 10, (1000,)) # 1000 fake labels, classes 0-9

dataset = ToyDataset(X, y)
loader = DataLoader(dataset, batch_size=32, shuffle=True)

for batch_x, batch_y in loader:
    print(batch_x.shape, batch_y.shape) # torch.Size([32, 784])
    torch.Size([32])
    break
```


CHECK YOURSELF

Why does the DataLoader use `shuffle=True` during training but you should use `shuffle=False` for a validation or test loader?

Reveal ▼

ANSWER

Shuffling during training stops the model from learning anything about the *order* examples arrive in (e.g. if all class-0 examples happened to be first) and gives each minibatch a fresh, independent random sample every epoch, which keeps gradient estimates unbiased. For validation or test, you're not updating any weights – you're just measuring performance – so order doesn't affect correctness, and leaving it fixed makes results reproducible and easier to compare run to run.

3 The canonical training loop

◆ INTUITION FIRST

Every PyTorch training script, no matter how large the model, boils down to the same handful of lines repeated over and over: look at a batch, guess, measure how wrong the guess was, work out how to improve, take a small step in that direction. Below is the whole loop with every line explained – memorize this shape, because you will type some version of it in every project you ever build.

🚂 EVERYDAY PARALLEL

Learning to throw darts: throw (forward pass), see how far you missed the bullseye (loss), figure out which way your arm was off and by how much (backward pass), adjust your stance a little (optimizer step), then throw again. Repeat hundreds of times, ideally against many different dartboards, not the same one over and over (that's why we loop over the whole dataset in batches, epoch after epoch).

◆ NOW THE MATH

Each step nudges every parameter θ in the model a small amount opposite its gradient, scaled by a learning rate α – the same gradient-descent update from [Section 12](#), just now computed by autograd instead of by hand:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L$$

In words: take each parameter, subtract a small step (learning rate α times its gradient), and that's the update. `optimizer.step()` below performs exactly this line for every parameter in the model, all at once.

◆ THE SAME THING IN PYTORCH, ANNOTATED

›_ THE SIX LINES THAT TRAIN (ALMOST) EVERY PYTORCH MODEL

```
import torch.nn as nn
import torch.optim as optim

model = TinyMLP(in_features=784, hidden=64, out_features=10)
loss_fn = nn.CrossEntropyLoss()          # for multi-class classification
-- combines softmax + log-loss
optimizer = optim.Adam(model.parameters(), lr=1e-3) # Adam: the adaptive
optimizer from Section 12

for epoch in range(10):                  # 1. one full pass over
    the data
        for batch_x, batch_y in loader:    # 2. one minibatch at a
            time

            optimizer.zero_grad()          # 3. clear old gradients
            -- see why below
            y_hat = model(batch_x)          # 4. forward pass: the
            model's current guess
            loss = loss_fn(y_hat, batch_y)  # 5. how wrong was the
            guess? one number
            loss.backward()                # 6. backward pass:
            fills in .grad for every parameter
            optimizer.step()               # 7. nudge every
            parameter opposite its gradient

    print(f"epoch {epoch}: loss = {loss.item():.4f}")
```

▲ WHY `ZERO_GRAD()` COMES FIRST

As you saw in [Section 13](#), calling `.backward()` adds to whatever gradient is already sitting in `.grad` — it never overwrites it. Without `optimizer.zero_grad()` at the top of the loop, each batch's gradient would pile on top of the last batch's, and the optimizer would take a step based on garbage. Clearing the gradients before every forward pass is what makes each step depend only on the current batch.

One line-by-line summary, since this shape repeats forever:

Line	What it does
<code>optimizer.zero_grad()</code>	Resets every parameter's <code>.grad</code> to zero before this batch.
<code>y_hat = model(batch_x)</code>	Forward pass — runs <code>forward()</code> , builds the computation graph.
<code>loss = loss_fn(y_hat, batch_y)</code>	Compares the guess to the true label, producing one scalar number to minimize.
<code>loss.backward()</code>	Backward pass — walks the graph, fills <code>.grad</code> on every parameter.
<code>optimizer.step()</code>	Uses each parameter's fresh <code>.grad</code> to update it (Adam's rule, building on plain gradient descent).

CHECK YOURSELF

What would go wrong, concretely, if you swapped the order and called `optimizer.step()` before `loss.backward()` ?

Reveal ▼

ANSWER

`optimizer.step()` reads whatever is currently sitting in each parameter's `.grad` and uses it to update the parameter. If you call it before `backward()` has run for this batch, `.grad` either holds gradients from a previous batch (if you forgot to zero them) or is empty/zero (if you did zero it), so the update would be based on stale or nonexistent information rather than this batch's actual loss. The order forward → loss → backward → step is not stylistic, it's required – each step's input is the previous step's output.

4 GPU basics: same code, different device

◆ INTUITION FIRST

A GPU (graphics processing unit) can do the same tensor arithmetic as a CPU, just with thousands of small cores working in parallel instead of a few fast ones – a huge speedup for the kind of bulk matrix multiplication neural networks do constantly. Moving your model to a GPU doesn't change any of the code above; it only changes *where* the numbers live.

🌂 EVERYDAY PARALLEL

A CPU is a few expert chefs who can each cook any dish. A GPU is a thousand line cooks who can each only chop one vegetable – individually limited, but if the job is "chop a thousand carrots," a thousand line cooks obliterate a few expert chefs on time. Matrix multiplication is exactly that kind of job: the same simple operation repeated an enormous number of times.

◆ THE ONE RULE THAT MATTERS

A tensor on the CPU and a tensor on the GPU cannot be combined in one operation – PyTorch will raise an error. The model's parameters and the data you feed it must always live on the **same device**. Move both, every time.

◆ THE SAME THING IN PYTORCH

```
import torch

device = "cuda" if torch.cuda.is_available() else "cpu"

model = TinyMLP(in_features=784, hidden=64, out_features=10)
model.to(device) # moves every parameter to the GPU (or
leaves it on CPU)

for batch_x, batch_y in loader:
    batch_x = batch_x.to(device) # data must move too, every batch
    batch_y = batch_y.to(device)

    optimizer.zero_grad()
    y_hat = model(batch_x) # now both model and data are on
'device' -- this works
    loss = loss_fn(y_hat, batch_y)
    loss.backward()
    optimizer.step()
```

▲ THE BUG YOU WILL HIT FIRST

`RuntimeError: Expected all tensors to be on the same device, but found at least two devices, cuda:0 and cpu!` . This means you moved the model but not the batch (or vice versa). The fix is always the same: find the tensor on the wrong device and add `.to(device)` .

CHECK YOURSELF

You call `model.to("cuda")` once before training, but inside the loop you forget to move `batch_x` and `batch_y`. What happens?

Reveal ▼

ANSWER

The forward pass raises a device-mismatch `RuntimeError` the moment the model's GPU-resident weights try to multiply against the CPU-resident batch. Moving the model once is not enough – every batch of data coming out of the `DataLoader` starts on the CPU by default and has to be moved explicitly, every single iteration.

5 What to carry forward

- **nn.Module** is a container: declare layers in `__init__`, define the computation in `forward`, and PyTorch finds every parameter for you.
- **Dataset** answers "how many, and give me item i "; **DataLoader** turns that into shuffled minibatches automatically.
- The **training loop** is always: `zero_grad` → forward → loss → `backward` → `step`. Zeroing first matters because gradients accumulate.
- **GPU basics**: `.to(device)` moves tensors; model and data must always be on the same device before you combine them.

Next we'll cover the debugging habits and shape-checking discipline that keep this loop from silently training the wrong thing.

Debugging & Good Habits

Almost every training bug announces itself the same handful of ways — learn to recognize them and a broken run stops being a mystery and becomes a five-minute fix.

22 min read

You'll learn: shape discipline · gradient checking · overfit-a-single-batch · logging & checkpointing

Training a neural network fails silently far more often than it fails loudly. The loss doesn't always crash with a red error — sometimes it just quietly refuses to go down, and nothing in the code looks wrong. This section is not theory. It's the exact sequence of checks a working engineer runs, in order, whenever a model isn't learning.

◆ THE ONE IDEA TO HOLD ONTO

Debugging a model is not different from debugging any other program — it's just that the failures are quieter. Make them loud on purpose: print shapes, check gradients by hand, and prove the model *can* learn something trivial before trusting it on something hard.

1 Shape discipline: print them everywhere

◆ INTUITION FIRST

The single most common bug in any deep learning codebase is a **shape mismatch** — a tensor coming out of one layer doesn't have the shape the next layer expects. We already met the rule that governs this in [Section 1](#): to multiply an $(m \times n)$ matrix by an $(n \times p)$ matrix, the inner numbers must match. Every time you write a new layer, model, or loss function, you are betting that this rule holds all the way through. Bet wrong, and PyTorch will tell you — but only if you read the error carefully.

🔧 EVERYDAY PARALLEL

It's like plumbing pipes of different diameters. If pipe A ends in a 2-inch opening and pipe B starts with a 3-inch fitting, water doesn't flow — no matter how good either pipe is on its own. A shape mismatch is exactly that: two perfectly correct layers that simply don't connect, because one hands over a different-sized "pipe" than the next one expects.

◆ READING A REAL SHAPE ERROR

Say you build a tiny two-layer network, but mistype the input size of the second layer:

```
import torch.nn as nn

layer1 = nn.Linear(784, 128)    # 784 pixels in, 128 features out
layer2 = nn.Linear(64, 10)     # BUG: should be nn.Linear(128, 10)

x = torch.randn(32, 784)      # a batch of 32 images
h = layer1(x)                  # h has shape (32, 128) -- correct so far
out = layer2(h)                # crashes
```

›_ WHAT PYTORCH ACTUALLY PRINTS

```
RuntimeError: mat1 and mat2 shapes cannot be multiplied (32x128 and 64x10)
```

Read it left to right: `mat1` is your data, shape (32, 128) — that's `h`, exactly as expected.

`mat2` is the weight matrix inside `layer2`, and its first number, 64, is wrong: it should have matched `h`'s last dimension, 128. The error message is telling you exactly which two tensors collided and what their shapes were at that moment — it is not a generic complaint, it's a diagnosis, if you read both numbers on each side.

▲ WATCH OUT

Don't guess at a fix by trial and error. Instead, add `print(x.shape)` after every line that changes a tensor's shape while you're debugging, then delete the prints once the model runs. Five seconds of printing beats twenty minutes of guessing.

CHECK YOURSELF

Using the buggy code above, what one-word change to

Reveal ▼

`layer2` fixes it, and why does 128 (not 64 or 10) belong there?

ANSWER

Change `nn.Linear(64, 10)` to `nn.Linear(128, 10)`. A

`nn.Linear(in_features, out_features)` layer's **first** number must match the last dimension of whatever tensor flows into it – here, `h` has shape (32, 128), so the incoming feature dimension is 128, regardless of what the previous layer's *output count* happened to be conceptually. The 10 stays as-is; it's the number of classes you want out, unrelated to the mismatch.

2 Gradient checking: does the math match reality

◆ INTUITION FIRST

Backpropagation (covered in [Section 10](#)) computes gradients *analytically* – by applying calculus rules symbolically. That's fast, but if you've written a custom loss or layer, it's easy to get one sign or one factor wrong. **Gradient checking** catches this by computing the same gradient a second, much slower way – by literally nudging one number and watching how much the output moves – and comparing the two answers.

🔼 EVERYDAY PARALLEL

You calculate a tip in your head (analytic: apply the 15% rule). Then, to double check, you actually add one dollar to the bill and see the total change by roughly the right amount (numeric: nudge and observe). If your mental math and the nudge-and-observe check disagree, trust the nudge — your mental shortcut has a bug in it.

◆ NOW THE MATH

The definition of a derivative already tells us how to check it: nudge the input by a tiny amount ϵ in both directions and see how the loss L moves.

$$\frac{\partial L}{\partial w} \approx \frac{L(w + \epsilon) - L(w - \epsilon)}{2\epsilon}$$

In words: nudge the weight w up by a tiny amount ϵ (epsilon) and evaluate the loss; nudge it down by the same tiny amount and evaluate again; the difference, divided by twice the nudge, is an estimate of the slope at that point — the numeric gradient. This uses *both* sides (a "centered" difference) because it cancels out more error than only nudging one way.

◆ THE SAME THING IN PYTORCH

```
import torch

def loss_fn(w):
    return (w ** 2).sum()  # a toy loss; true gradient is 2*w

w = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)

# 1) the analytic gradient, from autograd
loss = loss_fn(w)
loss.backward()
analytic = w.grad.clone()

# 2) the numeric gradient, one entry at a time
eps = 1e-4
numeric = torch.zeros_like(w)
for i in range(len(w)):
    w_plus = w.detach().clone(); w_plus[i] += eps
    w_minus = w.detach().clone(); w_minus[i] -= eps
    numeric[i] = (loss_fn(w_plus) - loss_fn(w_minus)) / (2 * eps)
```

```
print(analytic)    # tensor([2., 4., 6.])
print(numeric)    # tensor([2.0000, 4.0000, 6.0000]) -- matches, within
~eps^2
```

CHECK YOURSELF

You run this check and the analytic gradient is consistently exactly 1000 times larger than the numeric one, across every entry. What kind of bug does that pattern suggest?

Reveal ▼

ANSWER

A random, scattered mistake wouldn't produce a clean, uniform ratio like 1000x on every entry. A constant multiplicative gap points to a **scale** bug in the analytic path – e.g. the loss is being summed over a batch of 1000 elsewhere without ever averaging, or a manually derived gradient is missing (or double-applying) a constant factor from the chain rule. Look for a stray `.sum()` that should be `.mean()`, or a dropped/extra constant in a hand-written derivative.

3 The single best debugging test: overfit one batch

◆ INTUITION FIRST

Before you trust a model on your real dataset, ask it to do something much easier: memorize **one small batch** – say, 8 or 16 examples – with nothing held back. Train on that exact same batch for a few hundred steps. A working model and training loop should drive the loss down to nearly zero (or accuracy to 100%) on that tiny batch, because there is nothing to generalize; it just has to remember 16 answers.

🔼 EVERYDAY PARALLEL

You don't test a new oven by baking 200 cookies at once. You bake *one* cookie first. If that one cookie comes out burnt or raw, you don't have a "cookie problem," you have an **oven problem** – and you'd rather find that out with one cookie than with 200.

Overfitting a single batch is baking the one test cookie before you commit the whole tray.

◆ WHY THIS TEST IS SO POWERFUL

If the model **cannot** drive the loss near zero on one tiny, memorizable batch, the bug is almost never about the model being "not powerful enough" or "needing more data." It's a real bug: a wrong loss function, a detached computation graph (gradients silently not flowing), mismatched or shuffled labels, or a learning rate that's absurdly wrong. Find it here, on one batch, in seconds – not three hours into a real training run.

◆ THE SAME THING IN PYTORCH

```
model = MyModel()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
criterion = torch.nn.CrossEntropyLoss()

xb, yb = next(iter(train_loader))  # grab ONE batch and reuse it every
step

for step in range(300):
    optimizer.zero_grad()
    out = model(xb)
    loss = criterion(out, yb)
    loss.backward()
    optimizer.step()
    if step % 50 == 0:
        print(step, loss.item())

# healthy run: loss falls toward ~0 well before step 300
# broken run: loss plateaus at a stubbornly high value -- go find the bug
```

CHECK YOURSELF

You run the overfit-one-batch test and the loss drops for a bit, then plateaus at a high value and never gets close to zero, even after 500 steps. Name three concrete things to check. Reveal ▼

ANSWER

(1) Are the labels actually aligned with the inputs – did shuffling or a data pipeline bug scramble `xb` and `yb` relative to each other? (2) Is the loss function right for the task (e.g. not using mean-squared-error on raw classification logits, or using the wrong reduction) and is gradient actually flowing – no accidental `.detach()` or a stray `.item()` / `.data` access that cuts the graph? (3) Is the learning rate reasonable – plateauing rather than diverging can mean the learning rate is too *small* to make progress in the number of steps you gave it.

4 Logging and reading loss curves

◆ INTUITION FIRST

Once the single-batch test passes, log the loss (and any accuracy metric) every epoch, for both the training set and a held-out validation set, and plot them. The shape of that curve is a diagnosis, not just a number.

🌂 EVERYDAY PARALLEL

A doctor doesn't just look at your temperature once – they look at the *trend* across the day. A fever that's climbing, one that's flat and high, and one that's falling all mean different things and call for different responses. A loss curve is a vital-signs chart for your training run.

What you see	What it usually means
Loss stays high, barely moves	Something is fundamentally broken — re-run the overfit-one-batch test if you haven't; also suspect a learning rate that's far too small.
Loss goes up, or to NaN	Diverging. Almost always the learning rate is too high, or an unstable operation (e.g. <code>log(0)</code> , exploding gradients) is involved. Try lowering the learning rate or adding gradient clipping.
Training loss keeps falling, validation loss flattens or rises	A growing gap between train and validation performance — this is overfitting: the model is memorizing the training set instead of learning general patterns.
Both curves fall together and flatten near the same value	Healthy. This is what a well-behaved training run looks like.

>_ MINIMAL LOGGING LOOP

```

history = {"train_loss": [], "val_loss": []}

for epoch in range(num_epochs):
    model.train()
    train_loss = run_one_epoch(model, train_loader, optimizer)  # your
                                                                training step

    model.eval()
    with torch.no_grad():
        val_loss = run_one_epoch(model, val_loader, optimizer=None)  # no
                                                                update, just measure

    history["train_loss"].append(train_loss)
    history["val_loss"].append(val_loss)
    print(f"epoch {epoch}: train={train_loss:.4f}  val={val_loss:.4f}")

```

5 Checkpointing, and train() vs eval()

◆ INTUITION FIRST

A **checkpoint** is a saved snapshot of your model's learned numbers, so a crashed run — or a finished one — doesn't cost you the training you already did. Saving is only half the story,

though: when you load a model back for evaluation, you must also tell it to *behave differently* than it did during training, using `model.eval()`.

🔦 EVERYDAY PARALLEL

Think of a restaurant kitchen during a chaotic dinner rush versus a calm plating for a food photographer. During the rush (training), the kitchen improvises, substitutes ingredients, and varies technique run to run. For the photo (evaluation), everyone follows the exact same, fixed, repeatable steps every time. Same kitchen, same recipe – deliberately different mode.

◆ SAVING AND LOADING

```
torch.save(model.state_dict(), "checkpoint.pt")

model = MyModel()
model.load_state_dict(torch.load("checkpoint.pt"))
```

Some layers – dropout and batch normalization, covered in [Section 20](#) – behave differently while training than while evaluating. Dropout randomly zeroes units only during training; batch norm uses statistics from the current batch while training, but a running average while evaluating. `model.train()` and `model.eval()` are how you tell PyTorch which mode those layers should use – they do not touch the weights, only the *behavior* of a few specific layer types.

```
model.train()                # dropout active; batchnorm uses batch
stats
for xb, yb in train_loader:
    ...                       # normal training step

model.eval()                  # dropout off; batchnorm uses running stats
with torch.no_grad():         # also skip building the graph -- we won't
    call backward()
    preds = model(x_val)
```

CHECK YOURSELF

You load a checkpoint and run inference, but predictions are noticeably worse and slightly different every time you re-run the exact same input. What did you most likely forget?

[Reveal ▼](#)**ANSWER**

`model.eval()`. Without it, the model is still in training mode: dropout is randomly zeroing a fraction of units on every call (which explains why the same input gives slightly different outputs each time), and batch normalization is using statistics from whatever small batch you happened to pass in, rather than the stable running averages collected during training. Both effects degrade and randomize predictions in exactly this way.

6 The usual suspects: a checklist

When a model won't train, walk this list before anything more exotic:

- **Forgot `optimizer.zero_grad()`**. Gradients accumulate (add up) across calls to `backward()` by default. Skip the reset and each step's gradient is contaminated by every previous step's.
- **Forgot `model.eval()` at evaluation time**. Dropout and batch norm keep training-time behavior, quietly corrupting validation/test numbers.
- **Learning rate too high**. Shows up as a diverging or `NaN` loss; the fix is almost always to lower it (or warm it up gradually).
- **Wrong loss reduction**. Mixing up `sum` and `mean` reduction silently rescales your effective learning rate by the batch size.
- **Data on the wrong device**. Mixing a CPU tensor with a GPU model (or vice versa) raises a clear error, but forgetting to move a *new* tensor you created mid-forward pass is an easy miss – watch for `RuntimeError: Expected all tensors to be on the same device`.

7 What to carry forward

- Print shapes constantly. Most bugs are a shape mismatch, and PyTorch's error message tells you exactly which two tensors collided if you read both sides.
- **Gradient checking** compares the analytic gradient (from `backward()`) against a numeric one from nudging weights by ϵ – use it whenever you write a custom loss or layer.
- **Overfit one small batch first.** If the loss won't go near zero on a batch of 16, the bug is in your code, not your data or your model's capacity.
- Log train and validation curves every epoch; a growing gap between them means overfitting, a flat high loss means something is broken, a rising loss means the learning rate is too high.
- Checkpoint with `state_dict()`, and always pair loading a model with the right `model.train()` / `model.eval()` call.

Next we put these habits to work on real architectures, starting with convolutional networks for images in [Section 16](#).

Convolutional Networks

A convolution is a dot product wearing a sliding-window costume — see that clearly, and you can derive the shape of every layer in a CNN by hand.

30 min read

You'll learn: convolution as an operation · output-shape math · pooling · ResNets & skip connections

Every image is a grid of numbers, so in principle you could feed it into the fully-connected layers from [Section 1](#) — flatten the grid into one long vector and multiply by a big matrix. In practice this is a bad idea, for reasons that turn out to be exactly the reasons convolutional networks exist. Once you see why fully-connected layers struggle with images, the convolution operation stops looking like a new trick and starts looking like the obvious fix.

◆ THE ONE IDEA TO HOLD ONTO

A convolution takes one small, reusable pattern-detector (a **kernel**) and slides it over every position in the input, asking the same question everywhere: “*how much of my pattern is here?*” Reusing the same detector everywhere is what makes CNNs both cheap (few parameters) and smart about images (a cat is a cat wherever it appears).

1 Why fully-connected layers struggle with images

◆ INTUITION FIRST

A fully-connected layer gives every input pixel its own private weight, for every output neuron. That has two consequences, and both are bad for images. First, images have a lot of pixels, so the weight matrix becomes enormous. Second — and this is the subtler problem — the layer has no idea that “a cat's ear near the top of the photo” and “a cat's ear near the bottom of the photo” are the *same feature*. It learns them as two unrelated facts, because they land on two different input slots with two different weights.

🔦 EVERYDAY PARALLEL

Imagine teaching someone to spot a stop sign by pointing at one photo and saying “the stop sign is the red octagon in the upper-left corner of *this exact photo*.” Show them a second photo where the sign has moved to the bottom-right, and they're lost – they never learned “stop sign,” they learned “red shape at one specific pixel location.” A fully-connected layer learns the same way: a separate weight per position, with nothing forcing it to reuse what it learned about a shape when that shape shows up somewhere else.

◆ NOW THE MATH

Take a small colour image: 224×224 pixels, 3 colour channels (red, green, blue). Flattened into a vector, that's:

$$224 \times 224 \times 3 = 150,528 \text{ numbers}$$

In words: one single image is already a vector with over 150,000 entries. Connect that vector to a modestly sized hidden layer of just 1,000 neurons with a fully-connected layer, and the weight matrix **W** needs shape (150,528 × 1,000) — over **150 million** numbers to learn, for one layer, before the network has looked at a single example. And every one of those weights is tied to one exact pixel position.

Convolution fixes both problems with one move: instead of a giant matrix that touches every pixel once, use a *tiny* matrix (the kernel) that touches every pixel *many times*, once per position it slides to. The next section makes that precise.

CHECK YOURSELF

Why does a fully-connected layer need to learn "cat ear near top" and "cat ear near bottom" as two separate things, when a human clearly doesn't?

Reveal ▼

ANSWER

Because a fully-connected layer's weights are indexed by input position: the weight connecting "pixel (10, 10)" to a given output neuron is a completely different number from the weight connecting "pixel (200, 200)" to that same neuron. Nothing in the layer says these two weights should be related, so the network must independently discover the "ear" pattern at every position it might appear – wasting both parameters and training data. A mechanism that reuses the same small detector at every position (a convolution) sidesteps this entirely.

2 Convolution: a sliding dot product

◆ INTUITION FIRST

A **kernel** (also called a **filter**) is a small grid of numbers – say 3×3 . To apply it, you place it in the top-left corner of the input, multiply it elementwise against the patch of input underneath it, and add up the products into a single number. That's exactly a **dot product** (Section 1), just computed on a 2D patch instead of a flat list. Then you slide the kernel one step over and repeat, and again, until you've swept the whole input. The result is a new grid – one number per position – telling you how strongly that pattern showed up there.

🔑 EVERYDAY PARALLEL

Picture searching a photo by sliding a small stencil or rubber stamp over it, checking at every position how well the stamp's shape lines up with what's underneath. Move the stencil one inch right, check again. Slide it down a row, check the whole row again. You never redraw the stencil – it's the same small shape, reused at every spot – you're just asking "does this pattern match *here*?" over and over across the whole page.

◆ NOW THE MATH

Three numbers control how a convolution slides:

DEFINITIONS

The **kernel size K** is the width (and height) of the small sliding grid of weights. The **stride S** is how many pixels the kernel jumps between positions (stride 1 = check every pixel; stride 2 = skip every other one). **Padding P** is a border of zeros added around the input so the kernel can also center itself near the edges, and so you can control how much the output shrinks.

Let's derive how large the output grid comes out, for an input of width W (assume a square input and kernel for simplicity – the same formula applies to height). First, padding adds P zeros on each side, so the effective width becomes $W + 2P$. The kernel's left edge can start at position 0, then S , then $2S$, and so on – the last legal starting position is the largest one where the kernel still fits inside the (padded) input:

$$\begin{aligned}\text{start} + K &\leq W + 2P \\ \text{start} &\leq W + 2P - K\end{aligned}$$

In words: the kernel's starting position, plus its own width K , can't run past the padded edge. That caps how far right the kernel is allowed to start.

The starting positions are $0, S, 2S, \dots$ up to that cap, so the *count* of valid positions – which is exactly the output size – is that cap divided by the stride, plus one for the starting position itself:

$$\text{out} = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

In words: take the input width, subtract the kernel width, add back twice the padding, divide by the stride, round down ($\lfloor \rfloor$ means "round down" — you can't have a fractional position), then add 1 to count the first position. This one formula tells you the exact output shape of any convolution before you run it.

›_ WORKED EXAMPLE

An input of width $W = 7$, kernel $K = 3$, stride $S = 2$, no padding ($P = 0$):

$$\text{out} = \left\lfloor \frac{7 - 3 + 0}{2} \right\rfloor + 1 = \lfloor 2 \rfloor + 1 = 3$$

In words: the kernel fits at starting positions 0, 2, and 4 (covering pixels 0-2, 2-4, and 4-6) — three positions, three output values, matching the formula.

A second, very common case: keeping the output the *same size* as the input (“same” padding), with **W = 5, K = 3, S = 1, P = 1**:

$$\text{out} = \left\lfloor \frac{5 - 3 + 2}{1} \right\rfloor + 1 = 4 + 1 = 5$$

In words: with a 3×3 kernel, padding of 1 on each side exactly cancels the 2-pixel shrinkage a 3-wide kernel would otherwise cause, so the output stays 5×5. This is why you'll so often see “kernel 3, padding 1” paired together in real architectures.

Channels, feature maps, and parameter sharing

A colour image doesn't have one grid of numbers – it has three (red, green, blue), called *channels*. A kernel for such an input isn't 3×3, it's 3×3×3: it spans the full depth of the input and produces one number per position by summing over all three channels at once. One kernel produces one output grid, called a *feature map* – a map of “how much did my pattern fire, at every position.” A convolutional layer normally learns many kernels in parallel (say, 64 of them), producing 64 stacked feature maps – which then become the 64 input channels for the next layer.

◆ PARAMETER SHARING

The crucial trick: the *same* kernel weights are reused at every position in the image. This single choice buys two things at once. It gives **translation invariance** – a pattern the kernel learned to detect in one corner is automatically detected anywhere else, since it's the same weights doing the checking. And it slashes the parameter count: a 3×3 kernel over 3 input channels producing 64 output feature maps needs $3 \times 3 \times 3 \times 64 = 1,728$ weights – regardless of whether the image is 32×32 or 3200×3200. Compare that to the 150-million-weight fully-connected layer from the start of this section: convolution is doing more useful work with four orders of magnitude fewer numbers.

◆ THE SAME THING IN PYTORCH

>_ NN.CONV2D AND THE SHAPE FORMULA IN ACTION

```
import torch
import torch.nn as nn

# batch of 8 RGB images, 32x32 pixels: shape (batch, channels, height, width)
x = torch.randn(8, 3, 32, 32)

conv = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, stride=1, padding=1)
y = conv(x)
print(y.shape) # torch.Size([8, 16, 32, 32])
# height/width: floor((32 - 3 + 2*1)/1) + 1 = 32 → "same" padding, confirmed
# channels: 16, one feature map per learned filter

print(conv.weight.shape) # torch.Size([16, 3, 3, 3]) → (out_ch, in_ch, K, K)
print(sum(p.numel() for p in conv.parameters())) # 16*3*3*3 + 16 bias = 448

# a downsampling conv: stride 2, no padding
down = nn.Conv2d(3, 16, kernel_size=3, stride=2, padding=0)
print(down(x).shape) # torch.Size([8, 16, 15, 15])
# floor((32 - 3 + 0)/2) + 1 = floor(29/2) + 1 = 14 + 1 = 15
```

CHECK YOURSELF

An input feature map is 28×28 . You apply a convolution with kernel size 5, stride 1, and padding 2. What is the output size, and how many weights does one such filter have if the input has 8 channels?

Reveal ▼

ANSWER

Output size: $\text{floor}((28 - 5 + 2 \cdot 2)/1) + 1 = \text{floor}(27/1) + 1 = 27 + 1 = 28$ – padding of 2 with a 5-wide kernel again gives "same" padding, so the output stays 28×28 . One filter spanning 8 input channels has $5 \times 5 \times 8 = 200$ weights (plus 1 bias), no matter how large the 28×28 map is – that independence from spatial size is the whole point of parameter sharing.

3 Pooling: shrinking on purpose

◆ INTUITION FIRST

After a few convolutions you often want to shrink the feature maps deliberately – fewer numbers to carry forward, and a little tolerance for the exact pattern shifting by a pixel or two. **Pooling** does this with no learned weights at all: it slides a small window over the input and, at each position, keeps just one summary number.

🔑 EVERYDAY PARALLEL

Think of a company town-hall: instead of forwarding every employee's individual comment up the chain, each team submits a single summary. **Max pooling** is a team forwarding its loudest, strongest opinion (the maximum value in the window) – great for asking “did this pattern show up anywhere in this neighbourhood?” **Average pooling** is a team forwarding its average sentiment (the mean of the window) – a smoother, less spiky summary.

◆ NOW THE MATH

Pooling uses the exact same sliding mechanics as convolution – and the exact same output-size formula from earlier – but with kernel size **K** and stride **S** usually set equal (so windows don't overlap), and no weights: the operation itself, not learned numbers, decides the output.

$$\text{out} = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

In words: same formula as convolution's output size, because pooling is structurally the same sliding-window operation — only the thing computed inside each window (max, or mean) differs.

◆ THE SAME THING IN PYTORCH

```
import torch
import torch.nn as nn

x = torch.randn(8, 16, 32, 32)

pool = nn.MaxPool2d(kernel_size=2, stride=2)
y = pool(x)
print(y.shape)  # torch.Size([8, 16, 16, 16])
# floor((32 - 2 + 0)/2) + 1 = 15 + 1 = 16, applied to height and width
```

CHECK YOURSELF

Why does 2×2 max pooling with stride 2 give a small amount of tolerance to the input shifting by one pixel, while a plain matrix multiply wouldn't?

Reveal ▼

ANSWER

Inside each 2×2 window, max pooling only cares which value is largest, not exactly where it sits within that window. So if the whole feature map shifts by a pixel, a strong activation that used to be, say, top-left of its window is now top-right of the same window – and max pooling still reports the same maximum. A fully-connected or purely positional operation has no such slack: every input slot maps to a fixed, distinct weight, so a one-pixel shift changes which weights get multiplied by what, and the output changes too.

4 ResNets and skip connections

◆ INTUITION FIRST

Once researchers had convolutions, the obvious next move was to stack more of them — deeper networks should be strictly more powerful, since a deep network can always just copy the input through unchanged if extra layers aren't helping. But in practice, plain stacks of convolutions got *harder* to train past a certain depth, and not because of overfitting — even *training* accuracy got worse. The culprit is the same one from [Section 1](#)'s eigenvalue discussion and [Section 12](#): gradients have to flow backward through every layer, and if each layer shrinks the gradient a little, dozens of layers shrink it to essentially zero — the **vanishing gradient** problem. Early layers stop learning because barely any error signal reaches them.

🌂 EVERYDAY PARALLEL

Picture a long road with a set of traffic lights at every block. If each light only lets 80% of cars through, by block 50 almost no car from the start of the road has made it to the end — not because the road is broken, just because the effect compounds. Now add an express lane that runs alongside the whole road, letting some traffic skip every light entirely and arrive untouched. Even if the light-by-light lane is badly congested, the express lane guarantees traffic still gets through. A skip connection is that express lane, for gradients.

◆ NOW THE MATH

A plain layer computes some function of its input, $F(x)$, and passes only that forward. A **residual block** instead computes:

$$y = x + F(x)$$

In words: $F(x)$ (a couple of convolutions) still gets computed and still gets used, but it's *added to* the original input x rather than replacing it. If F learns nothing useful, $F(x) \approx 0$ and the block simply passes x through untouched — the identity is always available as a fallback, which is exactly the “deeper should never hurt” property plain stacks failed to get in practice.

Here's why this fixes the gradient problem. During backpropagation (see [Section 12](#)) we need the derivative of the output y with respect to the input x , by the chain rule:

$$\frac{\partial y}{\partial x} = \frac{\partial x}{\partial x} + \frac{\partial F(x)}{\partial x} = I + \frac{\partial F}{\partial x}$$

In words: the gradient splits into two paths, added together: an **identity** path (**I**, exactly 1 — the derivative of **x** with respect to itself) and whatever the convolutions inside **F** contribute. Stack many residual blocks and the gradient through the whole stack is a product of terms that each look like $(I + \partial F / \partial x)$. Even if every $\partial F / \partial x$ is tiny or poorly scaled, the “**1 +**” guarantees each factor stays close to 1 instead of collapsing toward 0 — so the gradient signal reaches early layers largely intact. This is the same shrink-or-explode story as repeated matrix multiplication in [Section 1](#)'s eigenvalue example and the recurrent networks in [Section 17](#); the skip connection is a structural fix rather than a tuning fix.

◆ THE SAME THING IN PYTORCH

>_ A MINIMAL RESIDUAL BLOCK

```
import torch
import torch.nn as nn

class ResidualBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.conv1 = nn.Conv2d(channels, channels, kernel_size=3,
padding=1)
        self.conv2 = nn.Conv2d(channels, channels, kernel_size=3,
padding=1)
        self.relu = nn.ReLU()

    def forward(self, x):
        out = self.relu(self.conv1(x))
        out = self.conv2(out)
        return self.relu(x + out)    # the skip connection: x + F(x)

block = ResidualBlock(channels=16)
x = torch.randn(8, 16, 32, 32)
y = block(x)
print(y.shape)    # torch.Size([8, 16, 32, 32]) → padding=1, kernel=3
keeps size unchanged
```

CHECK YOURSELF

In a residual block $y = x + F(x)$, suppose training has driven F 's weights to nearly zero everywhere (the block hasn't found anything useful to add yet). What does the block output, and what is the gradient flowing back through it?

Reveal ▼

ANSWER

The output is $y \approx x + 0 = x$: the block behaves like an identity function, passing its input through essentially unchanged – harmless, not broken. The gradient is $\partial y / \partial x = I + \partial F / \partial x \approx I$: with F contributing almost nothing, the gradient passes through at essentially full strength (multiplied by 1), rather than being crushed by whatever a plain (non-residual) layer would have done. This is exactly why very deep ResNets can train at all: untrained or unhelpful blocks default to "do nothing" instead of "actively destroy the signal."

5 What to carry forward

- Fully-connected layers give images too many parameters and no reuse of learned patterns across positions; convolution fixes both.
- A **convolution** slides a small kernel over the input, computing a dot product at each position; output size follows $\lfloor (W - K + 2P) / S \rfloor + 1$ – derive it, don't memorize it.
- **Parameter sharing** (the same kernel reused everywhere) gives translation invariance and cuts parameter counts by orders of magnitude versus a fully-connected layer.
- **Pooling** downsamples with no learned weights, using the same sliding-window formula, and adds a little shift tolerance.
- **Skip connections** ($y = x + F(x)$) keep gradients flowing in very deep networks because the identity path contributes a "+1" that a shrinking $\partial F / \partial x$ can't cancel out.

Convolutions assume a grid with local structure – great for images, but text and audio have a different structure: order along a single sequence. Next we'll see how networks handle that.

Sequence Models: RNNs & LSTMs

A recurrent network reads one step at a time and carries a memory forward — and the very thing that makes that possible is also what makes it so hard to train over long stretches.

24 min read

You'll learn: RNNs & hidden state · unrolling · vanishing/exploding gradients · LSTMs · why we moved on

Convolutional networks in [Section 16](#) assume their input is a grid where nearby pixels matter, but position order isn't inherently meaningful in any particular direction. Text, audio, and time series are different: they're **sequences**, where *order* carries the meaning. “Dog bites man” and “man bites dog” use the same words in a different order to say something completely different. Recurrent networks are the first architecture built specifically to respect that.

◆ THE ONE IDEA TO HOLD ONTO

A recurrent network reads a sequence one element at a time and keeps a running **hidden state** — a summary of everything seen so far — updating it at every step. That single reused update rule, applied over and over, is both the source of its power and, as you'll see, the source of its biggest weakness.

1 RNNs: a hidden state that carries memory forward

◆ INTUITION FIRST

A **recurrent neural network** (RNN) processes a sequence step by step — one word, one audio sample, one time-tick at a time. At each step it looks at the current input *and* a running summary of everything it has seen before (the **hidden state**), and produces an updated summary to carry into the next step.

🔼 EVERYDAY PARALLEL

Think of reading a sentence word by word, out loud, to someone who can't see the page. By the time you reach the word “bank” in “I walked along the river bank,” you already have a running sense of the sentence's meaning from the words before it – that running sense is what lets you interpret “bank” as a riverbank, not a place to deposit money. You don't re-read the whole sentence from scratch at every word; you carry a summary forward and update it. That carried summary is the hidden state.

◆ NOW THE MATH

At each time step t , the network takes the current input $\mathbf{x}_t$ and the previous hidden state $\mathbf{h}_{t-1}$, and produces the new hidden state $\mathbf{h}_t$:

$$\mathbf{h}_t = \tanh(\mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b})$$

In words: $\mathbf{W}$ is a matrix that transforms the current input (exactly the $\mathbf{W}\mathbf{x}$ from [Section 1](#)); $\mathbf{U}$ is a matrix that transforms the previous hidden state, folding in what's been remembered so far; $\mathbf{b}$ is a bias vector; and $\tanh$ squashes the result into a bounded range so the hidden state can't grow without limit. Crucially, the *same* $\mathbf{W}$, $\mathbf{U}$, and $\mathbf{b}$ are reused at every single time step — one small set of weights processes a sequence of any length, just like one convolution kernel is reused at every position in [Section 16](#).

Unrolling and backprop-through-time

To picture what's happening – and to train the network – it helps to draw the RNN “unrolled”: one copy of the same update rule per time step, laid out left to right, each feeding its hidden state into the next.

$$x_1 \rightarrow h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow \cdots \rightarrow h_T$$

In words: it looks like a very deep feedforward network — except every copy shares the exact same weights $\mathbf{W}$ and $\mathbf{U}$. Training uses ordinary backpropagation (see [Section 12](#)) applied to this unrolled picture, which is why it's called **backpropagation through time**: the error at the last step has to flow backward through every earlier time step, through the same weight matrix, again and again, to reach the beginning of the sequence.

◆ THE SAME THING IN PYTORCH

```
import torch
import torch.nn as nn

rnn = nn.RNN(input_size=8, hidden_size=16, batch_first=True)

x = torch.randn(4, 10, 8)          # (batch=4, sequence length=10,
features=8)
h0 = torch.zeros(1, 4, 16)         # initial hidden state, all zeros

outputs, h_last = rnn(x, h0)
print(outputs.shape)    # torch.Size([4, 10, 16]) → hidden state at every
one of the 10 steps
print(h_last.shape)     # torch.Size([1, 4, 16]) → just the final hidden
state
```

CHECK YOURSELF

Why can the same RNN, with a fixed number of parameters, process both a 5-word sentence and a 500-word sentence?

Reveal ▼

ANSWER

Because the parameters (W , U , b) don't describe the whole sequence – they describe a single *update step*, applied once per token regardless of how many tokens there are. A 500-word sentence just runs that same small update 500 times instead of 5, reusing identical weights each time. This is the sequence analogue of a convolution kernel being reused at every pixel position rather than needing one new kernel per image size.

2 The failure mode: vanishing and exploding gradients

◆ INTUITION FIRST

Here's the problem with reusing the same weight matrix at every step: backpropagation through time has to multiply by that *same* matrix once per time step, all the way back to the

start of the sequence. Multiplying by the same thing over and over doesn't stay neutral — it compounds, the same way repeatedly applying a stretch or a shrink compounds.

🚂 EVERYDAY PARALLEL

Whisper a sentence down a long line of people, and by the tenth person it's already garbled — small distortions compound with every relay. Now imagine instead that each person is instructed to say everything 5% louder than they heard it: after twenty people, the message is a deafening roar. Repeated small changes, applied over and over along a long chain, either fade to nothing or blow up — there's rarely a stable middle.

◆ NOW THE MATH

Recall from [Section 1](#) that a matrix's **eigenvalues** are the stretch factors along its favoured directions, and that applying the same matrix repeatedly raises those stretch factors to a power. Backpropagation through time does exactly this to the recurrent weight matrix $\mathbf{U}$: the gradient flowing back $\mathbf{T}$ steps involves a term that looks roughly like $\mathbf{U}$ multiplied by itself $\mathbf{T}$ times.

$$\frac{\partial h_T}{\partial h_0} \approx \prod_{t=1}^T U^\top \cdot (\text{local terms})$$

In words: the gradient connecting the last hidden state back to the first has to pass through $\mathbf{T}$ copies of (roughly) the same matrix, multiplied together. If $\mathbf{U}$'s dominant eigenvalue is less than 1, that product shrinks toward zero as $\mathbf{T}$ grows — the **vanishing gradient** problem, identical in spirit to the eigenvalue example in [Section 1](#) and the depth problem in [Section 16](#). If it's greater than 1, the product blows up — the **exploding gradient** problem instead. Either way, the network's early time steps get almost no usable training signal, which means it struggles to learn any dependency that spans a long stretch of the sequence (see [Section 12](#) for how this plays out during training).

▲ WATCH OUT

This is exactly why plain RNNs are bad at tasks like “does the subject introduced at the start of a long paragraph agree with the verb at the end?” — by the time the gradient (or, at inference time, the hidden state itself) has been pushed through dozens of update steps, information about that early subject has often faded to nearly nothing.

CHECK YOURSELF

If the recurrent weight matrix's dominant eigenvalue is 0.9, roughly what fraction of the original gradient survives after 50 time steps, and what does that imply about learning long-range dependencies?

Reveal ▼

ANSWER

$0.9^{50} \approx 0.005$ – about half a percent of the original signal survives. Any dependency that requires connecting something 50 steps back to the current step gets a gradient signal roughly 200 times weaker than a dependency just one step back, so the network effectively can't learn it: the training signal needed to strengthen that long-range connection is drowned out before it arrives. This is the concrete reason plain RNNs are said to have a "short memory" in practice, even though the hidden state is, in principle, carrying information forward from every step.

3 LSTMs: gates that choose what to remember

◆ INTUITION FIRST

The **LSTM** (Long Short-Term Memory) is an RNN variant designed directly against the vanishing-gradient problem. Instead of squashing everything through one **tanh** update every step, it keeps a separate **memory cell** and uses three learned **gates** – small networks that output a number between 0 and 1 – to control, at every step, what gets forgotten, what gets added, and what gets read out.

🔼 EVERYDAY PARALLEL

Picture keeping a notebook while reading a long report. At every page you make three deliberate decisions: what old notes to *cross out* because they're no longer relevant (the **forget gate**), what new fact to *write down* (the **input gate**), and how much of what's in the notebook to actually *read back out* right now to use (the **output gate**). The notebook itself – the memory cell – persists across pages largely untouched except where you deliberately edit it, instead of being rewritten from scratch every single page.

DEFINITION

A **gate**, in an LSTM, is a small learned layer (a sigmoid, so its output is always between 0 and 1) that acts as a per-element “volume knob”: near 0 blocks information from passing, near 1 lets it through unchanged. The forget, input, and output gates are three independent such knobs, each learned to serve a different job.

We won't derive the full LSTM update here – it's three gates plus a candidate memory update, and the algebra is more bookkeeping than new insight. The one thing worth carrying forward: the memory cell is updated additively (old memory times the forget gate, *plus* new information times the input gate), which is structurally similar to the $x + F(x)$ skip connection from [Section 16](#) – an additive path that lets gradients flow back through many steps without being forced through a repeated multiplication every time. This is what eases (though doesn't fully solve) the vanishing-gradient problem from the section above.

◆ THE SAME THING IN PYTORCH

```
import torch
import torch.nn as nn

lstm = nn.LSTM(input_size=8, hidden_size=16, batch_first=True)

x = torch.randn(4, 10, 8)          # (batch=4, sequence length=10,
features=8)
h0 = torch.zeros(1, 4, 16)
c0 = torch.zeros(1, 4, 16)         # LSTMs carry a memory cell alongside
the hidden state

outputs, (h_last, c_last) = lstm(x, (h0, c0))
```

```
print(outputs.shape) # torch.Size([4, 10, 16])
print(c_last.shape)  # torch.Size([1, 4, 16]) → the memory cell,
                    separate from the hidden state
```

CHECK YOURSELF

A plain RNN updates its entire hidden state by squashing it through tanh every step. Why does an LSTM's additive memory-cell update ease the vanishing-gradient problem better than that?

Reveal ▼

ANSWER

A plain RNN's hidden state is *rewritten* at every step by passing it through a matrix multiply and a squashing nonlinearity – the gradient backward through that step always has to go through that multiplication, which compounds over many steps exactly as shown above. An LSTM's memory cell is updated *additively* (forget-gated old memory plus input-gated new information), so there's a path for the gradient that doesn't require repeated multiplication by the same matrix – much like the identity path in a residual connection. The gates still have to learn good values, so LSTMs don't eliminate the problem, but the additive path gives gradients a much easier route over long sequences than a plain RNN offers.

4 Why we moved on: the case for attention

◆ INTUITION FIRST

LSTMs eased the gradient problem, but they left a different limitation untouched: an RNN, by construction, must process a sequence **one step at a time** – step 50 needs the hidden state from step 49, which needs step 48, all the way back to the start. You cannot compute step 50 before step 49 is done.

📌 EVERYDAY PARALLEL

It's the difference between one person reading a book aloud, page by page, versus handing every page to a different reader in a room and letting them all read simultaneously, then comparing notes. The second approach only works if readers can directly reference any other page whenever they need to – not just whatever the previous reader happened to pass along.

◆ NOW THE MATH

This sequential dependency has a very concrete cost: on modern hardware (GPUs), which are built to do enormous numbers of independent calculations *at once*, an RNN can't use that parallelism across the time dimension – step 50's calculation must wait for step 49's to finish, no matter how many idle processors are sitting around. Training on long sequences becomes slow simply from this forced serial order, on top of the fact that even LSTMs still degrade on very long-range dependencies.

◆ THE OPENING FOR ATTENTION

What if, instead of forcing information to pass through every intermediate step to get from position 3 to position 300, a model could let position 300 look directly at position 3 – no relay required, and no inherent obstacle to computing many positions at once? That's the idea [Section 18](#) builds on: attention mechanisms and the transformer architecture, which replace the sequential hidden state with a way for every position to directly attend to every other position.

5 What to carry forward

- An **RNN** reads a sequence step by step, keeping a **hidden state** that's updated by the same reused weights ($\mathbf{h}_t = \tanh(\mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b})$) at every step.
- **Unrolling** turns the recurrence into a deep chain of identical steps; training it is backpropagation through time.

- Because the same weight matrix is applied repeatedly, gradients over long sequences **vanish or explode** depending on that matrix's eigenvalues – the same phenomenon as Section 1's eigenvalues and Section 16's deep-network gradient problem.
- **LSTMs** add gates and an additive memory cell to ease (not eliminate) that problem, at the cost of more parameters and computation per step.
- RNNs (and LSTMs) are inherently **sequential** – they can't be parallelized across time steps – which, together with their still-limited long-range memory, is exactly what motivates attention and transformers next.

Next, we drop the recurrence entirely and see how a model can look at an entire sequence at once.

Attention & Transformers

The architecture behind every modern LLM – derived, not diagrammed, with every matrix shape spelled out. By the end you could rebuild the core of GPT from memory.

42 min read

You'll learn: scaled dot-product attention (from scratch) · multi-head attention · positional encodings · the full transformer block, dimension by dimension

Every large language model you have heard of – GPT, Claude, Llama, Gemini – is a tall stack of one repeated part: the **transformer block**. And the beating heart of that block is a single operation called **attention**. If you truly understand attention, the rest of modern AI stops being magic and becomes engineering. So we are going to build it here from nothing but the dot product you already know, and we will write down the shape of every single matrix as we go – because in this subject, the shapes *are* the understanding.

◆ THE ONE IDEA TO HOLD ONTO

Attention lets **every position in a sequence look directly at every other position** and pull in the information it needs – all at once, in parallel. That single capability is what replaced the slow, forgetful recurrent networks of [Section 17](#) and unlocked models with billions of parameters. Everything below is the detail of how that lookup is computed.

1 Why we needed something new

◆ INTUITION FIRST

In [Section 17](#) we met **recurrent neural networks** (RNNs): models that read a sentence one word at a time, carrying a running summary – a *hidden state* (a fixed-size memory vector) –

from each word to the next. They work, but they fight two problems at once.

- **Long-range forgetting.** To connect the last word of a paragraph to the first, the information must survive being squeezed through every step in between. It fades – the vanishing-gradient problem we previewed in [Section 1](#).
- **No parallelism.** Word 50 cannot be processed until word 49 is done, because it needs the hidden state coming out of word 49. Reading is strictly sequential, so it cannot be spread across the thousands of cores in a modern GPU. Training is slow.

We want the impossible-sounding combination: **direct connections between far-apart words** (so nothing has to survive a long relay) **and** the ability to **process every word at the same time** (so training is fast). Attention delivers both, and it does so with an idea that turns out to be surprisingly simple.

EVERYDAY PARALLEL

Imagine reading a mystery novel two ways. The **RNN way**: you may only keep a single sticky-note of memory, and you must rewrite it after every sentence – by the last chapter the early clues are smudged beyond reading. The **attention way**: the whole book lies open on a huge table, and to understand any sentence you may glance directly at any other sentence, however far back, instantly. No relay, no fading, and you can have a room full of readers each studying a different sentence at the same moment.

CHECK YOURSELF

In the sentence “The trophy didn't fit in the suitcase because it was too big,” what does “it” refer to, and why is that hard for an RNN?

Reveal ▼

ANSWER

“It” means the **trophy** (change “big” to “small” and it flips to the suitcase – the meaning depends on distant words). To resolve it, the model must link “it” back to “trophy,” which sits many words earlier. An RNN can only reach that word through the whole chain of hidden states in between, where the signal weakens. Attention lets “it” look *straight at* “trophy” in one hop, no matter the distance. This exact example is the classic test of long-range reference.

2 Scaled dot-product attention, from scratch

◆ INTUITION FIRST

Here is the whole mechanism in one picture, and it is a picture of a **lookup**. Every position asks a question, every position advertises what it is about, and every position offers some content. A position collects its answer by taking a *blend* of everyone's content, weighted by how well their advertisement matches its question. That is attention. Three roles do all the work, and they have names:

- The **query (q)** – what this position is looking for. (“I'm a pronoun; I need the noun I refer to.”)
- The **key (k)** – a label each position advertises about itself. (“I'm a concrete singular noun.”)
- The **value (v)** – the actual content a position hands over once it is selected. (“Here is my meaning: *trophy*.”)

📌 EVERYDAY PARALLEL

A library search. Your **query** is the phrase you type into the catalogue. Each book on the shelf has a spine label – its **key** – and its actual pages – its **value**. A normal computer picks the single book whose label matches best and hands you exactly that one. Attention is the *soft* version: instead of one winner, it returns a smooth blend – 70% of this book's pages, 20% of that one's, 10% of a third – weighted by how well each spine label matched what you asked for. It is a **soft dictionary lookup**: a dictionary returns one value for an exact key, attention returns a weighted mixture of all values, graded by key similarity.

◆ NOW THE MATH

We start from an input we already have: a sequence of vectors, one per position (per word), stacked into a matrix. Call it $\mathbf{X}$.

$$\mathbf{X} \in \mathbb{R}^{T \times d_{\text{model}}}$$

In words: $\mathbf{X}$ has one row per position. T is the *sequence length* — how many words (or tokens) we are processing. d_{model} is the *model dimension* — the length of the vector we use to represent each word (in the original transformer, 512). So $\mathbf{X}$ is T rows, each row a d_{model} -long vector.

Now the one trick: we do not use $\mathbf{X}$ directly as query, key, and value. We *project* it into three different spaces with three learned weight matrices. This lets the model decide, on its own, what makes a good question, a good label, and good content — they need not be the same.

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V$$

Track every shape with the shape rule from [Section 1](#) (inner dimensions cancel, outer survive):

Object	Shape	What it is
$\mathbf{W}_Q$	$d_{\text{model}} \times d_k$	learned: how to build a query
$\mathbf{W}_K$	$d_{\text{model}} \times d_k$	learned: how to build a key
$\mathbf{W}_V$	$d_{\text{model}} \times d_v$	learned: how to build a value
$\mathbf{Q} = \mathbf{XW}_Q$	$T \times d_k$	one query per position
$\mathbf{K} = \mathbf{XW}_K$	$T \times d_k$	one key per position
$\mathbf{V} = \mathbf{XW}_V$	$T \times d_v$	one value per position

In words: d_k is the length of each query and key (they must match, because we are about to compare them); d_v is the length of each value. Query and key live in the same space so they can be measured against each other; the value can be any size. Very often $d_k = d_v$, and we'll assume that from now on to keep the picture clean.

Step 1 — scores: how well does each query match each key?

To compare query i with key j we use the **dot product** from [Section 1](#) — our alignment meter. A big dot product means “this query and this key point the same way; strong match.” We want *every* query against *every* key, and that is exactly one matrix multiply:

$$\mathbf{S} = \mathbf{QK}^\top \quad \text{shape } (T \times d_k)(d_k \times T) = (T \times T)$$

In words: $\mathbf{K}^\top$ is $\mathbf{K}$ transposed (rows and columns swapped), so the shared d_k lines up and cancels. Entry S_{ij} is the dot product of query i with key j — a single number saying “how much should position i attend to position j .” The result is a square $L \times L$ table: every position scored against every position. This one matrix is where the “look at everything at once” promise is kept.

Step 2 — the scaling, and why it is not optional

We do not use $\mathbf{S}$ raw. We divide it by $\sqrt{d_k}$. This looks like a fudge factor; it is actually forced on us by statistics. Let us derive it, because it is the kind of detail this course refuses to hand-wave.

Suppose the entries of a query $\mathbf{q}$ and a key $\mathbf{k}$ are independent random numbers, each with mean 0 and variance 1 (a fair starting assumption for freshly initialised weights). Their dot product is a sum of d_k products:

$$\mathbf{q} \cdot \mathbf{k} = \sum_{i=1}^{d_k} q_i k_i$$

Take one term $q_i k_i$. Because q_i and k_i are independent with mean 0:

$$\begin{aligned}\mathbb{E}[q_i k_i] &= \mathbb{E}[q_i] \mathbb{E}[k_i] = 0 \cdot 0 = 0, \\ \text{Var}(q_i k_i) &= \mathbb{E}[q_i^2 k_i^2] - (\mathbb{E}[q_i k_i])^2 = \mathbb{E}[q_i^2] \mathbb{E}[k_i^2] - 0 = 1 \cdot 1 = 1.\end{aligned}$$

In words: each little product has average 0 and variance 1. (We used $\mathbb{E}[q_i^2] = \text{Var}(q_i) = 1$, since the mean is 0.) $\mathbb{E}$ is “expected value” — the long-run average. **Var** is variance — how spread out the values are.

Now add up d_k of these independent terms. Variances of independent things simply add:

$$\text{Var}(\mathbf{q} \cdot \mathbf{k}) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = d_k \quad \implies \quad \text{typical size} \approx \sqrt{d_k}.$$

In words: the raw score has variance d_k , so its typical magnitude grows like $\sqrt{d_k}$. With $d_k = 64$ the scores swing by roughly ± 8 instead of ± 1 — eight times too hot.

Why does an oversized score hurt? Because the next step is a **softmax** (defined below), and softmax of large numbers *saturates*: one entry rockets to nearly 1, the rest collapse to nearly 0. At that extreme the function is almost flat, so its **gradient** (the slope the network learns from — see [Section 2](#)) is almost zero. A zero gradient means *no learning signal*. Dividing by $\sqrt{d_k}$ rescales the variance back to 1, keeping scores in the gentle, high-gradient middle of the softmax:

$$\text{Var}\left(\frac{\mathbf{q} \cdot \mathbf{k}}{\sqrt{d_k}}\right) = \frac{1}{d_k} \text{Var}(\mathbf{q} \cdot \mathbf{k}) = \frac{d_k}{d_k} = 1.$$

In words: dividing a random quantity by a constant divides its variance by that constant *squared*. So dividing by $\sqrt{d_k}$ divides the variance by d_k , pulling it from d_k back to 1. That is the entire reason for the “scaled” in *scaled* dot-product attention.

▲ WATCH OUT

Drop the $\sqrt{d_k}$ and small models still train – just worse. But scale up $\mathbf{d}_k$ and the unscaled version quietly stops learning: the softmax locks into a near-one-hot spike from the first step, the gradient vanishes, and the loss plateaus. It looks like a mysterious optimisation failure; it is really this missing division.

Step 3 — softmax: turn scores into a blend that sums to 1

The scaled scores are still arbitrary real numbers – some negative, none adding up to anything tidy. We want *weights*: all positive, summing to 1, so they describe a genuine blend. The **softmax** function does exactly that, applied to each row independently.

DEFINITION

The **softmax** of a list of numbers exponentiates each (making it positive and amplifying the largest), then divides by the total so the results sum to 1:

$\text{softmax}(z)_j = \frac{e^{z_j}}{\sum_m e^{z_m}}$. It turns any list of scores into a set of probabilities. We met it in [Section 4](#); here it converts match-scores into attention weights.

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \quad \text{shape } (T \times T), \text{ each row sums to 1.}$$

In words: row i of $\mathbf{A}$ is position i 's attention distribution — how much of its answer it will draw from each of the T positions. “Softmax *over each row*” matters: we normalise across the keys a single query is comparing itself to, so each query spends a total attention budget of exactly 1.

Step 4 — the weighted blend of values

Finally, each position builds its output by mixing all the values with its attention weights. Another matrix multiply:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \underbrace{\text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)}_{\mathbf{A} : (L \times T)} \underbrace{\mathbf{V}}_{(L \times d_v)} \quad \text{shape } (T \times d_v).$$

In words: multiply the $L \times L$ weight table by the $L \times d_v$ value stack; the shared T cancels and out comes an $T \times d_v$ result — one new vector per position. Row i of the output is $\sum_j A_{ij} \mathbf{v}_j$: a weighted average of every value, using position i 's weights. Positions it matched strongly contribute most; the rest barely register.

◆ THE WHOLE FORMULA IN ONE LINE

$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}$. Read left to right it is: *compare every query to every key ($\mathbf{Q}\mathbf{K}^\top$), calm the numbers ($\div \sqrt{d_k}$), turn them into a blend (softmax), then mix the values by that blend ($\times \mathbf{V}$).* Four steps, one line, and it is the engine of the entire field.

A concrete walk-through: $\text{seq} = 3$, $d_k = 4$

Let us make it real with tiny numbers. Three positions ($L = 3$), query/key/value length four ($d_k = d_v = 4$). Then:

- $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ are each 3×4 – three rows, four numbers per row.
- $\mathbf{Q}\mathbf{K}^\top$ is $(3 \times 4)(4 \times 3) = 3 \times 3$ – the shared 4 cancels. Nine scores: every position against every position.
- Divide all nine by $\sqrt{4} = 2$.
- Softmax each of the 3 rows $\rightarrow$ a 3×3 table whose rows each sum to 1. Say position 1's row comes out $[0.7, 0.2, 0.1]$: it will take 70% of its answer from position 1, 20% from position 2, 10% from position 3.
- Multiply $(3 \times 3)(3 \times 4) = 3 \times 4$. The shared 3 cancels; out come three new 4-long vectors – one refreshed representation per position. Position 1's output is $0.7 \mathbf{v}_1 + 0.2 \mathbf{v}_2 + 0.1 \mathbf{v}_3$.

Notice the output has the *same shape* as the value input (3×4). Attention refreshes each position's vector by letting it gather context from the others; it does not change how many positions there are. That shape-preserving property is what lets us stack these blocks dozens deep.

◆ THE SAME THING IN PYTORCH

›_ SCALED DOT-PRODUCT ATTENTION — THE FOUR STEPS, VERBATIM

```

import torch
import torch.nn.functional as F

def scaled_dot_product_attention(Q, K, V):
    # Q, K, V: [..., seq, d_k] (the leading dims can be batch, heads,
    # etc.)
    d_k = Q.size(-1) # length of each
    # query/key
    scores = Q @ K.transpose(-2, -1) # [..., seq, seq]
    # QKT: every query · every key
    scores = scores / d_k ** 0.5 # [..., seq, seq]
    # the √d_k scaling
    weights = F.softmax(scores, dim=-1) # [..., seq, seq]
    # each ROW sums to 1
    return weights @ V # [..., seq, d_v]
    # blend the values

# tiny sanity check: seq = 3, d_k = 4
Q = torch.randn(3, 4)
K = torch.randn(3, 4)
V = torch.randn(3, 4)
out = scaled_dot_product_attention(Q, K, V)
print(out.shape) # torch.Size([3,
4]) → same as V

```

In words: `transpose(-2, -1)` swaps the last two axes to form K^T . `dim=-1` makes softmax normalise across keys (the last axis), which is exactly “over each row.” The leading `...` is deliberate — the same function will handle batches and heads untouched in a moment.

CHECK YOURSELF

If a query's scaled scores against three keys are `[5, 5, 5]` (a perfect tie), what attention weights does softmax give, and what is the resulting output?

Reveal ▼

ANSWER

Softmax of three equal numbers is uniform: `[1/3, 1/3, 1/3]` (the constant cancels top and bottom). The output is the plain average of the three values, $\frac{1}{3}(\mathbf{v}_1 + \mathbf{v}_2 + \mathbf{v}_3)$. This is the honest behaviour when a query has no reason to prefer anyone: it listens to everyone equally. Attention only becomes selective when the scores actually differ – which is why the learned $\mathbf{W}_Q$, $\mathbf{W}_K$ matter so much.

3 Multi-head attention: several lookups at once

◆ INTUITION FIRST

One attention operation produces one blend per position – one kind of relationship. But a word relates to its neighbours in many ways at once: grammar (which noun does this verb agree with?), position (what came just before?), meaning (what does this pronoun refer to?). Asking a *single* query to capture all of those at once is asking too much. So we run several attentions in parallel – each called a **head** – each with its own $\mathbf{W}_Q$, $\mathbf{W}_K$, $\mathbf{W}_V$, and let each specialise in a different kind of relationship. Then we stitch their answers back together.

☂ EVERYDAY PARALLEL

A panel of experts reviewing the same job application. The grammar expert checks one thing, the experience expert another, the culture-fit expert a third. Each reads the *same* application but attends to different parts of it. Afterwards a manager collects all their notes and merges them into one decision. Multi-head attention is that panel: several specialists looking at the same sentence through different lenses, then a final matrix combining their findings.

◆ NOW THE MATH

We fix a total budget d_{model} and split it across h heads. Each head gets a slice of size:

$$d_k = \frac{d_{\text{model}}}{h}$$

In words: with $d_{\text{model}} = 512$ and $h = 8$ heads, each head works in a small $d_k = 64$ -dimensional space. Eight cheap 64-wide lookups cost about the same as one expensive 512-wide lookup — we get variety *for free*, not at extra cost.

For each head $i = 1 \dots h$ we run ordinary scaled dot-product attention in its slice:

$$\text{head}_i = \text{Attention}(\mathbf{X}\mathbf{W}_Q^{(i)}, \mathbf{X}\mathbf{W}_K^{(i)}, \mathbf{X}\mathbf{W}_V^{(i)}) \quad \text{shape } (T \times d_k).$$

Each head's own projection matrices $\mathbf{W}_Q^{(i)}, \mathbf{W}_K^{(i)}, \mathbf{W}_V^{(i)}$ have shape $d_{\text{model}} \times d_k$. So each head outputs an $T \times d_k$ matrix. We lay all h of them side by side (**concatenate** — glue along the width):

$$\text{concat}(\text{head}_1, \dots, \text{head}_h) \quad \text{shape } (T \times h d_k) = (T \times d_{\text{model}}).$$

In words: because $h \cdot d_k = d_{\text{model}}$ by construction, gluing the heads back together restores the full width. We are exactly where we started, size-wise — but the vector now carries h different views blended in.

One last step. The concatenated heads sit in separate lanes; nothing has yet let a discovery in head 3 talk to a discovery in head 7. A final learned matrix $\mathbf{W}_O$ mixes them:

$$\text{MultiHead}(\mathbf{X}) = \text{concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}_O, \quad \mathbf{W}_O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}.$$

In words: $\mathbf{W}_O$ takes the stacked-up head outputs and blends them into one coherent d_{model} -wide vector per position. It is the “manager” in the panel analogy — the layer that turns separate expert notes into a single decision. Final output shape: $(T \times d_{\text{model}})$, same as the input, so the block stacks cleanly.

◆ WHY HEADS EARN THEIR KEEP

When researchers inspect trained transformers, different heads really do specialise: some track the previous word, some link verbs to their subjects, some connect a pronoun to the noun it refers to. No one programmed these roles – the split into heads simply gave the model *room* to discover several relationship types instead of averaging them all into one muddy signal.

CHECK YOURSELF

A model has `d_model = 768` and `h = 12` heads. What is `d_k`? And what shape is each head's output for a sequence of length 100? Reveal ▼

ANSWER

$d_k = 768/12 = 64$. Each head runs attention in a 64-wide space and outputs shape $(100, 64)$. Concatenating all 12 heads gives $(100, 12 \times 64) = (100, 768)$, and $\mathbf{W}_O$ of shape $(768, 768)$ maps that back to $(100, 768)$. These are the actual numbers inside BERT-base and GPT-2 small (two landmark early transformers) – you just sized a real model.

4 Positional encodings: attention can't tell order

◆ INTUITION FIRST

Attention has a hidden flaw, and it is a big one. Look again at how a position builds its output: $\sum_j A_{ij} \mathbf{v}_j$ – a sum over the other positions. Addition doesn't care about order. Shuffle the input words and you get the same set of outputs, just shuffled the same way. Attention treats a sentence as a *bag* of words. But “dog bites man” and “man bites dog” are very different sentences. We must hand the model some sense of position, because the mechanism itself has none.

🔼 EVERYDAY PARALLEL

Hand a chef a basket of loose ingredients with no recipe steps and no order – flour, eggs, sugar, heat. They can see *what* they have but not *in what sequence* to use it, and sequence is the whole difference between a cake and a mess. Positional encodings are the little numbered stickers we put on each ingredient – “I’m step 1, I’m step 2” – so order is no longer lost.

◆ NOW THE MATH — FIRST, THE FLAW, PRECISELY

Let $\mathbf{P}$ be a **permutation matrix** – an identity matrix with its rows shuffled – so $\mathbf{P}\mathbf{X}$ reorders the input positions. Because attention computes $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ row by row and then softmax-blends, feeding it $\mathbf{P}\mathbf{X}$ gives:

$$\text{Attention}(\mathbf{P}\mathbf{X}) = \mathbf{P} \text{Attention}(\mathbf{X}).$$

In words: reorder the inputs and you get *exactly* the same outputs, in the reordered order – nothing else changes. This property is called *permutation invariance* (a *permutation* is just a reshuffle). It is perfect for a set of items with no natural order, and disastrous for language, where order is meaning. The fix is not to change attention – it is to make each word's vector *already contain* its position before attention ever sees it.

So we build a position vector $\mathbf{PE}_{\text{pos}}$ for each slot $\text{pos} = 0, 1, 2, \dots$ and simply add it to the word's vector. The original transformer uses a fixed pattern of sines and cosines at many frequencies:

$$\begin{aligned}\mathbf{PE}_{(\text{pos}, 2i)} &= \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right), \\ \mathbf{PE}_{(\text{pos}, 2i+1)} &= \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right).\end{aligned}$$

In words: pos is which slot in the sentence (0, 1, 2, ...); i indexes the dimension inside the vector. Even dimensions use a sine, odd dimensions the matching cosine. The $10000^{2i/d_{\text{model}}}$ in the denominator makes the wavelength stretch as i grows: early dimensions wiggle fast, later dimensions wiggle slowly.

◆ WHY SINES AT MANY FREQUENCIES

Think of a bank of clocks with different hand speeds – seconds, minutes, hours, days. Read all their hands together and you can pin down a unique moment in time. The fast hand distinguishes nearby moments; the slow hand distinguishes far-apart ones. The sinusoidal encoding is exactly this: each pair of dimensions is a clock at its own speed, and a position's full set of hand-readings is a fingerprint no other position shares. A bonus falls out of the trig: the code for position $\text{pos} + k$ is a fixed linear function of the code for pos , so the model can learn to reason about *relative* offsets like “three words back.”

A common alternative is the **learned positional embedding**: instead of the fixed sine formula, keep a lookup table with one trainable vector per position and let training decide what each should be. GPT and BERT use this. It is simpler and works as well over the lengths seen in training; the sinusoidal version has the edge that it extends smoothly to positions longer than any seen during training, since the formula is defined for every pos . Either way the shape is the same ($T \times d_{\text{model}}$) and the move is the same: add it to the input.

◆ THE SAME THING IN PYTORCH

›_ SINUSOIDAL POSITIONAL ENCODINGS, ADDED TO THE INPUT

```

import torch, math

def positional_encoding(seq_len, d_model):
    pos = torch.arange(seq_len).unsqueeze(1)           # [L, 1] slot
    index 0,1,2,...
    i     = torch.arange(0, d_model, 2)               # [d/2] even
    dimension indices
    freq = 1.0 / (10000 ** (i / d_model))              # [d/2] one
    frequency per dim-pair
    pe = torch.zeros(seq_len, d_model)                # [L, d_model]
    pe[:, 0::2] = torch.sin(pos * freq)                # even dims →
    sine
    pe[:, 1::2] = torch.cos(pos * freq)                # odd  dims →
    cosine
    return pe                                          # [L, d_model]

X = torch.randn(10, 512)                             # 10 word
vectors, d_model = 512
X = X + positional_encoding(10, 512)                  # [10, 512]
position now baked in
print(X.shape)                                         #
torch.Size([10, 512])

```

In words: we add, not concatenate — the position rides along inside the existing d_{model} width, so nothing downstream changes shape. After this line, two identical words in different slots have different vectors, and permutation invariance is broken exactly where we wanted it broken.

CHECK YOURSELF

Without positional encodings, would a transformer give different outputs for “dog bites man” and “man bites dog”?

Reveal ▼

ANSWER

No – and that is the whole problem. With the same three word-vectors in a different order, permutation invariance guarantees the *same set* of output vectors, merely reordered. The model literally could not tell who bit whom. Adding positional encodings gives “dog” in slot 0 a different vector from “dog” in slot 2, so the two sentences finally produce genuinely different representations. Position is not a nice-to-have here; it is what makes language modelling possible at all.

5 The full transformer block, dimension by dimension

◆ INTUITION FIRST

We now have the two moving parts. **Multi-head attention** lets positions share information across the sequence – the “mixing across words” step. But it is mostly a weighted average; it does no heavy per-word computation on its own. So each block pairs it with a **feed-forward network** (FFN) – a small two-layer network applied to each position separately – that does the “think hard about this one word” step. Wrap both in two safety devices – **residual connections** and **LayerNorm** – and you have the complete transformer block. Stack it N times and you have a transformer.

🔼 EVERYDAY PARALLEL

A team working a problem in rounds. Each round has two phases: a **meeting**, where everyone shares what they know and updates their view (that's attention – mixing information between people), and **desk work**, where each person goes off alone and processes what they just heard (that's the FFN – per-position computation). Do many such rounds and a rich shared understanding builds up. A transformer is that team; the block is one round.

◆ NOW THE MATH — THE TWO SUPPORT DEVICES

Residual connections. Instead of replacing the input with a sub-layer's output, we *add* the output back to the input: $\mathbf{x} + \text{Sublayer}(\mathbf{x})$. This is the same skip-connection trick that made very deep image networks trainable in [Section 16](#). Two reasons it matters here: it gives each sub-layer an easy “do nothing” option (output zero and the input passes straight through), and it hands the gradient a clean shortcut straight back through the addition – so signal reaches the earliest layers without fading. Shapes must match for the add, which is why every sub-layer preserves d_{model} .

LayerNorm. Before each sub-layer we *normalise* each position's vector: subtract its mean, divide by its standard deviation, then rescale with two learned numbers. This keeps the numbers flowing through the network at a steady scale – not exploding, not vanishing – which is the difference between training that converges and training that diverges. We treat it properly in [Section 20](#); here just hold that it is a per-position “volume normaliser” that keeps every layer's input well-behaved.

The feed-forward network. Two linear layers with a nonlinearity between, applied identically to every position:

$$\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2$$

with the deliberate widen-then-shrink shape:

Step	Shape in → out	Matrix
expand	$d_{\text{model}} \rightarrow 4 d_{\text{model}}$	$\mathbf{W}_1 : d_{\text{model}} \times 4d_{\text{model}}$
nonlinearity	$4 d_{\text{model}} \rightarrow 4 d_{\text{model}}$	ReLU / GELU (no weights)
project back	$4 d_{\text{model}} \rightarrow d_{\text{model}}$	$\mathbf{W}_2 : 4d_{\text{model}} \times d_{\text{model}}$

In words: the FFN blows each position's vector up to four times the width, runs a nonlinearity ($\max(0, \cdot)$ is ReLU — “keep positives, zero out negatives”), then squeezes it back down. The wide middle gives the network room to compute richer features per word; the factor of 4 is the standard choice from the original paper. Crucially the FFN acts on each position *independently* — it does no mixing between positions. All the cross-word talking already happened in attention.

Assembling the block (pre-norm order)

Modern transformers put the LayerNorm *before* each sub-layer (“pre-norm” — it trains more stably than the original “post-norm”). The full block is four lines:

$$\begin{aligned}\mathbf{x} &\leftarrow \mathbf{x} + \text{MultiHead}(\text{LayerNorm}(\mathbf{x})) \\ \mathbf{x} &\leftarrow \mathbf{x} + \text{FFN}(\text{LayerNorm}(\mathbf{x}))\end{aligned}$$

In words: normalise, mix across words with attention, add the result back in. Then normalise again, think per word with the FFN, add that back in. Input shape $(T \times d_{\text{model}})$; output shape $(T \times d_{\text{model}})$ — identical, on purpose, so the next block can consume it with no adjustment.

◆ THE SHAPE NEVER CHANGES — THAT'S THE POINT

Every sub-layer takes $(T \times d_{\text{model}})$ in and returns $(T \times d_{\text{model}})$ out. Attention preserves it, the FFN preserves it, the residual add requires it, LayerNorm preserves it. That invariance is what lets you stack 12, 48, or 96 identical blocks like LEGO bricks — the entire depth of a large model is this one shape, repeated.

◆ THE SAME THING IN PYTORCH

›_ MULTI-HEAD ATTENTION AND A FULL TRANSFORMER BLOCK — SHAPES ON EVERY LINE


```

import torch
import torch.nn as nn
import torch.nn.functional as F

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_heads):
        super().__init__()
        assert d_model % n_heads == 0 # heads must

divide the width evenly


        self.n_heads = n_heads
        self.d_k = d_model // n_heads # slice size per

head



# one big Linear does all heads at once (cheaper than n_heads  

separate ones)


        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model) # the final

mixing matrix W_O



    def forward(self, x): # x: [B, L,

d_model]


        B, L, _ = x.shape


# project, then split the width into heads: [B, L, d_model] →  

[B, n_heads, L, d_k]


        def split(t):
            return t.view(B, L, self.n_heads, self.d_k).transpose(1, 2)
        Q, K, V = split(self.W_q(x)), split(self.W_k(x)),
        split(self.W_v(x)) # [B, n_heads, L, d_k]

        scores = Q @ K.transpose(-2, -1) / self.d_k ** 0.5 # [B,

n_heads, L, L]


        weights = F.softmax(scores, dim=-1) # [B,

n_heads, L, L] rows sum to 1


        out = weights @ V # [B,

n_heads, L, d_k]



# concat heads back together: [B, n_heads, L, d_k] → [B, L,  

d_model]


        out = out.transpose(1, 2).contiguous().view(B, L, self.n_heads *
        self.d_k)
        return self.W_o(out) # [B, L,

d_model]


```

```

class TransformerBlock(nn.Module):
    def __init__(self, d_model, n_heads, d_ff=None):
        super().__init__()
        d_ff = d_ff or 4 * d_model          # the standard
        4x wide middle
        self.attn = MultiHeadAttention(d_model, n_heads)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.ffn = nn.Sequential(
            nn.Linear(d_model, d_ff),        # [B, L,
d_model] → [B, L, d_ff]
            nn.GELU(),                       # smooth cousin
of ReLU
            nn.Linear(d_ff, d_model),        # [B, L, d_ff] -
> [B, L, d_model]
        )

    def forward(self, x):                   # x: [B, L,
d_model]
        x = x + self.attn(self.norm1(x))    # pre-norm
attention + residual add
        x = x + self.ffn(self.norm2(x))    # pre-norm FFN +
residual add
        return x                           # [B, L,
d_model] → same shape in and out

# smoke test: batch 2, sequence 10, width 512, 8 heads
block = TransformerBlock(d_model=512, n_heads=8)
x = torch.randn(2, 10, 512)
print(block(x).shape)                     # torch.Size([2,
10, 512])

```

In words: the batch dimension **B** rides along in front the whole way — our earlier `scaled_dot_product_attention` used a leading `...` for exactly this reason. Notice the code contains no loop over heads: reshaping to `[B, n_heads, L, d_k]` lets one batched matrix multiply do all heads at once. That is the parallelism we promised at the start of this section, cashed out.

◆ WHERE THIS GOES NEXT

This block is the *encoder* form: every position may look at every other. To turn a transformer into a text *generator*, we add one line – a **causal mask** that stops each position from peeking at words it hasn't produced yet. That single change is the subject of [Section 19](#). And when models grow huge, the FFN is often replaced by a **mixture of experts** – many FFNs with a router picking a few per token – which is [Section 22](#). Only the feed-forward sub-layer changes there; the masked self-attention works exactly as it does here.

CHECK YOURSELF

In the block, why must every sub-layer output the shape

Reveal ▼

`[B, L, d_model]` – what specifically breaks if the FFN returned `[B, L, 2*d_model]` instead?

ANSWER

The **residual add** `x = x + ffn(...)` requires both sides to have the same shape – you cannot add a `[B, L, d_model]` tensor to a `[B, L, 2*d_model]` one; PyTorch would raise a shape error. More broadly, the *next* block also expects `d_model` width, so a changed width would break the whole stack. That is why the FFN widens to `4*d_model` in the middle but must project back to `d_model` at the end – the residual pathway keeps the width pinned from input to output.

6 What to carry forward

- **Attention is a soft lookup.** Queries ask, keys advertise, values are the content; each position gets a weighted blend of all values, graded by query–key match.
- **The formula:** $\text{softmax}(\mathbf{QK}^\top / \sqrt{d_k}) \mathbf{V}$ – compare, calm, blend, mix. Shapes: $(L \times d_k)(d_k \times T) \rightarrow (L \times T)$, then $(L \times T)(L \times d_v) \rightarrow (L \times d_v)$.

- **The $\sqrt{d_k}$ is derived, not decorative:** raw scores have variance d_k ; dividing by $\sqrt{d_k}$ restores variance 1 and keeps softmax out of its flat, no-gradient zone.
- **Multi-head** splits d_{model} into h heads of size d_{model}/h , runs attention in each, concatenates, and mixes with $\mathbf{W}_O$ – many relationship types for the price of one.
- **Attention is permutation-invariant**, so we add **positional encodings** (sinusoidal or learned) to inject order.
- **The block** is: LayerNorm $\rightarrow$ multi-head attention $\rightarrow$ residual add $\rightarrow$ LayerNorm $\rightarrow$ FFN ($d_{\text{model}} \rightarrow 4d_{\text{model}} \rightarrow d_{\text{model}}$) $\rightarrow$ residual add. Shape in equals shape out, so blocks stack without limit.

You now hold the complete core of every modern LLM. Next we add a causal mask and turn this machine into something that *writes* – that's [Section 19](#), language modelling.

Language Modeling & GPT

How a stack of transformer blocks becomes something that writes — end to end, from characters to sampled text.

32 min read

You'll learn: tokenization · embeddings · next-token prediction · the GPT-style decoder & causal masking

A model that writes essays, answers questions, and drafts code sounds like magic. It is not. Underneath, it does one small thing over and over: given the text so far, it **guesses the next piece**. Do that guess well enough, feed each guess back in, and coherent paragraphs fall out. This section builds that machine from the ground up — from raw characters, to numbers, to a stack of transformer blocks, to sampled words on a screen.

◆ THE ONE IDEA TO HOLD ONTO

A language model is a **next-token predictor**. Everything else — tokenization, embeddings, attention, the causal mask, sampling — exists to make that one prediction accurate, and then to run it in a loop. Learn to see the loop and the whole field stops looking mysterious.

1 What a language model actually is

◆ INTUITION FIRST

A **language model** is a function that reads a stretch of text and outputs a probability for *every possible next piece of text*. Given “the cat sat on the”, it says: “mat” is very likely, “roof” is plausible, “democracy” is almost impossible. That's the entire job. A *token* is the small unit it predicts — for now, think of it loosely as a word; we make it precise in a moment.

📌 EVERYDAY PARALLEL

Your phone keyboard's autocomplete. You type “I’ll be there in five” and it offers “minutes” before you’ve touched another key. It has learned, from mountains of past texts, which word tends to come next. A language model is that same guessing game, scaled up enormously and trained to keep the guesses coherent across whole paragraphs instead of a single word.

◆ NOW THE MATH

Write the text as a sequence of tokens $x_1, x_2, \dots, x_T$. The model estimates the probability of the whole sequence by multiplying together the probability of each token given everything before it:

$$P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, x_2, \dots, x_{t-1})$$

In words: the chance of a whole sentence is the chance of the first token, times the chance of the second *given* the first, times the chance of the third *given* the first two, and so on. The big $\prod$ (“product”) just means “multiply all these together.” The one factor we actually train the model to produce is $P(x_t \mid x_{<t})$ — the next token given the past. Get that factor right and the whole product takes care of itself.

DEFINITION

An **autoregressive** model is one that generates a sequence one step at a time, feeding each output back in as input for the next step. “Auto” = self, “regressive” = predicting from earlier values. Every GPT-style model works this way: predict a token, append it, predict the next.

CHECK YOURSELF

Why do we condition each token on the tokens *before* it, rather than modelling $P(x_t)$ on its own?

Reveal ▼

ANSWER

Because language is about context. On its own, “bank” is a coin flip between a river edge and a place for money. But after “I deposited the cheque at the”, “bank” becomes overwhelmingly the money kind. Conditioning on the past $(x_1 \dots x_{t-1})$ is what lets the model use context to sharpen every prediction. Modelling each token in isolation would throw all that context away and produce word salad.

2 Tokenization: turning text into tokens

◆ INTUITION FIRST

A computer cannot multiply matrices by the letter “q”. Before anything else, we must chop raw text into a fixed list of pieces and give each piece a number. That chopping step is **tokenization**. The obvious question is: what should a piece be – a letter, a word, or something in between?

DEFINITION

A **token** is one unit of text the model reads and predicts. The **vocabulary** is the fixed, finite set of all tokens the model knows – say 50,000 of them – each assigned a unique integer id from 0 to (vocab size – 1). Tokenizing means rewriting a string as a list of these ids.

◆ NOW THE TRADE-OFF, LAID OUT

There are three natural choices for “what is a piece”, and each has a cost:

Scheme	A token is...	Vocabulary size	The catch
Character	one letter/symbol	tiny (~100)	sequences get very long; the model must relearn spelling from scratch, so it wastes capacity on “how are letters glued into words”.
Word	one whole word	huge (millions)	any word not seen in training becomes an <i>unknown token</i> (a single catch-all id for “I’ve never seen this”) — the model goes blind on new names, typos, and rare words.
Subword (BPE)	a frequent chunk of characters	fixed, chosen (~30k–100k)	the sweet spot used by every modern model.

The winning scheme is **subword** tokenization, most often via an algorithm called **BPE** — *Byte-Pair Encoding*. It starts from individual characters and repeatedly merges the most frequent adjacent pair into a new token. Common words like “the” end up as a single token; a rare word like “antidisestablishmentarianism” gets split into several familiar pieces. Because the pieces are chunks of characters, *any* string can be spelled out from them — so there is never a true unknown word.

🔑 EVERYDAY PARALLEL

Think of LEGO. Whole-word tokens are like buying a pre-built model — great until you need a piece the set doesn't include, and then you're stuck. Single-character tokens are like having only 1×1 bricks — you can build anything, but it takes forever. Subword tokens are the everyday LEGO set: a mix of small and medium bricks. Frequent shapes come pre-assembled, anything unusual you build up from smaller pieces, and you can construct *any* object without a special part. Fixed box of bricks, unlimited things to build.

◆ WHY SUBWORD WINS, IN ONE LINE

It gives you a **fixed vocabulary** (so the model's size is bounded), **no unknown words** (any text can be spelled from the pieces), and it **handles rare words by breaking them into common pieces** the model already understands. Small model, total coverage – that's why everyone uses it.

◆ THE SAME THING IN CODE

To keep this section self-contained and runnable with no downloads, we'll use the simplest possible tokenizer – **character-level** – for all the code below. The ideas are identical for BPE; only the vocabulary is bigger.

>_ A CHARACTER-LEVEL TOKENIZER: TEXT ⇌ TOKEN IDS

```
text = "hello world"

# the vocabulary = the set of distinct characters, sorted for a stable
# order
chars = sorted(set(text))          # [' ', 'd', 'e', 'h', 'l', 'o',
'r', 'w']
vocab_size = len(chars)            # 8

stoi = {c: i for i, c in enumerate(chars)} # string → integer id
itos = {i: c for c, i in stoi.items()}     # integer id → string

def encode(s): return [stoi[c] for c in s] # text → list of ids
def decode(ids): return "".join(itos[i] for i in ids) # ids → text

print(encode("hello"))             # [3, 2, 4, 4, 5]
print(decode([3, 2, 4, 4, 5]))     # 'hello' → round-trips exactly
```

CHECK YOURSELF

A word-level model is trained on news from 2010 and then reads a 2024 article mentioning “COVID”. What happens, and how would subword tokenization save it?

Reveal ▼

ANSWER

“COVID” never appeared in 2010 training text, so a word-level model has no id for it – it collapses to the single *unknown* token and loses all information about the word. A subword tokenizer instead spells it from pieces it already knows, e.g. `"CO" + "VID"` or even letter-by-letter. The model still receives real, meaningful input and can reason about the new word from its parts. This is exactly why subword tokenization keeps working on names, typos, and words invented after training.

3 Embeddings: token ids become vectors

◆ INTUITION FIRST

A token id like `3` is just a name tag – the number itself carries no meaning (id 3 is not “more” than id 2 in any useful sense). The model needs each token to be a rich **vector** (a list of numbers; see [Section 1](#)) that it can learn to fill with meaning. An **embedding** is a lookup table that maps each id to its own learned vector.

🔑 EVERYDAY PARALLEL

A coat check at a theatre. You hand over your coat and get a numbered ticket – that number is the token id, meaningless on its own. But behind the counter, the number points to a hook holding your actual coat. The embedding table is that rack of hooks: each id points to a full vector of learned features. During training, the staff quietly improve what hangs on each hook so that similar coats end up near each other.

◆ NOW THE MATH

The embedding table is a matrix $\mathbf{E} \in \mathbb{R}^{V \times d}$, where V is the vocabulary size and d is the *embedding dimension* (how many numbers describe each token). To embed token id i , you simply pick out row i of the table:

$$\text{embed}(i) = \mathbf{E}[i, :] \in \mathbb{R}^d$$

In words: “the embedding of token i is the i -th row of the table $\mathbf{E}$.” No multiplication, just a row lookup. Those rows are ordinary parameters, so [backpropagation](#) nudges them during training — tokens that play similar roles drift toward similar vectors.

There is one thing embeddings alone don't capture: **order**. A plain lookup gives “dog bites man” and “man bites dog” the exact same set of vectors, just shuffled — and the attention that follows ([Section 18](#)) treats its inputs as a set, not a sequence. So we *add* a second vector that encodes each token's **position**:

$$\mathbf{h}_t = \underbrace{\mathbf{E}[x_t, :]}_{\text{what the token is}} + \underbrace{\mathbf{P}[t, :]}_{\text{where it sits}}$$

In words: the vector that enters the network for position t is the token's meaning vector plus a position vector $\mathbf{P}[t]$ that says “this is slot 0, slot 1, slot 2...”. Now the model can tell a first word from a last word. The details of positional encoding live in [Section 18](#); here we just need to know it gets added in.

◆ THE SAME THING IN PYTORCH

> _ TOKEN + POSITION EMBEDDINGS

```

import torch
import torch.nn as nn

vocab_size, d_model, max_len = 8, 32, 16  # 32 numbers per token, up to
16 positions

tok_emb = nn.Embedding(vocab_size, d_model)  # the lookup table E (8 x
32)
pos_emb = nn.Embedding(max_len, d_model)  # one vector per position

ids = torch.tensor([[3, 2, 4, 4, 5]])  # (batch=1, seq_len=5) –
"hello"
B, T = ids.shape

positions = torch.arange(T)  # [0, 1, 2, 3, 4]
h = tok_emb(ids) + pos_emb(positions)  # (1, 5, 32): meaning +
position
print(h.shape)  # torch.Size([1, 5, 32])

```

In words: `tok_emb(ids)` looks up a 32-number vector for each of the 5 tokens, giving shape `(1, 5, 32)`. `pos_emb(positions)` has shape `(5, 32)` and is *broadcast-added* across the batch — PyTorch stretches it to match. Every token now knows both what it is and where it sits.

4 The objective: next-token prediction

◆ INTUITION FIRST

How do we *train* a next-token predictor? We never need a human to label anything — the text labels itself. At every position, the correct answer is simply the token that actually came next. We slide over the text, and at each spot ask the model to predict the following token, then penalise it for being wrong. This is why language models can learn from the raw internet: the supervision is baked into ordinary text.

📌 EVERYDAY PARALLEL

Flashcard practice where you write your own cards from a book you're reading. Cover the next word, guess it, then slide your hand to reveal the truth and see if you were right. The book grades you for free – every sentence is thousands of self-checking flashcards. That's exactly how a language model studies: the next word is always the answer key.

◆ NOW THE MATH

At each position the model outputs *logits* – one raw score per vocabulary token (a *logit* is an unnormalised score, before it's turned into a probability). We pass the logits through **softmax** (a function that squashes a list of scores into positive numbers that sum to 1, i.e. a probability distribution) to get the predicted probability of each possible next token:

$$\hat{p}_t = \text{softmax}(\mathbf{z}_t), \quad \hat{p}_{t,j} = \frac{e^{z_{t,j}}}{\sum_{k=1}^V e^{z_{t,k}}}$$

In words: $\mathbf{z}_t$ is the row of $\mathbf{V}$ raw scores at position t . Softmax exponentiates each score (making it positive) and divides by the total, so the outputs are all between 0 and 1 and add to 1 — a real probability for every token in the vocabulary.

We want the probability the model assigned to the *true* next token to be high. The standard penalty for that is **cross-entropy loss** (introduced in [Section 4](#)) – the negative log probability of the correct token, averaged over every position:

$$L = -\frac{1}{T} \sum_{t=1}^T \log \hat{p}_{t, x_{t+1}}$$

In words: for each position t , look up the probability the model gave to the token that actually came next (x_{t+1}), take its **log**, flip the sign, and average over all positions. If the model was confident and right, that probability is near 1, its log is near 0, and the loss is tiny. If it was confident and *wrong*, the probability is near 0, its log dives to $-\infty$, and the loss is huge. Minimising this drives the model to put probability on the real next token.

DEFINITION

Teacher forcing is the training trick of feeding the model the *true* previous tokens at every position – not the model's own earlier guesses. So when predicting token 5, it sees the real tokens 1–4 even if it would have gotten token 4 wrong. This lets every position be predicted in parallel from correct context, which makes training fast and stable. (At generation time there is no answer key, so we must feed back the model's own outputs – the mismatch is a known quirk, but teacher forcing is worth it.)

◆ THE SAME THING IN PYTORCH

›_ NEXT-TOKEN CROSS-ENTROPY WITH A SHIFTED TARGET

```
import torch
import torch.nn.functional as F

# logits: the model's raw scores. shape (batch, seq_len, vocab_size)
logits = torch.randn(1, 5, 8)          # 1 sequence, 5 positions, vocab
of 8
targets = torch.tensor([[2, 4, 4, 5, 1]]) # the TRUE next token at each
position

# cross_entropy wants (N, vocab) logits and (N,) targets, so flatten the
batch+time axes
B, T, V = logits.shape
loss = F.cross_entropy(logits.reshape(B * T, V), targets.reshape(B * T))
print(loss)      # a single scalar; ~log(8)=2.08 at random init, driven
down by training
```

In words: `F.cross_entropy` does softmax *and* the negative-log-probability step in one numerically safe call, so we hand it raw logits, never softmaxed ones. We flatten the 5 positions into 5 independent (prediction, true-next-token) pairs and average their losses. The targets are just the inputs shifted left by one — position `t` is asked to predict what really sat at `t+1`.

CHECK YOURSELF

At the very start of training, weights are random, so the model spreads probability evenly over all $V = 50257$ tokens. What loss do you expect, and why is that a good sanity check?

Reveal ▼

ANSWER

If every token gets probability $1/V$, the loss at each position is $-\log(1/V) = \log(V)$. For $V = 50257$ that's $\log(50257) \approx 10.8$. Seeing your loss start near $\ln(V)$ and fall from there confirms the model is initialised sanely and the loss is wired up correctly. If it starts far from $\ln(V)$, something is wrong – often a bug in how logits or targets are shaped.

5 The GPT-style decoder, and the causal mask

◆ INTUITION FIRST

Now we assemble the model. A **GPT** (Generative Pre-trained Transformer) is *decoder-only*: it is just a stack of identical **transformer blocks** (the attention-plus-feed-forward units from [Section 18](#)), followed by one final linear layer that turns each position's vector into vocab-size logits. Embeddings go in the bottom, logits come out the top, and every block in between lets each position gather information from the others.

◆ THE WHOLE SHAPE, TOP TO BOTTOM

token ids $\rightarrow$ **embeddings + positions** $\rightarrow$ **N transformer blocks** (each = masked self-attention, then a feed-forward layer, with residual connections) $\rightarrow$ **final linear layer** to size $V \rightarrow$ **logits** $\rightarrow$ softmax $\rightarrow$ probabilities. Training minimises next-token cross-entropy over those probabilities; generation samples from them.

But there is a trap. Attention lets every position look at every other position. During training we hand the model the whole sentence at once (teacher forcing). If position 3 is allowed to

peek at position 4 while trying to *predict* position 4, it will just copy the answer – and learn nothing. We must forbid looking ahead.

🚫 EVERYDAY PARALLEL

An exam where the answers are printed on the next page. If students may flip ahead, the test measures nothing – everyone “predicts” perfectly by reading the answer. The **causal mask** is the invigilator's rule: you may reread everything you've already written, but you may not turn the page. Each word must be guessed from the words before it, never the ones after.

◆ DERIVING THE CAUSAL MASK

Recall from [Section 18](#) that self-attention first computes a grid of *attention scores* – a matrix $\mathbf{S} \in \mathbb{R}^{T \times T}$ where S_{ij} says how much the token at position i (the one asking) wants to attend to the token at position j (the one being read):

$$\mathbf{S} = \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}$$

In words: $\mathbf{Q}$ holds each position's “query” (what am I looking for?) and $\mathbf{K}$ holds each position's “key” (what do I offer?). Their dot products fill the grid $\mathbf{S}$; the $\sqrt{d_k}$ just keeps the numbers from getting too large. Row i of $\mathbf{S}$ is position i 's wish-list over all positions j .

“Position i must not attend to future position $j > i$ ” means: kill every entry where the column is to the right of the diagonal. Softmax is applied to each row to turn scores into weights, and softmax sends $-\infty$ to a weight of exactly 0 (because $e^{-\infty} = 0$). So we don't delete the future entries – we set their *scores* to $-\infty$ before the softmax:

$$S_{ij}^{\text{masked}} = \begin{cases} S_{ij} & j \leq i \quad (\text{past or self}) \\ -\infty & j > i \quad (\text{future}) \end{cases}$$

In words: keep a score if the column j is at or before the row i (a token may read itself and everything earlier); replace it with $-\infty$ if j is later. After softmax those $-\infty$ entries become zero weight, so no information flows backward in time.

For a length-5 sequence the mask is this lower-triangular pattern – a $\checkmark$ means “allowed to attend”, a $-\infty$ means “blocked”:

$i \backslash j$	0	1	2	3	4
0	✓	$-\infty$	$-\infty$	$-\infty$	$-\infty$
1	✓	✓	$-\infty$	$-\infty$	$-\infty$
2	✓	✓	✓	$-\infty$	$-\infty$
3	✓	✓	✓	✓	$-\infty$
4	✓	✓	✓	✓	✓

In words: row 0 (the first token) may look only at itself. Row 4 (the last token) may look at all five. Every row sees itself and everything to its left, nothing to its right. This lower-triangular shape is *the* defining feature of a decoder — it is what makes the model causal, i.e. only influenced by the past.

◆ THE MASK IN PYTORCH

›_ BUILDING THE CAUSAL MASK AND APPLYING IT TO SCORES

```
import torch

T = 5
# lower-triangular ones: allowed[i, j] = 1 when j ≤ i, else 0
allowed = torch.tril(torch.ones(T, T))
print(allowed)
# tensor([[1., 0., 0., 0., 0.],
#         [1., 1., 0., 0., 0.],
#         [1., 1., 1., 0., 0.],
#         [1., 1., 1., 1., 0.],
#         [1., 1., 1., 1., 1.]])

scores = torch.randn(T, T) # raw attention scores S
scores = scores.masked_fill(allowed == 0, float("-inf")) # blocked → -inf
weights = torch.softmax(scores, dim=-1) # each row sums to 1;
future = 0
print(weights[0]) # tensor([1., 0., 0., 0., 0.]) → token 0 attends
only to itself
```

In words: `torch.tril` keeps the lower triangle (past and self) and zeroes the upper triangle (future). `masked_fill` writes $-\infty$ everywhere the future lives, and softmax over the last dimension turns each row into weights that sum to 1 with the future entries at exactly 0. Row 0 attends only to itself, which is precisely what the mask math promised.

▲ WATCH OUT

The mask must be applied to the *scores*, before softmax – not to the weights after. If you zero out the weights afterwards instead, the surviving weights no longer sum to 1 (softmax already normalised them including the future), and every prediction is silently miscalibrated. Mask first, softmax second.

CHECK YOURSELF

Without the causal mask, a decoder trains to almost zero loss in a few steps but generates gibberish at test time. Why?

Reveal ▼

ANSWER

With no mask, position t can attend to position $t+1$ – which holds the very token it's being asked to predict. The model learns the trivial shortcut “copy the next input”, so training loss collapses. But at generation time the future doesn't exist yet (that's what we're trying to produce), so the shortcut has nothing to copy and the model falls apart. The causal mask forces it to actually *learn to predict* instead of cheating, which is the only skill that transfers to generation.

6 Generation: from logits to written text

◆ INTUITION FIRST

A trained model is a next-token predictor; writing is just running it in a loop. Feed in the prompt, get logits for the next token, turn them into probabilities, **pick one**, append it to the sequence, and feed the whole thing back in. Repeat until you've written enough. The only real decision is *how* you pick from the probabilities.

🔼 EVERYDAY PARALLEL

Improvising a story around a campfire, one word at a time. After each word your mind holds a shortlist of plausible next words with rough odds. Always blurting out the single most likely word makes for a flat, repetitive tale; occasionally taking a less obvious word makes it lively but riskier. **Temperature** is the dial between “play it safe” and “surprise me”.

◆ GREEDY, SAMPLING, AND TEMPERATURE

Greedy decoding always takes the highest-probability token (**arg max**). It's deterministic and safe but tends to loop and sound robotic. **Sampling** instead draws a token at random *in proportion to its probability* – likely tokens are picked often, rare ones occasionally – which gives variety and life. **Temperature** (τ) reshapes the distribution before we sample by dividing the logits:

$$\hat{p}_j = \text{softmax}\left(\frac{\mathbf{z}}{\tau}\right)_j = \frac{e^{z_j/\tau}}{\sum_k e^{z_k/\tau}}$$

In words: divide every logit by τ before softmax. When $\tau < 1$ the gaps between scores grow, so the top token dominates — the model is *more confident and greedier*. When $\tau > 1$ the gaps shrink, probabilities flatten toward equal, and the model is *more random and adventurous*. As $\tau \rightarrow 0$ sampling collapses to pure greedy; at $\tau = 1$ you sample from the model's raw distribution.

DEFINITION

Temperature is a single positive number that controls randomness in sampling. Low temperature (e.g. 0.2) = confident, focused, repetitive. High temperature (e.g. 1.3) = diverse, creative, error-prone. It is the most common knob you'll turn when using a language model.

◆ THE GENERATION LOOP IN PYTORCH

›_ SAMPLE TOKENS ONE AT A TIME, WITH TEMPERATURE

```

import torch

@torch.no_grad()                                # generation is inference: no
gradients needed
def generate(model, idx, max_new_tokens, block_size, temperature=1.0):
    # idx: (batch, current_length) tensor of token ids – the prompt
    for _ in range(max_new_tokens):
        idx_cond = idx[:, -block_size:]          # keep only the last
        block_size tokens
        logits = model(idx_cond)                  # (batch, length, vocab)
        logits = logits[:, -1, :] / temperature  # take the LAST position,
        scale by temp
        probs = torch.softmax(logits, dim=-1)     # (batch, vocab) – a
        distribution
        next_id = torch.multinomial(probs, num_samples=1) # sample one
        token per row
        idx = torch.cat([idx, next_id], dim=1)    # append and loop
    return idx

```

In words: at each step we run the model, keep only the logits at the *last* position (the prediction for what comes next), divide by temperature, softmax to a probability distribution, and `torch.multinomial` draws one token in proportion to those probabilities. We glue it onto the sequence and repeat. `idx_cond` trims the input to the model's maximum context length `block_size` so it never overflows. Swap the sampling line for `logits.argmax(dim=-1)` and you have greedy decoding.

▲ WATCH OUT

Only the logits at the **last** position matter for generation – that's the model's guess for the next token. Beginners sometimes sample from every position or from position 0; you'd be sampling predictions for tokens you already have. Always slice `logits[:, -1, :]`.

CHECK YOURSELF

You set `temperature = 0.01`. How does the output behave, and is it different from greedy decoding?

Reveal ▼

ANSWER

Dividing logits by 0.01 multiplies them by 100, blowing the gaps between scores wide open. After softmax, the single top token gets essentially all the probability mass, so `multinomial` picks it almost every time. The output becomes effectively deterministic and matches greedy decoding in practice – because $\tau \rightarrow 0$ is the limit that turns sampling into `arg max`. Very low temperature is a common way to get near-greedy, focused output while keeping the same sampling code.

7 Putting it together: a minimal GPT block

◆ ONE DECODER BLOCK, WIRED END TO END

Here is a single transformer block with *built-in causal masking*, plus the tiny model head that produces logits. It reuses the attention idea from [Section 18](#); the only new ingredient is the mask we derived just above. This is exactly the unit you'll scale up – and route between experts – in the capstone, [Section 22](#).

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class CausalSelfAttention(nn.Module):
    def __init__(self, d_model, n_heads, block_size):
        super().__init__()
        assert d_model % n_heads == 0
        self.n_heads = n_heads
        self.qkv = nn.Linear(d_model, 3 * d_model)  # produce Q, K, V in
one matmul
        self.proj = nn.Linear(d_model, d_model)      # mix heads back
together
        # a constant lower-triangular mask, saved with the model but not
```

```

trained
    self.register_buffer("mask", torch.tril(torch.ones(block_size,
block_size)))

    def forward(self, x):
        # x: (B, T, d_model)
        B, T, C = x.shape
        q, k, v = self.qkv(x).split(C, dim=2)
        # each (B, T, C)
        # reshape into heads: (B, n_heads, T, head_dim)
        q = q.view(B, T, self.n_heads, C // self.n_heads).transpose(1, 2)
        k = k.view(B, T, self.n_heads, C // self.n_heads).transpose(1, 2)
        v = v.view(B, T, self.n_heads, C // self.n_heads).transpose(1, 2)

        att = (q @ k.transpose(-2, -1)) / (k.size(-1) ** 0.5)
        # scores
        S: (B, h, T, T)
        att = att.masked_fill(self.mask[:T, :T] == 0, float("-inf"))
        # block the future
        att = F.softmax(att, dim=-1)
        # weights, future →
        0
        y = att @ v
        # (B, h, T, head_dim)
        head_dim)
        y = y.transpose(1, 2).reshape(B, T, C)
        # re-merge the heads
        return self.proj(y)

class Block(nn.Module):
    def __init__(self, d_model, n_heads, block_size):
        super().__init__()
        self.ln1 = nn.LayerNorm(d_model)
        # normalise before
        attention
        self.attn = CausalSelfAttention(d_model, n_heads, block_size)
        self.ln2 = nn.LayerNorm(d_model)
        self.mlp = nn.Sequential(
            # position-wise
            feed-forward
            nn.Linear(d_model, 4 * d_model), nn.GELU(),
            nn.Linear(4 * d_model, d_model),
        )

    def forward(self, x):
        x = x + self.attn(self.ln1(x))
        # residual add-on
        around attention
        x = x + self.mlp(self.ln2(x))
        # residual add-on
        around the MLP
        return x

class MiniGPT(nn.Module):
    def __init__(self, vocab_size, d_model=64, n_heads=4, n_layers=2,

```

```

block_size=32):
    super().__init__()
    self.block_size = block_size
    self.tok_emb = nn.Embedding(vocab_size, d_model)
    self.pos_emb = nn.Embedding(block_size, d_model)
    self.blocks = nn.ModuleList(
        [Block(d_model, n_heads, block_size) for _ in
range(n_layers)]
    )
    self.ln_f = nn.LayerNorm(d_model)
    self.head = nn.Linear(d_model, vocab_size)  # final linear →
logits

    def forward(self, idx):  # idx: (B, T) token
ids
        B, T = idx.shape
        pos = torch.arange(T, device=idx.device)
        x = self.tok_emb(idx) + self.pos_emb(pos)  # (B, T, d_model)
        for block in self.blocks:
            x = block(x)
        x = self.ln_f(x)
        return self.head(x)  # (B, T, vocab_size)
logits

# smoke test: shapes must line up end to end
model = MiniGPT(vocab_size=8)
idx = torch.zeros(1, 5, dtype=torch.long)  # a length-5 prompt
of zeros
logits = model(idx)
print(logits.shape)  # torch.Size([1, 5,
8])

```

In words: ids become token+position vectors; each `Block` runs masked self-attention and a feed-forward layer, each wrapped in a *residual connection* (`x + ...`), which lets gradients flow cleanly, see [Section 18](#)) and preceded by *layer norm* (which rescales each vector to a stable range). The final linear `head` maps every position to `vocab_size` logits. The smoke test confirms the promised shape `(batch, seq_len, vocab)` — feed those logits into cross-entropy to train the model, or into the sampling loop from earlier in this section to generate text.

◆ THAT'S A REAL LANGUAGE MODEL

The code above, trained on enough text with the cross-entropy loss and run through the generation loop, genuinely writes. GPT-scale models differ only in size – thousands of times more blocks, dimensions, and data – plus engineering to train them. The architecture you just read is the same one. In [Section 22](#) we swap each block's single feed-forward layer for a *mixture of experts* to grow capacity without growing the cost of every token.

8 What to carry forward

- A **language model** predicts the next token given the previous ones; run that predictor in a loop and it writes.
- **Subword tokenization** (BPE) is the sweet spot: a fixed vocabulary, no unknown words, rare words handled by breaking them into familiar pieces.
- **Embeddings** turn each token id into a learned vector via a lookup table; we *add* positional vectors so order matters.
- Training is **next-token prediction** – cross-entropy over the vocabulary at every position – with **teacher forcing** feeding the true past.
- A **GPT** is a stack of transformer blocks plus a final linear layer to logits. The **causal mask** sets future scores to $-\infty$ so no position can cheat by reading ahead.
- **Generation** = logits $\rightarrow$ softmax $\rightarrow$ sample $\rightarrow$ append $\rightarrow$ repeat, with **temperature** dialing between greedy/confident and random/creative.

You now have every moving part of a GPT. Next we scale it up cleverly: the capstone builds a [mixture-of-experts](#) model that adds capacity while keeping each token cheap.

Training at Scale

The gap between a model that trains on paper and one that trains on real hardware, in real time, on real data – normalization, dropout, half-precision numbers, and splitting work across machines.

28 min read

You'll learn: batch norm vs layer norm · dropout · mixed precision · gradient accumulation · distributed training

Everything so far has assumed you can just call `loss.backward()` and `optimizer.step()` and be done. At real scale, a handful of practical techniques stand between that idea and a model that actually trains stably, fits in memory, and finishes before the deadline. None of it is exotic; all of it is necessary.

◆ THE ONE IDEA TO HOLD ONTO

Every technique in this section solves one of two problems: keeping the numbers flowing through the network **well-behaved** (normalization, dropout, mixed precision), or making the most of **limited hardware** (gradient accumulation, distributed training). Know which problem you're solving and the right tool is obvious.

1 Normalization: the general recipe

◆ INTUITION FIRST

As data flows through a deep network, the scale of the numbers at each layer can drift – growing too large, shrinking too small, or shifting off-center – which makes training unstable. **Normalization** is a layer that resets the numbers back to a well-behaved range (centered around zero, with a controlled spread) at each step, then lets the network relearn whatever scale and offset it actually needs.

🔧 EVERYDAY PARALLEL

A sound engineer normalizes every microphone's volume to the same baseline level before a concert, so no one instrument drowns out the others just because it happened to start too loud or too quiet. Then, deliberately, they turn some instruments up and others down to get the mix they actually want. Reset to a common baseline, then re-adjust on purpose — that's normalization.

◆ NOW THE MATH

The recipe is always the same two steps: first normalize, then let the network scale and shift the result by its own learned amount.

$$z = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$
$$y = \gamma z + \beta$$

In words: μ (mu) is the mean and σ^2 (sigma-squared) is the variance (how spread out the values are) — subtracting the mean and dividing by the spread gives z , a version of x centered at 0 with spread 1. ϵ (epsilon) is a tiny constant added purely so we never divide by zero when the variance happens to be 0. Then γ (gamma) and β (beta) are *learned* numbers that rescale and re-shift the normalized value — they let the network undo the normalization partially or fully if that turns out to be what's best, rather than being stuck with "always centered at 0."

DEFINITION

The only real design choice in any normalization layer is *which numbers you average together* to compute μ and σ^2 . Batch norm and layer norm are the same formula above, applied over two different groups of numbers.

2 Batch norm vs layer norm: which numbers get averaged

◆ INTUITION FIRST

Batch normalization computes μ and σ^2 for each feature separately, averaging *across all examples in the mini-batch*. **Layer normalization** computes μ and σ^2 for each example

separately, averaging *across all features of that one example*. Same formula, one crucial difference: which direction you slice the data to gather statistics from.

🔦 EVERYDAY PARALLEL

Batch norm is grading on a curve, per subject: your score on the math test gets adjusted based on how the *whole class* did on that test – it depends on your classmates. **Layer norm** is normalizing one student against their own report card: comparing your math score to your own scores in history, art, and gym – it only ever depends on you, never on anyone else in the room.

◆ THE SAME THING IN PYTORCH

```
import torch, torch.nn as nn

x = torch.randn(32, 4)    # 32 examples in the batch, 4 features each

bn = nn.BatchNorm1d(num_features=4)
y_bn = bn(x)              # mean/var for each of the 4 features, computed
                           across all 32 examples

ln = nn.LayerNorm(normalized_shape=4)
y_ln = ln(x)              # mean/var for each example, computed across
                           its own 4 features
```

◆ WHY TRANSFORMERS USE LAYER NORM

Layer norm's statistics never depend on other examples in the batch, which buys three things transformers (Section 18) need: it works fine with a **batch size of 1** (batch norm degenerates there, since a single example gives a variance of essentially zero); it doesn't care that different sequences in a batch have **different lengths**, since it never mixes examples together; and it behaves **identically at training and test time**, with no separate "running statistics" bookkeeping that batch norm requires to make evaluation match training.

▲ WATCH OUT

Batch norm needs to behave the same on a single test example as it did on a full training batch, so it tracks a *running average* of μ and σ^2 during training and switches to using that running average at evaluation time – which is exactly what

`model.eval()` tells it to do (see [Section 15](#)). Layer norm has no such distinction: it computes real statistics from the actual input every single time, train or eval, so there is nothing extra to get wrong.

CHECK YOURSELF

Why does batch normalization behave badly (or fail outright) with a batch size of 1?

Reveal ▼

ANSWER

Batch norm computes each feature's mean and variance *across the examples in the batch*. With only one example, the "mean" of a single number is just that number, and the variance is exactly 0 – the statistics carry no real information about spread, and dividing by `sqrt(0 +  $\epsilon$ )` makes the normalized value dominated entirely by the tiny constant ϵ , not by anything meaningful in the data. Layer norm is unaffected, because it always has many features within the one example to average over, regardless of how many examples are in the batch.

3 Dropout: forcing redundancy

◆ INTUITION FIRST

Dropout randomly zeroes out a fraction of a layer's units on every forward pass during training. The network can no longer lean too heavily on any one unit, because that unit might vanish on the very next step – so it's forced to spread the useful signal across many units instead, which reduces overfitting.

🔼 EVERYDAY PARALLEL

A sports team that rotates who starts each game, even its best player, ends up with a squad where several players can competently fill any role. If the team only ever depended on one star, losing that player to injury would be catastrophic. Dropout is that rotation policy, applied to a neural network's units instead of a roster.

◆ NOW THE MATH

During training, each unit is independently zeroed with probability p , and the survivors are scaled up by $1/(1-p)$ so the expected total signal stays the same:

$$\text{dropout}(x_i) = \begin{cases} 0 & \text{with probability } p \\ \frac{x_i}{1-p} & \text{with probability } 1-p \end{cases}$$

In words: flip a biased coin for every unit. Heads (chance p): that unit outputs 0 this step. Tails: the unit's value is divided by $1-p$ to make up for the units that got zeroed, so the layer's total output stays roughly the same size on average whether or not dropout is active. This rescaling is called *inverted dropout*, and it's what lets dropout be a no-op at test time with zero extra work.

◆ THE SAME THING IN PYTORCH

```
drop = nn.Dropout(p=0.5)

model.train()
y = drop(x)      # roughly half the entries become 0; survivors are
                 # doubled (1/(1-0.5))

model.eval()
y = drop(x)      # identity -- dropout does nothing at eval time, x passes
                 # through unchanged
```

CHECK YOURSELF

You forgot to call `model.eval()` before running validation.

Reveal ▼

Predictions come out worse than expected and change slightly every time you re-run the same input. Why, specifically?

ANSWER

In training mode, dropout is still active: each forward pass zeroes a different random subset of units, so the same input produces a different, noisier output every call – and you're effectively evaluating with a random, incomplete slice of the network rather than the full model. Calling `model.eval()` turns dropout into the identity function, so every unit contributes every time, giving stable, deterministic, and stronger predictions.

4 Mixed precision: same math, half the bits

◆ INTUITION FIRST

Every number in a network is normally stored as a 32-bit float (`fp32`). Modern GPUs can run much faster, and store roughly twice as much in memory, using a 16-bit format instead – either `fp16` or `bf16` (two different ways of packing a number into 16 bits). **Mixed precision** training runs most of the network's math in 16-bit numbers while keeping a few sensitive spots – like the master copy of the weights – in 32-bit, to get the speed without losing correctness.

🔧 EVERYDAY PARALLEL

A tailor doesn't measure every cut to the nearest micrometer – a millimeter of precision is plenty for cloth, and using a coarser tool is faster and cheaper. But for one critical piece, like fitting a watch gear, they switch back to the precise tool. Mixed precision is using the coarser, faster measurement everywhere it's safe, and the precise one only where it truly matters.

◆ NOW THE MATH: WHY LOSS SCALING EXISTS

`fp16` can represent numbers only within a much narrower range than `fp32` before rounding them to zero. Many gradients during training are legitimately tiny – say, `1e-8` – and a value that small can **underflow** in `fp16` (round down to exactly 0.0), silently deleting a real gradient. **Loss scaling** fixes this with one multiplication and one division:

$$\text{scaled_loss} = s \cdot L, \quad \nabla_{\text{used}} = \frac{1}{s} \nabla(\text{scaled_loss})$$

In words: multiply the loss `L` by a large scale factor `s` (e.g. 1024 or higher) before calling `backward()`. By the chain rule, every gradient computed from that scaled loss is also multiplied by `s`, which pushes tiny gradients up out of the underflow zone and into a range `fp16` can represent. Right before the optimizer uses them, divide back by the same `s` so the actual weight update is correct. `bf16` has the same exponent range as `fp32` (it just drops precision, not range), so it rarely needs this trick at all.

◆ THE SAME THING IN PYTORCH

```
scaler = torch.cuda.amp.GradScaler()

for xb, yb in loader:
    optimizer.zero_grad()
    with torch.cuda.amp.autocast(dtype=torch.float16):
        out = model(xb)
        loss = criterion(out, yb)

    scaler.scale(loss).backward()    # scale the loss up before backward
    scaler.step(optimizer)           # unscales gradients, then steps
    # (skips the step if any are inf/NaN)
    scaler.update()                 # adjusts the scale factor for the
    next iteration
```

CHECK YOURSELF

A gradient of $3e-8$ is a real, non-zero value in fp32 but becomes exactly 0.0 in fp16. What does loss scaling do about it, mechanically?

Reveal ▼

ANSWER

Loss scaling multiplies the loss by a large constant s before `backward()` runs, so by the chain rule every downstream gradient – including this one – is also multiplied by s . A gradient of $3e-8 \times 1024 \approx 3e-5$ is comfortably inside `fp16`'s representable range and no longer rounds to zero. After `backward()`, the scaled gradients are divided by s again before the optimizer applies them, so the final weight update is mathematically the same as if you'd never scaled anything – you only used the scaling to survive the trip through 16-bit storage.

5 Gradient accumulation: a bigger batch on smaller memory

◆ INTUITION FIRST

Sometimes the batch size you want doesn't fit in GPU memory. **Gradient accumulation** fakes a large batch by processing several small "micro-batches" one at a time, adding up their gradients without resetting them, and only calling `optimizer.step()` once every few micro-batches – as if all of them had been one big batch all along.

🌂 EVERYDAY PARALLEL

You want to deposit 400 dollars but your wallet only holds 100 at a time. You make four trips to the ATM, but you don't deposit each 100 separately into different sub-accounts – you add them up and it's exactly as if you'd deposited 400 once. Gradient accumulation is four small trips that add up to the one big deposit you actually wanted.

◆ THE SAME THING IN PYTORCH


```

accumulation_steps = 4          # 4 micro-batches of 32 = an effective
                                batch of 128
optimizer.zero_grad()

for i, (xb, yb) in enumerate(loader):
    out = model(xb)
    loss = criterion(out, yb) / accumulation_steps  # average, not sum,
    across micro-batches
    loss.backward()                                # gradients
    ACCUMULATE in .grad (no zero_grad yet)

    if (i + 1) % accumulation_steps == 0:
        optimizer.step()
        optimizer.zero_grad()

```

▲ WATCH OUT

Dividing the loss by `accumulation_steps` is not optional. Skip it and four accumulated micro-batch gradients add up to roughly 4× the gradient a true batch of 128 would have produced – which behaves like secretly training with a 4× larger learning rate, and can destabilize training in a way that looks like an unrelated bug.

CHECK YOURSELF

You accumulate gradients over 4 micro-batches of size 32 Reveal ▼ but forget to divide the loss by 4. How does this actually change what the optimizer does, compared to a true batch of 128?

ANSWER

A true batch of 128 with a mean-reduced loss produces one gradient that's the average over 128 examples. Without dividing, summing 4 un-averaged micro-batch losses (each already averaged over its own 32) produces a gradient that's roughly 4x too large before the optimizer step. The optimizer then applies an update as if the learning rate were 4x higher than configured – which can cause instability or divergence that has nothing to do with the model itself, only with this missing division.

6 Distributed training: an honest overview

◆ INTUITION FIRST

When a model or dataset is too large for one machine, the work is split across several. There is no single "distributed training" technique – there are a few different things you can split, and real systems often combine them. None of it is magic: it all costs communication between machines, and that communication is usually the actual bottleneck, not the computation.

☂ EVERYDAY PARALLEL

A restaurant kitchen with one overloaded chef can add more chefs. You could give each chef an identical copy of the full menu and have them each cook different tables' orders (data parallelism). Or you could have one chef only do sauces and hand plates to a chef who only does grilling, in a chain (pipeline parallelism). Either way, the moment plates need to move between chefs is the moment things can slow down – coordination has a cost.

◆ DATA PARALLELISM, THE WORKHORSE

The most common setup by far is **data parallelism**: every device (GPU) holds a full, identical copy of the model. Each device processes a different slice of the data and computes its own gradients independently. Then, before anyone updates their weights, all devices combine their gradients with an **all-reduce** – a collective operation where every device ends up with the same combined (typically averaged) gradient – so every replica applies an identical update and stays in sync.

- **Model / tensor parallelism** – the model itself is too big to fit on one device, so individual layers, or even individual matrices within a layer, are split across devices, and a single forward pass requires those devices to exchange intermediate results.
- **Pipeline parallelism** – different layers of the model live on different devices, and data flows through them like a factory assembly line, with each device handing its output to the next.

◆ THE HONEST BOTTLENECK

Adding devices doesn't automatically make training proportionally faster. Every one of these strategies requires devices to send data to each other – gradients in data parallelism, activations in tensor and pipeline parallelism – over a network that is almost always slower than the computation itself. Past a certain scale, training speed is limited by how fast machines can talk to each other, not by how fast any one of them can compute. This is why hardware interconnect (the physical links between devices) matters as much as raw compute in large-scale training.

7 What to carry forward

- Every normalization layer is **normalize, then scale-and-shift**:
$$y = \gamma(x - \mu) / \sqrt{\sigma^2 + \epsilon} + \beta.$$
 The only difference between variants is which numbers μ and σ^2 are computed over.
- **Batch norm** normalizes each feature across the batch (depends on other examples); **layer norm** normalizes each example across its own features (independent of the batch) –

which is why transformers use layer norm.

- **Dropout** randomly zeroes units during training to prevent over-reliance on any one of them, and is switched off entirely by `model.eval()`.
- **Mixed precision** trades numeric range for speed and memory; **loss scaling** keeps tiny `fp16` gradients from rounding to zero.
- **Gradient accumulation** simulates a large batch on limited memory by summing (averaged) micro-batch gradients before one optimizer step.
- **Distributed training** splits either the data (data parallelism) or the model itself (model/tensor/pipeline parallelism) across devices – and communication between devices, not raw compute, is usually the real limiting factor.

Next we switch from building to reading – how to pull a real algorithm out of a research paper in [Section 21](#), the last skill before the capstone.

Reading a Paper Like an Engineer

You don't need to read every word of a research paper to steal everything useful from it — you need a method, and about twenty minutes.

22 min read

You'll learn: the three-pass reading method · what to extract from any paper · dissecting an MoE / Switch Transformer paper · habits that make it stick

Most people try to read a paper the way they read a novel: start at the top, plough through every sentence, and hope understanding arrives by the last page. It doesn't, and worse, it's slow — you'll burn two hours on a paper's tenth-most-important paragraph before you even know if the paper is relevant. Engineers read papers differently: in passes, each one deeper than the last, extracting only what that pass is for. This section gives you that method, then applies it to the exact papers behind our capstone project.

◆ THE ONE IDEA TO HOLD ONTO

You never read a paper start to finish, once. You read it in **increasing passes of depth** — skim, then structure, then reproduce — and you stop as soon as the pass you're on has given you what you needed.

1 The three-pass method

◆ INTUITION FIRST

This method has a name in the research world — it's close to what computer scientist S. Keshav called "How to Read a Paper" (2007), and practicing engineers converge on something like it independently. The idea: do three passes over the same document, each one asking a different question and costing more time than the last. You're allowed to stop after pass one if the paper turns out not to matter to you.

🔼 EVERYDAY PARALLEL

You don't understand a recipe by reading it – you understand it by cooking it once. Reading it first tells you what dish it is and roughly what's involved (pass one). Reading it again slowly tells you the one step that will make or break the dish – say, the exact moment you add the egg (pass two). Actually cooking it, once, is the only way you truly learn why that step matters (pass three). Skipping straight to "make it fifty times" without ever reading the recipe wastes ingredients; reading it fifty times without cooking teaches you nothing your hands will remember.

1. **Pass 1 – skim (about 5 minutes).** Read only the title, the *abstract* (the short paragraph at the top summarizing the whole paper), every figure and its caption, and the conclusion. Skip the method section entirely. Goal: decide what the paper claims and whether it's relevant to you at all. If you can't state the paper's one-sentence claim after this pass, either the paper is poorly written or you're not its intended reader – in either case, that's useful information before you invest more time.
2. **Pass 2 – structure and the one key equation (30–60 minutes).** Now read the body, in order, but don't chase every citation or verify every derivation. Note the structure: what problem section 1 sets up, what section 3 proposes, what section 4 measures. Find the *one* equation that is actually load-bearing for the paper's claim – usually there's a single formula everything else exists to justify – and find the main architecture or results figure. Redraw that figure from memory on paper; if you can't, you haven't understood it yet.
3. **Pass 3 – reproduce (hours to days, only for papers you're building on).** Re-derive the key equation yourself, line by line, and implement the smallest unit of the method in code. Not the whole system – the one component that carries the paper's actual idea. This is the pass that turns "I read it" into "I understand it," because code and math don't let you fool yourself the way a nodding-along read-through does.

CHECK YOURSELF

You finish a 5-minute pass-1 skim of a paper and still can't [Reveal ▼](#) state its main claim in one sentence. What should you do next?

ANSWER

Don't jump into a full pass-2 read hoping it clicks. First re-check the pass-1 material specifically for the claim: reread the abstract's last sentence and the conclusion's first paragraph, since that's almost always where it's stated plainly. If it's still unclear, that's a signal the paper may be poorly scoped for you right now – check whether it assumes background you don't have, or whether it simply isn't the paper you need. Either way, deciding this in 5 minutes is the entire point of pass one: it's far cheaper than discovering the same thing after an hour of pass two.

2 What to extract from any paper

◆ INTUITION FIRST

Regardless of the field, almost every research paper is answering the same six questions. If you extract clean answers to these six, you've captured the paper's engineering value even if you never touch the surrounding prose again.

- **The problem.** What specifically didn't work before, in one sentence? Not "language models are hard" – something falsifiable, like "dense models need a proportional increase in compute for every increase in parameters."
- **The key idea, in one sentence.** If you can't compress the contribution to one sentence, you haven't found it yet – you're still describing the paper's title.
- **The central equation.** The formula that, if you deleted it, would make the rest of the paper meaningless. Usually there's exactly one.
- **The architecture diagram.** The picture that shows data flowing through the proposed system. If the paper has no diagram, draw one yourself from the method section – it will expose anything you glossed over.

- **The ablations** – what they tell you the idea is *for*.
- **What they don't tell you** – what the idea *costs*.

DEFINITION

An **ablation** is an experiment where the authors remove or swap out one single piece of their system and re-measure performance, to prove that piece was actually necessary rather than incidental. ("Ablate" literally means to cut away.)

🔧 EVERYDAY PARALLEL

Back to the recipe: if you want to prove the yeast is what makes bread rise, you bake one loaf with yeast and one identical loaf without it, changing nothing else. The flat second loaf is your ablation – it's proof, not assertion. A paper's ablation table is doing exactly this: baking the same model over and over with one ingredient removed each time, so you can see which ingredients were load-bearing.

⚠️ WATCH OUT

Papers report what worked. They rarely headline what it cost to get there: how many failed runs preceded the reported one, how much hyperparameter tuning was needed, what hardware and how much wall-clock time the "efficient" method actually used, or which results didn't reproduce on a different dataset. Read the "Limitations" section if there is one – it's often the most honest paragraph in the paper. If there isn't one, assume the costs exist and were left out, and go looking for them in the appendix or the released code.

CHECK YOURSELF

A paper proposes a new component X and reports one number: "model with X gets 91% accuracy." No ablation table. What's missing, and why does it matter?

Reveal ▼

ANSWER

Missing is the number for "model without X, everything else identical" – without that baseline, 91% tells you nothing about whether X caused it. Maybe a bigger learning rate or three extra epochs would have gotten 91% anyway. An ablation isn't a nice-to-have footnote; it's the actual evidence for the paper's claim. A single headline number with no matched baseline is a marketing figure, not an engineering one – treat it with real skepticism until you find the ablation, or run it yourself.

3 Dissecting the MoE / Switch Transformer papers

◆ INTUITION FIRST

A **Mixture of Experts** (MoE) model replaces one big feed-forward layer with many smaller feed-forward layers, called *experts*, plus a small decision maker called the *router* that picks which one or two experts handle each token. The point is to grow a model's total number of parameters – its capacity to store knowledge – without growing the compute spent on every single token, since each token only visits a few experts instead of one enormous layer.

☂ EVERYDAY PARALLEL

A large call center has dozens of specialists – billing, technical support, returns, legal – but a caller never talks to all of them. A receptionist (the router) listens to the first sentence of your call and sends you to the one or two specialists (the experts) most likely to solve your problem. The building can grow enormous specialist capacity while any single call still only occupies one or two people's time. That receptionist's judgment call is precisely what the router is learning to make.

Two papers define almost all the vocabulary you'll meet here: Shazeer et al., 2017, "*Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer*" (the modern MoE layer, with top-2 routing), and Fedus, Zoph & Shazeer, 2021, "*Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*" (simplifies routing to top-1 and scales it into a Transformer). Four things to hunt for specifically when you open either one:

DEFINITION

The **router** is a small learned layer (usually just one linear layer plus a softmax) that looks at a token's vector and outputs a probability for every expert: "how suited is this token to expert 1, to expert 2, ..." Those probabilities decide routing and also weight the expert's output.

◆ NOW THE MATH

The one equation worth re-deriving from the Switch Transformer paper is the **auxiliary load-balancing loss** – added on top of the normal training loss to stop the router from lazily sending every token to the same one or two experts:

$$L_{\text{aux}} = \alpha \cdot N \sum_{i=1}^N f_i P_i$$

In words: N is the number of experts. f_i is the fraction of tokens in the batch that actually got dispatched to expert i . P_i is the average router probability the router assigned to expert i across the whole batch (a smooth, differentiable stand-in for the routing decision, since "which expert got picked" itself isn't differentiable). α is a small weighting constant (0.01 in the paper). Multiplying two probability-like vectors this way and summing is minimized exactly when both are spread evenly across experts — so minimizing this loss *is* minimizing imbalance.

You can see why with two clean cases. With 4 experts and perfectly even routing, $f_i = P_i = 0.25$ for every expert: $\sum f_i P_i = 4 \times (0.25 \times 0.25) = 0.25$, so $L_{\text{aux}} = \alpha \times 4 \times 0.25 = \alpha$. If instead everything piles onto expert 1 ($f_1 = P_1 = 1$, the rest zero): $\sum f_i P_i = 1$, so $L_{\text{aux}} = \alpha \times 4 \times 1 = 4\alpha$ – four times higher, purely as a penalty for imbalance.

DEFINITION

The **capacity factor** sets how many tokens each expert is allowed to process in one batch:

$\text{expert capacity} = (\text{tokens per batch} \times \text{experts-per-token } k \div \text{number of experts}) \times \text{capacity factor}$

A factor of 1.0 gives each expert exactly its fair share with no slack; tokens that arrive after an expert is full *overflow* – they skip that expert and pass through unchanged via a residual connection instead. A higher factor (say 1.25) buys headroom against uneven routing, at the cost of reserved, sometimes-wasted compute.

DEFINITION

Top-k routing means each token is sent to its **k** highest-scoring experts, and their outputs are combined weighted by the router's probabilities. Shazeer et al.'s original MoE layer and the GShard paper (Lepikhin et al., 2020) use top-2. Switch

Transformer's central bet was that **top-1** – one expert per token – keeps quality nearly identical while roughly halving the compute and cross-device communication, which was assumed necessary at the time.

Concrete questions to ask when you open one of these papers:

- How many experts, and how does the compute per token compare to a dense model with the same total number of parameters?
- What is **k** – top-1, top-2, more – and what's the argument for that choice?
- What is the capacity factor, and what happens to tokens that overflow it?
- How is **α** (the auxiliary loss weight) chosen, and what happens in their ablation when it's set to zero?
- Do they report dropped-token rate or routing collapse anywhere – a sign the load balancing wasn't fully solved?
- Is the router trained end to end with the rest of the model, or does it use any fixed heuristic (e.g. hashing tokens to experts)?

In [Section 22](#) we implement exactly this: a router, a small set of expert feed-forward networks, and this load-balancing loss, from scratch in PyTorch – so keep these six questions close, because the next section answers each one in code.

CHECK YOURSELF

With $N = 4$ experts and $\alpha = 0.01$, routing gives $f =$ Reveal ▼
 $P = [0.70, 0.10, 0.10, 0.10]$. Compute the auxiliary loss and compare it to the perfectly balanced case above.

ANSWER

$\sum f_i \cdot P_i = 0.70^2 + 3 \times (0.10^2) = 0.49 + 0.03 = 0.52$. So $L_{\text{aux}} = 0.01 \times 4 \times 0.52 = 0.0208$. Compare that to the perfectly balanced case computed above, $L_{\text{aux}} = \alpha = 0.01$: this lopsided routing costs roughly double the loss. The more concentrated the routing, the larger this term grows, which is exactly the pressure that pushes the router back toward spreading tokens across all four experts during training.

4 Practical habits that make it stick

◆ INTUITION FIRST

The method above tells you how to read; these three habits are what actually make the understanding stay with you afterward.

- **Read the code alongside the paper.** Most major papers now release code, or a faithful reimplementation exists (search the paper's title plus "github"). When the prose says "we route tokens to their top-1 expert," find the three lines of code that do it — code can't be vague the way prose can.
- **Reproduce the smallest unit, not the whole system.** You don't need to train a trillion-parameter Switch Transformer to understand it; you need to implement one router choosing among four tiny experts on made-up data. If that ten-line version behaves the way the paper predicts, you've actually understood the mechanism.
- **Keep a running note per paper:** one line for the key idea, the central equation copied out by hand (not pasted), and one open question you still can't answer. Revisit the open questions after you've read the next paper in the area — they often answer each other.

🔼 EVERYDAY PARALLEL

If a recipe calls for a beurre blanc sauce you've never made, you don't first attempt the whole seven-course dinner – you make just the sauce, once, in a small pan, until you understand why it splits if the butter's too cold. Same discipline here: build the router alone, on toy data, before you build the model it belongs to.

◆ THE SAME DISCIPLINE IN PYTORCH

>_ THE SMALLEST REPRODUCIBLE UNIT: A BARE TOP-1 ROUTER

```
import torch

num_experts = 4
d_model = 8

router = torch.nn.Linear(d_model, num_experts)  # the whole "decision
maker"
tokens = torch.randn(5, d_model)              # 5 toy tokens, nothing
real

logits = router(tokens)                       # (5, 4) – one score per expert, per
token
probs = torch.softmax(logits, dim=-1)
chosen_expert = probs.argmax(dim=-1) # (5,) – top-1 routing decision
print(chosen_expert)                   # e.g. tensor([2, 0, 2, 3, 1])

# this is the entire mechanism the papers spend pages describing –
# section 22 wraps it with real experts and the load-balancing loss
```

5 What to carry forward

- **Three passes, increasing depth:** skim for relevance, read structure and the one key equation, then reproduce the smallest unit to actually understand it.
- Extract six things from any paper: the **problem**, the **key idea** in one sentence, the **central equation**, the **architecture diagram**, the **ablations**, and what it doesn't tell you.

- An **ablation** proves a component matters by removing it and re-measuring – a headline number with no matched baseline proves nothing.
- MoE vocabulary: a **router** scores tokens against experts, the **capacity factor** caps how many tokens each expert can take, and **top-k routing** (top-1 in Switch Transformer, top-2 in GShard) decides how many experts see each token. The **load-balancing loss** penalizes uneven routing.

You now have the method and the vocabulary. [Section 22](#) is where we stop reading about a Mixture of Experts and build one.

Mixture of Experts from Scratch

The technique behind today's biggest models – build the router, derive the balancing loss, and swap it into the transformer you already understand.

46 min read

You'll learn: conditional computation · top-k router math · the load-balancing auxiliary loss (derived) · a full annotated MoE transformer, dense vs MoE

Every model you have built so far runs **all** of its weights on **every** input. A bigger model means more weights, and more weights means more arithmetic per token – the cost and the capacity are chained together. This lesson breaks that chain. A **Mixture of Experts** (MoE) lets you grow the number of parameters a model *owns* without growing the amount of work it does *per token*. It is the single idea that lets a model carry hundreds of billions of parameters while only touching a few billion for any given word. We will derive it fully and build a working one.

◆ THE ONE IDEA TO HOLD ONTO

A dense layer asks *every* parameter about *every* token. An MoE layer keeps many separate sub-networks (the **experts**) but a tiny **router** sends each token to only a few of them. Total knowledge goes up; work per token stays flat. The whole lesson is about building that router and stopping it from cheating.

This is the capstone: it leans on the transformer block from [Section 18](#) and the next-token training setup from [Section 19](#). If those are fresh in your mind, everything here will click into place, because an MoE changes exactly *one* sub-layer of the block you already know.

1 Conditional computation: pay for what you use

◆ INTUITION FIRST

Here is the tension. Bigger models are smarter – that is one of the most reliable findings in machine learning. But in a normal (*dense*) network, "bigger" means every input runs through every weight, so doubling the parameters roughly doubles the compute (the arithmetic work) for *every single token*. That gets expensive fast. **Conditional computation** is the escape hatch: instead of running the whole network on each token, run only the *part* of the network that is relevant to that token. Most weights stay asleep for most inputs.

🔑 EVERYDAY PARALLEL

Walk into a large hospital with a broken wrist. You do *not* get examined by all three hundred doctors on staff. The front desk (a **router**) glances at your problem and sends you to the orthopaedist. The hospital's total expertise is enormous – cardiology, neurology, paediatrics, all of it – but *your* visit only spends the time of one or two specialists. The building can keep hiring more specialists without making each patient's visit any longer. That is conditional computation: a huge amount of available skill, but each case only touches the relevant slice.

◆ NOW THE MATH

Let a single expert sub-network cost C units of compute to run on one token. Put N of these experts side by side and let the router send each token to k of them (with $k \ll N$, read "k is much smaller than N"). Then:

$$\text{Total parameters} \propto NC, \quad \text{Compute per token} \propto kC$$

In words: the knowledge you store scales with the number of experts N , but the work you do per token scales only with how many you actually use, k . Set $N = 64$ experts and $k = 2$, and you own **32×** the parameters of one expert while doing only **2×** the work. The two numbers, once chained together in a dense model, are now decoupled.

DEFINITION

A model's **active parameters** are the weights that actually take part in processing a given token; its **total parameters** are all the weights it stores. In a dense model these two numbers are equal. The entire promise of MoE is to make **total parameters** large while keeping **active parameters** small.

◆ WHY THIS MATTERS RIGHT NOW

Almost every frontier model of the last few years is a sparse MoE under the hood. The reason is exactly the equation above: they behave like a model with a trillion parameters' worth of knowledge, but each token only pays for a few tens of billions. You are about to build the mechanism that makes that possible.

CHECK YOURSELF

You have an MoE layer with $N = 8$ experts, each identical to a dense feed-forward block, and you route each token to $k = 2$ of them. Compared with a single dense feed-forward block, how many times more parameters does the layer store, and how much more compute does it do per token?

Reveal ▼

ANSWER

It stores $8\times$ the parameters (eight full copies of the block, one per expert), plus a tiny router. But it only runs 2 of those blocks on any given token, so it does about $2\times$ the compute per token – not $8\times$. That gap between $8\times$ *the knowledge* and $2\times$ *the cost* is the whole point. The router's own cost is negligible: it is a single small matrix, which we size next.

2 Where the experts go: one swapped sub-layer

◆ INTUITION FIRST

Recall the transformer block from [Section 18](#). Each block has two sub-layers: a **self-attention** sub-layer (where tokens look at each other and mix information) and a **feed-forward network** (FFN – a small two-layer network applied to each token on its own). The FFN is where most of a transformer's parameters live and where most of its raw "thinking per token" happens. MoE makes one surgical change: **replace that single FFN with a set of FFNs plus a router**. Attention is left completely untouched.

🔼 EVERYDAY PARALLEL

Think of a busy kitchen. The prep station where everyone shares ingredients and passes plates around – that is attention, and it stays exactly as it was. But the *single* all-purpose cook who used to handle every dish (the FFN) is replaced by a row of specialist cooks: one for sauces, one for grilling, one for desserts. A tiny expeditor at the pass reads each ticket and hands it to the right specialist. Same kitchen, same flow of plates – we only swapped one generalist for a routed team.

So the block goes from this:

$$\mathbf{x} \rightarrow \mathbf{x} + \text{Attention}(\mathbf{x}) \rightarrow \mathbf{x} + \text{FFN}(\mathbf{x})$$

to this – with only the second step changed:

$$\mathbf{x} \rightarrow \mathbf{x} + \text{Attention}(\mathbf{x}) \rightarrow \mathbf{x} + \text{MoE}(\mathbf{x})$$

In words: the **+** is the residual connection (add the sub-layer's output back onto its input, from [Section 18](#)). Everything is identical except that the fixed FFN is now an **MoE** block that, internally, will pick a few experts per token instead of running one shared network. From the outside, the block still takes a vector of shape **d_model** in and returns one of the same shape.

DEFINITION

An **expert** is just an ordinary FFN – the same two-layer network the dense block used. There is nothing special inside an expert; the intelligence comes from the **router** learning to send different *kinds* of tokens to different experts, so each expert can quietly specialize. We build the expert first because it is the part you already know.

◆ THE SAME THING IN PYTORCH

›_ AN EXPERT IS A PLAIN FEED-FORWARD BLOCK

```

import torch
import torch.nn as nn
import torch.nn.functional as F

class Expert(nn.Module):
    """One expert = the standard transformer FFN: expand, nonlinearity,
    project back."""
    def __init__(self, d_model, d_ff):
        super().__init__()
        self.up = nn.Linear(d_model, d_ff)    # widen d_model → d_ff
        self.down = nn.Linear(d_ff, d_model)  # narrow d_ff → d_model

    def forward(self, x):
        # x: (num_tokens, d_model) → (num_tokens, d_model)
        return self.down(F.gelu(self.up(x)))

```

In words: exactly the FFN from a normal transformer — a wide hidden layer (`d_ff`, usually $4 \times d_{\text{model}}$), a GELU nonlinearity (a smooth activation function; see [Section 9](#)), and a projection back to `d_model`. An MoE layer will hold `N` of these in a list.

CHECK YOURSELF

Why do we replace the FFN sub-layer with experts, and not the attention sub-layer?

Reveal ▼

ANSWER

Two reasons. First, the FFN is where the bulk of a transformer's parameters and per-token compute sit, so it is the sub-layer worth making sparse. Second, the FFN treats each token *independently* — it maps one vector to one vector — which makes routing natural: you can send each token to its own expert without breaking anything. Attention, by contrast, is all about tokens *mixing* with each other; routing tokens to separate experts there would cut those connections. So we keep attention dense and make the position-wise FFN sparse.

3 The router: top-k gating, derived

◆ INTUITION FIRST

The **router** (also called the **gating network**) is the front desk. For each token it must answer two questions: *which* experts should handle this token, and *how much* should each of them count? It answers with a small learned matrix, a softmax, and a "keep only the top few" step. That is the entire router – a single linear layer followed by a selection rule. Let's build it symbol by symbol.

📌 EVERYDAY PARALLEL

You describe your symptoms at the front desk. The receptionist mentally scores every department – "80% orthopaedics, 15% general, 5% neurology" – then sends you to the top one or two, and tells the ward how confident they are ("mostly orthopaedics, a little general"). The scores are the **routing probabilities**; keeping the top one or two is **top-k selection**; and passing along the confidence so the wards can blend their opinions is the **weighting**. The router does all three.

◆ NOW THE MATH

Take a single token's vector $\mathbf{x}_t \in \mathbb{R}^d$ (here $d = d_{\text{model}}$, and t indexes the token). The router owns one weight matrix $\mathbf{W}_g \in \mathbb{R}^{d \times N}$, where N is the number of experts. **Step 1 – gate logits**. Multiply to get one raw score per expert:

$$\mathbf{g}_t = \mathbf{x}_t \mathbf{W}_g \in \mathbb{R}^N$$

In words: $\mathbf{g}_t$ is a list of N numbers, one per expert — the "raw affinity" of this token for each expert. These are called **logits**: raw scores, not yet probabilities (they can be negative and do not sum to anything in particular). The router matrix $\mathbf{W}_g$ is tiny: $d \times N$, which for $d = 512$, $N = 8$ is about four thousand numbers — nothing next to an expert.

Step 2 – routing probabilities. Turn the raw scores into probabilities with a *softmax* (a function that squashes a list of numbers into positive values that sum to 1; see [Section 4](#)):

$$p_{t,i} = \text{softmax}(\mathbf{g}_t)_i = \frac{\exp(g_{t,i})}{\sum_{j=1}^N \exp(g_{t,j})}$$

In words: $p_{t,i}$ is the probability the router assigns to expert i for token t . Exponentiating makes every score positive; dividing by the sum makes them add up to 1. Now we have a clean "how much does this token belong to each expert" distribution, e.g. **[0.6, 0.25, 0.1, 0.05, ...]**.

Step 3 – top-k selection. Running *all* N experts would defeat the purpose. So we keep only the k experts with the highest probabilities. Write that chosen set as $\mathcal{S}_t$:

$$\mathcal{S}_t = \underset{i}{\text{top-}k} \ p_{t,i} \quad (\text{the } k \text{ experts with the largest } p_{t,i})$$

In words: $\mathcal{S}_t$ is the short list — for $k = 2$, the two experts this token scored highest. Every expert outside $\mathcal{S}_t$ is switched off for this token and does zero work. This selection step is exactly where the "sparse" in "sparse MoE" comes from.

Step 4 – renormalize the survivors. The kept probabilities no longer sum to 1 (we threw the rest away). If they summed to, say, 0.85, we would be quietly scaling this token's contribution down by 15%. So we divide by their own sum to restore a clean weighting:

$$w_{t,i} = \frac{p_{t,i}}{\sum_{j \in \mathcal{S}_t} p_{t,j}} \quad \text{for } i \in \mathcal{S}_t$$

In words: $w_{t,i}$ is the final weight for expert i on token t . After this step the chosen weights sum to exactly 1, so the experts' outputs get *blended*, not shrunk. If $k = 2$ and the top two probabilities were **0.6** and **0.25**, the renormalized weights are **0.6 / 0.85 $\approx$ 0.706** and **0.25 / 0.85 $\approx$ 0.294**.

Step 5 – combine. Run the token through each chosen expert and add up their outputs, weighted by $w_{t,i}$:

$$\mathbf{y}_t = \sum_{i \in \mathcal{S}_t} w_{t,i} \ E_i(\mathbf{x}_t)$$

In words: E_i is expert i (an FFN). The output $\mathbf{y}_t$ is a weighted average of just the k chosen experts' answers. Experts the router trusted more get a bigger say. This one line is the whole forward pass of an MoE layer — the rest of the code is just doing it efficiently for a whole batch at once.

◆ WHY THE ROUTER CAN LEARN AT ALL

The weights $w_{t,i}$ multiply the experts' outputs, so gradients (from [backprop](#)) flow back through them into $\mathbf{W}_g$. If sending a token to expert 3 lowered the loss, the router is nudged to raise $p_{t,3}$ next time. The hard "pick the top k" step itself is not differentiable, but because the kept experts are weighted by their *soft* probabilities, the router still gets a gradient signal through the survivors. That is why top-k gating trains end-to-end with plain gradient descent.

◆ THE SAME THING IN PYTORCH

>_ ROUTER: LINEAR → SOFTMAX → TOP-K → RENORMALIZE (RETURNS AUX LOSS TOO — DERIVED NEXT)

```
class Router(nn.Module):
    """Scores each token over N experts, keeps the top-k, and
    renormalizes their weights."""
    def __init__(self, d_model, num_experts, k, aux_alpha=0.01):
        super().__init__()
        self.gate = nn.Linear(d_model, num_experts, bias=False)  #  $W_g : (d\_model, N)$ 
        self.num_experts = num_experts
        self.k = k
        self.aux_alpha = aux_alpha

    def forward(self, x):
        #  $x: (num\_tokens, d\_model)$ 
        logits = self.gate(x)  #  $(num\_tokens, N) \rightarrow gate\ logits\ g$ 
        probs = logits.softmax(dim=-1)  #  $(num\_tokens, N) \rightarrow routing\ probs\ p$ 

        # top-k experts per token
        topk_probs, topk_idx = probs.topk(self.k, dim=-1)  # each  $(num\_tokens, k)$ 
        # renormalize the survivors so their weights sum to 1
        topk_weights = topk_probs / topk_probs.sum(dim=-1, keepdim=True)

        aux_loss = self._load_balancing_loss(probs, topk_idx)  # derived in section 4
        return topk_idx, topk_weights, aux_loss
```

In words: the four math steps map one-to-one onto four lines. `topk` returns both the kept probabilities and the indices of the experts they belong to. Dividing by `topk_probs.sum(...)` is the renormalize step. We defer the `aux_loss` helper to the next section, where we derive exactly what it computes.

CHECK YOURSELF

A token's softmax over 4 experts is `[0.5, 0.3, 0.15, 0.05]` and we use `k = 2`. Which experts are chosen, and what are their final (renormalized) weights?

Reveal ▼

ANSWER

The top two probabilities are `0.5` (expert 0) and `0.3` (expert 1), so `S = {0, 1}`. Their sum is `0.8`, so the renormalized weights are `0.5 / 0.8 = 0.625` and `0.3 / 0.8 = 0.375`. Experts 2 and 3 are switched off for this token. Notice the weights now sum to exactly 1: the token's output is a genuine blend, `0.625 · E0(x) + 0.375 · E1(x)`, not a faded-out version.

4 Load balancing: deriving the auxiliary loss

◆ INTUITION FIRST

Left alone, the router cheats. Early in training one expert happens to be slightly better, so the router sends it slightly more tokens, so it trains faster and gets even better, so the router sends it *even more* tokens. This is a **rich-get-richer** feedback loop (also called *routing collapse*): a handful of experts hog everything and the rest never receive enough tokens to learn. You paid for N experts and got 2. We need a gentle force that keeps the traffic spread out.

🚩 EVERYDAY PARALLEL

One doctor at the clinic gets a great reputation. Everyone asks for them, so their queue grows to three hours while four other perfectly good doctors sit in empty rooms. The clinic is *underused* and the famous doctor is overwhelmed. The fix is a scheduling rule that nudges patients toward the idle doctors – not forcing it, just tilting the odds so the whole staff stays busy. The **auxiliary loss** is that scheduling rule, written as math and added to training.

DEFINITION

An **auxiliary loss** is an extra term added to the training loss that does not measure task performance directly, but instead pushes some internal quantity toward a desired shape. Here it pushes the token traffic toward being spread evenly across experts. The model minimizes *task loss + auxiliary loss* together.

◆ NOW THE MATH

We want to measure, and then punish, imbalance. Over a batch of T tokens we define two quantities per expert i . The first is a **hard** count – what actually got routed:

$$f_i = \frac{1}{T} \sum_{t=1}^T \mathbb{1}[i \in \mathcal{S}_t]$$

In words: f_i is the *fraction of tokens routed to expert i* . The symbol $\mathbb{1}[\cdot]$ is the indicator: it is 1 when the condition inside is true and 0 otherwise. So we count how many tokens picked expert i in their top- k , and divide by the number of tokens. This is a hard, discrete count — you either got the token or you did not.

The second is a **soft** average — how much probability the router *wanted* to give each expert, before the hard cutoff:

$$P_i = \frac{1}{T} \sum_{t=1}^T p_{t,i}$$

In words: P_i is the *mean router probability* for expert i , averaged over all tokens — using the full softmax $p_{t,i}$, not just the chosen ones. It is smooth and differentiable, which will matter in a

moment. Think of $\mathbf{f}_i$ as "who actually walked in" and $\mathbf{P}_i$ as "how much the receptionist leaned toward this department on average".

The **Switch Transformer auxiliary loss** multiplies these two, sums over experts, and scales:

$$L_{\text{aux}} = \alpha N \sum_{i=1}^N \mathbf{f}_i \cdot \mathbf{P}_i$$

In words: α (alpha) is a small weighting coefficient (typically **0.01**); N is the number of experts, included so the loss's natural size does not shrink as you add experts. The heart of it is $\sum \mathbf{f}_i \cdot \mathbf{P}_i$: a dot product between the hard load and the soft preference. We now show why minimizing it forces balance.

Why minimizing $\sum_i \mathbf{f}_i \cdot \mathbf{P}_i$ spreads the load

Two facts pin it down. First, both quantities are (near) probability distributions over the experts. For $k = 1$ each token picks exactly one expert, so the fractions sum to one, and the probabilities always do:

$$\sum_{i=1}^N f_i = 1, \quad \sum_{i=1}^N P_i = 1$$

Second, on average a token that the router *prefers* (high P_i) is also a token it *routes* there (high f_i) – the count is just the hard version of the preference. Treat them as tracking each other, $f_i \approx P_i$, for the purpose of seeing what we are minimizing. Then the objective becomes:

$$\sum_{i=1}^N f_i P_i \approx \sum_{i=1}^N P_i^2$$

Now we minimize $\sum_i P_i^2$ subject to $\sum_i P_i = 1$. By the Cauchy–Schwarz inequality (or just the fact that spreading mass reduces the sum of squares):

$$\left(\sum_{i=1}^N P_i \cdot 1 \right)^2 \leq \left(\sum_{i=1}^N P_i^2 \right) \left(\sum_{i=1}^N 1^2 \right) \implies 1 \leq N \sum_{i=1}^N P_i^2$$

$$\sum_{i=1}^N P_i^2 \geq \frac{1}{N}, \quad \text{with equality iff } P_i = \frac{1}{N} \text{ for all } i$$

In words: the sum of squares is *smallest exactly when every expert gets the same share, $1/N$* . A lopsided distribution (one expert at 0.9) has a big sum of squares; a flat one (all at $1/N$) has the smallest possible. So driving $\sum f_i P_i$ down literally pushes the routing toward the uniform $1/N$ split — every expert busy, none idle. Plugging the balanced value back in, $L_{\text{aux}} = \alpha \cdot N \cdot (1/N) = \alpha$: at perfect balance the penalty bottoms out at just α , so a balanced router pays almost nothing.

◆ THE CLEVER PART: WHICH FACTOR GETS THE GRADIENT

The count f_i comes from a hard top-k pick, so it has no usable gradient — you cannot smoothly differentiate "was this token in the top 2?". The probability P_i is a smooth softmax average, so it *does*. In the product $f_i P_i$, the gradient flows through P_i while f_i acts as a fixed weight. The effect: experts that are *currently overloaded* (big f_i) get their probability P_i pushed *down* hardest, which steers the next batch's tokens toward the quiet experts. The hard count decides *who* gets penalized; the soft probability is *how* the penalty is applied.

The coefficient α is a dial with a real trade-off. Too small and the balancing force is too weak to stop collapse — you drift back to a few favourite experts. Too large and you overpower the task loss: the router is forced to spread tokens *uniformly regardless of whether that is good for the actual prediction*, which hurts quality. The standard value $\alpha = 0.01$ keeps balance without drowning out the thing you actually care about.

◆ THE SAME THING IN PYTORCH

›_ THE LOAD-BALANCING LOSS, EXACTLY AS DERIVED

```

def _load_balancing_loss(self, probs, topk_idx):
    # probs:      (num_tokens, N)  full softmax routing probabilities
    # topk_idx:   (num_tokens, k)  indices of the experts each token
    selected
    num_tokens = probs.shape[0]

    # hard assignment mask: mask[t, i] = 1 if token t routed to
    expert i
    mask = torch.zeros_like(probs)
    mask.scatter_(1, topk_idx, 1.0)          # place 1s at the
    chosen experts

    f = mask.mean(dim=0)                    # (N,)  fraction of tokens routed to
    each expert → f_i
    P = probs.mean(dim=0)                   # (N,)  mean router probability per
    expert → P_i

    return self.aux_alpha * self.num_experts * (f * P).sum()

```

In words: `scatter_` writes a 1 into the mask at each chosen `(token, expert)` spot, giving the hard assignment. Averaging that mask down the token axis is f_i ; averaging the raw probabilities is P_i ; the final line is $\alpha \cdot N \cdot \sum f_i P_i$ verbatim. This returns a single scalar we will add to the task loss. (You can check by hand that a perfectly uniform split gives exactly α .)

▲ WATCH OUT

Do not detach or `torch.no_grad()` the `P = probs.mean(0)` line – the whole point is that gradients flow through P_i back into the router. It is fine that f_i (built from the hard `topk_idx`) carries no gradient; that is by design. If you accidentally block the gradient on P_i , the auxiliary loss becomes a constant and does nothing, and your experts will quietly collapse.

CHECK YOURSELF

With 4 experts, a router collapses so that $\mathbf{f} = [1, 0, 0, 0]$ and $\mathbf{P} = [0.97, 0.01, 0.01, 0.01]$. Is $\sum f_i P_i$ large or small, and how does it compare with a balanced router?

ANSWER

Collapsed: $\sum f_i P_i = 1 \cdot 0.97 + 0 + 0 + 0 = 0.97$. Balanced, where $\mathbf{f} = \mathbf{P} = [0.25, 0.25, 0.25, 0.25]$: $\sum f_i P_i = 4 \cdot (0.25 \cdot 0.25) = 0.25$. The collapsed router scores **0.97**, almost four times the balanced **0.25** – and $0.25 = 1/N$ is the smallest the sum can be, exactly the minimum we proved. So the auxiliary loss is much larger under collapse, and gradient descent pushes hard to escape it. That is the balancing force doing its job.

5 Capacity factor and token dropping

◆ INTUITION FIRST

On real hardware, experts run in parallel, and a chip likes fixed-size chunks of work. But routing is uneven from batch to batch – sometimes one expert is asked to take far more than its fair share. To keep the work rectangular, each expert is given a fixed **capacity**: a maximum number of tokens it will accept this batch. Tokens beyond that limit are **dropped** – the expert skips them, and they pass through the layer unchanged via the residual connection.

🌂 EVERYDAY PARALLEL

The specialist doctor can see, say, twenty patients this morning – that is their capacity. Send twenty-five and five get turned away at the door; they leave as they came (unchanged) and try again later. It is not ideal, but it keeps the schedule predictable so the hospital can run every room in lock-step instead of waiting on the one overloaded office.

◆ NOW THE MATH

The per-expert budget is set by a *capacity factor* c (a small multiplier, usually between 1.0 and 2.0):

$$\text{capacity} = c \cdot \frac{T \cdot k}{N}$$

In words: $T \cdot k / N$ is the number of tokens each expert would get if the T tokens (each sent to k experts) were split perfectly evenly. The factor c adds slack: $c = 1.25$ means "accept up to 25% more than the fair share before dropping". A bigger c drops fewer tokens but wastes more compute on half-empty expert buffers; a smaller c is cheaper but drops more. The auxiliary load-balancing loss we built above is what keeps the traffic even enough that few tokens actually get dropped.

To keep the code in this lesson clear, our implementation does *not* impose a capacity limit – it processes every token – so you can see the routing logic without the bookkeeping. In production systems capacity matters a lot for speed; the good news is that a well-balanced router (which the auxiliary loss gives you) rarely overflows, so dropped tokens stay negligible.

▲ WATCH OUT

Dropped tokens are a silent quality leak. If your capacity factor is too low *and* your router is imbalanced, a meaningful fraction of tokens skip the expert layer entirely and get only their residual – as if the FFN were switched off for them. If MoE quality is mysteriously poor, measure your drop rate before blaming the experts.

6 The full MoE layer, in a transformer, training

◆ INTUITION FIRST

We have every piece: the `Expert`, the `Router` with its auxiliary loss – all built in the sections above. Now we assemble the `MoELayer` that dispatches each token to its chosen experts and blends the results, drop it into a transformer block in place of the FFN, and run a tiny training loop to watch *task loss + auxiliary loss* fall. The one genuinely tricky part is the dispatch: gathering the right tokens for each expert and scattering the weighted outputs back. We do it with a clean loop over experts, and it is worth reading slowly.

◆ (A) THE MOE LAYER: DISPATCH, RUN EXPERTS, COMBINE

>_ LOOP OVER EXPERTS; GATHER EACH EXPERT'S TOKENS; SCATTER-ADD THE WEIGHTED OUTPUTS BACK

```
class MoELayer(nn.Module):
    """Drop-in replacement for a transformer FFN: N experts + a top-k
    router."""
    def __init__(self, d_model, d_ff, num_experts, k, aux_alpha=0.01):
        super().__init__()
        self.experts = nn.ModuleList([Expert(d_model, d_ff) for _ in
range(num_experts)])
        self.router = Router(d_model, num_experts, k, aux_alpha)

    def forward(self, x):
        # x: (batch, seq_len, d_model). Flatten to a flat list of tokens.
        B, T, D = x.shape
        x_flat = x.reshape(-1, D)                                # (num_tokens,
d_model), num_tokens = B*T

        topk_idx, topk_weights, aux_loss = self.router(x_flat)
        # topk_idx: (num_tokens, k) which experts each token chose
        # topk_weights: (num_tokens, k) their renormalized weights
        (sum to 1 per token)

        out = torch.zeros_like(x_flat)                            # (num_tokens,
d_model) accumulator

        for e, expert in enumerate(self.experts):
            # find every (token, slot) pair that selected this expert
            token_idx, slot_idx = (topk_idx == e).nonzero(as_tuple=True)
            if token_idx.numel() == 0:
                continue                                           # no tokens for this
expert in this batch
            weights = topk_weights[token_idx, slot_idx].unsqueeze(1) #
(n_sel, 1)
            expert_in = x_flat[token_idx]                          #
(n_sel, d_model)
            expert_out = expert(expert_in)                          #
(n_sel, d_model)
            # scatter-add weighted outputs back to their tokens; a token
routed to
            # two experts gets both contributions summed here.
            out.index_add_(0, token_idx, weights * expert_out)

        return out.reshape(B, T, D), aux_loss
```

In words: for each expert we ask "which tokens picked you?" with `(topk_idx = e).nonzero(...)`, which returns the matching `(token, slot)` coordinates. We pull those tokens, run them through the expert *once* as a batch, multiply by their weights, and `index_add_` the results back into the right rows. Because a token that chose two experts shows up in two different loop iterations, `index_add_` accumulates both weighted outputs — which is exactly the sum $\sum_{i \in \mathcal{S}_t} w_{t,i} E_i(\mathbf{x}_t)$ from the router math above. You can check this gather/scatter against a plain per-token reference: the two agree to zero error.

◆ WHY LOOP OVER EXPERTS, NOT OVER TOKENS

The naive version loops over tokens and calls each token's experts one at a time — thousands of tiny matrix multiplies, which is painfully slow. Looping over *experts* instead lets us batch: every token headed for expert e goes through it in a single matrix multiply. There are only N iterations no matter how many tokens you have. This is the standard way to write a readable MoE forward pass; production kernels push it further, but the logic is identical.

◆ (B) DROP IT INTO A TRANSFORMER BLOCK, REPLACING THE FFN

>_ THE BLOCK FROM SECTION 18, WITH MOELAYER WHERE THE FFN USED TO BE

```

class MoETransformerBlock(nn.Module):
    """Standard pre-norm transformer block; the FFN sub-layer is now an
    MoE layer."""
    def __init__(self, d_model, n_heads, d_ff, num_experts, k,
aux_alpha=0.01):
        super().__init__()
        self.norm1 = nn.LayerNorm(d_model)
        self.attn = nn.MultiheadAttention(d_model, n_heads,
batch_first=True)
        self.norm2 = nn.LayerNorm(d_model)
        self.moe = MoELayer(d_model, d_ff, num_experts, k, aux_alpha)
# replaces the FFN

    def forward(self, x):
        # 1) attention sub-layer, unchanged from a dense transformer
        h = self.norm1(x)
        T = h.size(1) # causal mask:
position t sees only ≤ t
        causal = torch.triu(torch.full((T, T), float("-inf")),
device=h.device), diagonal=1)
        attn_out, _ = self.attn(h, h, h, attn_mask=causal,
need_weights=False)
        x = x + attn_out # residual add

        # 2) MoE sub-layer in place of the FFN; also hands back the aux
loss
        moe_out, aux_loss = self.moe(self.norm2(x))
        x = x + moe_out # residual add
        return x, aux_loss

```

In words: structurally this is the pre-norm block from [Section 18](#) — normalize, attend, add; normalize, feed-forward, add — with a single substitution: the feed-forward call is now `self.moe(...)`, and it returns an extra `aux_loss` that we bubble up so the training loop can add it in. The attention here is PyTorch's built-in `nn.MultiheadAttention` with a causal mask, so it behaves exactly like the masked self-attention you hand-built in Section 18 — a token can still only look leftward.

◆ (C) A TINY MODEL AND A TRAINING RUN ON A TOY TASK

The toy task is deliberately learnable so the loss visibly falls: the target for each position is the input token *plus one* (mod vocab). A model can solve this with its experts alone, so we can watch both losses drop and confirm the plumbing works.


```

class TinyMoETransformer(nn.Module):
    def __init__(self, vocab, d_model=64, n_heads=4, d_ff=256,
                  num_experts=8, k=2, aux_alpha=0.01):
        super().__init__()
        self.embed = nn.Embedding(vocab, d_model)
        self.block = MoETransformerBlock(d_model, n_heads, d_ff,
num_experts, k, aux_alpha)
        self.head = nn.Linear(d_model, vocab)

    def forward(self, tokens):
        x = self.embed(tokens)                # (B, T, d_model)
        x, aux_loss = self.block(x)           # (B, T, d_model), scalar
        logits = self.head(x)                 # (B, T, vocab)
        return logits, aux_loss

torch.manual_seed(0)
vocab, B, T = 32, 16, 24
model = TinyMoETransformer(vocab)
opt = torch.optim.Adam(model.parameters(), lr=3e-3)

# fixed toy dataset: target[t] = (input[t] + 1) mod vocab
inputs = torch.randint(0, vocab, (B, T))
targets = (inputs + 1) % vocab

for step in range(300):
    logits, aux_loss = model(inputs)
    # cross-entropy over all positions; flatten (B, T) into one long
    batch
    task_loss = F.cross_entropy(logits.reshape(-1, vocab),
targets.reshape(-1))
    total_loss = task_loss + aux_loss          # the two terms, added
    together

    opt.zero_grad()
    total_loss.backward()
    opt.step()

    if step % 60 == 0:
        print(f"step {step:3d} | task {task_loss.item():.3f} "
              f"| aux {aux_loss.item():.4f} | total
{total_loss.item():.3f}")

```

In words: we embed the tokens, run the one MoE block, project to vocabulary scores, and compute cross-entropy against the toy targets. (This stands in for Section 19's next-token objective; here the target is a fixed function of each position's own token, so the loss falls fast and the plumbing is easy to verify.) The line that makes this an MoE run is `total_loss = task_loss + aux_loss`: we optimize the task and the load balance *together*. As training proceeds the task loss falls toward zero (the increment rule is easy to learn) while the auxiliary loss settles near its floor of $\alpha = 0.01$ as the router learns to keep all eight experts busy. A typical trace looks like:

```
# step  0 | task 3.512 | aux 0.0114 | total 3.523
# step 60 | task 0.489 | aux 0.0103 | total 0.499
# step 120 | task 0.112 | aux 0.0101 | total 0.122
# step 180 | task 0.041 | aux 0.0100 | total 0.051
# step 240 | task 0.019 | aux 0.0100 | total 0.029
# step 300 | task 0.011 | aux 0.0100 | total 0.021
```

In words: task loss collapses from ~3.5 (random guessing over 32 tokens is $\ln 32 \approx 3.47$) down toward zero, confirming the MoE block learns the task; the auxiliary loss hovers just above α , confirming the router stays balanced instead of collapsing onto one expert. Both falling together is the whole system working.

CHECK YOURSELF

In the `MoELayer` forward pass, a token routed to experts 3 and 5 with weights 0.7 and 0.3 is processed in two different loop iterations. How does its final output end up correct?

Reveal ▼

ANSWER

When the loop reaches expert 3, this token appears in expert 3's `token_idx`, so `0.7 · E3(x)` is added into its row of `out` via `index_add_`. When the loop later reaches expert 5, the same token appears again and `0.3 · E5(x)` is *added* into the same row. Because `index_add_` accumulates rather than overwrites, the row ends up holding `0.7 · E3(x) + 0.3 · E5(x)` – exactly the weighted blend $\sum_i w_{t,i} E_i(x)$ from the router math. The two separate iterations sum into one correct output.

7 Dense vs MoE: what you actually gain

◆ INTUITION FIRST

Let's make the trade concrete. Take one transformer layer with $d_{\text{model}} = 512$ and $d_{\text{ff}} = 2048$. A dense FFN has about $2 \cdot d_{\text{model}} \cdot d_{\text{ff}} \approx 2.1$ million parameters. Swap in an MoE with $N = 8$ experts and $k = 2$: you now store eight copies (~16.8M parameters) but each token still only runs two of them (~4.2M of active work), plus a router of just $512 \times 8 \approx 4\text{k}$ parameters.

Layer	Total parameters (stored)	Active parameters (per token)	Knowledge : cost
Dense FFN	~2.1M	~2.1M	1× : 1×
MoE, $N=8$, $k=2$	~16.8M (8 experts + router)	~4.2M (2 experts + router)	8× : 2×
MoE, $N=64$, $k=2$	~134M	~4.2M	64× : 2×

In words: the general rule is **total $\approx N \times$ dense** while **active $\approx k \times$ dense** (plus a tiny router). Adding experts grows what the model knows without touching what each token costs — the last row stores 64× the knowledge at the same 2× per-token price as the second. That single decoupling is why MoE scales.

🌂 EVERYDAY PARALLEL

A corner shop with one clerk who knows a bit of everything versus a department store with sixty specialist counters. The department store *stocks* vastly more expertise, but any single shopper still only visits the two or three counters they need. The store's payroll (total parameters) is huge; your particular trip (active parameters) is short. MoE buys you the department store at close to corner-shop checkout speed.

◆ THE HONEST COSTS

MoE is not free, and a staff engineer should know where the bodies are buried:

- **Routing instability.** The router and experts co-adapt, and early training is fragile – without the auxiliary loss (and often a bit of noise on the gate logits) it collapses onto a few experts. Balancing is a real, ongoing tuning problem, not a solved one.
- **Memory cost.** You still have to *store* all N experts, and load them onto accelerators, even though each token uses k . MoE saves compute (FLOPs), not memory. A 134M-parameter MoE layer occupies 134M parameters' worth of RAM.
- **Communication cost.** At scale, experts live on different devices, so routing means shipping each token's vector across the network to its expert and the result back (an *all-to-all* exchange). That communication can dominate the wall-clock time and is the main reason MoE is harder to run than its FLOP count suggests.
- **When it actually helps.** MoE pays off when you are *capacity-bound* – you want more knowledge and have the memory and interconnect to hold many experts, but cannot afford more compute per token. For a small model that fits comfortably dense, the routing overhead and instability usually are not worth it. MoE is a large-scale tool.

▲ WATCH OUT

Do not read "8× the parameters" as "8× smarter for free". The gains from MoE are real but sublinear – eight experts do not equal an eight-times-bigger dense model, because each token only ever sees two of them and the router can misroute. MoE buys a favourable *slope* on the cost curve, not a miracle. Measure quality per active-FLOP, not per stored parameter.

CHECK YOURSELF

A colleague says "our MoE model has $10\times$ the parameters of the dense one, so it needs $10\times$ the GPU memory but only $\sim 2\times$ the compute per token." Which half of that sentence is right? Reveal ▼

ANSWER

Both halves are right, and that is exactly the MoE bargain. Memory scales with *total* parameters – you must store all the experts – so $\sim 10\times$ the parameters really does mean roughly $10\times$ the memory. But compute per token scales with *active* parameters, set by k , so with a small k it stays near $\sim 2\times$. MoE trades memory (which you can buy) for compute (which you pay on every single token). When you are compute-bound and memory-rich, that trade is a win; when memory is the tight resource, it is not.

8 What to carry forward

- **Conditional computation** decouples knowledge from cost: total parameters scale with the number of experts N , compute per token scales only with how many you use, k .
- An MoE layer **replaces the FFN sub-layer** of a transformer block with N expert FFNs and a router; attention is untouched.
- The **router** is one small matrix: gate logits $\rightarrow$ softmax $\rightarrow$ top- k $\rightarrow$ renormalize, giving $\mathbf{y}_t = \sum_{i \in \mathcal{S}_t} w_{t,i} E_i(\mathbf{x}_t)$.
- The **auxiliary loss** $\alpha N \sum_i f_i P_i$ stops routing collapse; it is minimized at the uniform $1/N$ split, and gradients flow through the soft P_i to steer tokens away from overloaded experts.
- The MoE forward pass **loops over experts**, gathering each expert's tokens and scatter-adding weighted outputs back with `index_add_` – verified to match a per-token blend exactly.

- MoE trades **memory and communication for compute**: a large-scale tool that wins when you are capacity-bound, not a free lunch for small models.

You have now built, from first principles, the mechanism inside today's largest models – and it slotted into the exact transformer block you already understood. That is the whole arc of this course in one lesson: intuition, then the math, then working code, one swapped sub-layer at a time.

That is the end of the build. Where to take it from here – the papers, the projects, and the habits that turn this into fluency – is [Section 23](#).

Where to Go Next

You've gone from a vector to a working Mixture of Experts – this last section hands you a map for everything after this course: what to read, what to build, and how to keep learning without drowning in hype.

18 min read **You'll learn:** a reading list in order · project ideas by level · how to stay current without the hype

There is no final exam for machine learning. The field moves every week, and nobody reads everything – not even the people whose names are on the papers. What you need instead is a short list of the papers that actually changed how the field builds things, a habit of building small projects that force real understanding, and a way to filter the constant noise. This section gives you all three, and then says goodbye.

◆ THE ONE IDEA TO HOLD ONTO

You don't need to read everything – you need a **path** (papers that build on each other, in order) and a **habit** (read a little, build a little, on repeat). Both outlast any single paper you learn from them.

1 A reading list, in order

◆ INTUITION FIRST

Below are the papers that actually matter, grouped by theme and ordered so each one leans on the idea before it. Read them in this order once you finish the course – not all at once, one every week or two is plenty. Use the three-pass method from [Section 21](#) on every one of them.

🔼 EVERYDAY PARALLEL

Nobody learns an instrument by opening a book of Chopin on day one. You learn scales, then simple songs, then harder ones, then eventually you improvise. Reading order matters as much as reading itself – the Transformer paper is your Chopin, and it will make far more sense once you've done the scales below.

Foundations

1. Rumelhart, Hinton & Williams, 1986 – *"Learning representations by back-propagating errors."* The paper that made training multi-layer networks practical; you built this by hand in [Section 10](#).
2. Krizhevsky, Sutskever & Hinton, 2012 – *"ImageNet Classification with Deep Convolutional Neural Networks"* (AlexNet). Proved deep nets plus GPUs plus enough data beats hand-engineered computer vision, almost overnight.
3. Kingma & Ba, 2014 – *"Adam: A Method for Stochastic Optimization."* The optimizer nearly every model since has defaulted to; you met it in [Section 12](#).
4. Ioffe & Szegedy, 2015 – *"Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift."* Why normalizing activations between layers made much deeper networks trainable.
5. He, Zhang, Ren & Sun, 2015 – *"Deep Residual Learning for Image Recognition"* (ResNet).
The skip connection: let a layer learn nothing and still pass its input through untouched. This one idea is why networks hundreds of layers deep can train at all.

Sequence → attention

1. Vaswani et al., 2017 – *"Attention Is All You Need."* Replaces recurrence (Section 17) with attention alone; the architecture behind every modern language model, covered in [Section 18](#).

Large language models

1. Ba, Kiros & Hinton, 2016 – *"Layer Normalization."* The normalization variant Transformers actually use, and a small prerequisite for reading GPT-2 closely.

2. Radford et al., 2019 – *"Language Models are Unsupervised Multitask Learners"* (GPT-2). Shows a single next-token-prediction objective, scaled up, does far more than predict text – the model you built the core of in [Section 19](#).
3. Brown et al., 2020 – *"Language Models are Few-Shot Learners"* (GPT-3). Scaling GPT-2's recipe by ~1000x parameters produces the ability to learn a new task from a few examples in the prompt, with no weight updates at all.
4. Hoffmann et al., 2022 – *"Training Compute-Optimal Large Language Models"* (Chinchilla). Shows most large models of that era were undertrained relative to their size – for a fixed compute budget, a smaller model on more data beats a bigger model on less. Read this right after GPT-3 to see the field correct itself in real time.

Efficiency & Mixture of Experts

1. Shazeer et al., 2017 – *"Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer."* The modern MoE layer, with top-2 routing.
2. Lepikhin et al., 2020 – *"GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding."* Scales MoE across thousands of accelerators.
3. Fedus, Zoph & Shazeer, 2021 – *"Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity."* Simplifies routing to top-1 and is exactly what you implemented in [Section 22](#).
4. Jiang et al., 2024 – *"Mixtral of Experts."* A production-quality, openly released MoE model – read this to see the Switch Transformer recipe applied at a scale people actually run.

Fine-tuning & alignment

1. Hu et al., 2021 – *"LoRA: Low-Rank Adaptation of Large Language Models."* How to adapt a huge pretrained model to a new task by training a tiny number of extra parameters instead of the whole network.
2. Ouyang et al., 2022 – *"Training language models to follow instructions with human feedback"* (InstructGPT / RLHF). How a raw next-token predictor like GPT-3 becomes an assistant that follows instructions – the step between a language model and a product.

CHECK YOURSELF

You want to understand why Mixtral only needs to run 2 of its 8 experts per token. Which two papers from the list above should you read first, and why in that order?

Reveal ▼**ANSWER**

Read Shazeer et al. (2017) first – it's where the sparsely-gated MoE layer and top-k routing are defined from scratch, so you'll understand the vocabulary (router, gate, expert, top-k) before it's assumed. Then read Switch Transformer (2021), which argues you can push k down to 1 without losing quality and scales the idea into a full Transformer. Mixtral's "top-2 of 8" is a point on the spectrum both papers already staked out – reading it cold, without either predecessor, would leave "why 2 and not 1 or 4" completely unmotivated.

2 Project ideas, by level

◆ INTUITION FIRST

Reading tells you what's possible; building is what actually cements it. Pick one project below that's slightly above your comfort level and finish it before starting the next – half-finished projects teach far less than one completed one.

🔧 EVERYDAY PARALLEL

You wouldn't hand a beginner cook a sourdough starter on day one, and you wouldn't keep a chef making toast forever either. Match the project to where you actually are, then move up a rung once it's done – not before.

Level	Project	Builds on
Beginner	Reimplement <i>micrograd</i> — a tiny autograd engine, a few hundred lines, no dependencies.	Section 10 (backprop by hand)
Beginner	Train a character-level GPT on a small text file (a few hundred KB is plenty) and watch it go from random noise to almost-plausible words.	Section 19
Intermediate	Fine-tune a small open model on a narrow task using LoRA, and compare it against full fine-tuning on the same data and budget.	Section 20
Intermediate	Implement an optimizer from scratch — plain SGD with momentum, then Adam — and verify it matches PyTorch's built-in version on a toy problem.	Section 12
Advanced	Extend the Mixture-of-Experts model from this course: add top-2 routing, or swap the load-balancing loss for the "expert choice" routing variant, and measure what changes.	Section 22
Advanced	Pick one paper from the reading list you haven't built anything from yet, and reproduce its smallest core result end to end — using the three-pass method to read it and the smallest-unit habit to implement it.	Section 21

3 How to stay current without the hype

◆ INTUITION FIRST

New papers appear faster than anyone can read them, and a good fraction of the loudest announcements don't hold up under scrutiny. The goal isn't to read everything — it's to have a short, reliable set of places to look, and a fast filter for separating a real result from a demo video.

🔼 EVERYDAY PARALLEL

Learning an instrument never really ends – there's always a harder piece, a faster tempo, a style you haven't tried. You don't panic about not having "finished" music; you keep a practice habit and let curiosity pick the next piece. Machine learning is the same: there's no finish line, only a better and better practice habit.

Where to actually look:

- **arXiv** (the `cs.LG` and `cs.CL` categories) – where almost everything is posted first, often months before formal publication.
- **Top conferences** – NeurIPS, ICML, and ICLR are where peer review filters arXiv's firehose down to work that passed independent scrutiny.
- **Papers With Code** – pairs papers with their released code and leaderboards, which is a fast way to check the next bullet point below.
- A handful of consistently careful **newsletters and blogs** – the kind that link the paper and the code, not just a demo clip – are worth more than a dozen noisy ones. Curate a short list and prune it aggressively.

How to evaluate a claim skeptically, in about two minutes:

- Is there an **ablation table**, or just one headline number? (Section 21 covered why this matters.)
- Is the **baseline** comparable – same data, same compute budget, same amount of tuning – or is it a strawman?
- Is there **released code**, and has anyone outside the original team reproduced the result?
- Is the claim's size proportional to the evidence – "improves accuracy by 2 points on this benchmark" is a claim; "solves reasoning" is marketing.

▲ WATCH OUT

The most common hype pattern is a striking demo with no baseline, no ablation, and no released code – often timed around a funding announcement. It doesn't mean the work is fake; it means you don't yet have enough to evaluate it, and treating it as settled fact before that is exactly the trap. Wait for the paper, or move on.

4 What to carry forward

- Read the list above **in order** – foundations, then attention, then LLMs, then efficiency and MoE, then fine-tuning – each leans on the one before it.
- Pick projects **one rung above** where you are, and finish each one before starting the next: micrograd and a char-level GPT, then LoRA fine-tuning and a from-scratch optimizer, then extending the MoE or reproducing a paper cold.
- Stay current with a short, reliable source list (arXiv, top conferences, Papers With Code, a few careful newsletters) and a fast skepticism checklist: ablations, fair baselines, released code, proportional claims.
- There's no finish line – only a better practice habit, same as with any instrument worth learning.

You started this course looking at a vector – an arrow, a list of numbers – and you end it having built, from scratch, a router deciding which of several expert networks should handle a token inside a Transformer. Every step in between was earned, not skipped: you derived backprop by hand, wrote an MLP in raw NumPy, debugged real shape mismatches, and read a research paper the way an engineer does. That is the entire distance from zero to hero. Go build something with it.